



Developing Solutions

STUDENT GUIDE

aras
— *INNOVATOR*

Copyright © 2023 by Aras Corporation. This material may be distributed only subject to the terms and conditions set forth in the Open Publication License, V1.0 or later (the latest version is presently available at <http://www.opencontent.org/openpub/>).

Distribution of substantively modified versions of this document is prohibited without the explicit permission of the copyright holder.

Distribution of the work or derivative of the work in any standard (paper) book form for a commercial purpose is prohibited unless prior permission is obtained from the copyright holder.

Aras Innovator, Aras, and the Aras Corp "A" logo are registered trademarks of Aras Corporation in the United States and other countries.

All other trademarks referenced herein are the property of their respective owners.

Microsoft, Office, SQL Server, IIS and Windows are either registered trademarks or trademarks of Microsoft Corporation in the United States and/or other countries.

Notice of Liability

The information contained in this document is distributed on an "As Is" basis, without warranty of any kind, express or implied, including, but not limited to, the implied warranties of merchantability and fitness for a particular purpose or a warranty of non-infringement. Aras shall have no liability to any person or entity with respect to any loss or damage caused or alleged to be caused directly or indirectly by the information contained in this document or by the software or hardware products described herein.

Revision January 2023

Table of Contents

Introduction.....	5
Unit 1 Overview	11
Unit 2 Exploring AML	21
Unit 3 Creating a Method	59
Unit 4 Debugging and Logging Client/Server Methods	83
Unit 5 Creating Custom Actions.....	99
Unit 6 Exploring the Innovator Class	113
Unit 7 Exploring the Item Class.....	129
Unit 8 Creating Server Event Methods	155
Unit 9 Exploring Aras Object Functions	175
Unit 10 Creating Client Event Methods	187
Unit 11 Configuring the User Interface.....	211
Unit 12 Using Federated Properties	249
Unit 13 Integrating a .NET Application	265
Unit 14 Implementing RESTful Services.....	275
Unit 15 Importing Data with Batch Loader.....	305
Unit 16 Configuring the Scheduler Service	323
Appendix A Custom Search Result Sets.....	333
Appendix B Creating Form Dialogs.....	341
Appendix C Customizing a Form.....	353
Appendix D Tracking Login and Logout	361
Appendix E Working with Relationship Grids.....	367
Appendix F Using Message Resources	385
Appendix G Passing Event Parameters.....	397
Appendix H Creating Methods with Aras VS Methods Plugin.....	413
Lab Solutions	429
Index.....	447

Developing Solutions

This page intentionally left blank.



Introduction

Overview

In this course, you learn how to customize and extend the Aras Innovator environment to include business logic in a solution. You gain a working knowledge of AML (Adaptive Markup Language) and learn how to establish integrations with other systems using the Aras Innovator Object Model (IOM) API.

Objectives

- Query, add, delete and modify Items using AML
- Use the IOM to create client and server Methods to extend solution behavior
- Run custom code using menu Actions, Server Events and Client Events
- Integration topics: Federation, .NET applications, RESTful Services, BatchLoader and Scheduling Method execution

Aras Innovator Overview

Business	Next Generation Enterprise Software Solutions
Company	Global Organization, PLM Executives & Technologists
Solutions	Aras Innovator® Suite
Out-of-the-Box	Comprehensive Enterprise PLM ▶ NPDI New Product Development & Introduction ▶ CMII Configuration & Change Management ▶ APQP Advanced Product Quality Planning ▶ PDM Product Data Management
Advantage	Highly Scalable, Extensible, Upgradable
Innovation	Advanced Model-based SOA Technology



Aras Innovator Overview

Aras takes a very different approach to PLM. We combined enterprise open source solutions with advanced model-based SOA technology to deliver a highly scalable, flexible and secure PLM solution suite. Our extensive out-of-the-box functionality and modern Web architecture enable you to deploy quickly and continuously enhance your PLM environment in a fraction of the time required by conventional enterprise PLM / PDM systems and at a Total Cost of Ownership far below that of any leading competitor.

Aras Customers



Aras Customers

Aras enterprise open source solutions are for performance driven companies of all sizes from Fortune 500 enterprises to midsize businesses looking to get an edge on the competition. Companies achieve better customer alignment, greater profit margins, and shorter development cycles with the Aras enterprise open source solutions.

Course Goals

When you have completed this course you should be able to:

- Query, add, delete and modify Items using AML
- Visualize Data using Query Builder/Tree Grid Views
- Use the IOM to create and debug client and server Methods to replace or extend solution behaviors
- Customize the User Interface
- Work with federated properties
- Integrate a .NET application
- Implement RESTful Services
- Schedule Method execution
- Import data using the Batch Loader



Topics

- Overview
- Exploring AML
- Creating a Method
- Debugging Client and Server Methods
- Creating Custom Actions
- Reviewing Innovator Class methods
- Reviewing Item Class methods
- Creating Server Methods with the IOM
- Creating Client Methods with the IOM
- Configurable User Interface



Topics

- Integration
 - Using Federated Properties
 - Integrating a .NET Application
 - Implementing RESTful Services
 - Using the Scheduler Service
 - Using the Batch Loader





Unit 1 Overview

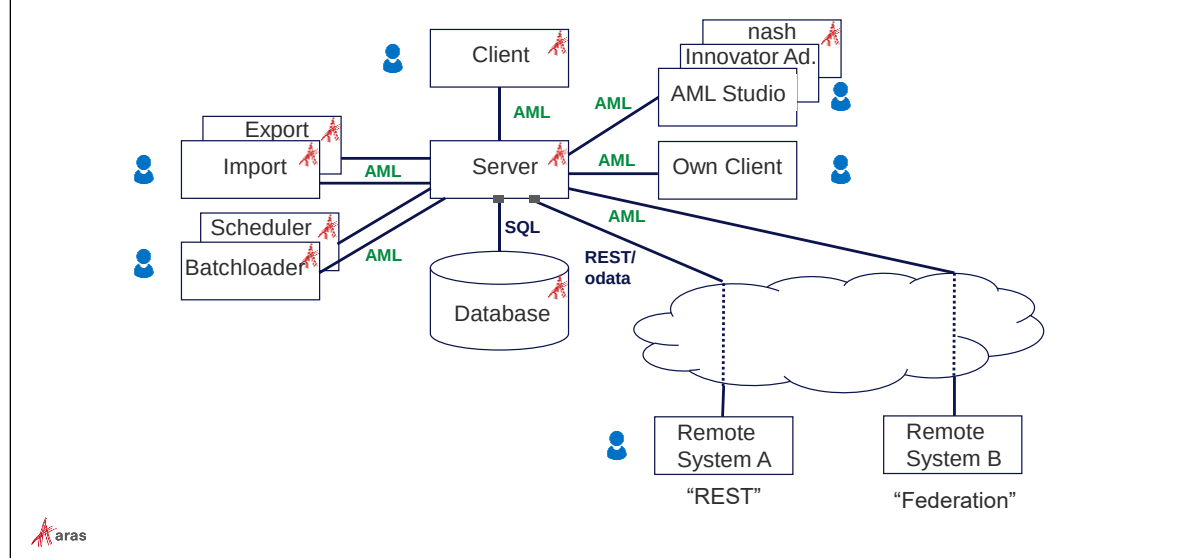
Overview

In this unit, you learn about the Aras Innovator Architecture and how you can programmatically interact with the Aras Innovator Server. You will also review the Aras Innovator Methodology, which describes the basic rules you should follow when developing solutions.

Objectives

- Reviewing Aras Innovator Architecture
- Interacting with Aras Innovator
- Defining the Aras Innovator Methodology

Overview Scenario



Overview Scenario

The internal language of the Aras Innovator Server is AML. Additionally we find AML with most of the interfaces of Aras Innovator, first of all the interface between the Client and the Server.

Aras itself offers further clients as individual executables, e.g. the packaging tools or the Batchloader.

You can create own Clients based on AML. In order to practice AML we are using in the training the client AML Studio, which was developed outside Aras and nowadays improved with the name Innovator Admin. Aras provides the integrated client nash for sending AML to the server.

Another interface offered by Aras Innovator Server is realized with RESTful services using the protocol odata. For the internal processing in the Innovator Server the REST requests are translated to AML when contacting our server. Similarly also the database language SQL is transformed to AML inside our Server.

A special type of interface is realized by the federation mechanism. In this case the Aras Innovator Client is used. It always connects to the Server. Then the server contacts a remote system for accessing data, optionally combining this federated data with data of the internal Aras Innovator database.

We will learn about all these mechanisms during this training. On top we will see how to call our self-made code inside our client by CUI and from outside by the Aras Scheduler.

Because Aras Innovator uses a SOA Architecture it can easily be incorporated into an enterprise solution with other providers.

Note to R22

The primary focus of Aras Innovator 22 is transitioning the Aras server components from .NET Framework to .NET Core. .NET Core has some significant advantages over .NET Framework, such as cross-platform support and better scalability.

Interacting with Aras Innovator

- Using Internal Innovator Methods
 - Allow for custom business logic
 - Can run on the client or server
 - Can react to system events
 - Can modify, extend or replace existing behavior
- Using External Programs
 - .NET integration in a Microsoft Visual Studio project
 - Other deployments including Android, IOS and Windows Surface RT
 - Using RESTful services



Interacting with Aras Innovator

You have several ways you can interact with the Aras Innovator Server including:

- Creating Methods in Aras Innovator
- Invoking requests and responses from a .NET application using the IOM
- Using Aras supplied SDK's that support Android, IOS and Windows Surface devices.
- Using RESTful services

In this course, you learn how to use several of the approaches available.

Defining the Aras Innovator Methodology

- Everything is an Item
- AML language describes Items
- AML messages are passed and returned from the Server
- Innovator Object Model (IOM) can be used to build AML messages
- Innovator Methods:
 - Define custom business logic
 - Modify, extend or replace Item behavior



Defining the Aras Innovator Methodology

AML is the language that drives the Aras Innovator Server.

- Every object is defined as an Item.
- Every Item is described using the AML language
- The Aras Innovator Server is a message-based system in that it accepts AML scripts as messages and returns AML messages. AML document, AML script and AML message all mean the same thing.
- The IOM is the Object API used to build and apply AML messages.
- Methods implement business logic using the IOM API.
- Methods extend the Item Class when used as an "Item Action". An Action is a relationship between a Method and an Item which is configured on the ItemType. This simulates Object Oriented programming, where the ItemType is the Class and "Item Action" relationships to Methods are the Class methods.
- Methods are also "generic" arbitrary business logic that can be called like a sub routine from other Methods using the IOM Innovator.applyMethod(...) method.

Defining the Aras Innovator Methodology

- Methods follow an Item Factory design pattern
- Client and Server events operate on a Context Item
- The Context Item should only be altered before it is processed by the Server
- The Context Item is referred to using the keyword *this* (C#, JavaScript) or *Me* (VB.NET) in a Method
- Use the appropriate attributes of AML when performing queries to optimize performance
- Actions define a relationship between the UI and an event



Defining the Aras Innovator Methodology

- Methods follow the Item Factories design pattern; they should return a new Item and not side affect the context Item.
- Server Events are the exception because the purpose is to intercept and operate on the AML before the server parses it, and before the AML is returned to the client after the server parses it. So, you do modify the context Item and return nothing.
- Implement changes/edits to the context Item in the OnBefore Event by altering the AML before the server parses it. Refrain from attempting to update the context Item after the server has already operated on it in the OnAfter Event. Use the OnAfter Events to update/include federated data in the response AML.
- Use the select, page, and pagesize attributes for the AML queries to optimize the performance for the request.
- Use the generic IOM methods to construct the AML queries rather than convenience methods like getItemBy_, getRelationships(), or use the levels attribute because the convenience methods typically return far more data than is required imposing a performance hit.
- The context Item is the keyword *this* Object for JavaScript, C# and the *Me* Object for VB.Net.
- Attributes are used to pass control switches to the Method. You can invent your own and they can be a simple way to pass additional meta-data to a Method.

Defining the Aras Innovator Methodology

- Every client window has access to the "aras" object
 - Additional JavaScript methods (in addition to IOM)
 - JavaScript file aras_object.js located in ..\Innovator\Client\javascript folder



Defining the Aras Innovator Methodology

In addition to the IOM API an Aras Innovator Client method has access to many additional JavaScript methods that are associated with an object named "aras". These methods have been created to interact with the Aras Innovator Server or provide convenience utilities while working on the client.

You learn about several of these methods in this course.

Comparing Aras Innovator Terms

Aras	Object Oriented	Business
ItemType	class	Object Template (<i>Work Order</i>)
Item	object instance	Single Occurrence (<i>WO-000331</i>)
Property	property	Attribute (<i>Priority</i>)
Method	method / function	Behavior (<i>Promote</i>)



Comparing Aras Innovator Terms

An Item in Aras Innovator is like an “object” in object-oriented terminology. It is used to represent real world ‘things’.

The nature and behavior of objects is defined in a blueprint or template, referred to as a ‘class’ in Object Oriented terminology. This is known as the ItemType in Aras Innovator.

Items are related to Properties, which store data with the Item like properties of an object in Object Oriented terms.

ItemTypes have “Item Action” and “Server Event” relationships, which describe their behavior - like methods defined for an object class.

Summary

In this unit, you learned about the Aras Innovator Architecture and different ways you can interact with the Aras Innovator Server. You also learned the basic rules for creating new programs in the environment.

You should now be able to:

- Describe the Aras Innovator Architecture
- Explain the different ways to interact with the Aras Innovator Server
- Describe the Aras Innovator Methodology

Lab Exercise

Goal

Be able to describe the class business case scenario and data model to be used in this course.

Description

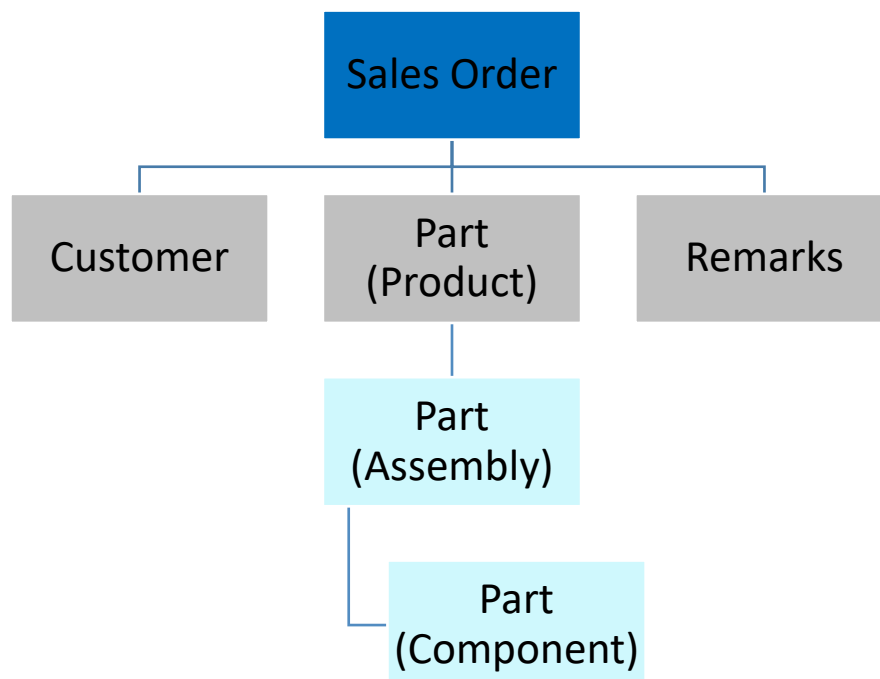
This course continues with the business Items you were introduced to in the *Configuring Solutions* course.

These include:

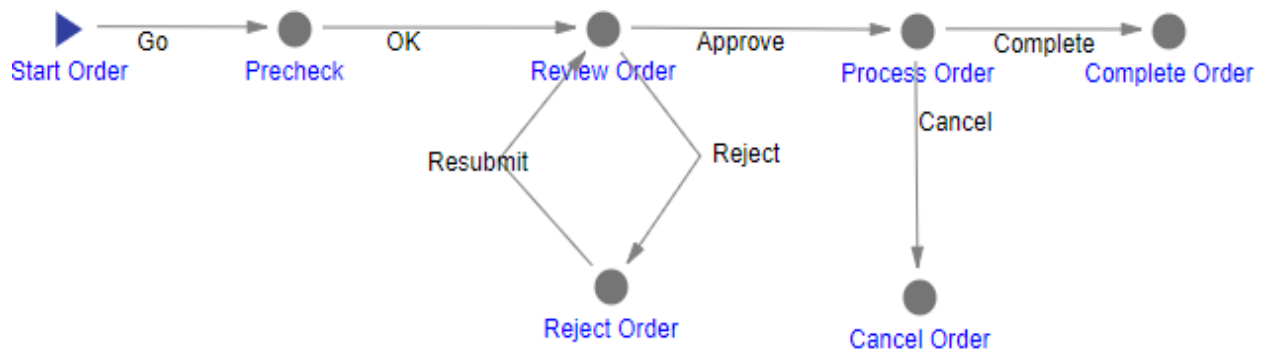
- Design Requests
- Change Requests
- Customers

This course introduces a new ItemType for Sales Orders.

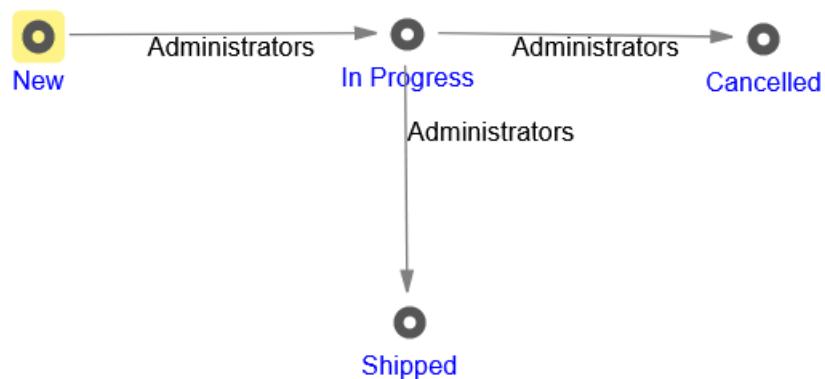
In addition, the Part ItemType is used to demonstrate Part to Part (Part BOM) relationships in the following hierarchy:



Each Sales Order is associated with the following Workflow:



Sales Orders use the following lifecycle:



You will provide custom business logic to the solution to validate consistency of data, implement specific business rules and provide data access to external client applications.

A small set of sample data has been provided to you that you can begin to access and modify business Items.

Please do not change any of the sample data Items as they will be used in subsequent labs in the course.



Unit 2 Exploring AML

Overview

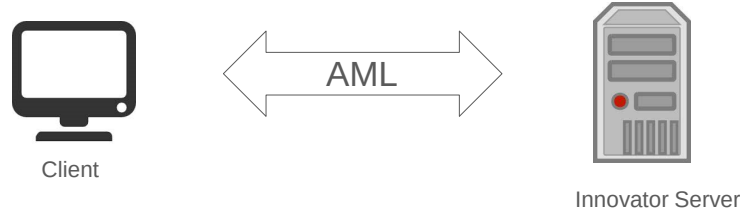
In this unit, you learn how to use Adaptive Markup Language (AML) to create custom search queries as well as how to add and modify Items in the Innovator database. AML is the language that drives Aras Innovator and all communication between client and server is done using the AML dialect. We can consider it the native language of Aras Innovator, understanding this language will also help you to work effectively with the IOM classes and methods.

Objectives

- Defining the AML Syntax
- Exploring AML Tags
- Reviewing AML Structure
- Building AML Queries
- Testing with AML Studio
- Reviewing Built-In Actions
- Adding, Editing, Promoting and Deleting an Item with AML
- Using AML with Extended Properties

Defining AML

- “Everything in Aras Innovator is an Item”
- Every Item is described using Adaptive Markup Language (AML)



Defining AML

As you have already learned, everything in Aras Innovator is considered an Item.

The Adaptive Markup Language (AML) is an extension to XML that allows you to perform operations like search, edit, update, and delete of Items without using the Aras Innovator user interface.

Any information that is exchanged between a client and the server is formatted in AML. So, every transaction you perform using the Innovator client generates AML that is sent to the Innovator Server and then returns to the client with results.

AML is used in integrations to other systems, for building advanced queries against the database as well as for batch or updates to the database.

In this unit, you will learn the basics of AML to build queries, as well as edit and delete Items. More information about AML is also available in the Aras Innovator’s Programmer’s Guide.

AML Options

- AML Allows Item:
 - Retrieval – complex queries
 - Additions – multi-level items
 - Modifications
 - Purging (of a version)
 - Deletions
 - Claiming/Unclaiming
 - Promotion



AML Options

AML provides a set of built-in operations that allow you to get, update and delete Items in the database. You can also extend these actions with logic of your own.

Note

These built-in actions always obey the security permissions that you have defined for your solution.

Reviewing AML Structure



Reviewing AML Structure

Each Item you create in the database can be related to another Item using a relationship Item.

A Relationship Item is created for each connection between two Items and contains the source and related ids of the connected Items. The properties `source_id` and `related_id` included with each relationship Item store the ids of the source Item and the related Item.

This simple structure is important to remember as you begin to work with AML. Many of the AML tags used in the language use these terms as part of the dialect.

Exploring AML Tags

Top Level Tag —→ **<AML>**

Item Tag —→ **<Item ...>**

Relationships Tag —→ **<Relationships>**

.

.

.

</Relationships>

</Item>

</AML>



Exploring AML Tags

<AML> Tag

This top-level tag must always surround the encompassed Items. Like XML, a matching closing tag (</AML>) is also required to define the document.

<Item> Tag

Contained within the AML tag are one or more Item tags that define each Item being expressed in the AML document. Item tags also contain nested property tags that you will learn about shortly.

If the Item tag is missing or misspelled (e.g., <item> with lowercase begin), then the response is in format of a fault containing the string "The XML for this operation is malformed".

<Relationships> Tag

Remember that each Item can be related to another Item using a relationship. The Relationships tag defines the connection between the source Item (the parent tag of this relationship) and the related Item.

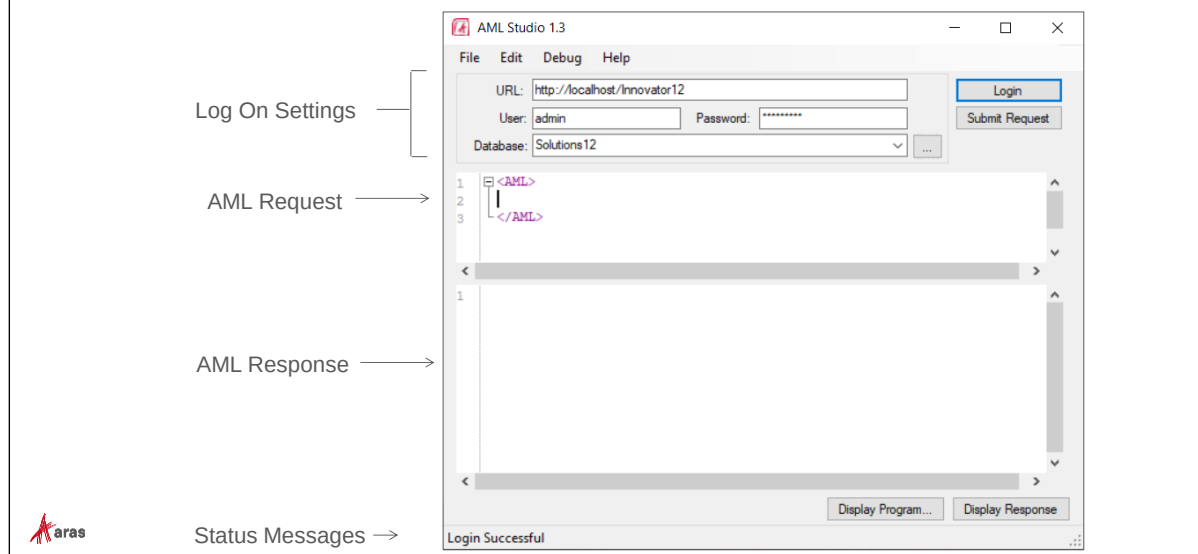
Notes

AML is case-sensitive. Much like XML, each tag must have a matching close tag. Unlike XML, there is no XML schema defined for AML. Because Aras uses a self-describing data model, a formal named workspace is not used.

You can also add comment tags to the AML statement using the XML syntax:

```
<!-- Comment text -->
```

Building AML Queries with AML Studio



Building AML Queries with AML Studio

To assist with creation of AML queries a simple tool is provided with this course. This utility can be used to help you build queries faster and includes a “type ahead” feature as well as a syntax check that steps you through the language as you enter an AML request.

The AML Studio window is divided into 4 main areas:

Log On Settings

Provide the URL to your Aras server as well as the user id, password, and desired database. After entering the URL in the field provided you can click the database lookup button to show you the currently configured database names in the drop list. Click the Login button to connect the server indicated. The message *Login Successful* will appear in the Status Message area.

AML Request

Enter a valid AML request in the field provided. Click Control + t (or choose Edit -> Tidy XML) from the menu to check the syntax for any errors and indent the request for easier viewing. Click the Submit Request button to send the message to the server.

You can resize the proportions between the request and response panes by dragging the scroll bar up or down on the screen. You can also increase the size of the request text font by pressing the Control key while moving the mouse wheel up or down.

AML Response

The AML response message from the server will appear in the AML Response pane when a message is submitted to the server. This will either contain the requested results or an error message.

To see the response message in an alternate viewer. Click the Display Program... button and choose an XML editor/viewer installed on your machine. When you click the Display Response button the response text will appear using your installed viewer.

Status Messages

The message area will indicate when you are connected to the server as well as the number of items found in a query when a search is successful.

Notes

The author of AML Studio continues increasing the functionality under the name [Innovator Admin](#).

In the training we stay with the old product focusing to basics.

Aras also supplies a standard AML tool (NASH) when you install the Aras Innovator Server. To access this tool, provide the following URL in the web browser:

```
http://[servername]/[ ServerAlias]/Client/Scripts/Nash.aspx
```

The NASH tool was created by Aras development. The tool provides advanced features not available in the AML Studio.

Primary Item Tag Attributes

- id – unique identifier for an Item
- type – ItemType name for the Item
- action – behavior (Method) to apply to the Item

```
<Item type="Part"
      id=[32 character identifier]
      action="get" />
```



Primary Item Tag Attributes

Every Item tag can contain attributes which further describe the kind of Item being expressed.

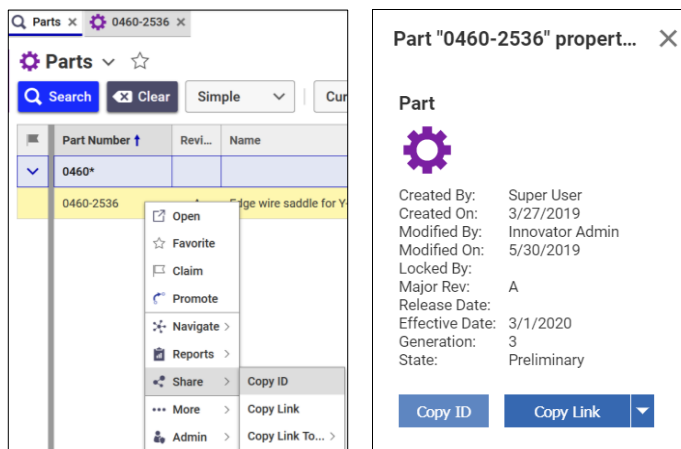
type – is used to specify the **ItemType of the Item**. In the example above we are describing a Part Item.

id – is used to specify which Part Item in the database we are describing. One way to locate an Item in the database is to provide the unique identifier.

action – is used to indicate **what action to take when this AML is sent to the server**. Aras has a set of built-in actions you will learn about later in the unit. **You can also specify a Method that you have written to perform some custom operation.**

Note

You can obtain the id of an Item in two ways: First, by selecting an Item in the Main Search Grid, opening the right-click-menu and from the Share category choosing the option Copy ID (copies id to the clipboard). Second, by opening the right-click-menu for an Item in the Main Search Grid and choosing the Properties menu. The properties dialog will display the “Copy ID” button.



First Request

- Define the attributes **type** and **action**

```
<Item type="Part" action="get">
</Item>
```

- Response

```
<Item type="Part" typeId="4F1AC04A2B484F3ABA4E20DB63808A88" id="D359AB7C4C4F49FB88668E75E9104B72">
  <classification>Component</classification>
  <config_id keyed_name="0460-2536" type="Part">C178DF9A420C4E8A8C9A058F27D662CF</config_id>
  <created_by_id keyed_name="Super User" type="User">AD30A6D8D3B642F5A2AFED1A4B02BEFA</created_by_id>
  <created_on>2021-08-25T12:48:59</created_on>
  ...
</Item>
```



First Request

In order to define a first AML Request define the attributes type and action and click to Submit.

In the response you will find nested inside a pair of Item tags information about the properties of the selected Item. The display comprises those properties, which are not NULL in the database and which are not xProperties.

Please note that the response contains also attributes. In the response the attribute keyed_name is of special interest for programming. It exists for all properties of data type Item, e.g. for created_by_id. Those properties show as value just the id, but the keyed_name most time is a good info for the users.

Item Property Tags

- Nested inside of an Item tag
- Property name is the tag name

```
<Item type="Part" action="get">  
  <item_number>8120-1378</item_number>  
</Item>
```



8120-1378



Item Property Tags

Within each Item tag you can specify one or more property tags. The property tag name is the name of the property that has been created for this Item in the database. Like Item properties, property tags cannot contain spaces and are always lower case.

In the example above, the *item_number* property is being used with a Part Item.

The *get* action will attempt to retrieve the Part Item with an item number of 8120-1378.

Notes

In case your search criteria do not result in a single hit, the output will be formatted as InnovatorFault. Please observe, that you will find in the output a concrete info "No items of type '...' found using the criteria: ...".

Property values listed in /Server/reserved_words.txt are not allowed.

Multiple Property Tags

- Implied "AND" condition

```
<Item type="Part" action="get">  
  <classification>Assembly</classification>  
  <name>Cable Assembly</name>  
</Item>
```



Multiple Property Tags

When you supply more than one property tag the implied operation is the AND operator. Only items that meet all conditions will be returned in the example.

Using Logical And/Or Operators

```

<Item type="Part" action="get">
  <or>
    <and>
      <cost>350</cost>
      <classification>Product</classification>
      <make_buy>Make</make_buy>
    </and>
    <and>
      <cost>100</cost>
      <classification>Assembly</classification>
      <make_buy>Buy</make_buy>
    </and>
  </or>
</Item>

```



Using Logical And/Or Operators

AML supports the ability to provide logical AND, OR and NOT operators as part of a query.

You can group property tags into logical condition sets to locate Items that meet those conditions.

In this example, any Part Items that have a cost equal to 350 AND a classification equal to Product AND a make/buy equal to Make – OR – any Part Items that have a cost equal to 100 AND a classification equal to Assembly AND a make/buy equal to Buy will be returned.

Note

To supply a NOT condition, wrap the enclosed tags with the <not> </not> tags.

Property Attributes

- Using Conditions:

```
<Item type="Part" action="get">
  <cost condition="gt">35</cost>
</Item>
```



8121-0811
Cost 55



8121-0868
Cost 70



8121-1036
Cost 36



Property Attributes

When retrieving Items, it is useful to be able to provide some conditional logic to indicate what Item (or Items) will be returned from the database. The condition attribute supports the following values:

Condition	Description	Example
eq	Property is equal to another value (default)	<name condition="eq">Peter</name>
ne	Property is not equal to another value	<name condition="ne">Peter</name>
ge	Property is greater than or equal to another value	<cost condition="ge">200</cost>
le	Property is less than or equal to another value	<cost condition="le">200</cost>
gt	Property is greater than another value	<cost condition="gt">200</cost>
lt	Property is less than another value	<cost condition="lt">200</cost>
like not like	Include % or * wild card symbols	<name condition="like">Admin%</name>
between not between	Use <i>and</i> keyword for range	<cost condition="between">10 and 90</cost>
in	Value is in comma delimited set	<name condition="in">'Admin','Peter','Bob'</name>

Condition	Description	Example
is null	Value is null or not null	<code><name condition="is null" /></code>
is not null		<code><name condition="is not null" /></code>

Working with Date and Time

When working with date and time in Aras, there are three main rules to keep in mind:

- If the `CorporateTimeZone` variable is set in the database, then all Date properties must be applied in the corporate time zone.
- If the `CorporateTimeZone` is NOT set, then all Date properties must be applied in the local time zone of the client session context that applied the AML.
- The Date applied in the AML must conform to the pattern `yyyy-MM-dd[Thh:mm:ss]`

Notes

The optional time portion is enclosed in "[]", and the format does not contain milliseconds as they are ignored by the server.

Searching by Date and Time

To search for Items by date and time you can use the *condition* property attribute to find items before and after a specific date (and time).

Examples

```
<created_on condition="ge">2013-02-01</created_on>
<modified_on condition = "lt">2012-03-01T12:00:00</modified_on>
<schedule_date condition="between">2013-03-01 and 2013-04-01</schedule_date>
```

Now() Function

There is one special case when applying updates to Date properties in AML to Aras. It is possible to specify '`__now()`' as the value for properties of type Date, instead of a specific value.

Example

```
<Item type="Document" id="ACD123C46C1BFE8ED4E8BDDE0B009812">
  <effective_date>__now()</effective_date>
</Item>
```

When a request like this is received by the server, it writes the value of the current moment of time into the database for the date property `effective_date`.

Relationships Tag

- "Container" for Related Items
- Has no attributes
- Nested ItemType refers to the Relationship Item

```
<Item type="Part" action="get">
  <item_number>C4723-69096</item_number>
  <Relationships>
    <Item type="Part BOM" action="get">
    </Item>
  </Relationships>
</Item>
```



Relationships Tag

Items can have relationships to other Items. The Relationships tag describes a relationship between the source Item and a related Item that has been defined with a RelationshipType. Remember that each relationship connection is defined by a Relationship Item in the database.

In the example above, we are attempting to get a Part Item with Item Number C4723-69096 that has a Relationship Item of the ItemType Part BOM. If successful, the AML returned will describe the source Part C4723-69096 as well as the Relationship Item(s) and the related Part(s).

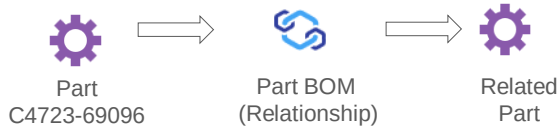
On one level only one Relationships tag can be used.

If you want to filter by several relationships of different Relationship ItemTypes, you will work nested inside the Relationships tags with multiple Item tags, one after the other. Each of them will indicate the Relationship ItemType you are using as filter.

If you try by adding a second Relationships tag inside the same Item tag, then the second one will just be ignored.

Related Id Tag

```
<Item type="Part" action="get">
  <item_number>C4723-69096</item_number>
  <Relationships>
    <Item type="Part BOM" action="get">
      <related_id>
        <Item type="Part" action="get" >
          </Item>
        </related_id>
      </Item>
    </Relationships>
  </Item>
```



Related Id Tag

A Relationship Item contains a source_id and a related_id property which define the connection between the two Items. To describe the related Item, you can use the <related_id> tag as shown above.

In this example, we are retrieving the same Part, getting the Relationship Item named Part BOM – and from that retrieving the related Part.

Searching on Related Items

```

<Item type="Part" action="get"> ①
  <Relationships>
    <Item type="Part BOM" action="get"> ②
      <related_id>
        <Item type="Part" action="get"> ③
          <item_number>C4704-60117</item_number>
        </Item>
      </related_id>
    </Item>
  </Relationships>
</Item>

```



① Part → ② Part BOM (Relationship) → ③ Part C4704-60117



Searching on Related Items

Using the `related_id` tag, you can describe the related Item and any properties that should be queried.

In this example, any Parts ① with a Part BOM ② relationship to a Part ③ with the item number of C4704-60117 will be returned.

Searching on Relationship Items

```
<Item type="Part BOM" action="get">
  <related_id>
    <Item type="Part" action="get">
      <item_number>C4704-60117</item_number>
    </Item>
  </related_id>
  <source_id>
    <Item type="Part" action="get">
    </Item>
  </source_id>
</Item>
```



Part BOM
(Relationship)



Part
C4704-60117



Searching on Relationship Items

Because relationships are also Items, you can use the AML "get" action to locate a relationship and the attached source and related Items.

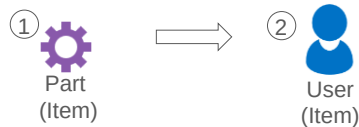
In this example, a search is performed to find any Part BOM relationships with a related Part number C4704-60117. If any are found, the source Part Item is also returned. Reverse lookups are very helpful when working with Configurable Grids.

Searching on Item Properties

```

<Item type="Part" action="get"> ①
  <created_by_id>
    <Item type="User" action="get">
      <last_name>Admin</last_name> ②
    </Item>
  </created_by_id>
</Item>

```



Name	Data Type	Data Source [...]
created_by_id	Item	User



Searching on Item Properties

Remember how you can create properties of data type Item that can be used to refer to another single Item in the system. The syntax above shows how you can use AML to describe this property and query on properties that belong to the connected Item.

In this example, the Part Items ① will be searched for a property named `created_by_id` that can contain an Item of type User ②. The search will attempt to find a User with the *last_name* of Admin that is associated with this Part. If successful, the Part(s) will be returned.

Additional Item Attributes

- select – choose the properties to return (get only)
- where – used instead of id attribute (for database modifications)
- doGetItem – boolean setting for returning item (for database modifications)
- orderBy – sort the returned results
- page – page number for the result set
- pagesize – size of returned page
- maxRecords – size of returned result set
- levels – Item level "depth" to return
- version – allow versioning (update only)
- idlist – comma delimited list of item id's



Additional Item Attributes

Attribute	Data Type	Description
where	String	Used instead of the id attribute to specify the WHERE clause for the retrieving data. Only supports simple queries on the same ItemType.
	Boolean	Indicates whether any items should be returned from the Query. If 0 then do not perform a final get action on the Item after the server performed that action as defined by the action attribute. Default = 1. This is useful to speed up update and add.
select	String	Specifies a comma delimited list of property names (column names) to return. This can be used to improve performance by suppressing unnecessary properties as well as to display xProperties or Properties which are NULL.
orderBy	String	A comma delimited list of property names (column names) to order the results. Use DESC keyword to sort descending order.
page	Integer	Defines the page number for the results set.
pagesize	Integer	Defines the page size for the results set.
maxRecords	Integer	Defines the maximum Items to be searched in the database.

Attribute	Data Type	Description
levels	Integer	Specifies the Item configuration depth to be returned for items related to the current item. Use the GetItemRepeatConfig action to define recursive queries for better performance.
serverEvents	Boolean	The value 0 can be used to suppress server events improving performance, if not protected in the Server Event item. Default is 1.
related_expand	Boolean	If 0 then do not expand the related_id Property for the relationship Items to include the related Item. Another word just returns its ID. Default is 1.
language	String	A comma-delimited list of language codes, or "*" to return all languages. Multilingual property values will be returned (if present) for all specified languages.
version	Boolean	If 0 then do not version an Item on update. Default is 1, which is version the Item (if it is a versionable Item) on update.
queryType	String	Effectivity Search – possible values are "Latest", "Released" or "Effective".
queryDate	String	Date of effective search based on queryType – the format must be "yyyy-mm-ddThh:mm:ss".
idlist	String	Is followed by a comma delimited list of item id's to act on.

Using Additional Item Attributes

```

<AML>
  <Item type="Part" action="get"
    select="item_number,name" orderBy="item_number"
    page="1" pagesize="10">
    <is_released>0</is_released>
  </Item>
</AML>

```



Part C5316-60000
PC Board Kit



Part C5316-60108
Scanner Assembly



Part C5316-60055
Power Assembly



Using Additional Item Attributes

You can reduce the amount of information returned from a query by using additional Item attributes.

In this example, the select attribute ① will retrieve the item_number and name values from the Part Items that have not been released and then order the results ② by the item_number. Only 1 page ③ with a maximum of 10 records ④ will be returned.

Reviewing Built-In Actions

- get – retrieves an Item
- GetItemRepeatConfig – supports recursive queries
- add – creates a new Item
- update – updates a claimed Item
- purge – deletes an Item version
- delete – removes the Item (and all versions)
- edit – claims, updates and unclaims an Item
- create – adds the Item if the id does not exist
- merge – edits the Item if the id exists, otherwise add
- lock/unlock – claims or unclaims an Item
- promoteItem – promote Item to next lifecycle state
- GetItemAllVersions – returns all versions, like using Versions menu from the UI



Reviewing Built-in Actions

The following actions are into the system to perform different kinds of operations against the database. You can also create your own Methods and supply them as an action (described later in the course).

Built-in Action Method	Description
add	Add the Item as an instance of an ItemType.
update	Updates the Item. The Item must be claimed. If the Item is versioned and is being updated the first time since being claimed, then the update versions the Item (creates a new generation). If the attribute version="0" then versioning is disabled.
purge	Delete a specific version of the Item.
delete	Delete all versions of the Item. The purge and delete are the same for non-versionable Items.
get	Gets the Item(s) and its configuration based on the AML Item configuration used to query the database.
GetItemConfig	This will return the Item configuration as described by the standard AML query. The AML in and out are no different than the standard action="get". The GetItemConfig is optimized by limiting the logic done between the SQL call and the AML result. The performance improvement is gained by limiting the features typically available in the "get" action (no server events).

Built-in Action Method	Description
GetItemRepeatConfig	This will allow deep recursive queries and is useful in multi-Part BOM's with repeating relationships.
edit	This will claim, update, and unclaim the Item.
create	This will act as a "get" if the Item exists, otherwise acts as an "add". Note that if an attempt is made to create an item that already exists with the same property values it will not create the item, just return the item(s) that match the existing criteria.
merge	This will act as an "edit" if the Item exists, otherwise acts as an "add".
lock	This will claim the Item and is the same as the Item.lockItem() method.
unlock	This will unclaim the Item and is the same as the Item.unlockItem() method.
promoteItem	This will promote an item to next lifecycle state.
version	This creates a new generation. Current item id (GUID) is required.
GetItemAllVersions	This returns all versions. Current item id (GUID) is required.

Note

The criteria to determine if an Item exists, is the existence of an id. Therefore, the differences dependent on the existence of an Item are only evaluated if the id is included in the AML.

Retrieving Recursive Relationships

```
<AML>
  <Item type="Part" action="GetItemRepeatConfig"
    select="item_number,name"
    id="14C6241EFD9049428B96F05EECCBC044">
    <Relationships>
      <Item type="Part BOM" action="get"
        repeatTimes="2" repeatProp="related_id"
        select="related_id,quantity">
      </Item>
    </Relationships>
  </Item>
</AML>
```



Retrieving Recursive Relationships

The Aras data model supports "multi-level" relationships where one item is related to a related item that is then related to another sub item, etc. The typical example used in many applications is a multi-level Parts BOM where a starting Part item is constructed with one or assemblies that point to subassemblies and components.

You can perform a recursive query on a collection of related items use the `GetItemRepeatConfig` action. This action allows you to "repeat" a query against a related Item property to return a "deep" structure. In addition, the AML select attribute allows you to define which properties at each level will be returned.

To Use `GetItemRepeatConfig`

1. Specify the starting item for the query using the required `id` item attribute. You can also specify which properties will be returned using the `select` attribute.
2. In the Relationship ItemType definition, use the `repeatTimes` attribute to define how many times to repeat the query.
3. Use the `repeatProp` attribute to specify which Item property will be repeatedly expanded. This property must be in the result, so please do not exclude it by using the `select` attribute.

A `repeatTimes` value of "0" indicates the query should execute recursively until all expansions have been executed.

Adding New Items

```
<AML>
  <Item type="Part" action="add">
    <item_number>2134-9099</item_number>
    <name>Paper Sensor</name>
    <description>Front feed sensor</description>
    <make_buy>Buy</make_buy>
    <classification>Component</classification>
  </Item>
</AML>
```



Part
(Item)



Adding New Items

The add action is used to add a new Item to the database (or to add new relationships to existing Items). In this example, a new Part will be generated with the property values specified by the property tags above.

Note

To improve performance, consider to use the attribute setting = "0" for all database modifications.

Adding Multiple Items

```

<AML>

  <Item type="Part" action="add">
    <item_number>1121-9011</item_number>
    <name>Front Roller</name>
  </Item>

  <Item type="Part" action="add">
    <item_number>1121-9012</item_number>
    <name>Rear Roller</name>
  </Item>

</AML>

```



Adding Multiple Items

A single AML request may contain multiple Item statements.

In the example both Item statements are for adding an item of the same ItemType. In general, it is allowed to mix statements for different ItemTypes, and to mix different actions.

For all such AML scrips with more than one statements please be aware of the following rule:

If in a minimum one statement fails, nothing will be changed to the database. More precisely, all database activities which happened before the error will be rolled back.

Editing a Single Item

- Supply the *id* or *where* attribute to act on an Item

```
<AML>
  <Item type="Part" action="edit"
        where="[part].item_number='1990-1972'">
    <make_buy>Buy</make_buy>
  </Item>
</AML>
```



Make/Buy
Buy

1990-1972



Editing a Single Item

Much like editing an Item in the Aras user interface, the edit action will:

1. Claim the Item specified (you must supply an Item id or use a where clause to locate the Item)
2. Make any changes described
3. Save the Item
4. Unclaim the Item

In this example, a Part is being modified so that the make_buy value is set to Buy.

Notes

You should specify the table name using the bracket notation shown above to avoid "colliding" with reserved terms in SQL Server. Remember that each ItemType is stored in its own table – the name of the table becomes the name of the ItemType (without spaces).

Starting in Aras version 11 SP9, the use of the “where” clause is restricted to the table (ItemType) used in the “type” attribute. The AML server performs an Item Analysis on the SQL statement and if security permissions are violated an error message will appear as the result. This prevents SQL injection into AML statements. If you are upgrading code from a previous release of Aras and are using SQL injection, please contact Aras Support via portal for assistance.

Editing Multiple Items

- Supply a where clause that returns multiple items
- Alternative: Use an idlist

```
<AML>
  <Item type="Part" action="edit"
        where="[part].classification='Assembly'">
    <make_buy>Buy</make_buy>
  </Item>
</AML>
```



Editing Multiple Items

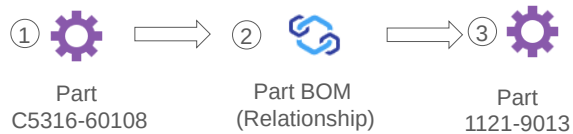
If the where clause returns more than one Item, then all Items in that result set are affected by the edit. In the example above, the where clause will return Assembly Part Items and set the make/buy to Buy.

Adding Related Items

```

<Item type="Part" action="edit" where="item_number='C5316-60108'"> ①
  <Relationships>
    <Item type="Part BOM" action="add"> ②
      <related_id>
        <Item type="Part" action="add"> ③
          <item_number>1121-9013</item_number>
        </Item>
      </related_id>
    </Item>
  </Relationships>
</Item>

```



Adding Related Items

As mentioned previously, the add action can also be used to add related Items to an existing Item.

Using familiar syntax when querying for related items, the `related_id` tag can be used to assist with this operation.

In this example,

- ① – Part C5316-60108 is being edited.
- ② – A new Relationship “Part BOM” is added to the Part.
- ③ – A new Part is added as the related Item for the Relationship.

Promoting Items

```
<AML>

  <Item type="Part"  action="promoteItem"
    where="[part].item_number='5063-1257'">
    <state>Released</state>
  </Item>
</AML>
```



Promoting Items

Several pre-requisites apply when working with the action promoteItem.

Prerequisites to promote an Item to the next lifecycle state:

- A where clause is required (or id/idlist attribute).
- The tag <state> indicating the target state of the promotion is required.
- The value of the state property must be the next logical state defined in the lifecycle map for the Item.
- The Item must be unclaimed.
- The current user calling the promoteItem action must be a member of the state transition Identity defined on the lifecycle map.

Deleting Items

```
<Item type="Part" action="delete"  
  where="[part].state='Preliminary'">  
</Item>
```



Deleting Items

If the where clause affects more than one Item, then all Items are deleted that meet the required criteria.

In this example, all Parts that are in the Preliminary lifecycle state will be deleted.

Using AML with Extended Properties

- Extended Classification Trees, Properties and their values are available using AML to:
 - Select and filter on xProperty values
 - Assign values to both xClass and Explicit xProperties
 - Filter items based on Defined/Is Not Defined xProperties
 - Determine and define Permissions on xProperties



Using AML with Extended Properties

Extended Classification allows you to share extended properties (xProperty) that are optionally associated with extended classes that are assembled into a hierarchy (xClassification Tree). xClassification Trees as well as individual xProperties can be assigned to one or more ItemTypes.

Working with individual xProperties is partly supported by the Aras Innovater Client, but all activities needed for managing them are supported by AML.

Extended properties work much the same way as standard custom properties but have a few unique features, which require some additional syntax in AML.

Using AML, you can e.g.:

- Retrieve items based on xProperty value filters
- Assign values to xProperties
- Determine whether an xProperty has been previously defined on an item
- Assign and determine xProperty permissions

Additional detailed information is available in the Aras *Extended Classification* documentation.

Selecting Extended Properties

- **select**="propertyname"

- Returns a specific xProperty value

```
<Item type="Design Request" action="get"
  select="xp-height,xp-width">
  <xp-height>5</xp-height>
</Item>
```

- **select**="xp-*

- Returns all xProperties defined on returned items



Selecting Extended Properties

Extended property values are not returned in a query result by default. To retrieve the value of an xProperty supply the name using the select attribute. To specify more than one property use the comma delimiter.

Return every xProperty value stored on an item by using the xp-* wildcard with the select attribute.

Summary

In this unit, you learned how to work with AML to query, add and update Items.

You should now be able to:

- Describe the AML Syntax
- Build AML Queries
- Use AML Studio to Test and Execute AML
- Use Built-In Actions to Add and Edit an Item using AML
- Use AML with Extended Properties

Review Questions

1. What prevents you from deleting an Item in AML?
2. What tag is useful for working with the Item that is related to the source Item?
3. What attribute(s) are required to edit an Item?

Lab Exercise

Goal

Familiarize with the use of AML commands. You will use AML Studio to create and execute custom requests to retrieve, add, edit, and delete items.

Scenario

In this exercise, you will use AML Studio to create several requests to retrieve, add and edit Sales Orders.

Using AML Studio, retrieve:

1. All Sales Orders that allow Multiple Shipments and use the Shipping Method of UPS.
2. All Sales Orders that allow Multiple Shipments OR uses the Shipping Method UPS.
3. All Sales Orders where the associated Customer is from the city of New York.

For performance reasons reduce the selected relationship properties to “related_id” only and the properties of the related item to “city” only. You can accomplish this by using the attribute *select*.

4. All Sales Orders where the associated customer is from New York and where the Sales Order Ship Date is greater than May 1, 2013. Remember the time format in AML is: yyyy-mm-ddThh:mm:ss.

Reuse the performance improvements of step 3.

Using AML Studio, add items as follows:

5. A new Sales Order and assign the Reviewing Manager (*managed_by_id* is an Item data type property) to the Identity of Peter Smith and the Shipping Method as UPS.

Reviewing Manager
 Peter Smith

Shipping Info
Shipping Method
 UPS
Ship Date

 Allow Multiple Shipments? ☐

6. Modify the request you created above so that it creates a new Sales Order and a relationship to the Customer named Brown Industries and the Part with the item_number "C4703A" with a quantity of 1.

Customers ▼ ☆

Name ↑	Main Phone	Contact Name
Brown Industries	213-988-100	Ken Allen

Parts ▼ ☆

Part Number ↑	Revi...	Name	Type	State	Unit	Chan...	Quantity
C4703A	A	DesignJet 2000CP Printer	Product	Preliminary	EA	☐	1

Using AML Studio, edit items as follows:

7. Change any Sales Orders that allows Multiple Shipments to use the UPS Shipping Method.
8. Modify the request you created above so that it also adds the Sales Order Remark "Modified by AML Administrator" (use the comment_text property) to any orders that are changed by this AML request.

Sales Order Remarks ▼ ☆

Date [...] ↑	User [...]	Comments
4/12/2021 3:...	Innovator Ad...	Modified by AML Administrator

This page intentionally left blank.



Unit 3 Creating a Method

Overview

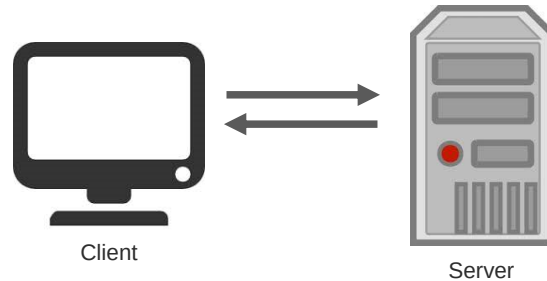
In this unit, you will learn how to create a Client or Server method using the Aras Innovator Solution Studio Editor. You will also learn about the Innovator Object Model and the various methods available to generate AML programmatically.

Objectives

- Exploring Method Types
- Introducing the Innovator Object Model (IOM)
- Exploring the API On-Line Reference
- Use the Editor to Create a Method
- Comparing IOM and AML
- Creating a Client and Server Method
- Using the Context Item
- Using the Item Factory Design Pattern
- Returning New Result or Error

Exploring Method Types

- Provide custom business logic to a solution
- Client Methods use JavaScript
- Server Methods use C# or VB.NET
- Innovator Object Model (IOM) can be used to interact with Items



Exploring Method Types

When you create a Method in Aras Innovator you decide whether Method will be used to execute client code in the browser (JavaScript) or server code in the Innovator Server (C# or VB.NET).

Innovator Methods are special Items that contain source code that gets executed either on the client or server based on which type is selected.

As mentioned earlier, the IOM is available on both the client and server and calls to the IOM methods are basically the same.

Note

Aras Innovator Methods created in the Solution Studio Editor will be referred to as Methods (capital "M") in this course while class methods in the IOM will use lower case "m".

Invoking a Method

- Server Method
 - Item Actions
 - Generic
 - Server Events
- Client Method
 - Item Actions
 - Generic
 - Form, Field, Grid Events
 - Client ItemType Events



Invoking a Method

Methods can be called in several ways depending on their purpose.

Server Methods can be called from an Action (menu driven), from another Method (Generic) or in response to a server event (e.g., OnAdd).

Client Methods can also be called from Actions as well as other client Methods. In addition, there are many events available in the user interface (Form, Field, Grid) that can trigger a client Method. The ItemType also supports several events that can respond to an Item as it is displayed (e.g., OnBeforeNew)

Introducing the Innovator Object Model (IOM)

- API used to generate AML
- Support both Client and Server methods
- Over 200 methods available to interact with Items
- Client and Server implementations share same codebase
 - Namespace: Aras.IOM
 - Contained in: IOM.dll
- On-line API reference available



Introducing the Innovator Object Model

Common codebase, namespace Aras.IOM in IOM.dll

- Referenced by Method Templates on Server
- Embedded as an Object on the Client
- IOM.dll can be referenced by .Net code
- Alternate compile for use as a COM object in VB6 and VBA etc.

Self-documented C# source

- User Menu > Admin > API Reference > Aras IOM

The standard location for the Server IOM.dll is: Innovator\Server\bin

Introducing the API On-Line Reference

The screenshot displays the 'Innovator 12.0 .NET API Reference' window on the left and the Aras user interface on the right. The API Reference window shows a tree view on the left with 'Item Class' selected, and a main pane on the right titled 'Item Class' containing a description and XML examples. The user interface on the right shows a sidebar menu with options like 'Preferences', 'Actions', 'Reports', 'Admin', 'Licenses', 'Activate Feature', 'Help', 'Session', 'About', and 'Logout'.

Innovator 12.0 .NET API Reference

Item Class

The class provides interface to AML structure in memory as well as means of communication with Innovator server. Depending of Item's dom internal structure, as well as its node and nodeList values the instance of this class may represents one of the following:

- **Single** Innovator Item of an arbitrary ItemType. In this case `Item.node1=null`, and inside of `Item.dom` this node can refer to any XmlNode with localName `<Item>`. Notice that in case of Single item the `Item.node.OuterXml` will look like this

```
<Item />
```

or this

```
<Item >
  <prop1>a</prop1>
  <prop2>b</prop2>
</Item>
```

Aras User Interface

- Preferences >
- Actions >
- Reports >
- Admin >
- Licenses >
- Activate Feature
- Help
- Session
- About
- Logout

Introducing the API On-Line Reference

The On-Line Help Reference documents available classes and methods available for your use.

There are two API references available:

- API Reference (.NET) – documents the Aras IOM classes and methods available.
- API Reference (JavaScript) – documents the classes and methods available to Client Methods only for some of the controls used in the Aras client user interface.

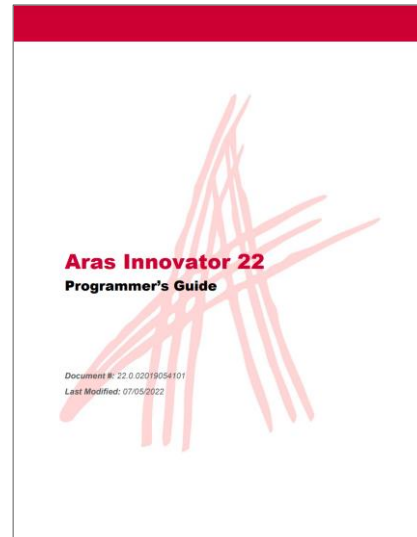
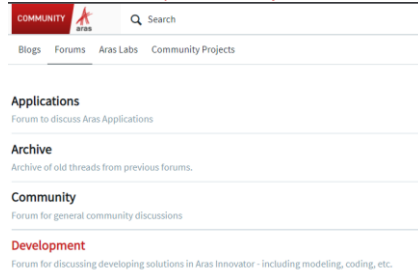
Note

Make sure your Aras working directory is assigned to a location with read/write access for the on-line help to operate correctly. (Tools > Preferences > Working Directory).

Locating Other References

- Additional Programming Resources
 - Developing Solutions Student Guide
 - Aras Innovator 22 Programmers Guide
 - API Training Examples Help
 - Aras.com Developers Forum

- Searchable at <https://community.aras.com/>



Locating Other References

In addition to this Student Guide, you can also find additional information about the Aras API by downloading the Programmer's Guide from the Aras.com website (Documentation page).

An additional help file is also provided with this course, which demonstrates each API call with a small example that can be used in your own development work.

Aras.com also has an actively moderated Developers Forum.

Reviewing the Context Item



DR-000010

Methods are executed on the instance of an Item

Use the keyword *this* to reference the Item
(*Me* in VB.NET)

Contains an authenticated connection to the server



MyMethod

```
string value = this.getProperty("_cost");
```



Reviewing the Context Item

Methods represent the behavior of Items. **Methods execute in the context of an Item and can refer to it using the keyword *this* (or *Me* using VB.NET).**

The context Item may be a single Item, a collection of Items, an Error, just a Result (a string) or be empty. Robust code will check for all these possibilities.

In general, it is a best practice not to alter the context Item, to avoid side effects. An exception to this rule is in certain Server Events where the purpose of the Method is to change the context Item.

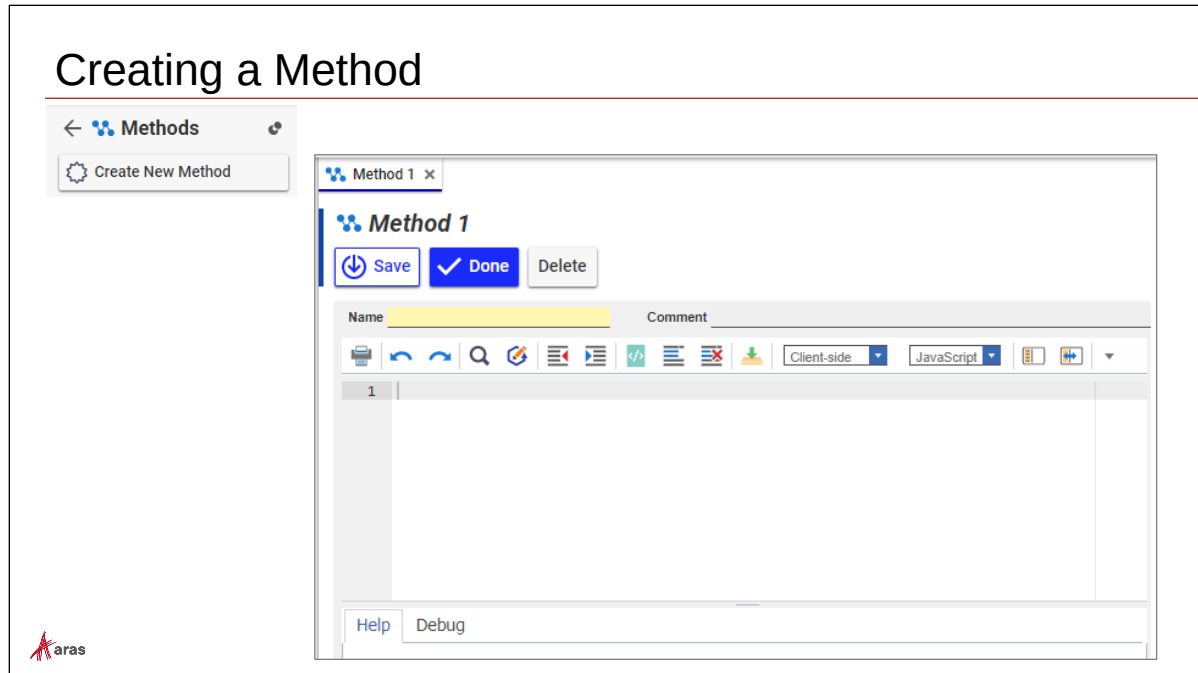
Reviewing the Item Factory Design Pattern

- Methods typically follow an Item Factory Design Pattern
- Method executes on Context Item
- Method should return an Item
 - Recommended for Client Methods
 - Required for Server Methods



Reviewing the Item Factory Design Pattern

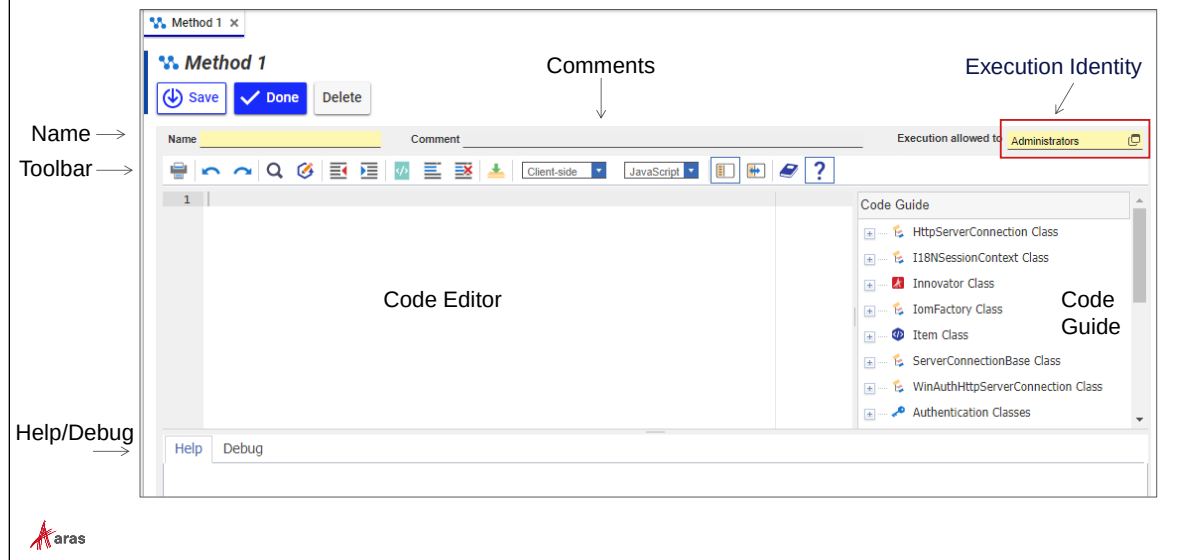
A basic rule in the Aras Innovator Methodology is that a Method should always return an Item. This is optional (but recommended) in client Methods. It is required for a server Method. If your Server Method does not return an Item, a compilation error will be raised when you perform a syntax check or attempt to run the Method.



Creating a Method

To create a Method, choose Administration > Method from the TOC. A Method Item is created in the database to store the source code of the procedure you will invoke from Aras Innovator.

Using the Method Editor



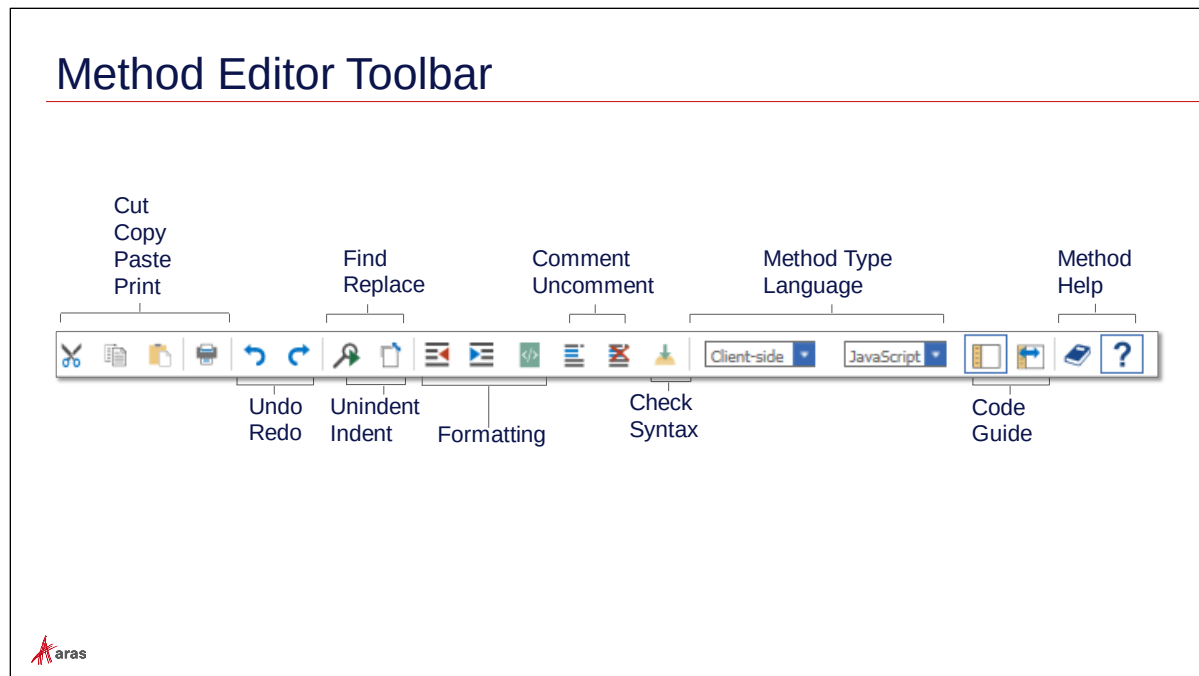
Using the Method Editor

When you create a Method in Aras Innovator, the editor allows you to enter source code and configure some basic settings.

Name	Name of the Method Item to be saved in the database. Note this name is case sensitive.
Comments	Text description of the Method's purpose.
Toolbar	A series of buttons for working with the editor – shown on the next page.
Code Guide	An abbreviated guide to the IOM – API Reference is more complete under User Menu > Admin.
Help/Debug	Two tabs – Debug shows any compilation errors and Help has an abbreviated display of IOM method parameters.
Execution Identity	Members of this identity are allowed to execute this Method.

Note

The default Execution Identity is the Administrators group when a Method is created. You will need to adjust this Identity based on who will need execute access to this program.



Method Editor Toolbar

Standard editing functions are available while working in the code editor. The Check Syntax button is a helpful resource that will do a compilation check on the method to look for obvious errors in your code.

Note

Server method syntax checking is more robust as the method is compiled on the server and any compile errors are returned as a message in the Debug window. Client syntax checking relies on the browser and is not as thorough (i.e. will not check for IOM method signature errors).

Previewing the Item and Innovator Classes

- **Innovator Class**
 - Represents the Innovator instance
 - Contains authenticated connection to Innovator Server
 - Includes operations that are not Item-related
- **Item Class**
 - Represents the Item instance
 - Includes operations that are Item-related
 - Contains a property field (dom) representing an Item DOM object



Previewing the Item and Innovator Classes

The Innovator and Item classes are the two main classes that contain many methods to generate and send AML requests to the Innovator Server. You learn about many of these methods in this class.

Comparing IOM and AML

- AML Example:

```
<Item type="Design Request" action="get">
  <_cost>100</_cost>
  <_patent_required>1</_patent_required>
</Item>
```

- Using the IOM in a Server Method

```
01 Item itm = this.newItem("Design Request", "get");
02 itm.setProperty("_cost", "100");
03 itm.setProperty("_patent_required", "1");
04 return itm.apply();
```



Comparing IOM and AML

Remember – everything in Aras Innovator is an Item and every Item is described using Adaptive Markup Language (AML). When you use the IOM on the client or server, you are generating AML that will be sent to the server in the form of a SOAP message. That request will in turn send a response back in the form of AML.

In the example above, an AML request is made to return any Items of type Design Request that have a _cost Property set to “100” and _patent_required set to “1”(true).

To create this request using the IOM you use appropriate IOM class methods to generate the AML.

In this example, a new object of type Item is created in the Method (line 1). Two properties are then set on the object using setProperty. Finally, the object is sent to the server, which will return an AML response containing any Items found.

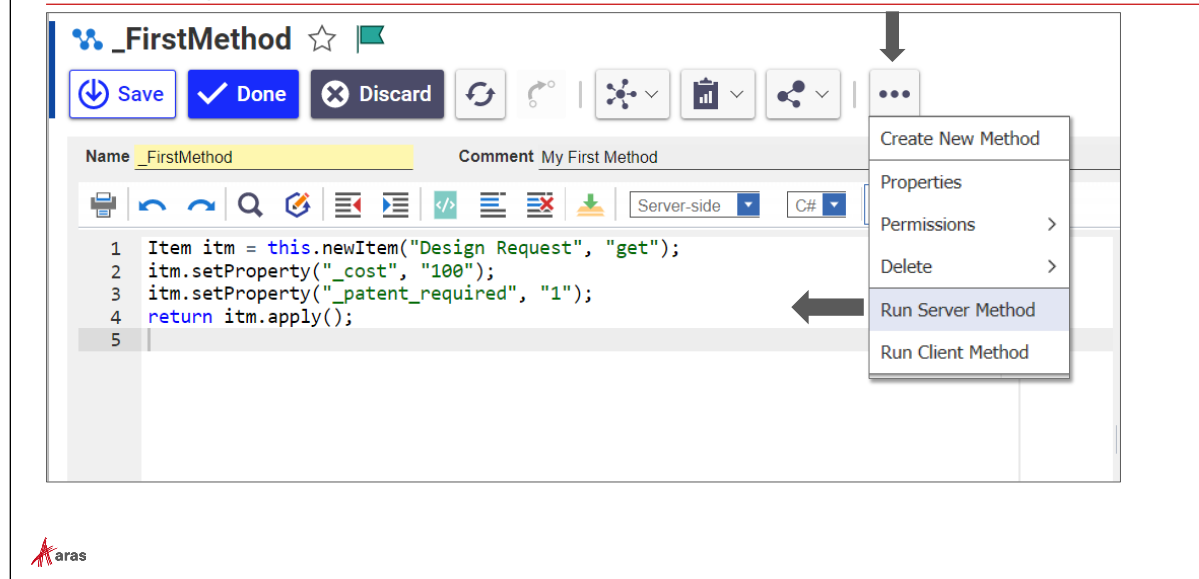
Note

Understanding AML is key to working with the IOM effectively.

VB.NET Example

```
Dim itm As Item = Me.newItem("Design Request","get")
itm.setProperty("_cost", "100")
itm.setProperty("_patent_required", "1")
Return itm.apply()
```

Running a Method



Running a Method (= Testing a Method)

The Run Server Method Action will execute a server Method and display a response window with the output of the response in a separate tab window. This is helpful for testing purposes.

This Action uses a mechanism which appends new results to previous ones foreseen the output tab is still open. If you do not want to scroll down in the output, close the previous tab before calling Run Server Method again.

Run Client Method is an Action that has been provided for training purposes and is not available in the standard product. Use this Action to execute a client Method and review the results. This Action uses an aras object JavaScript method that you will learn about later in this course.

Note

You cannot run a Method until it has been saved. When a Method is executed, the copy saved in the database is the current source of the Method. Remember to always save a Method first before attempting to execute it.

Comparing IOM and AML

- AML Example:

```
<Item type="Design Request" action="get" select="id">
  <_cost>100</_cost>
  <_patent_required>1</_patent_required>
</Item>
```

- Using the IOM in a Client Method

```
01 const itm = this.newItem("Design Request", "get");
02 itm.setProperty("_cost", "100");
03 itm.setProperty("_patent_required", "1");
04 itm.setAttribute("select", "id");
05 return itm.apply();
```



Comparing IOM and AML

In this example an attribute is set in the AML request to select the id property from the Design Requests found. Using the *select* attribute can greatly reduce the amount of data that needs to be returned from the server and can increase performance of a solution.

The setAttribute IOM method can be used to set the same attribute and generate the AML displayed in this example.

Creating an Item

▪ Client (JavaScript)

```
01 const myItem = this.newItem("Design Request", "add");
02 myItem.setProperty("_title", "New PC Type");
03 myItem.setProperty("_cost", "500");
04 return myItem.apply();
```

▪ Server (C#)

```
01 Item myItem = this.newItem("Design Request", "add");
02 myItem.setProperty("_title", "New Mobile Phone");
03 return myItem.apply();
```



Creating an Item

Everything is defined as an Item in Aras Innovator including server requests and responses.

The newItem method allows you to create an Item AML request in a program, set various properties and attributes, and apply them against the Innovator Server. You will learn about more options and other methods throughout the course.

VB.NET

```
Dim myItem As Item = Me.newItem("Design Request", "add")
myItem.setProperty("_title", "New Mobile Phone")
Return myItem.apply()
```

Editing an Item Object

▪ Client (JavaScript)

```
const myItem = this.newItem("Design Request", "edit");
myItem.setAttribute("where",
    "[design_request]._item_number='DR-000010'");
myItem.setProperty("_cost", "250");
return myItem.apply();
```

▪ Server (C#)

```
Item myItem = this.newItem("Design Request", "edit");
myItem.setAttribute("where",
    "[design_request]._item_number='DR-000010'");
myItem.setProperty("_cost", "250");
return myItem.apply();
```



Editing an Item Object

As you learned in a previous unit of the course, to change an existing Item you use the edit action which requires a “where” clause to provide the required criteria.

The Item class `setAttribute` method can be used to assign the where clause to the edit request, and `setProperty` can be used to set the changed property value.

VB.NET

```
Dim myItem As Item = Me.newItem("Design Request", "edit")
myItem.setAttribute("where",
    "[design_request]._item_number='DR-000010'")
myItem.setProperty("_cost", "250")
Return myItem.apply()
```

Supported IOM Item Types

- **Single**
 - Single Instance of an ItemType
- **Collection**
 - A set of Items (e.g. "get" action that returns multiple Items)
- **Error**
 - Request fails against the Innovator Server
- **Result**
 - Arbitrary text wrapped by <Result> tags
- **Logical**
 - Set of property wrapped in logical statements (query)



Supported IOM Item Types

An Item object can represent several types of data structures.

The IOM is intended to be a generic and compact API for modeling the Item structure of the AML as abstract Objects. Most methods for the IOM deal with memory management of the AML document for the Item Object. The AML is a script sent as a message to the Aras Innovator Server; the IOM is an Object API to build the AML messages, submit them to the Innovator Server and parse an AML document that is returned by the Server.

Using the Innovator Class

- Obtain the Innovator instance from the Context Item:

- Client (JavaScript)

```
const inn = this.getInnovator();
```

- Server (C#)

```
Innovator inn = this.getInnovator();
```



Using the Innovator Class

The Innovator class provides fundamental methods for working in the IOM. You will learn about these methods later in this course.

The Innovator object is easily obtained from the Method context Item by using the `getInnovator()` IOM method.

VB.NET

```
Dim inn As Innovator = Me.getInnovator()
```

Returning A New Result Item

▪ Client (JavaScript)

```
const inn = this.getInnovator();
const uid = inn.getUserID();
return inn.newResult(uid);
```

▪ Server (C#)

```
Innovator inn = this.getInnovator();
string uid = inn.getUserID();
return inn.newResult(uid);
```



Returning a New Result Item

The newResult method takes a string as its argument which it then transforms into an Item object.

The string will be enclosed in a <Result> tag as follows:

```
<Result>30B991F927274FA3829655F50C99472E</Result>
```

VB.NET

```
Dim inn as Innovator = Me.getInnovator()
Dim uid as String = inn.getUserID()
Return inn.newResult(uid)
```

Returning a New Error Item

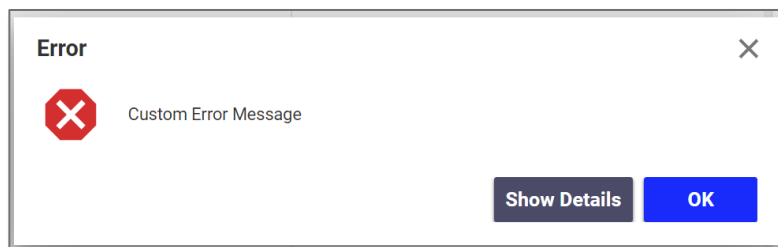
- Server (C#)

```
Innovator inn = this.getInnovator();  
return inn.newError("Error message!");
```



Returning a New Error Item

A server Error Item will display a dialog on the and can be configured with additional attributes you will learn about later in this course. An error item raises an exception on the server blocking the current transaction.



VB.NET

```
Dim inn As Innovator = Me.getInnovator()  
Return inn.newError("Error message!")
```

Summary

In this unit you learned how to create new and server Methods and some fundamental IOM methods that are used to generate AML.

You should now be able to:

- Use the Editor to Create a Method
- Understand the Basics of the Innovator Object Model (IOM)
- Compare IOM and AML
- Execute a simple or Server Method
- Explore the API On-Line Reference
- Use the Context Item in a Method
- Use the Item Factory Design Pattern
- Return a New Result or Error

Review Questions

1. What are some basic differences between a Method and a server Method?
2. What should every Method return? When is it required vs. recommended?
3. What does every IOM method generate?
4. What IOM methods can you use to retrieve an Item(s) from the database using the IOM?

Lab Exercise

Goal

Be able to create basic and server Methods and test them in the Innovator .

Scenario

In this exercise, you will create several Methods that use the IOM to generate AML to the server. You will run these Methods using the Run Server and Run Method Actions available from the Solution Studio Editor.

Retrieve Items

1. Create a Server Method named **Server GetOrders** that retrieves all Sales Orders that allow Multiple Shipments and use the Shipping Method of UPS. Unit test the Method using the Run Server Method Action in the Method editor.
2. Create a Method named **GetOrders** that performs the same command as above but only retrieves the Sales Order number. You must use the *select* attribute to accomplish this. What IOM method allows you to assign an attribute? _____
Unit test the Method using the Run Method Action in the Method editor.

Add Items

3. Create a Server Method named **Server AddOrder** that will create a new Sales Order with the Shipping Method of Federal Express. Unit test in the Method editor and make sure the item is created.
4. Modify the Method you created above to also assign a Purchase Order number (using today's date and time) using the following .NET function:

C#:

```
string po = DateTime.Now.ToString("yyMMddhhmmss");
```

VB:

```
Dim po As String = DateTime.Now.ToString("yyMMddhhmmss")
```

Edit Items

5. Create a Server Method named **Server EditOrders** that will modify any Sales Orders that allow Multiple Shipments to use the UPS Shipping Method. You must use the "where" attribute to edit the item.
For performance reasons take care that the system does not respond with the complete data of the modified Sales Order, instead returns a minimum. You must use the *doGetItem* attribute to accomplish this.

This page intentionally left blank.



Unit 4 Debugging and Logging

Client/Server Methods

Overview

In this unit you learn how to debug client and server Methods. Aras Innovator relies on industry-standard just-in-time debugging tools to allow you to single step through a program and examine variables and values. You will learn how to configure Microsoft Visual Studio and the internet browsers to support both client and server-side debugging. You will also learn how to activate logging features to capture information about a running application.

Configuring a debugger for client and server Methods is probably one of the most important steps you will take when you begin to create custom solutions (unless you are a perfect coder!).

Objectives

- Debugging Server Methods
- Debugging Client Methods
- Configuring Logging
- Clearing Cache
- Debugging Production Code

Configuring the Server Method Debugger: Overview

- Using Visual Studio
 - Visual Studio Community, Pro or Enterprise Edition
- Activating Aras Server Debugging
- Attaching to the IIS Worker Process
- Setting a Breakpoint in a Server Method



Configuring the Server Method Debugger: Overview

Because Aras Innovator is a Microsoft .NET server application, you can easily configure Microsoft Visual Studio to execute at a break point in your Method.

Visual Studio supports both local and remote debugging if you are not developing Methods on the server machine.

Required Software

To debug server-side Methods, you will need to download and install Microsoft Visual Studio. The Community edition is free for local non-enterprise development. For more information on Visual Studio licensing options go to <https://visualstudio.microsoft.com/>

Activating Aras Server Debugging

To allow server side debugging of Aras Methods requires a change to the system configuration file.

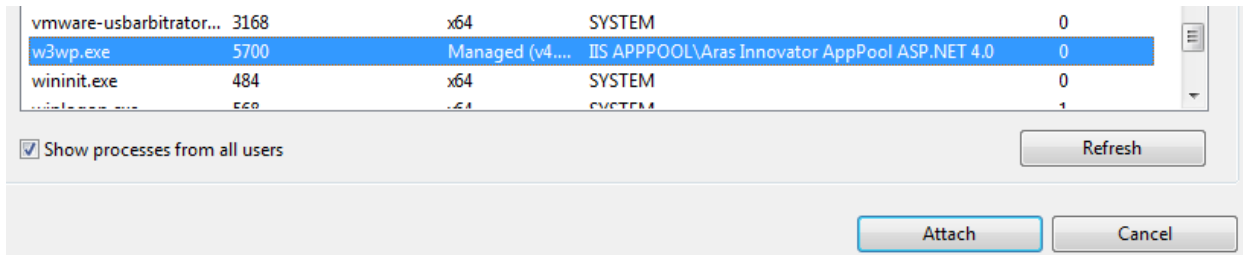
Launching the Visual Studio Debugger from a Method

If the Aras Server is installed locally on your development machine, you can use the .NET launch command to attach to the Aras IIS process by providing the following line of code in your server Method.

Attaching to the Aras IIS Worker Process

Otherwise, to prepare Visual Studio to access Aras Server Methods you must attach to the Aras IIS worker process named **w3wp.exe** on the server which is associated with the Aras Innovator Application pool.

From the Visual Studio main menu select Debug > Attach To Process and locate the process in the list. If the process does not appear, make sure you have clicked the *Show processes from all users* box and that the Aras Innovator client is up and running.



Attach to Process

When prompted click the *Attach* button and accept any warning prompts.

Note: Visual Studio also supports remote server debugging if you are not developing Methods directly on the same server machine. Go to <https://docs.microsoft.com/> and search on Visual Studio Remote Debugger for more information about available tools and setup.

Setting a Breakpoint

Enter the statement `System.Diagnostics.Debugger.Break()` in a server Method to set a breakpoint. When the Method is executed, the attached Visual Studio debugger is executed.

IIS Settings

The Windows IIS server default debug timeout on a single Method step is 90 seconds. This can cause the client to hang when time expires. You can change this behavior by starting the IIS Manager and selecting Advanced Settings of the Aras Application Pool. From the context menu set Process Model/Ping Enabled to False (default true) or increase the Ping Maximum Response Time.

Configuring the Server Method Debugger: Details

- **Activating Aras Server Debugging**

- Include the following in the file **InnovatorServerConfig.xml**:

```
<operating_parameter key="DebugServerMethod" value="true"/>
<operating_parameter key="ServerMethodTempDir" value="[path]" />
```

- **Setting a Breakpoint in a Server Method**

- Include the following at start of Server Method:

```
if (System.Diagnostics.Debugger.Launch())
    System.Diagnostics.Debugger.Break();
```



Configuring the Server Method Debugger: Details

Activating Aras Server Debugging

You enable the server side debugging of Aras Methods via a change to the system configuration file **InnovatorServerConfig.xml** file located on the server.

The following tags must be set to support server Method debugging:

```
<operating_parameter key="DebugServerMethod" value="true" />
<operating_parameter key="ServerMethodTempDir" value="[path]" />
```

The default setting for the key **DebugServerMethod** is "false".

Each time a server Method is checked for syntax, a corresponding dll assembly file as well as a program database file (pdb) will be created containing the compiled program code for debugging purposes and written to the directory assigned in the **ServerMethodTempDir** key above.

The names of the dll and pdb files follow a naming scheme:

[release and build number].[database_name].[Method_name].[Method_GUID]

For example, if a server Method is created named *SampleMethod* in the **InnovatorSolutions** database the following files will be generated in the location indicated by the **ServerMethodTempDir** key:

12.0.0.0000.InnovatorSolutions.SampleMethod.17FBE8EAA7DD41329FE52D6E3CEE135A.dll

12.0.0.0000.InnovatorSolutions.SampleMethod.17FBE8EAA7DD41329FE52D6E3CEE135A.pdb

Launching the Visual Studio Debugger from a Method

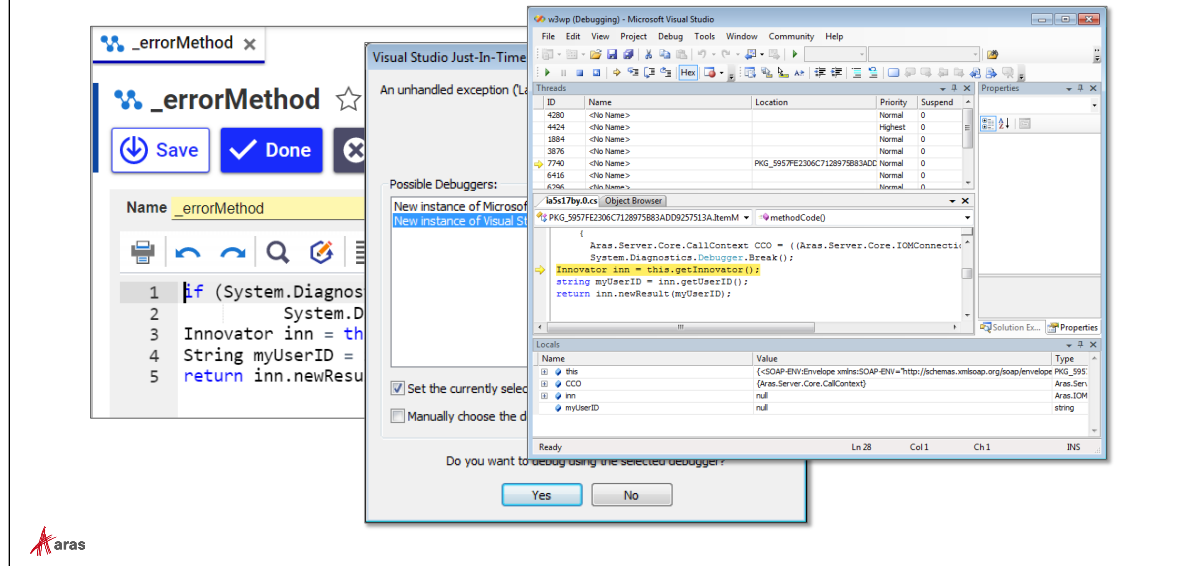
If the Aras Server is installed locally on your development machine, you can use the .NET launch command to attach to the Aras IIS process by providing the following line of code in your server Method:

```
if (System.Diagnostics.Debugger.Launch())  
    System.Diagnostics.Debugger.Break();
```

VB.NET

```
If System.Diagnostics.Debugger.Launch() Then  
    System.Diagnostics.Debugger.Break()  
End If
```

Debugging a Server Method



Debugging a Server Method

You can use all the Visual Studio debugging features available (single step, quick watch, etc.) while examining your Method code.

Notes

Your Method runs inside of the Aras Framework – you will also see the framework code while examining your own Method.

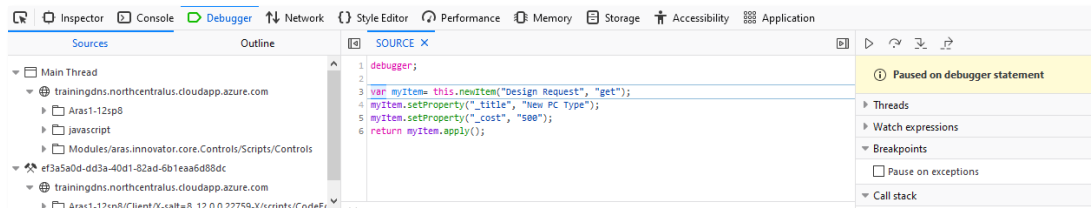
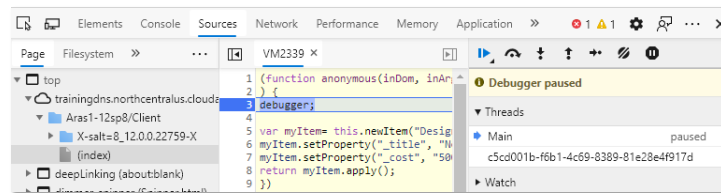
Note that any changes you make to the Method code must be made in the Aras Innovator Method editor. Changes made in the Visual Studio debugger are not saved permanently to the Method.

You can also write optional debug messages from a server Method to the Visual Studio Output window by using the `.NET Write` method of the `System.Diagnostics.Debug` class :

```
System.Diagnostics.Debug.WriteLine("Result is" + var);
System.Diagnostics.Debug.WriteLine(item);
```

Browser Debuggers

- Microsoft Edge
- Mozilla Firefox
- Google Chrome



Browser Debuggers

To access the developer tools in each browser, press the F12 key while logged into Aras Innovator. All three browsers supported by Aras Innovator have built-in debugging tools to allow you to single step through an Aras Client Method (JavaScript) and inspect variables, etc.

To Debug a Client Method

1. Add the *debugger;* statement to the Method to be examined.
2. Start the browser debugging tools by pressing the F12 key.
3. Execute the Client Method. The Script tag will display the running Method and break at the *debugger* statement. You can then Step Over each line to examine the results (F10).

Note

More information about the features of each browser debugger is available from the respective web sites of each browser supplier.

Debugging Client Methods

- Use the *debugger* command
- Sets breakpoint in JavaScript Client Method

 Debugger Command

```
debugger ;  
const myItem = this.newItem("Design Request", "get");  
myItem.setProperty("_title", "New PC Type");  
myItem.setProperty("_cost", "500");  
return myItem.apply();
```



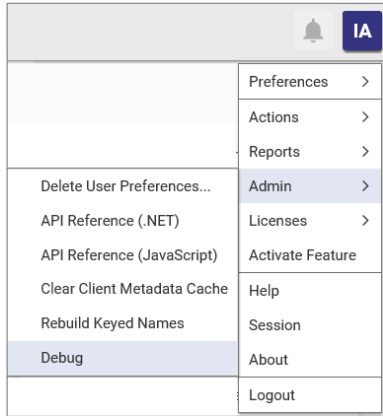
Debugging Client Methods

The debugger statement invokes any available debugging functionality, such as setting a breakpoint. If no debugging functionality is available, this statement has no effect.

The debugger statement can be used in Mozilla Firefox, Microsoft Edge and Google Chrome to allow you to debug a Client Method that uses JavaScript.

Setting Client Debug Option

- aras.DEBUG is set to TRUE



Example Code to evaluate:

```
if (aras.DEBUG) {
    debugger;
}
```

Setting Client Debug Option

The Debug menu is a toggle that allows you to set a reserved boolean iable in the aras object. If the menu is checked then aras.DEBUG=true else, it equals false.

This can be used to enable or disable client debugging without modifying the Client Method.

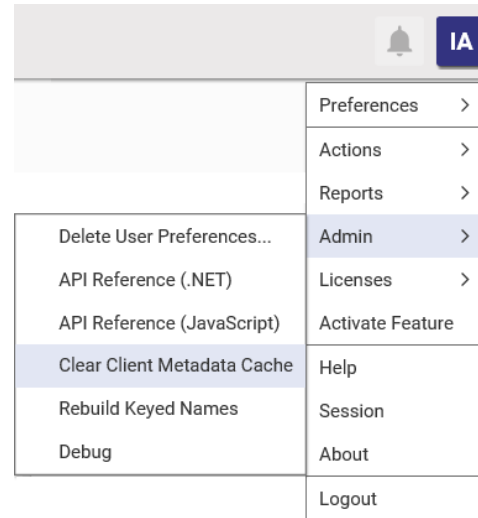
Example

```
if (aras.DEBUG) {
    debugger;
}
```

Clearing Client Cache

- Client Cache
 - Close all Browser Windows
 - Close any Browser processes
 - Delete all temporary Internet Files

Clear Client Metadata Cache →



Clearing Client Cache

Aras Innovator caches the Item DOM in memory along with other browser information. At times, it may be necessary to clear this cache to resolve a problem or prevent the inspection of data. When a client is started, Aras Innovator will cache metadata about commonly used ItemTypes that a user works with in a session to improve performance. You can clear this cache by selecting *Clear Client Metadata Cache* from *User Menu > Admin*

You should also delete any temporary files using the browser delete files/history feature.

Inspecting an Item

- Item contains:
 - dom – Document Object Model for Item
 - node – item node (if a single Item)
 - nodelist – item nodes (if more than one Item returned)

[-] item	{<SOAP-ENV:Envelope xmlns:SOAP-ENV:
[+] dom	{Document}
node	null
[+] nodeList	{System.Xml.XPathNodeList}



Inspecting an Item

Every item object supports an XML dom, node and nodelist object. The dom object represents the Document Object Model of the current item(s). The node object is populated if the item represents a single instance. The nodelist object will be populated if the item represents multiple instances.

Exact steps to display the item dom, node and nodelist will differ between browser debugger interfaces.

Configuring Logging

- Writing to a log file:
 - `CCO.Utilities.WriteDebug("logfile", stringvar)`
- Appends *stringvar* text to log file
- Log files written to "temp_folder" location
 - Configured in InnovatorServerConfig.xml file
- Example:


```
01 string msg = "Changed variable value";
02 CCO.Utilities.WriteDebug("DebugMsg", msg);
```



Configuring Logging

Logging options are available to help identify an issue in a running system without halting the system using a debugger.

The CCO class (Core Context Object) contains several useful utility methods including the ability to write debug messages to a log file. The WriteDebug method accepts two arguments – the name of the log file (the .log extension is automatically applied) and a string which represents the log text to append to the file. In the example above, the log file DebugMsg.log will be created (if it does not already exist) and the message will be added to the file with a date and time stamp.

The location of the debug log is indicated by the temp_folder setting in the InnovatorServerConfig.xml file.

Note

The CCO.Utilities.WriteDebug method has been deprecated as of Version 11 SP 11 and will be replaced with a new method in a future service pack.

Debugging Production Code

- Logging Server Requests
- Enable Logging – InnovatorServerConfig.xml


```
<operating_parameter key="debug_log_flag" value="true">
</operating_parameter>
<operating_parameter key="debug_log_prefix"
value="C:\\Program Files (x86)\\Aras\\Innovator12\\Server\\Logs\\">
</operating_parameter>
```



Debugging Production Code

Debugging production code presents its own set of issues as it may not be possible to set breakpoints or disturb the system as it is running. The following options allow you to log requests made to the Aras server.

To log requests made by all clients logged into the system, access the InnovatorServerConfig.xml file in the Aras installation directory. The following parameter keys are available in the InnovatorServerConfig.xml file to invoke and configure server logging:

debug_log_flag

Set to true to invoke server logging.

debug_log_limit

Maximum number of server requests per log file (default 10000). When exceeded, a new log file is created.

debug_log_pretty

Set to true to display the log contents using indentation.

debug_log_prefix

File path location to create log files. Defaults to Aras Server\\logs directory. A new log file is created for each connected session using the date and time and session id as the file name. This includes individual user sessions as well as system processes.

Note

To obtain the Aras Session id for a logged in user access the database table named application.ACTIVEUSER. The ARAS_SESSION_ID column holds the id value and LOGIN_NAME shows the associated user.

Other Tools

There are also other supplemental tools you can download and use to help when debugging server requests.

Summary

In this unit you learned how to configure Aras Innovator to work with a Just in Time Debugger like Microsoft Visual Studio to be able to debug both client and server Methods.

You should now be able to:

- Debug Server Methods using Visual Studio
- Debug Client Methods in a Browser
- Configure and Review Logs
- Clear Cache
- Understand the implications of debugging a production problem

Lab Exercise

Goal

Be able to debug server Methods using the Visual Studio debugger.

Scenario

In this exercise, you will debug a server Method to determine and correct a problem.

Steps

1. Locate the Method named **DebugMe**. This server Method was created to return the current purchase order number of a Sales Order as a Result (and does not execute).
2. Check the Method syntax first to correct any issues before trying to run or debug the Method and then insert the appropriate statement to start the Visual Studio Debugger and run the Method.
3. Use Single Step (F10) to step over each statement in your Method and notice how the AML is generated at each statement for the Item you are working with.
4. Step past the apply() method statement in the code and inspect the Item(s) returned.
5. Compare the property names used in the Method with the Sales Order ItemType properties.
6. Correct the Method so that it operates as intended.



This page intentionally left blank.



Unit 5 Creating Custom Actions

Overview

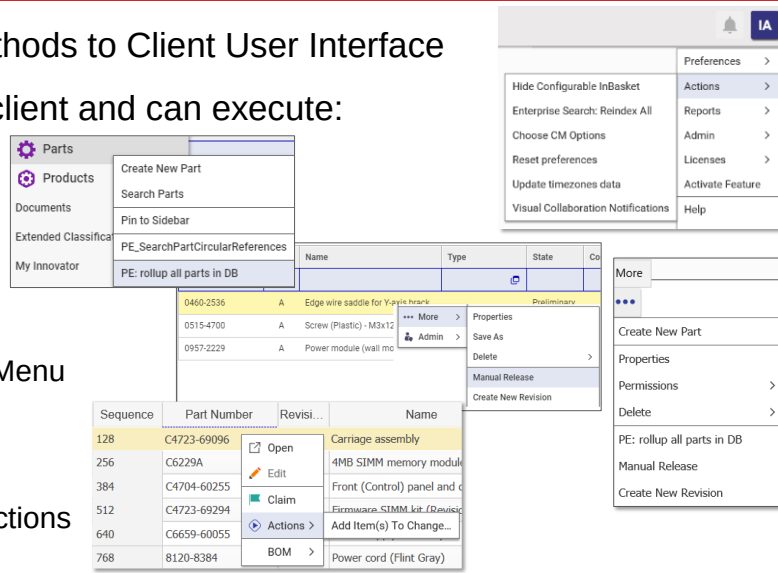
In this unit, you learn how to build Actions that appear on client menus to allow a user to perform an operation. You will learn about the three kinds of Actions available and use all of them.

Objectives

- Defining Actions
- Defining Action Types
- Creating Generic Actions
- Creating ItemType Action
- Creating Item Actions

Defining Actions

- Action Items bind Methods to Client User Interface
- Action is invoked on client and can execute:
 - Client Method
 - Server Method
- User selects from:
 - User Menu > Actions
 - Search Grid Context Menu
 - Item More Menu
 - TOC Context Menu
 - Relationship Grid > Actions



Defining Actions

An Action Item is used to bind Methods to the Aras client user interface (menus). An Action appears as a menu entry and is invoked on the client side but can call either a client or server-side Method.

Action menu entries appear in various locations depending on the Action type configuration.

These include the:

- User Menu (button)
- Search Grid (context menu)
- Item More Menu (button)
- Table of Contents (context menu)
- Relationship Grid (context menu)

Defining Action Types

- Generic – Action applies to any Item(s)
- ItemType – Action applies to **all** Item(s) of a specific ItemType
- Item – Action applies to current Item(s) **selected** by a user (one by one)



Defining Action Types

Generic Actions are useful for defining Methods that do not require a context to an existing Item or ItemType. Typical use cases for Generic Actions are programs that perform system-wide calculations or assign system settings. Generic Actions are always available to a user in the User Menu. Reset preferences is an example of a Generic Action.

ItemType Actions are assigned to ItemTypes. The Action menu entry will only be displayed when the ItemType category is selected from the TOC by a user. The ItemType Action Method does not have any context to items that may be open or selected in the client. A parts cost roll-up program is an example of an ItemType Action that calculates the cost of all parts in the database.

Item Actions are used to process or change one (or more) item(s) that a user has selected from the search grid. The Item Action passes the selected item (context) to the corresponding Method which can then be used to act on the item. The Add Item(s) to Change Action is an example of an Item Action.

Creating a New Action

Creating a New Action

Select Action from Administration in the TOC and create a new Action Item. The following fields are available:

- ① **Name**
Name of the Action (used for Menu Item entry text if no label supplied)
Aras Innovator uses the name of the Action item to sort the Action menu choices in ascending order. To change the order of a menu choice you can name the Action item accordingly.
- ② **Type**
Generic, Item, ItemType (Note: URL is not supported)
- ③ **Location**
Location of Method – Client or Server
- ④ **Label**
Used in installations that support multiple languages
- ⑤ **Method**
Name of the Client or Server Method
- ⑥ **Target**
Defines, where the Method result will be displayed. No impact to error messages
None – No results displayed when Method executes
Window – Method results appear in a tab – subsequent calls to the Action display new tabs
Main – Currently same as Window
One Window – Method results appear in a tab – subsequent calls to the Action append to the previous tab, if kept open

⑦ **On Complete**

Optional JavaScript Method that executes following completion of Action Method

⑧ **Body**

Additional arguments to be passed to the Action Method, applies to Generic and ItemType Actions only

⑨ **Query**

Define context Item for the Action Method, applies to Item Actions only, syntax different for client or server Methods

⑩ **Can Execute**

Method that can be used to disable the Action menu choice; useful for providing additional security restrictions on an Action; the Method needs to have a Boolean return

Client Method for client-side Actions, Server Method for server-side Actions

Can Execute Example Method

This Method checks to see if the current user has an email address. If there is no address the Action menu item is disabled.

JavaScript:

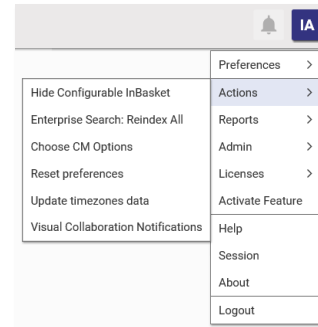
```
const inn = this.getInnovator();
const id = inn.getUserID();
const user = inn.getItemById("User", id);
if (user.getProperty ("email", "") === "") {
    return false;
}
else {
    return true;
}
```

C#:

```
Innovator inn = this.getInnovator();
string id = inn.getUserID();
Item user = inn.getItemById("User", id);
if (user.getProperty ("email", "") == "") {
    return inn.newResult("false");
}
else {
    return inn.newResult("true");
}
```

Defining Generic Actions

- Appear in top Actions User Menu regardless of Item or ItemType selection
- Useful for working with any ItemType
- Server Method Context Item is the executing Method
 - *this* keyword (or Me in VB.NET)



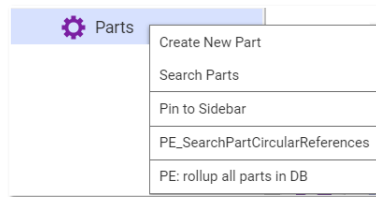
Defining Generic Actions

Generic Actions are used for general activities that are not associated with a specific Item or ItemType. You define access to the context Item (the Method itself) using the keyword *this*.

Generic Actions appear to a user in the User Menu > Action menu.

Defining ItemType Actions

- Associated with an ItemType
- Action Menu item only appears when user selects ItemType from TOC
- Method Context Item is the executing Method
 - *this* keyword (or Me in VB.NET)



Defining ItemType Actions

ItemType Actions define some logic for all Items of the same type.

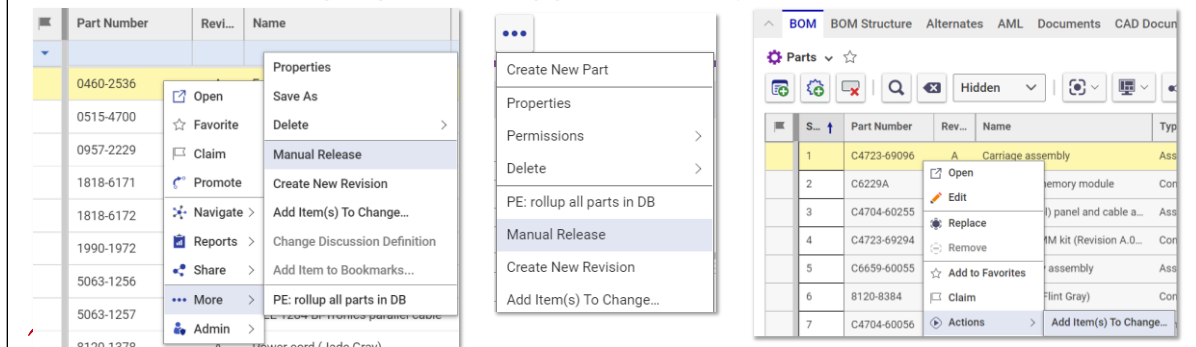
The context item of the associated Method is the Method itself.

ItemType Actions appear to a user in the following locations:

- TOC context menu on a selected ItemType category
- Search Grid context menu
- More toolbar menu on an open item

Defining Item Actions

- Are associated with the current selected Item from:
 - Item Search Grid
 - Relationship Grid
- Context Item is (are) the Item(s) selected by a user in a Grid



Defining Item Actions

An Item Action operates within the context of a single Item. An Item Query is generated by default, which returns the current item the user has selected. The associated Method has access to the context item using the variable `this` (or `Me` in VB.Net). Two different queries are defined based on whether a client or server Method will be used with the Action.

Item Actions appear to a user on the item Search Grid context menu, the More menu in an item window, and on the relationship item context menu in the Relationship Grid.

The grid on which they appear just depends on the fact, whether the ItemType connected to the Action is a relationship ItemType or not.

The big majority of Actions delivered with the Aras Innovator are Item Actions.

Item vs. ItemType Action – Differences in Methods

Name	Comment
TRN example for an Item Action	Sets the shipdate for SELECTED Sales Order
<pre> 1 //Sets the shipdate for SELECTED Sales Orders to the current date 2 3 this.setAttribute("action", "edit"); 4 this.setProperty("ship_date", "__now()"); 5 return this.apply(); </pre>	
TRN example for ItemType Action	Sets ALL Sales Orders with multiple shipmen
<pre> 1 //Sets ALL Sales Orders with multiple shipment to UPS 2 Item so = this newItem("Sales Order", "edit"); 3 so.setAttribute("where", "[sales_order].multiple_shipment='1'"); 4 so.setProperty("shipping_method", "UPS"); 5 return so.apply(); </pre>	

Item vs. ItemType Action – Differences in Methods

A default query is supplied in the Action form when you choose type Item and have entered the location. The query is used to populate the context item variable (this or Me) in the associated Method. Like that the context item contains the selected item. If the user selects more than one item, the Method will be called individually for the selected items one after the other.

The query syntax is defaulted automatically, differently for client vs. server to retain backward compatibility.

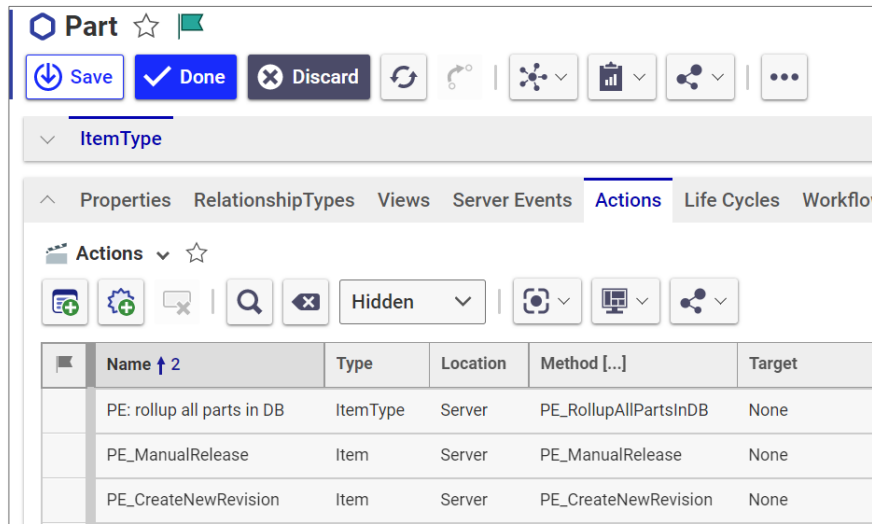
When a user selects an Item to execute an Action from the search grid, the current Item's AML action attribute is set to *get*. You must change the "action" attribute of the context item to *edit* to allow changes to be made to an existing Item – and apply the changes to the server.

Methods for ItemType Actions normally define via newItem an item with a specified ItemType and the "action" *edit*. These Methods, too, typically set an attribute, but in this case it is the "where" attribute in order to define the items to be changed, and it is set on the item which was returned from the newItem-method.

Note

Methods which can be used with an Item Action cannot be tested from the More button, because they need the real context item.

Relating Item and ItemType Action to ItemType



To Relate an Action to an ItemType

1. Edit an ItemType and select the Actions tab.
2. Add the Action to the ItemType.

Aras automatically sorts the Actions based on the Sort Order column. You can define a custom menu sort order for the Actions by supplying a ranking number in the Sort Order column.

Relationship Actions

To configure an Item Action for relationship items, you must assign the Action to the desired Relationship ItemType. The context of the associated Method will be an item representing the selected relationship in the following format:

```
<Item id="[current id]" type="[Relationship ItemType Name]" typeid="[ItemType id]" levels="#">
```

The Action menu entry will appear on the Relationship Grid context menu.

The following example Method accesses the related item of the currently selected Sales Order Part relationship:

```
const inn = this.getInnovator();

//Get ID of the relationship item
const i = this.getID();

//Get the Relationship Item from the database
const relationship = inn.getItemById("Sales Order Part", i);

//Get the Related Item
const related_item = relationship.getRelatedItem();
```


Action Type Summary

	Generic	ItemType	Item
Assignment to ItemType	-	✓	✓
Ordered by	name	sequence no	sequence no
Displayed from the User Menu Button	✓	-	-
Displayed from Search Grid Context Menu	-	✓	✓
Displayed from the More Button (...)	-	✓	✓
Displayed from the TOC Context Menu	-	✓	-
Displayed from the Relationship Grid	-	✓	✓
“this” contains	Method	Method	selected Item(s) in Grid
Passing Parameters	in Body	in Body	-



Action Type Summary

The chart above summarizes the features of the three Action Types discussed in this unit.

Summary

In this unit, you learned how to build custom Actions that are available from the Innovator client to perform custom operations.

You should now be able to:

- Define Actions and Action Types
- Create Generic Actions
- Create ItemType Action
- Create Item Actions
- Associate ItemType and Item Actions with an ItemType

Lab Exercise

Goal

Be able to create an Action that calls a Method to perform a custom user function.

Scenario

In this exercise, you will use Server and Client Methods to create custom Actions that users can execute.

Generic Action

1. Create a Server Method named **Server-GetMachineName** that will retrieve the name of your current server. You can obtain this using the .NET instruction:

C#

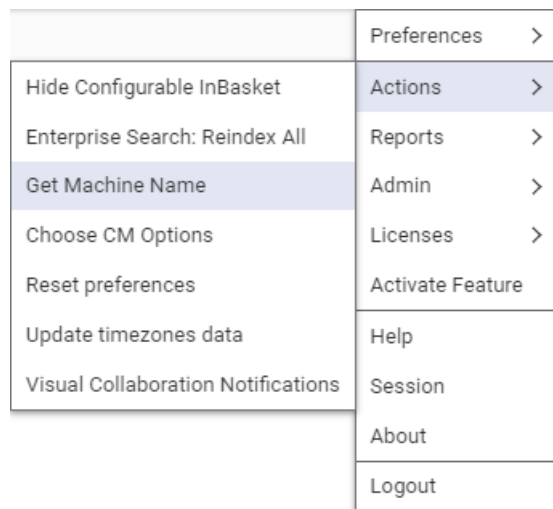
```
string machine = System.Environment.MachineName;
```

VB:

```
Dim machine As String = System.Environment.MachineName
```

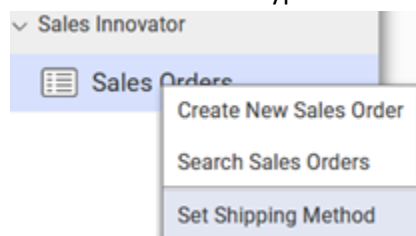
Return the machine name as a new Result item. See *Creating a Method* unit to review how to return a Result item.

Attach this method to a Generic action named Get Machine Name. Make sure the Action Target is Window.



ItemType Action

2. Create an ItemType Action named **Set Shipping Method** that uses the **Server-EditOrders** Method you created in an earlier exercise. Set the Action Target to None. Attach the Action to the Sales Order ItemType.



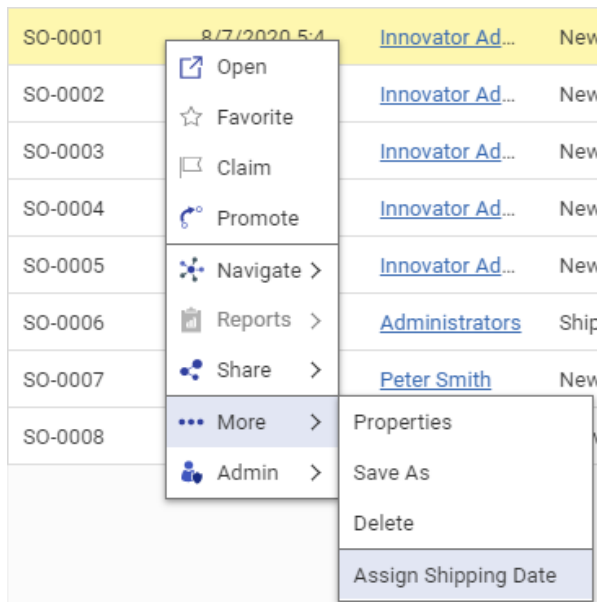
Item Action

3. Create a client Method that will assign today's date to the Shipping Date of an existing selected Sales Order. Assign the Method to an Item Action named **"Assign Shipping Date"**. Attach the newly created action to the Sales Order ItemType.

A date property is always set using a neutral date string and must be formatted as yyyy-mm-dd.
Example: 2021-06-10

Retrieve today's date and time using the Aras __now() function:

```
setProperty("ship_date", "__now()");
```





Unit 6 Exploring the Innovator Class

Overview

In this unit you will learn how use the various methods available in the Innovator Class. These methods allow you to execute other Methods. You will also be able to hand over parameters to called Methods and use Server Methods in AML.

Objectives

- Classifying IOM Innovator method types:
- Creating Generic Methods
- Creating AML Action Methods

Base methods

- **applyAML**
 - Submit AML string to server and return an Item response
- **applyMethod**
 - Execute server Method
- **applySQL**
 - Submit SQL string to server and returns an Item response



Base methods Examples

applyAML

Item applyAML(string AML)

```
Innovator inn = this.getInnovator();
string AML = "<AML><Item type='Customer' action='get' /></AML>";
Item result = inn.applyAML(AML);
return result;
```

VB.NET

```
Dim inn as Innovator = Me.getInnovator
Dim AML as String = _
    "<AML><Item type='Customer' action='get' /></AML>"
Dim result as Item = inn.applyAML(AML)
Return result
```

applyMethod

Item applyMethod(string methodName, string body)

```
Innovator inn = this.getInnovator();
string body = "<itm_type>Customer</itm_type><name>IBM</name>";
Item res = inn.applyMethod("GetItem", body);
return res;
```

VB.NET

```
Dim inn as Innovator = Me.getInnovator
Dim body as String = _
    "<itm_type>Customer</itm_type>" & "<name>IBM</name>"
Dim res as Item = _
    inn.applyMethod("GetItem", body)
Return res
```

applySQL

Item applySQL(string SQL)

```
Innovator inn = this.getInnovator();
Item gry = inn.applySQL("select login_name from
    innovator.[user]");
return gry;
```

VB.NET

```
Dim inn As Innovator = Me.getInnovator()
Dim gry As Item = _
    inn.applySQL("select login_name from innovator.[user]")
Return gry
```

Returns:

```
<Result>
    <Item><login_name>admin</login_name> </Item>
    <Item><login_name>root</login_name></Item>
    <Item><login_name>psmith</login_name></Item>
    <Item><login_name>vadmin</login_name></Item>
</Result>
```

Note

The applySQL method is intended for execution in *server Methods only*. Attempting to execute SQL that modifies data in a client Method (by a user without administrative credentials) will result in an error.

As a security feature Aras Innovator parses SQL upon receipt from client machines. Additional information is available in the *Aras AML Security Settings* documentation.

Item methods

- Retrieve an Item
 - getItemById
 - getItemByKeyedName
 - getItemInDom
- Get a New ID
 - getNewID
- Get next Sequence
 - getNextSequence
- Create new Item
 - newItem
- Create new Error Item
 - newError
- Create new Result Item
 - newResult



Item method Examples

getItemById

Item getItemById(string Name, string id)

```
Innovator inn = this.getInnovator();
string id = "02898329898912891289918299129d";
Item myItem = inn.getItemById("Customer", id);
```

VB.NET

```
Dim inn as Innovator = Me.getInnovator()
Dim id as String = "02898329898912891289918299129d"
Dim myItem as Item = inn.getItemById("Customer", id)
```

getItemByKeyedName

Item getItemByKeyedName(string Name, string keyedName)

```
Innovator inn = this.getInnovator();
Item myItem = inn.getItemByKeyedName("Document",
    "Attached Document");
```

VB.NET

```
Dim inn as Innovator = Me.getInnovator()
Dim myItem as Item = _
    inn.getItemByKeyedName("Document", "Attached Document")
```


getItemInDom**Item getItemInDom(XmlDocument DOM)**

```
Innovator inn = this.getInnovator();
Item myItem = inn.getItemInDom(this.dom);
```

VB.NET

```
Dim inn as Innovator = Me.getInnovator()
Dim myItem as Item = inn.getItemInDom(Me.dom)
```

getNewID**string getNewId()**

```
string id = this.getNewID();
return this.getInnovator().newResult(id);
```

VB.NET

```
Dim id as String = Me.getNewID()
return Me.getInnovator().newResult(id)
```

getNextSequence**string getNextSequence(string sequence_name)**

```
Innovator inn = this.getInnovator();
string nextseq = inn.getNextSequence("Design Request");
```

VB.NET

```
Dim inn as Innovator = Me.getInnovator()
Dim nextseq as String = inn.getNextSequence("Design Request")
```

Utility methods

- `getI18NSessionContext`
- `newXMLDocument`
- `ScalcMD5`
- `getUserID`



Utility Method Examples:

`getI18NSessionContext`

`I18NSessionContext getI18NSessionContext()`

```
Innovator inn = this.getInnovator();
I18NSessionContext i = this.getInnovator().getI18NSessionContext();
return inn.newResult(i.GetLanguageCode());
```

`ScalcMD5`

`static string ScalcMD5(string val)`

```
string md5_string = Innovator.ScalcMD5("mypassword");
```

`newXMLDocument`

`XMLDocument newXMLDocument()`

```
XmlDocument doc = this.newXMLDocument();
```

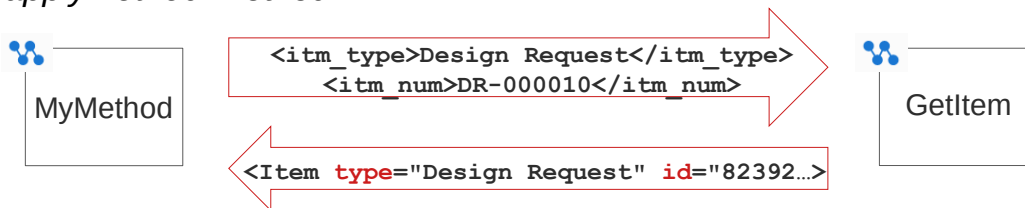
`getUserID()`

`string getUserID()`

```
string myid = this.getInnovator().getUserID();
```

Creating Generic Methods

- Represent a piece of business logic
- Must return an Item, e.g. NewResult or NewError
- Can accept formatted XML as arguments
- Can be called from other Methods using the Innovator Class *applyMethod* method



Creating Generic Methods

Generic Methods are useful for creating reusable modules. They can accept input and return an Item. The `applyMethod` method allows you to call one Method from another and pass parameters as shown above.

The XML argument list is parsed, and the arguments and values become temporary properties of the Method being passed to a called Method. The starting Method becomes the context item in the called Method.

This allows you to access the value of the argument properties in the called Method and use them for processing.

The called Method then returns an Item back to the starting Method for inspection.

Defining Generic Methods

▪ MyMethod (Calling Method)

```
Innovator inn = this.getInnovator();
string body = "<itm_type>Design Request</itm_type>
              <itm_num>DR-000010</itm_num>";
Item res = inn.applyMethod ("GetItem", body);
return res;
```

Define arguments

Call Method

▪ GetItem (Called Method)

```
string item_number = this.getProperty("itm_num");
string item_type = this.getProperty("itm_type");
Item myItem = this.newItem(item_type, "get");
myItem.setProperty("_item_number", item_number);
return myItem.apply();
```

Passed Context Item

Return Item to Calling Method



Defining Generic Methods

Calling Method

The calling method creates a parameter, that contains two arguments named *itm_type* and *itm_num*. The first Method then calls the second Method named *GetItem* using *applyMethod*.

VB.NET

```
Dim inn As Innovator = Me.getInnovator()
Dim body As String = "<itm_type>Design Request</itm_type>" & _
    "<itm_num>DR-000010</itm_num>"
Return inn.applyMethod("GetItem", body)
```

Called Method

The context Item (ME or this) is *MyMethod* (calling Method) in this example. In addition, the called method accesses the two parameters using the *getProperty* method and uses them to return an Item from the server.

VB.NET

```
Dim item_number As String = Me.getProperty("itm_num")
Dim item_type As String = Me.getProperty("itm_type")
Dim myItem As Item = Me.newItem(item_type, "get")
myItem.setProperty("_item_number", item_number)
Return myItem.apply()
```

Defining Generic Methods

▪ TRN_CalculateShipDate – Server Method

```

string sonum = this.getProperty("order");
string days = this.getProperty("days");

//.NET Functions
int daynum = Convert.ToInt16(days);
DateTime date = DateTime.Now.AddDays(daynum);
string strdate = date.ToString("yyyy-MM-dd");

Item itm = this.newItem("Sales Order", "edit");
itm.setAttribute("where", "[sales_order].so_number='" + sonum + "'");
itm.setProperty("ship_date", strdate);
return itm.apply();

```

Retrieve arguments

Return new result



Defining Generic Methods

This example shows how to create a server Method named **TRN_CalculateShipDate** that will return a Sales Order with an assigned ship date that has been calculated by the Method. This method can then be called from a client Method (next page). Note that the Method expects the parameters named "days" and "order".

VB.NET

```

Dim sonum As String = Me.getProperty("order")
Dim days As String = Me.getProperty("days")

'.NET Functions
Dim daynum As Integer = Convert.ToInt16(days)
Dim [date] As DateTime = DateTime.Now.AddDays(daynum)
Dim strdate As String = [date].ToString("yyyy-MM-dd")

Dim itm As Item = Me.newItem("Sales Order", "edit")
itm.setAttribute("where", "[sales_order].so_number='" & sonum & "'")
itm.setProperty("ship_date", strdate)
Return itm.apply()

```

Defining Generic Methods

- Call Server Method from Client

```
var inn = this.getInnovator();  
const body = "<days>10</days><order>SO-0005</order>";  
const result = inn.applyMethod("TRN_CalculateShipDate",  
    body);  
return result;
```



Defining Generic Methods

A client (or server) Method can then be used to call the TRN_CalculateShipDate server Method to access server data.

Passing Arguments to an Action Method

▪ Server Method

Body

```
<Discount>TEST</Discount>
```

```
string disc = this.getProperty("Discount", "0");
```

▪ Client Method

Body

```
<Item><Discount>TEST</Discount></Item>
```

```
var disc = this.getProperty("Discount", "0");
```



Passing Arguments to an Action Method

Generic Methods can also be passed arguments from an Action. **Handing over arguments is not supported for Item Actions.** The Body field of the Action item can be used to supply one or more arguments to a Method.

Server Methods

To pass an argument to a Server Method in an ItemType or Generic Action use an AML property tag to define the argument name. The value of the argument can then be retrieved using the getProperty method.

Client Methods

To pass a value to a Client Method requires that the argument be enclosed in an Item tag. Values can then be retrieved from the "Item" established in the Body.

Using Generic Methods as AML Actions

- Server Method is also available as AML action

```

<AML>
  <Item type="Sales Order"
    action="TRN_CalculateShipDate">
    <order>SO-0005</order>
    <days>10</days>
  </Item>
</AML>

```

Diagram annotations:

- A callout box labeled "Server Method" points to the `action="TRN_CalculateShipDate"` attribute.
- A callout box labeled "Property arguments" points to the `<order>SO-0005</order>` and `<days>10</days>` elements.



Using Generic Methods as AML actions

Remember an AML statement contains an "action" attribute that instructs the server on what operation should take place (add, edit, delete, etc.). There is a collection of built-in actions available. You can also create your own custom action by creating a server Method.

This allows you to easily extend the AML actions available to other developers or users.

To call the `TRN_CalculateShipDate` server method the action attribute is used to supply the server Method name. The property tags become arguments to the server Method and can be obtained by using the `getProperty` method of the `Item` class.

Passing an Attribute From AML

- Additional attributes added to the calling AML expression can be retrieved in the called Method
- AML Alternative:

```
<AML>
<Item type="Sales Order"
      action="TRN_CalculateShipDate"
      order="SO-0005" days="10">
</Item>
</AML>
```

Attribute arguments



Passing an Attribute from AML

You can also pass additional parameters to a custom AML action by adding one or more attributes that then capture the values using the `getAttribute` method Item class method.

To accept the attributes displayed in the AML example above two lines of the server Method would be changed to:

C#:

```
string sonum = this.getAttribute("order");
string days = this.getAttribute("days");
```

VB.NET

```
Dim sonum As String = Me.getAttribute("order")
Dim days As String = Me.getAttribute("days")
```

Summary

In this unit, you learned about the various IOM methods available in the Innovator class.

You should now be able to:

- Identify and Classify IOM Innovator Class methods
- Create Generic Methods
- Create AML Action Methods

Lab Exercise

Goal

Be able to locate and use the basic IOM methods available in the Innovator class.

Scenario

Users would like the ability to set a login preference to display their In Basket at login. This is normally handled by an Aras administrator who edits the Starting Page preference:

Steps:

1. Create a Server Method named **Server_LoginPreference** to do the following:
 - Get the id of the current User with the `getUserID` Innovator class method.
 - Get the current User item with the `getItemById` Innovator class method.
 - Change the action attribute of the current User item to *edit*.
 - Assign the `starting_page` property on the User item to *InBasket Task* (not to *Configurable InBasket*) and apply changes.
 - Set the *World Identity* for the *Execution Allowed To* field of the Method Item (top right)
2. Create a new Generic Action that calls the new Server Method.
3. Run the Action and then log out and in again to see the change.
4. Now make the program more generic by allowing the Method to accept an "option" argument to set the `starting_page` property.
Use the `getProperty` Item class method to get the value of the "option" argument and assign it to the `starting_page` property.

5. Add the option argument as tag to the Body of the Action.

Name	Type	Location
ActionAssignInBasket	Generic	Server

Label
Show InBasket at Login

Method	Target	Can Execute	On Complete
server_LoginPreference	None		

Body
<option>InBasket Task</option>

6. Create an additional Action that clears the login setting by passing an option argument with no value:

Body
<option> </option>

This page intentionally left blank.



Unit 7 Exploring the Item Class

Overview

In this unit you will learn how use the various methods available in the Item Class. These foundation methods allow you to set and get property and attribute values, define relationships, iterate through Item collections and test for various Item conditions.

Objectives

Classify IOM Item method types:

- Base
- Boolean
- Attribute
- Property
- File
- Relationship
- Collection
- Logical
- New
- Error

Base methods

- **apply**
 - Submit AML request to server using Context Item and returns new Item
- **loadAML**
 - Load AML into Item dom property – does not return a new Item
- **clone**
 - Creates a copy of Item (with new id)
 - Can optionally clone relationships



Base method Examples:

apply

Item apply()

Item apply(string actionName)

The apply method can accept a string argument which defines a custom AML action to run on the item in context as it is submitted to the server. You will use this method signature later in the course.

```
Item resultResult = this.apply("MyCustom Action");
```

VB.NET

```
Dim resultResult as Item = Me.apply("MyCustom Action")
```

loadAML

void loadAML(string AML)

The loadAML method sets the AML string to the item DOM but does not submit it to the server. You must use the apply() method to submit the item after the AML is loaded.

```
Item result = this.newItem();
String AML =
    @"<Item type='Customer' action='get'></Item>";
try {result.loadAML(AML);}
catch (System.Xml.XmlException ex) {
    return this;
}
return result.apply();
```

VB.NET

```
Dim result As Item = Me.newItem()
Dim AML As String = "<Item type='Customer' action='get'></Item>"
Try
    result.loadAML(AML)
```

```

Catch ex As System.Xml.XmlException
    Return Me
End Try
Return result.apply()

```

clone

Item clone(bool cloneRelationships)

If cloneRelationships is true the relationships must be fetched in the item before it is copied.

```

Item result = this.newItem("Customer" , "get");
result.setProperty("name", "EDA");
Item customer = result.apply();

Item i_copy = customer.clone(false);
i_copy.setProperty("name", "EDA - Copy");
return i_copy.apply();

```

VB.NET

```

Dim result as Item = Me.newItem("Customer", "get")
result.setProperty("name", "EDA")
Dim customer as Item = result.apply()

Dim i_copy as Item = customer.clone(false)
i_copy.setProperty("name", "EDA - Copy")
Return i_copy.apply()

```

Boolean methods

- isError
- isCollection
- isEmpty
- isLogical
- isNew
- isRoot



Boolean method Examples

```
string sResult = "The following is true about this item: ";
Item i = this.newItem("User", "get");
Item result = i.apply();
if (result.isError()) {sResult += "Error Item ";}
if (result.isCollection()) {sResult += "Item Collection ";}
if (result.isEmpty()){sResult += "Empty Item ";}
if (result.isNew()){sResult += "New Item ";}
if (result.isRoot()) {sResult += "Root Item";}
return this.getInnovator().newResult(sResult);
```

VB.NET

```
Dim sResult as String = "The following is true about an Item: "
Dim i As Item = Me.newItem("User", "get")
Dim result As Item = i.apply()
If result.isError() Then sResult += "Error Item " End If
If result.isCollection() Then sResult += "Item Collection " End If
If result.isEmpty() Then sResult += "Empty Item "End If
If result.isNew() Then sResult += "New Item " End If
If result.isRoot() Then sResult += "Root Item" End If
Return Me.getInnovator().newResult(sResult)
```


Attribute methods

- getAttribute
- setAttribute
- getID
- setID
- setNewID
- getType
- setType
- getAction
- setAction
- removeAttribute



Attribute method Example

```
Item result = this.newItem();
result.setType("Customer");
result.setAction("get");
result.setAttribute("select", "name");
result.setProperty("name", "EDA");
Item resultDB = result.apply();
string id = resultDB.getID();
string type = resultDB.getType();
return this.getInnovator().newResult("ID: " + id + " - " + "Type: " +
type);
```

VB.NET

```
Dim result as Item = Me.newItem()
result.setType("Customer")
result.setAction("get")
result.setAttribute("select", "name")
result.setProperty("name", "EDA")
Dim resultDB as Item = result.apply()
Dim id as String = resultDB.getID()
Dim type as String = resultDB.getType()
Return _
Me.getInnovator().newResult("ID: " + id + " - " + "Type: " + type)
```

Property methods

- getProperty
- setProperty
- removeProperty
- getPropertyAttribute
- setPropertyAttribute
- removePropertyAttribute
- getPropertyItem
- setPropertyItem
- createPropertyItem
- setPropertyCondition
- getPropertyCondition
- fetchDefaultPropertyValues



Property methods

The getProperty method offers two argument signatures:

```
getProperty(property_name);
getProperty(property_name, default_value);
```

The second argument signature allows you to specify a default value to be returned from the method if the property is not defined in the Item.

Examples:

```
string s = this.getProperty("details", "");
Dim s as String = Me.getProperty("details", "")
```

The value of s would be equal to "" if the details property was not provided in the current Item. Using the default value makes it easier for testing purposes in a condition statement.

```
if (s == "") {property not defined logic}
If (s="") property not defined logic End If
```

Property method Examples

setProperty

```
Item result = this.newItem("Design Request", "get");
//Set Cost Value to 50
result.setProperty("_cost", "50");
```

setPropertyAttribute/setPropertyCondition

```
//Add condition greater than (in this example > 50)
result.setPropertyCondition("_cost", "gt");

//Set a property value to NULL
result.setPropertyAttribute("locked_by_id", "is_null", "1");

return result.apply();
```

getPropertyAttribute

```
string state =
    result.getPropertyAttribute("current_state", "keyed_name");
```

getPropertyItem

```
Item customer = result.getPropertyItem("_customer");
```

getProperty

```
string id = customer.getProperty("id", "");

return this.getInnovator().newResult("ID: " + id);
```

fetchDefaultPropertyValues

Sets the default values of all properties on the item and returns the updated item. To overwrite current property values set the first argument to true.

```
return result.fetchDefaultPropertyValues(true);
```

VB.NET

```
Dim result as Item = Me.newItem("Design Request", "get")
result.setProperty("_cost", "50")
result.setPropertyAttribute("_cost", "condition", "gt")
result.setPropertyAttribute("locked_by_id", "is_null", "1")

Return result.apply();

Dim state as String = _
    result.getPropertyAttribute("current_state", "keyed_name")

Dim customer as Item = result.getPropertyItem("_customer")
Dim id as String = customer.getProperty("id", "")

Return Me.getInnovator().newResult("ID: " + id)
```

Property methods can be also used in combination with Extended Properties and extended Classification. Detailed information therefore is available in the Aras *Extended Classification* documentation.

File methods

- `setFileProperty`
- `fetchFileProperty`



File methods

`setFileProperty (property, filepath)`

`fetchFileProperty (property, targetpath, mode)`

File methods are used to programmatically upload and download files to and from an Aras vault.

The type of Method used (client versus server) determines how the file operations behave. The *setFileProperty* method is used to assign a file to a property of type Item (ItemType File) which causes an upload to occur. The *fetchFileProperty* method is used to obtain a file reference from a property and retrieve a file to a user's local client.

Using Client Methods (JavaScript)

Uploading

To allow a user to upload a selected file to the vault requires that the user select the file under program control. The returned file Object is then used by the IOM to copy the file to the vault associated with the current user. The following example is used as an Item Action to upload a selected file associated with a CAD item to the vault. The *native_file* property is defined as an Item property (ItemType File).

```
const thisItem = this;

const vlt = aras.vault;
vlt.selectFile().then(
  function (fileObject)
  {
    thisItem.setAction('edit');
    thisItem.setProperty("description", "File Updated");
    thisItem.setFileProperty("native_file", fileObject);
    thisItem.apply();
  }
);
```

This program uses the JavaScript *then* prototype to make a call back to a function once the file has been selected by the user using a function from the `aras.vault` object.

Downloading

The following example downloads a file from a selected file property.

```
var result = this.fetchFileProperty("native_file", "", 0);
```

For client downloads, only the first parameter is used. The internet browser will prompt the user for the location to copy the retrieved file.

Using Server Methods

Server methods can also upload and download files from an Aras vault to a location on the Aras server. Note that directory locations specified refer to the Aras server, not the user's client.

Uploading

This example creates a new CAD item and associates a file on the server to the `native_file` property:

```
Item d = this.newItem("CAD", "add");
d.setProperty("item_number", "DFG 1000");
d.setFileProperty("native_file", @"C:\DFG1000.dwg");
return d.apply();
```

Downloading

This example retrieves a file associated with the `native_file` property of a CAD item and copies the file to a server directory named "Temp".

```
Item return = this.fetchFileProperty("native_file", @"C:\Temp\",
FetchFileMode.Normal);
```

FetchFileMode.Normal copies file to client, *Mode = FetchFileMode.Dry* just returns an item with the *checkedout_path* property set which contains the name of the file if it was streamed. You can then use this path name to determine if the file already exists before using *FetchFileMode.Normal* to stream the file.

Relationship methods

- addRelationship
- createRelationship
- removeRelationship
- createRelatedItem
- setRelatedItem
- getRelationships
- getRelatedItem
- getRelatedItemID
- getParentItem
- fetchRelationships



Relationship methods

Relationship methods allow you define a Relationship as part of a query and also return Relationships or related items.

Remember, that these methods are generating AML that will be submitted to the server for processing.

Note

The “fetch” methods in the IOM are designed to access the database directly rather than construct an AML request in memory prior to processing. The fetchRelationships method will access the database directly to assign the Relationships to the current context Item. The next page demonstrates this method.

Using fetchRelationships

- Retrieves current relationships from server

```
01 Item itm = this.newItem("Design Request", "get");
02 itm.setProperty("_item_number", "DR-000010");
03 Item result = itm.apply();

//Go to server to fetch relationships
04 result.fetchRelationships("Design Request Document");
05 return result;
```



Using fetchRelationships

In this example the “fetch” has been done after the apply – fetch retrieves from data from the server while the other Relationships methods work with the current DOM. In the initial apply no relationship information will be included in the returned AML for the Design Request Item.

```
<Result>
  <Item type="Design Request" ... >
    ...
  </Item>
</Result>
```

Once the fetchRelationships method is invoked, the Design Request Document Relationships will then include the relationship information.

```
<Result>
  <Item type="Design Request" ... >
    ...
    <Relationships>
      <Item type=" Design Request Document" ... />
    </Relationships>
  </Item>
</Result>
```

VB.NET

```
Dim itm as Item = Me.newItem("Design Request", "get")
itm.setProperty("_item_number", "DR-000010")
Dim result as Item = itm.apply()
'Go to server to fetch relationships
result.fetchRelationships("Design Request Document")
Return result
```

Note

The *fetchRelationships* method changes the current item (and automatically does an apply to the server). In the example above the variable result is changed by the *fetchRelationships* method. This is not true for the *apply* method which requires that you return a result to inspect changes.

Retrieving Items using Relationships

```

01 Item itm = this.newItem("Design Request", "get");
02 Item relationship =
03     this.newItem("Design Request Document", "get");
04 Item relitm = this.newItem("Document", "get");
05 relitm.setProperty("item_number", "SPEC-00001");
06 relationship.setRelatedItem(relitm);
07 itm.addRelationship(relationship);
08 return itm.apply();

```



Retrieving Items using Relationships

The method above will generate the AML shown below. It is important to understand how the IOM constructs AML requests that need to use Relationships:

```

<Item isNew="1" isTemp="1" type="Design Request" action="get">
  <Relationships>
    <Item isNew="1" isTemp="1" type="Design Request Document"
action="get">
      <related_id>
        <Item isNew="1" isTemp="1" type="Document" action="get">
          <item_number>SPEC-00001</item_number>
        </Item>
      </related_id>
    </Item>
  </Relationships>
</Item>

```

VB.NET

```

Dim result as Item = Me.newItem("Design Request", "get")
Dim relationship as Item = Me.newItem("Design Request Document", "get")
Dim relresult as Item = Me.newItem("Document", "get")
relresult.setProperty("item_number", " SPEC-00001")
relationship.setRelatedItem(relresult)
result.addRelationship(relationship)
Return result.apply()

```


Adding New Relationships

```

01 Item itm = this.newItem("Design Request", "edit");
02 itm.setAttribute("where",
03     "design_request._item_number='DR-000020'");
04 Item relationship =
05 itm.createRelationship("Design Request Document", "add");
06 Item relitm = this.newItem("Document", "get");
07 relitm.setProperty("item_number", "SPEC-00001");
08 relationship.setRelatedItem(relitm);
09 return itm.apply();

```



Adding New Relationships

This example demonstrates how to add a new Relationship to an Item.

In this example, a new Document is related to the Design Request using the Design Request Document relationship. The IOM createRelationship method is used in this example rather than addRelationship shown on the previous page. This saves a step, since it will create the appropriate AML item tags as well as connect the relationship to the source item (result). The addRelationship method requires that you define the relationship item first.

Can you write down what the generated AML would look like that is applied to the server?

VB.NET

```

Dim result As Item = Me.newItem("Design Request", "edit")
result.setAttribute("where", _
    "design_request._item_number='DR-000020'")
Dim relationship As Item = _
    Me.newItem("Design Request Document", "add")
Dim relresult As Item = Me.newItem("Document", "get")
relresult.setProperty("item_number", "SPEC-00001")
relationship.setRelatedItem(relresult)
result.addRelationship(relationship)
Return result.apply()

```

Retrieving Related Items

```

01 Item itm = this.newItem("Design Request", "get");
02 itm.setProperty("_item_number", "DR-000010");
03 Item result = itm.apply();
04
05 result.fetchRelationships("Design Request Document");
06 Item relationships =
07     result.getRelationships("Design Request Document");
08 int count = relationships.getItemCount();
09 for (int i=0; i<count; i++) {
10     Item doc_relationship = relationships.getItemByIndex(i);
11     Item doc_itm = doc_relationship.getRelatedItem();
12 }

```



Retrieving Related Items

This example demonstrates how to retrieve related items from a source Item.

In the example, after the “fetchRelationships” has been applied to the Design Request item the getRelationships method will return the Relationships on the Design Request (Design Request Document).

A for loop is then used to cycle through each Relationship and the getRelatedItem method is used to get the current Item and assign it to rel_result.

Further processing on each Item could then take place as required.

The next page describes how to use Collection methods to process a list of Items.

VB.NET

```

Dim inn as Innovator = Me.getInnovator()
Dim itm as Item = Me.newItem("Design Request", "get")
itm.setProperty("_item_number", "DR-000010")
Dim result as Item = itm.apply()

result.fetchRelationships("Design Request Document");
Dim relationships As Item = _
    result.getRelationships("Design Request Document")
Dim count As Integer = relationships.getItemCount()
Dim i As Integer
For i = 0 To count - 1
    Dim doc_relationship As Item = relationships.getItemByIndex(i)
    Dim doc_result As Item = doc_relationship.getRelatedItem()
Next i

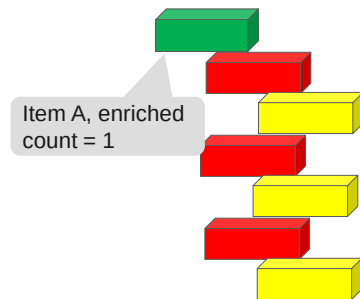
```

fetchRelationships vs. getRelationships

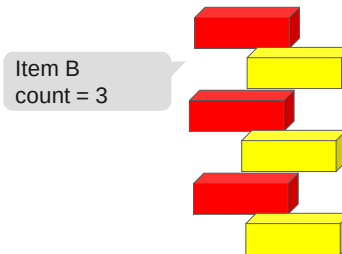
Starting Point:



fetchRelationships



getRelationships



fetchRelationships vs. getRelationships

When comparing fetchRelationships to getRelationships we should consider some facts about fetchRelationships:

- fetchRelationships retrieves data from the database without the need of an additional apply().
- fetchRelationships enriches the calling item by the relationships found for the indicated Relationship ItemType.
- Root Item will be still the one on which we had run fetchRelationships, i.e. it still is a single item with count = 1.

The method getRelationships bases on the item as it is after fetchRelationships. For the result of getRelationships applies:

- getRelationships does not modify the calling item.
- Instead use the return value and store it in a new item.
- This new item is a collection item consisting of the retrieved relationships without the parent item.
- The count of the collection item is the count of relationships, so you can use it for working through the relationships.

Overall: Simply use first fetchRelationships, then getRelationships on the same item, and store the result in an item with which you do the rest.

Collection methods

- getItemCount
- getItemByIndex
- getItemsByXPath
- appendItem
- removeItem



Collection methods Example:

Collection methods allow you to process a collection of Items. This example builds a comma delimited list of Users:

```
Innovator inn = this.getInnovator();
Item itm = this.newItem("User", "get");
Item results = itm.apply();

int count = results.getItemCount();
string ids_str = "";

for (int i=0; i<count; i++) {
    Item result_idx = results.getItemByIndex(i);
    string id = result_idx.getID();
    ids_str += id + ",";
}
return inn.newResult(ids_str);
```

VB.NET

```
Dim inn as Innovator = Me.getInnovator()
Dim itm as Item = Me.newItem("User", "get")
Dim results as Item = itm.apply()

Dim count as Integer = results.getItemCount()
Dim ids_str as String = ""
Dim i as Integer
For i = 0 to count-1
    Dim result_idx as Item = results.getItemByIndex(i)
    Dim id as String = result_idx.getID()
    ids_str += id + ","
Next i

Return inn.newResult(ids_str)
```

Using XPath

XPath can be used to search for values in the DOM and return specific data. In this example, a list of Design Requests is requested from the server. XPath is then used to locate a specific Item and assign it to a variable

```
Item qry = this.newItem("Design Request", "get");
Item result = qry.apply();
Item result_find =
result.getItemsByXPath("//Item[_item_number='DR-000010']");
return this.getInnovator().newResult(result_find.getID());
```

VB.NET

```
Dim qry as Item = Me.newItem("Design Request", "get")
Dim result as Item = qry.apply()
Dim result_find =
result.getItemsByXPath("//Item[_item_number='DR-000010']")
Return Me.getInnovator().newResult(result_find.getID())
```

Append/Remove Item

Append Item can be used to build a collection in a Method. Remove Item will remove an Item from the collection.

```
Item itm1 = this.newItem("Customer", "get");
itm1.setProperty("name", "IBM");
Item itm2 = this.newItem("Customer", "get");
itm2.setProperty("name", "GHC");
Item result1 = itm1.apply();
Item result2 = itm2.apply();
result1.appendItem(result2); //Appends Item to collection
result1.removeItem(result2); //Removes the Item from the collection
return result1;
```

VB.NET

```
Dim itm1 as Item = Me.newItem("Customer", "get")
itm1.setProperty("name", "IBM")
Dim itm2 as Item = Me.newItem("Customer", "get")
itm2.setProperty("name", "GHC")
Dim result1 as Item = itm1.apply()
Dim result2 as Item = itm2.apply()
result1.appendItem(result2) 'Appends Item to collection
result1.removeItem(result2) 'Removes the Item from the collection
Return result1
```

Logical methods

- newOR
- newAND
- newNOT
- removeLogical
- getLogicalChildren



Logical methods

Logical methods allow you to create Boolean queries using AND, OR or NOT. The appropriate AML tags are generated based on how the methods are used.

In this example, a query is built to get all Design Requests that have a status of “New” AND patent required = true – OR – have a status of “In Review” and a patent required of false.

```
Item qry = this.newItem("Design Request", "get");

Item logicResult = qry.newOR();

Item logicAND1 = logicResult.newAND();
logicAND1.setProperty("state", "New");
logicAND1.setProperty("_patent_required", "1");

Item logicAND2 = logicResult.newAND();
logicAND2.setProperty("state", "In Review");
logicAND2.setProperty("_patent_required", "0");

Item retResult = qry.apply();
```

VB.NET

```
Dim qry as Item = Me.newItem("Design Request", "get")

Dim logicResult as Item = qry.newOR()
Dim logicAND1 as Item = logicResult.newAND()
logicAND1.setProperty("state", "New")
logicAND1.setProperty("_patent_required", "1")

Dim logicAND2 as Item = logicResult.newAND()
logicAND2.setProperty("state", "In Review")
logicAND2.setProperty("_patent_required", "0")

Dim retResult as Item = qry.apply()
```

Which creates the following AML request:

```
<Item type="Design Request" action="get">
<or>
  <and>
    <state>New</state>
    <_patent_required >1</_patent_required>
  </and>
  <and>
    <state>In Review</state>
    <_patent_required >0</_patent_required>
  </and>
</or>
</Item>
```

To modify the request above you can access the logical children of the Design Request query and remove the “and” logic:

```
Item lchildren = qry.getLogicalChildren();

for( int i = 0; i < lchildren.getItemCount(); i++ )
{
  Item lchild = lchildren.getItemByIndex(i);
  qry.removeLogical(lchild);
}
```

VB.NET

```
Dim lchildren As Item = qry.getLogicalChildren()

For i As Integer = 0 To lchildren.getItemCount() - 1
  Dim lchild As Item = lchildren.getItemByIndex(i)
  qry.removeLogical(lchild)
Next i
```

New methods

- newItem
- newXMLDocument



New methods

The newItem method is one of the most common Item methods used as you have seen in previous examples. You can create an “empty” Item (no parameters), an Item of a specified type with no action assigned (one parameter) or an Item of a specific type with the action attribute set.

```
Item result = this.newItem("Design Request", "delete");
```

VB.NET

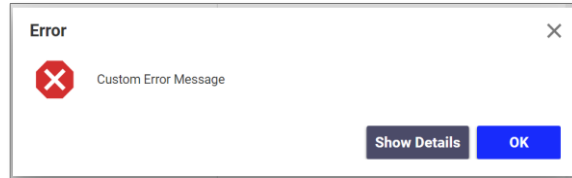
```
Dim result as Item = Me.newItem("Design Request", "delete")
```

The newXMLDocument method can be used to create a new DOM which can then be parsed using standard XML .NET methods.

```
XmlDocument doc = this.newXMLDocument();
XmlElement root = doc.CreateElement("AML");
XmlElement item = doc.CreateElement("Item");
item.SetAttribute("type", "Design Request");
item.SetAttribute("action", "get");
doc.AppendChild(root);
root.AppendChild(item);
Item result = this.newItem();
result.loadAML(doc.OuterXml);
return result.apply();
```


Error methods

- `getErrorString`
- `setErrorString`
- `getErrorCode`
- `setErrorCode`
- `getErrorDetail`
- `setErrorDetail`
- `getErrorSource`
- `setErrorSource`



Error methods

A series of methods are available to configure a message. You must first define a new Error object to set the appropriate attributes:

```
Innovator inn = this.getInnovator();
Item err = inn.newError("");

err.setErrorCode("-1");
err.setErrorDetail("My custom message");
err.setErrorSource("My program");
err.setErrorString(" Message");
return err;
```

VB.NET

```
Dim inn as Innovator = Me.getInnovator()
Dim err as Item = inn.newError("")
err.setErrorCode("-1")
err.setErrorDetail("My custom message")
err.setErrorSource("My program")
err.setErrorString("Message")
Return err
```

Extended methods

- Claiming
 - lockItem
 - unlockItem
 - fetchLockStatus
 - getLockStatus
- Lifecycle
 - promote
- Result Item
 - getResult
- Workflow
 - instantiateWorkflow
- Email
 - email
- Item to String
 - ToString



Extended methods

These methods are described as “extended” as they do not fit in any other “category” described previously.

Claiming

To change the claim on an item you can use the lockItem and unlockItem methods which apply or remove a claim to the item in the database. To obtain the status of a claim in the database use the fetchLockStatus method passing the id of the item to be checked. Use the getLockStatus method to check the status of a claim on an item that has already been retrieved.

Lifecycle

To promote an item to the next lifecycle state, use the promote method which changes the current state of the item in the database.

Result Item

To obtain the results from a Result item (which has been returned from the Innovator newResult method) use the getResult method.

Workflow Example

The example below shows how a workflow process can be created for a controlled item using the instantiateWorkflow method. Here it is assumed that the Method is being invoked from the controlled Item.

The last line uses the action startWorkflow to start the workflow process, not the deprecated method.

```
Item result = this.newItem("Workflow Map", "get");
result.setAttribute("select", "id");
result.setProperty("name", "Design Request");
result = result.apply();
string wf_id = result.getID();
Item wf_process = this.instantiateWorkflow(wf_id);
return wf_process.apply("startWorkflow");
```

VB.NET

```

Dim result as Item = Me.newItem("Workflow Map", "get")
result.setAttribute("select", "id")
result.setProperty("name", "Design Request")
result = result.apply()
Dim wf_id as String = result.getID()
Dim wf_process as Item = Me.instantiateWorkflow(wf_id)
Return wf_process.apply("startWorkflow")

```

Note

Once a workflow has been instantiated, it must be started using the action *startWorkflow*. To invoke an action, use the Item *apply* method. You can also cancel a running workflow using the *cancelWorkflow* or close a workflow using the *closeWorkflow* action.

E-mail Messages

To send an e-mail message, you must first create the message and ensure that the From User and To User identities have been specified and that all users involved have valid e-mail addresses in their User Profiles.

```

Innovator inn = this.getInnovator();
Item email_msg = this.newItem("Email Message", "get");
email_msg.setProperty("name", "Service Reminder");
email_msg = email_msg.apply();
//Get admin Identity
Item iden = this.newItem("Identity", "get");
iden.setProperty("name", "Innovator Admin");
iden = iden.apply();
try {
    bool result = this.email(email_msg, iden);
}
catch (Exception e) {
    return inn.newError(e.Message);
}
return this;

```

VB.NET

```

Dim inn as Innovator = Me.getInnovator()
Dim email_msg as Item = Me.newItem("Email Message", "get")
email_msg.setProperty("name", "Service Reminder")
email_msg = email_msg.apply()
'Get admin Identity
Dim iden as Item = Me.newItem("Identity", "get")
iden.setProperty("name", "Innovator Admin")
iden = iden.apply()
Try
    Dim result as Boolean = Me.email(email_msg, iden)
Catch (Exception e)
    return inn.newError(e.Message)
End Try
return Me

```

Note

The email message has access only to the properties of the calling item. In the example above, this (or Me) is the current item reference. If the email method is used in a workflow activity, then the controlling item should be used to invoke the email method.

Summary

In this unit you learned about the IOM Item methods available to create, edit and remove Items from the database.

You should now be able to:

- Identify and ify IOM Item methods

Lab Exercise

Goal

Be able to use IOM Item methods to create and manipulate Items in a solution.

Scenario

In this exercise, you will use Item methods to create a Part cost calculator that can be called from an existing Sales Order to show the total of all Part line items on an order.

Create Server Calculate Method

Create a Server Method that will total up the cost of all of the Part line items in a Sales Order. The Method will need to establish a line item cost first (quantity * part unit cost) and then sum all of the line items for a total cost. Attach the Method to an Item Action to allow a user to see the result in a target Window.

The steps for this Method follow the same pattern discussed in the "Retrieving Related Items" example discussed in this unit. The value of quantity is stored in a property on the relationship item (between the Sales Order and the related Part item).

1. Create a Server Method named **Server-CalculateCost**. This server Method will be used as an Item Action so what does the context Item (this or Me) represent? _____
2. Retrieve an Innovator instance using the `getInnovator` IOM method and store it in a variable named `inn`. You will use this instance later in the program to return a new Result item.
3. Declare the following variables that will be used to calculate cost and set them to 0:

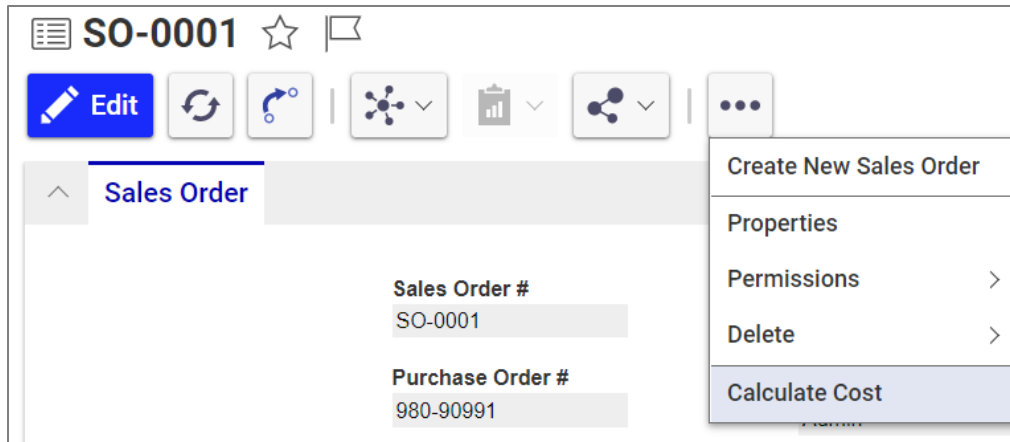
Data Type C#	Data Type VB	Variable Name
decimal	Decimal	total_cost
decimal	Decimal	unit_cost
decimal	Decimal	line_cost
int	Integer	quantity
int	Integer	count

4. Use the `fetchRelationships` IOM method to retrieve the "Sales Order Part" relationships from the current Sales Order item (`this`).
5. Now use the `getRelationships` IOM method to retrieve the Relationship collection from the current item (`this`) and store this information in an Item variable named `relationships`.
6. Get the count of `relationships` using the `getItemCount` method and store it in `count`.
7. Create a for-loop to iterate through each item in `relationships`. (See the example shown in Retrieving Related Items in this unit if you need help with the syntax).
8. Within the for-loop, access and store the currently indexed `relationships` item in an Item variable named `partRel` using the `getItemByIndex` method.
9. Get the value of the `quantity` property from the current `partRel` relationship item using `getProperty`, convert the returned string to an integer and store in `quantity`:
`quantity= Convert.ToInt16(partRel.getProperty("quantity", "0"));`
10. Access the related Item from the current `partRel` Relationship using the `getRelatedItem` method and store it in an item variable named `part`.
11. Get the value of the `cost` property from the current related `part` item using `getProperty`, convert it to a decimal variable and store in `unit_cost`:
`unit_cost = Convert.ToDecimal(part.getProperty("cost", "0"))`

12. Multiply the `quantity * unit_cost` and store in `line_cost`.
13. Add the `line_cost` to `total_cost`.
14. When the for-loop completes, convert `total_cost` to a string and return it using the Innovator `newResult` method:
`inn.newResult(total_cost.ToString())`

Create Action

15. Create a new Item Action named **Calculate Cost** that calls the **Server-CalculateCost** Method and Targets a Window. Attach the Action to the Sales Order ItemType.





Unit 8 Creating Server Event Methods

Overview

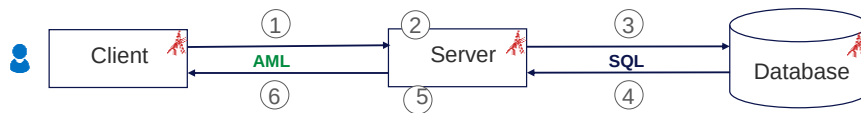
In this unit, you learn how to build and assign server methods to customize and extend the standard behavior of the system. This includes Item validation, setting Item property values as well as workflow dynamic routing and assignment.

Objectives

- Reviewing Server Events
- Developing ItemType Validation Routines
- Developing Workflow Validation Routines
- Creating Dynamic Workflow Routes
- Creating Workflow Dynamic Assignments

Reviewing Server Events

- Allow custom logic in a Server Method to be applied:
 - Before standard logic (OnBefore***** - cmp.②)
 - After standard logic (OnAfter***** cmp.⑤)
 - Instead of standard logic (On*****)



Reviewing Server Events

Server Events enable you to assign a server Method that prevents, extends, or replaces the standard behavior in an Aras Innovator solution.

"OnBefore" events are typically used for validation to prevent invalid data from being saved to the database. They have access to AML Request.

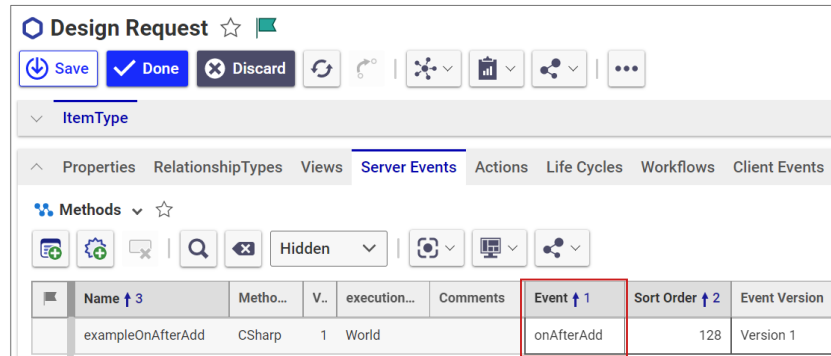
"OnAfter" events can be used to extend the standard behavior and perform additional operations once the AML response has been sent back to the client. They have access to AML Response.

"On" events are used to completely replace the standard behavior and require that you create an AML response Item that is acceptable to the client. On event Methods typically take the most work to employ in a solution. Note that when you assign a Method to an On event that behavior is disabled by Aras Innovator and you are responsible for replacing the task behavior. They must create and return a valid AML Response.

ItemType Server Events

- Assigned to an ItemType
- Allow prevention, validation and replacement of Item events including:

- Add
- Update
- Delete
- Get
- Promote
- Copy
- Lock
- Unlock
- Version
- Method



ItemType Server Events

The following events have been defined to prevent, extend, or override standard Item behaviors:

ItemType Events

Before	After	Instead Of
OnBeforeAdd	OnAfterAdd	OnAdd
OnBeforeUpdate	OnAfterUpdate	OnUpdate
OnBeforeDelete	OnAfterDelete	OnDelete
OnBeforeGet	OnAfterGet	OnGet
OnBeforePromote	OnAfterPromote	OnPromote
OnBeforeCopy	OnAfterCopy	
OnBeforeLock	OnAfterLock	
OnBeforeUnlock	OnAfterUnlock	
OnBeforeVersion	OnAfterVersion	
OnBeforeMethod	OnAfterMethod	

The GetKeyedName Event is also available and is invoked when an item is added or updated to programmatically assign the keyed_name property (override the Keyed Name Order).

OnBeforeMethod runs before a Server Action Method, OnAfterMethod after the Action Method respectively.

Configuring the ItemType Server Event

ItemType Server events contain two options when a Method is attached to the ItemType.

Sort Order

Sort Order is used to rank the execution order of a Method when two (or more) Methods are attached to the same Server Event. Method execution occurs based on a ranking from low to high. For example, if two Methods are associated with the OnAfterUpdate Method and one Method has a Sort Order of 10 and the second Method has a Sort Order of 20 the Method with the Sort Order of 10 will execute first.

Sort Order is useful for modularizing Server Methods. You may want to run the same Method on many different ItemTypes but apply special logic to only a few specific ItemTypes. Rather than creating custom Methods for each type you can "layer" Method execution.

Event Version

An additional setting is available to override standard event behavior for several specific uses cases. The default setting of Event 1 indicates that the Server Methods will execute using the standard behavior defined for all previous releases of Aras Innovator.

Setting the Event Version to Event 2 changes the following specific behaviors:

1. The OnAfterAdd, OnAfterUpdate and OnAfterVersion event behavior is changed when an AML Request is made to modify or add a group of Items. The default event behavior (Event 1) executes the server event Method on each individual Item that was modified or added. If the Event 2 option is selected, then the event Method is executed once for the collection. The context item (this or ME) contains the collection of items that can be processed more efficiently in a single Method.
2. When a "source" Item is cloned or versioned, the existing relationships attached to the source item may also be cloned or versioned. The OnBeforeCopy and OnAfterCopy events will be triggered on the Relationship ItemType if a Method is attached, and the Event 2 option is selected. This behavior does not exist for the Event 1 option.

Notes

The OnBeforePromote and OnAfterPromote events can be used to define a standard promote operation for an item regardless of the lifecycle transition selected by the user. Both events are triggered BEFORE the standard lifecycle pre/post events are invoked. This allows you to create a baseline rule for all promotions and to add additional behavior for specific transitions on the lifecycle map.

Both promote events only pass a simple context item to the Method in the following form:

```
<Item type="[name]" idlist="[GUID]" />
```

Refer to the *Aras Programmer's Guide* for a full description of each server event.

Lifecycle Transition Events

- Assigned to Lifecycle Map

The screenshot shows a web interface for configuring lifecycle transitions. At the top, there are two tabs: 'State' and 'Transition', with 'Transition' being the active tab. Below the tabs, there is a 'Role' section with a dropdown menu currently showing 'Aras PLM'. To the right of the role is a checkbox labeled 'Get Comment'. Below the role section is a 'Server Methods' section. It contains two rows: 'Pre' and 'Post'. The 'Pre' row has a text input field containing 'checks before release' and a copy icon. The 'Post' row has an empty text input field and a copy icon. To the right of the 'Server Methods' section is a link labeled 'Configure E-Mail'. The Aras logo is visible in the bottom left corner of the interface.

Lifecycle Transitions

A Server Method can be used before and after a lifecycle state is changed to prevent a promotion (Pre) or extend the behaviors (Post) of a completed promotion.

Workflow Events

- Workflow Activity:
 - OnActivate
 - OnAssign
 - OnRefuse
 - OnDelegate
 - OnVote
 - OnRemind
 - OnDue
 - OnEscalate
 - OnClose
- Workflow Path
 - Pre Method
 - Post Method



Workflow Events

Workflow Events are enabled for each **Workflow Activity as well as each Path**.

Workflow Events are useful for performing Item validation before an Activity can proceed in a workflow. They are also used to **perform Workflow Dynamic Routing** (decided which path to take at runtime) and **Workflow Dynamic Assignments** (creating Activity assignments at runtime).

Developing ItemType Validation

Design Request

OnBeforeAdd
OnBeforeUpdate



Innovator Server

Developing ItemType Validation

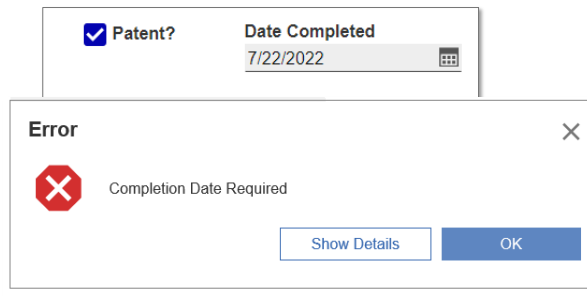
A common use of server Methods is in Item validation to prevent invalid data from being inserted into the database based on business rules.

You can then review the data being sent to the server and prevent acceptance by returning a new Error Item.

Defining Server Validation Method

```
Innovator inn = this.getInnovator();
string p = this.getProperty("_patent_required", "0");
string c = this.getProperty("_completion_date", "");
if (p=="1" && c=="") {
    return inn.newError("Completion Date Required");
}
return this;
```

Note Default Parameters



Defining Server Validation Method

In this example, the `_patent_required` and `_completion_date` properties are assigned to variables from the context of the current Design Request Item and then tested using a condition statement.

This Method can then be attached to an `onBeforeAdd` or `onBeforeUpdate` method to prevent the data from being added or saved to the database.

A new Error Item is returned if the condition is true, and a dialog is displayed to the user.

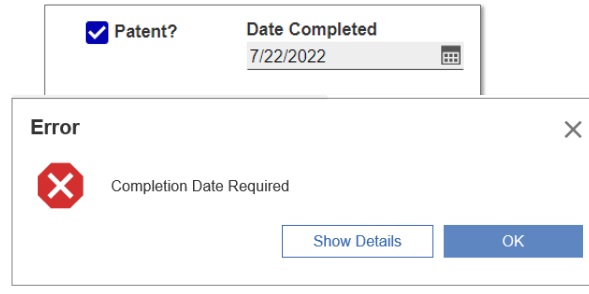
VB.NET

```
Dim inn As Innovator = Me.getInnovator()
Dim p As String = Me.getProperty("_patent_required", "0")
Dim c As String = Me.getProperty("_completion_date", "")
If p = "1" AndAlso c = "" Then
    Return inn.newError("Completion Date Required")
End If
Return Me
```

Defining Lifecycle Validation Method

```
Innovator inn = this.getInnovator();
string p = this.getProperty("_patent_required", "0");
string c = this.getProperty("_completion_date", "");
if (p=="1" && c=="") {
    return inn.newError("Completion Date Required");
}
return this;
```

Note Default Parameters



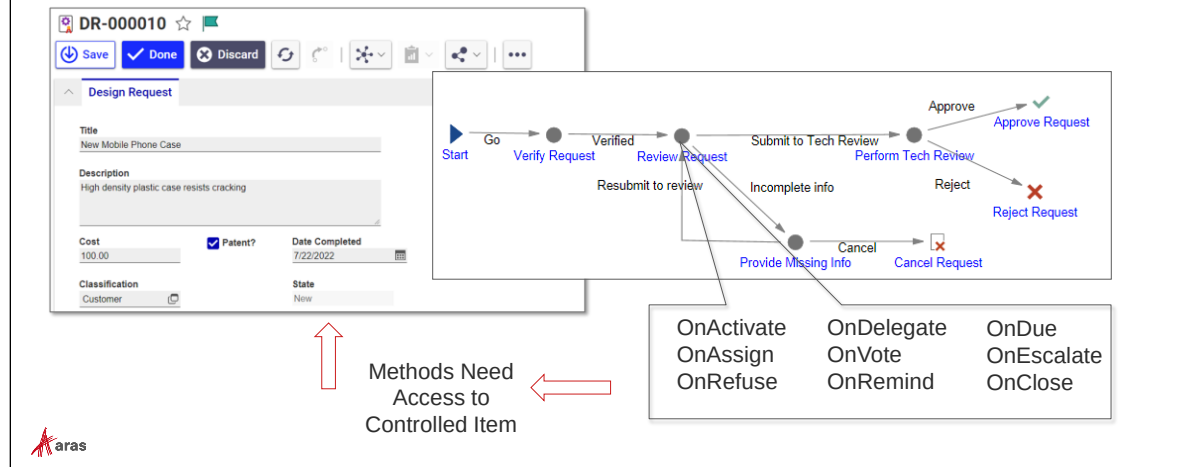
Defining Lifecycle Validation Method

In this example, the same Method as in the previous example is used.

It can be attached to the Pre event of a lifecycle transition, too, to prevent a user from promoting an item to the next state.

Developing Workflow Validation

- Context Item for Workflow events is current Workflow Activity



Developing Workflow Validation

Testing for Item conditions in a Workflow also may require validation.

If you attach a Method to an Activity event, the context Item is the actual Activity Item, which is typically not what you are attempting to validate. Typically we want to validate properties of the item to which the Workflow Process is related. Remember: This item is called the Controlled Item.

Defining Workflow Validation Method

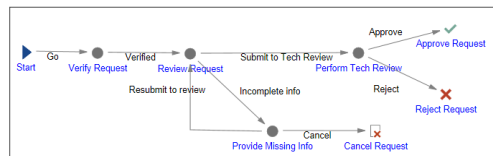
```

Innovator inn = this.getInnovator();
Item controlledItem = this.apply("Get Controlled Item");
string subj = controlledItem.getProperty("_title", "");
if (subj=="") {
    return inn.newError("Subject required");
}
return this;

```

Call custom action on this Item

Workflow Process



Controlled Item

Defining Workflow Validation Method

Accessing the controlled Item from a Workflow Activity requires an additional step.

An AML Action Method named **Get Controlled Item** is available on the Aras.com Community Projects site that retrieves the controlled Item from a w Activity@

You can use the Item apply method to run a custom action. The apply method will pass the context of the current activity to the Get Controlled Item Method.

Once you have a handle to the controlled Item you can then proceed to validate any properties on that Item.

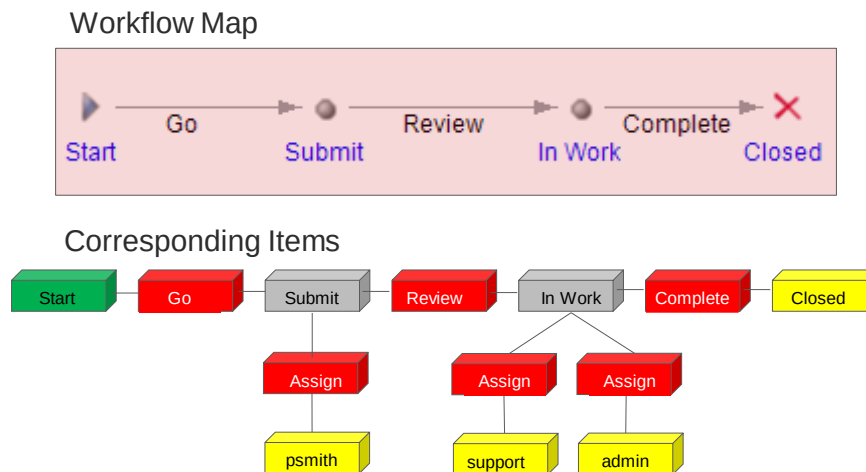
VB.NET

```

Dim inn as Innovator = Me.getInnovator()
Dim controlledItem as Item = _
    Me.apply("Get Controlled Item")
Dim subj as String = controlledItem.getProperty("_title", "")
If subj=""
    return inn.newError("Subject required")
Else
    Return Me
End if

```

Understanding Workflow Definition Items



Understanding Definition Workflow Items

When you create and save a Workflow Map, Aras Innovator generates corresponding Items and Relationships in the database to define the Workflow process.

The following ItemTypes are defined by the Workflow Map:

Activity

Defines the Activity created on the Workflow Map.

Workflow Process Path

Relationship between Activities – represented as the path on the Workflow Map.

Activity Assignment

Relationship from an Activity to an Identity.

Activity Email

Relationship from an Activity to an E-mail message.

Activity Method

Relationship from an Activity to a Server Event Method.

Activity Task

Null relationship from an Activity which defines each Task on the Activity.

Activity Transitions

Relationship from the Activity to a Lifecycle Transition for promotion.

Activity Variable

Null relationship defining an Activity variable that can appear on the Workflow Completion dialog.

Use Cases

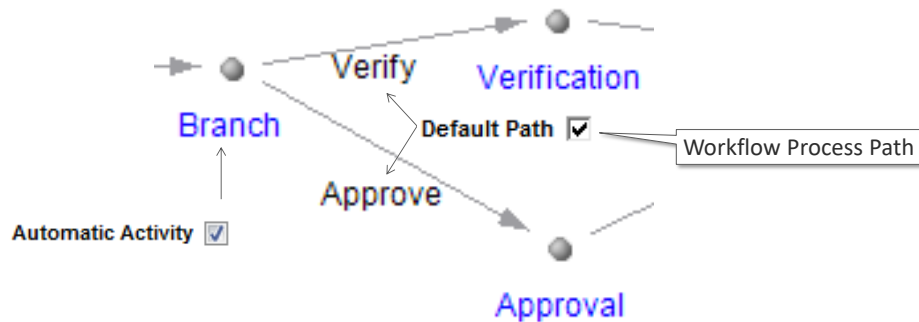
When you define Server Events on a Workflow Activity, you have access to the Items described above to make additions, modifications, or deletions while the Workflow is in process on a controlled item.

Two of the common techniques used in customizing a Workflow are to create Assignments dynamically on an Activity as well as change to a default process route defined on the Workflow Map.

The following sections describe these options.

Creating Dynamic Workflow Routes

- Change Path Route Based on Conditions
- All Paths from Automatic Activity should be "Default"
- Method will "remove" a Default path



Creating Dynamic Workflow Routes

Another common problem when creating a solution with Workflows is dynamically assigning a route based on custom business logic.

Remember that when the Default Path setting is used on a path following an Automatic Activity the Workflow will follow that path regardless of any other settings.

You can use this feature to dynamically "switch" paths so that only one path becomes the Default when the Workflow Activity becomes active.

In the example above, the Workflow is originally set so that both paths following the Branch activity are Default. If an activity is created to change the Default setting on one of the paths, then only one path will be followed.

Note

A Workflow path is represented by the ItemType "Workflow Process Path".

Defining Dynamic Workflow Route Method

```
//Remove a "Default" path
string path_name="Approve";
Item qry = this.newItem("Workflow Process Path", "get");
qry.setProperty("source_id", this.getID());
qry.setProperty("name", path_name);
Item path = qry.apply();
path.setAction("edit");
path.setProperty("is_default", "0");
return path.apply();
```

Get path

Deselect default

Default Path



Defining Dynamic Workflow Route Method

This example shows how the Default path can be turned “off” by setting the `is_default` property on the Workflow Process Path Item to false (“0”).

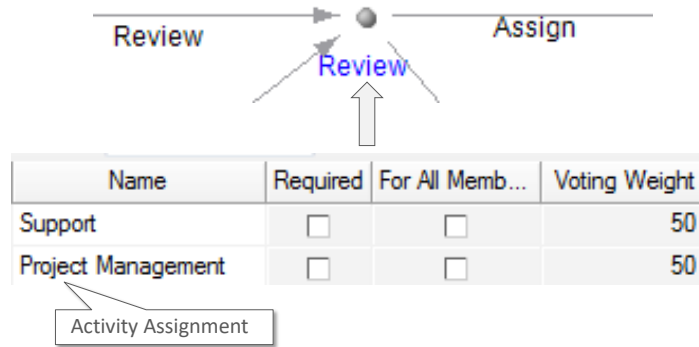
If you assign this Method to an Automatic Activity you can indicate the “direction” the Workflow should take based on some business rule.

VB.NET

```
Dim path_name as String = "Approve"
Dim qry as Item = Me.newItem("Workflow Process Path", "get")
qry.setProperty("source_id", Me.getID())
qry.setProperty("name", path_name);
Dim path as Item = qry.apply()
path.setAction("edit")
path.setProperty("is_default", "0")
Return path.apply()
```

Developing Workflow Dynamic Assignments

- Add, modify or remove assignments based on business rules



Developing Workflow Dynamic Assignments

There may be times when the recipients of an Activity Assignment are unknown until the Workflow is in progress.

You can create a dynamic assignment on an activity that indicates who should receive the assignment and what voting weight they should receive.

Note

Assignments are represented by the ItemType "Activity Assignment".

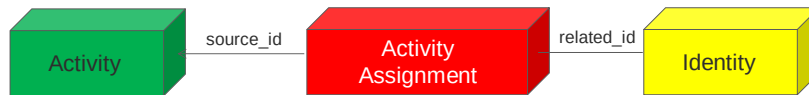
Defining Dynamic Assignments Method

```
//Add assignment to an activity
Item asgn = this.newItem("Activity Assignment", "add");
asgn.setProperty("source_id", this.getID());
Item iden = this.newItem("Identity", "get");
iden.setProperty("keyed_name", "Peter Smith");
asgn.setRelatedItem(iden);
asgn.setProperty("voting_weight", "100");
Item res = asgn.apply();
```

Add Assignment

Set Assignment to this Activity

Relate to Identity



Defining Dynamic Assignments Method

In this example, a new Activity Assignment relationship is created between the current Activity and an existing Identity (Peter Smith).

Note how both the source and related id's are set on the Relationship. The Relationship also stores the voting weight of the Identity in a property named "voting weight".

VB.NET

```
Dim asgn as Item= Me.newItem("Activity Assignment", "add")
asgn.setProperty("source_id", this.getID())
Dim iden as Item = Me.newItem("Identity", "get")
iden.setProperty("keyed_name", "Peter Smith")
asgn.setProperty("related_id", iden.getID())
asgn.setProperty("voting_weight", "100")
Dim res as Item = asgn.apply()
```

Summary

In this unit, you learned how to apply server Methods to customize a solution.

You should now be able to:

- Develop ItemType and Lifecycle Validation Routines
- Develop Workflow Validation Routines
- Create Dynamic Routes in a Workflow
- Create Dynamic Assignments in a Workflow

Lab Exercise

Goal

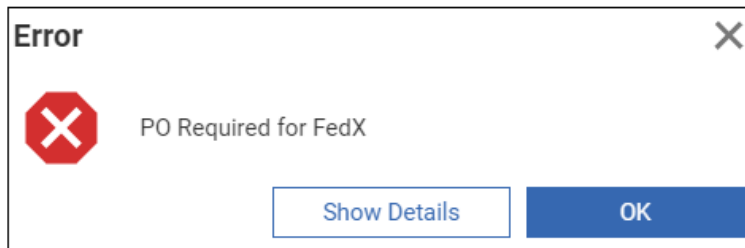
Be able use to create server Methods and associate them with various events to customize a solution.

Scenario

In this exercise, you will create several Methods to provide custom business logic to a solution. You will then associate each Method with the appropriate event to extend the standard behavior.

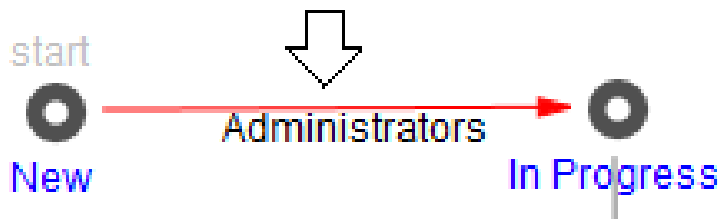
Before Saving a Sales Order

1. A Sales Order with Federal Express Shipping Method requires a P.O. number be provided before the order can be saved.



Before Promoting Sales Order

2. A Sales Order requires a Shipping Date before it can be promoted from the **New** to **In Progress** lifecycle state. Create a Method to stop the promotion if this is not true and attach it to the Sales Order lifecycle transition between New and In Progress.



3. When the Sales Order **Reject Order** Workflow Activity is activated (onActivate), the following Remark should automatically be added to the Sales Order Remarks tab:

Sales Order Remarks ▼ ☆

Hidden ▼

Date [...]	User [...]	Comments
4/12/2021 4:30:09 PM	Innovator Admin	Sales Order Rejected

4. Create a server Method and obtain the controlled item from the Workflow activity as shown in this unit:

C#

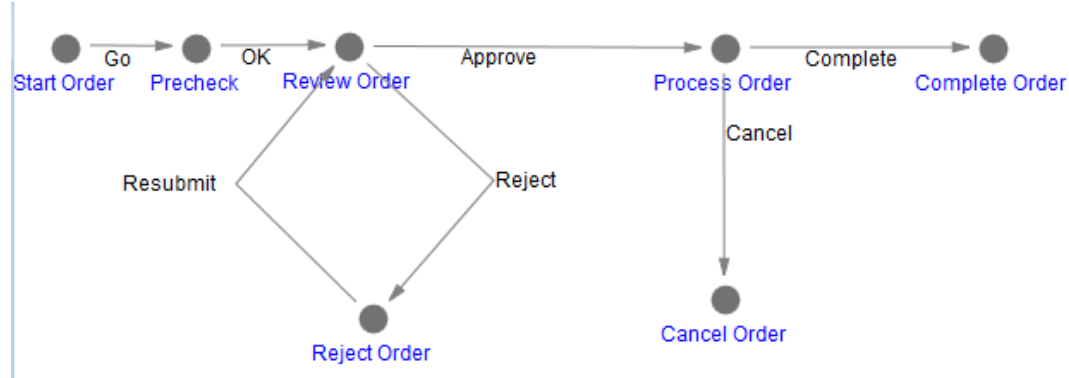
```
Item order = this.apply("Get Controlled Item");
```

VB.Net

```
Dim order as Item = Me.apply("Get Controlled Item")
```

5. Use the `createRelationship Item` method to add a new "Sales Order Remarks" relationship on the `order`.
6. Set the `comment_text` property on the relationship item to "Sales Order Rejected" and apply the change to the `order` Item.
7. Attach the Method to the Server Event of the **Reject Order** activity (onActivate).
8. To test the behavior, create a NEW Sales Order and then go the MyInnovator > InBasket to Reject it from the Review Order activity.

Workflow OnActivate Activity





Unit 9 Exploring Aras Object Functions

Overview

In this unit you will learn how use the various client JavaScript methods available in the aras Object. Every window in Aras Innovator has access to the aras Object, which contains over 200 utility functions that can be used in client Methods.

Objectives

- Classifying Aras Object Functions

Reviewing Aras Object methods

- Every Aras Innovator window has a reference to the "aras" object
- Object references JavaScript methods (in addition to IOM)
- Source file: aras_object.js
- Contains client utilities for Client Methods only

aras.methodname()



Reviewing Aras Object methods

The aras_object.js file is in the /Innovator/Client/javascript folder.

You access the aras Object from any JavaScript client Method using the syntax:

aras.*methodname* ()

Where *methodname* is a JavaScript function defined in the library file.

Displaying Success and Errors

- aras.AlertSuccess



- aras.AlertError



Displaying Success and Errors

AlertSuccess

Displays a success message window in the right corner of the current window. The message is cleared in several seconds or when a user clicks on the message.

```
aras.AlertSuccess("Item Saved Successfully");
```

AlertError

Displays an error window with a message and optional fault code and stack trace. The AlertError method uses JavaScript promise technology to allow a callback once the user has accepted the message.

```
function callback() {
    alert("handle Error");}

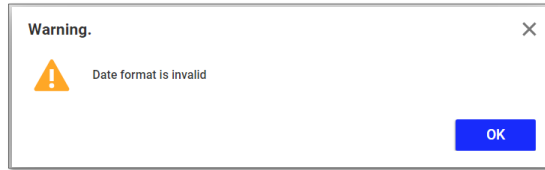
aras.AlertError("Operation Failed").then (
    function() {
        callback(); }
    );
```

Note

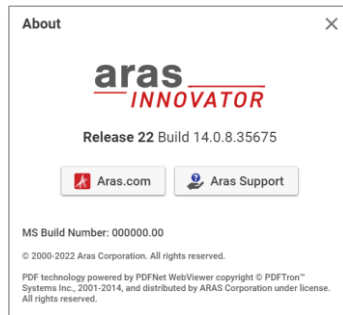
The AlertError is a non-modal window. The Client Method continues once the dialog is displayed.

Displaying Alert Warning and Alert About Window

- `aras.AlertWarning`



- `aras.AlertAbout`



Displaying Alert Warning and Alert About Window

```
aras.AlertWarning("Date format is invalid");  
aras.AlertAbout();
```

Displaying Confirmation and Prompt Dialog

- **aras.confirm**



- **aras.prompt**



Displaying Confirmation Dialog

Displays a modal confirmation dialog box, which contains a message and OK and Cancel buttons. OK returns true, Cancel returns false.

```
const result = aras.confirm("Remove Document?");
```

Displaying Prompt Dialog

Displays window to enter and capture text. Accepts a title and a default value.

```
const result = aras.prompt("Enter Company Name", "");
```

The prompt window is displayed asynchronously using JavaScript promise technology which requires a callback function to get the results:

```
function callback(results) {
  alert(results);}
aras.prompt("Enter Company Name:").then (
  function(dialogResult) {
    callback(dialogResult);}
  );
```

Executing a JavaScript Client Method

▪ evalMethod

- Executes Client Method from a Client Method
- Optional arguments include:
 - AML string to be loaded into context of called Method
 - JavaScript Object containing named arguments

```
var inArgs = new Object();
inArgs.firstoperand=100;
inArgs.secondoperand=200;
inArgs.operation="+";
inArgs.property="_cost";

var result=aras.evalMethod("Calculate", this.node.xml,
  inArgs);
```



Executing a Client Method from another Client Method

To run an Aras Client Method *from* a Client Method requires the evalMethod function.

The evalMethod can accept up to 3 arguments:

- Client Method Name (required) – name of the Client Method to execute.
- inDom (optional) – an AML string which is loaded into the context item of the called Client Method
- inArgs (optional) – a JavaScript object to be passed to the Client Method

Example

The calling Method above is associated with a Design Request Item Action. The AML of the current context is passed to the called Method named Calculate below to assign the value of _cost property based on a simple arithmetic expression.

```
const a = inArgs.firstoperand;
const b = inArgs.secondoperand;
const op = inArgs.operation;
const prop = inArgs.property;

const result = evalMethod( a + op + b );

this.setAction('edit');
this.setProperty("_cost", result);
return this.apply();
```


Getting Paths

- `getServerBaseUrl`
 - Get URL of Innovator Server
- `getBaseUrl`
 - Get URL of Innovator Client



Getting Paths Examples

```
const url = aras.getServerBaseUrl();  
const base = aras.getBaseUrl();
```

Getting a File

- **vault.selectFile**
 - Prompt the user to select a local file
 - Returns a File object asynchronously

```
aras.vault.selectFile().then(  
  function (fileObject){  
    f = fileObject;  
    alert("Filename:" + f.name);  
  });
```



Getting a File

To obtain a file from a user's local machine requires a method available on the vault object.

The selectFile method uses a JavaScript Promise which requires a call back function to capture the passed file Object. This method is typically used to upload a selected file to a user vault. See the Item class setFileProperty method for more examples.

Getting User Information

- **getLoginName**
 - Returns current User's login name
- **getUserType**
 - Returns user or admin
- **isAdminUser**
- **getUserID**
- **getDatabase**
- **getIdentityList**
 - Returns list of Identity ID's that User belongs to



Getting User Information Examples

```
const r0 = aras.getLoginName();
const r1 = aras.getUserType();
const r2 = aras.isAdminUser();
const r3 = aras.getUserID();
const r4 = aras.getDatabase();
const r5 = aras.getIdentityList();

aras.AlertSuccess("Login: " + r0 + " Database: " + r4);
```

Summary

In this unit, you learned about the various IOM methods available in the Innovator class.

You should now be able to:

- Identify and Classify Aras Object Functions

Lab Exercise

Goal

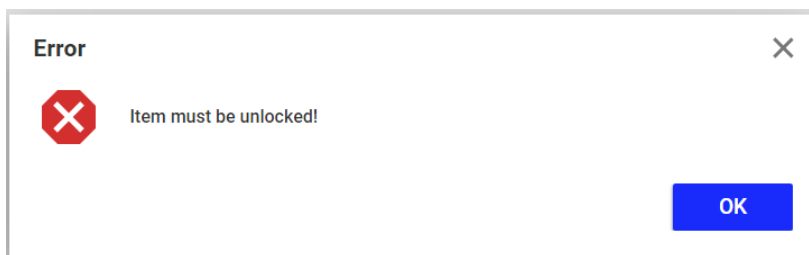
Be able to use locate and use the Aras Object Functions in a client Method.

Scenario

In this exercise, you will create a Method that use the functions discussed in this unit.

Steps

1. Locate the Client Method you created in an earlier exercise that is called by the **Assign Shipping Date** action to assign the ship date property to today's date.
2. Add a condition statement at the top of the Method that display an Aras error if the item is currently locked. Use an Item class method discussed earlier to determine the lock status.



This page intentionally left blank.



Unit 10 Creating Client Event Methods

Overview

In this unit, you will learn how to build client methods to customize and extend the standard client behavior of the system. This includes client validation and customizing the user interface.

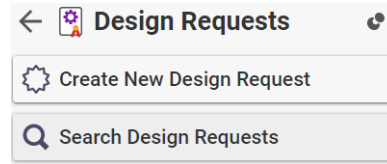
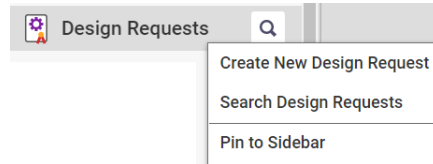
Objectives

- Customizing New Item Functionality
- Reviewing Client Events
- Developing Form and Field Client Methods
- Predefine Search Criteria

Customizing New Item Functionality

- Client events triggered by creating a new Item from:

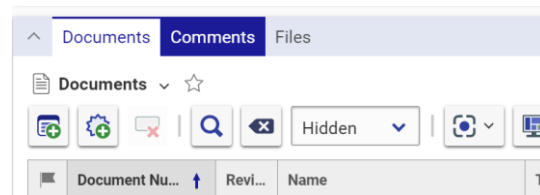
- TOC



- Search Grid



- Relationship Grid



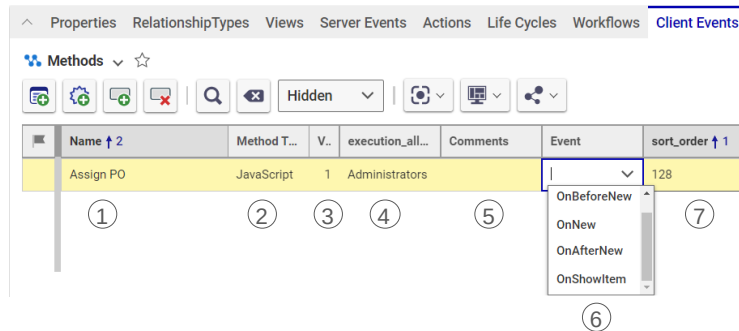
Customizing New Item

Client events are available on each Item and are triggered when a user attempts to create an Item for the first time (add).

You can create logic to prevent the Item from being created or alter the server response before the user sees the form appear on the client.

Configuring ItemType Client Events

- Four client events supported
- Three of them support the New Item Functionality:
 - OnBeforeNew
 - OnNew
 - OnAfterNew
- OnShowItem



Configuring ItemType Client Events

Four events are available that are configured on the ItemType, the first three out of them support the New Item Functionality:

- **OnBeforeNew** – runs before a new Item is created and can be used to cancel a new Item request as well as prevent display of the new Item Form. This is useful for preventing new Item creation if certain conditions have not been met.
- **OnNew** – replaces the standard new Item behavior. A client Method must be provided that provides custom logic to replace the standard add operation.
- **OnAfterNew** – runs after the Item has been created. Useful for populating a new Item with data or opening custom dialogs, etc. The new request has been processed by the server and the context Item contains the current values that will be displayed on the form. If you change (or add properties) to the AML response they will be displayed instead (or in addition to the existing properties) on the form.
- **OnShowItem** – replaces the standard logic for displaying a tear off window. A client Method must provide the custom logic to display an appropriate window.

Defining ItemType Client Method – OnBeforeNew

```
const user =  
  this.getInnovator().getItemById("User", aras.getUserID());  
  
if (user.getProperty("email", "")=== "")  
{  
  aras.AlertError("User Profile missing email address!");  
  return false;  
}
```



Defining ItemType Client Method – OnBeforeNew

In this example, a logged in user must have an email address to create a new item.

If the logged in user does not have an email address, the program displays an error and returns false. Otherwise, the program returns true. A false return will block the ability for the Aras client to create a new Item.

Defining ItemType Client Method – OnAfterNew

```
const po = "PO-" + new Date().getTime();
this.setProperty("po_number", po);
return this;
```

Context Item
Altered



Defining ItemType Client Method – OnAfterNew

This example demonstrates modification of the context Item returned from the server (response). In this case, we want to assign a dynamic purchase order number with the Method above.

Example adding a Relationship

```
const msg = "Item Added " + new Date().toLocaleDateString();
const rel = this.newItem("Design Request Comments", "add");
rel.setProperty("_comments", msg);
this.addRelationship(rel);
return this;
```

Note

Take care when modifying a response context Item. If you provide invalid data, the response may cause unpredictable results on the client.

Exploring User Interface Events

- Used to provide custom business logic for:
 - Form
 - Field
 - Relationship Grid



Exploring User Interface Client Events

The Aras Innovator client displays forms to the user to allow them to view and edit data relevant to an Item.

You can customize the interaction between the user and the form with a series of client events on the form, each field of the form as well as the rows and columns of the relationship grid (tab).

Developing Form and Field Client Event Methods

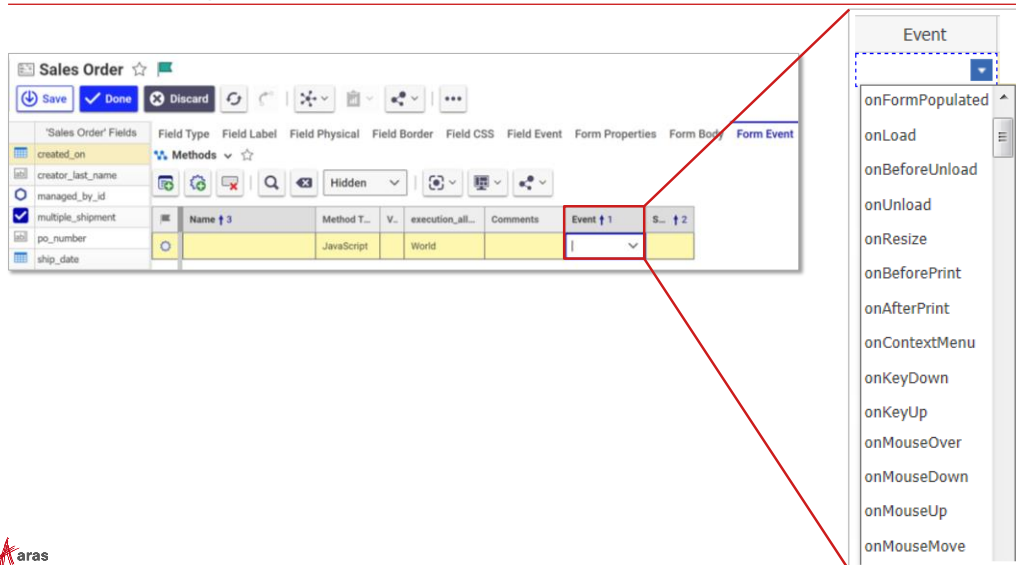
- Binding Method to Form and Field Events
- Useful for:
 - Populating/copying data
 - Validating client input
 - Providing additional feedback



Developing Form and Field Client Event Methods

Form and Field client event Methods are useful for populating data on a form as well as providing cross-field validation before a user attempts to save Item information to the database.

Reviewing Form Client Events



Reviewing Form Client Events

Many of the Form events are familiar to developers who have worked with HTML forms previously. Aras Innovator has some additional events that you can configure for your solution:

Form Loading Events

- **onFormPopulated** – runs when all Fields have been populated with data from the server.
- **onLoad** – runs when the HTML form has been loaded into the window.
- **onBeforeUnload** – runs just before the form is removed from view.
- **onUnload** – runs when the form is removed from view (on a window close).

Form Resize Event

- **onResize** – runs when window height or width is modified.

Form Print Events

- **onBeforePrint** – runs before the form is sent to the printer – allows for changes or modification before sending a form to the printer.
- **onAfterPrint** – runs after the form has been sent to the printer – allows form to be reverted back to previous view once form is sent to printer.

Form Menu Event

- **onContextMenu** – runs when the user right clicks the mouse button to display the context menu.

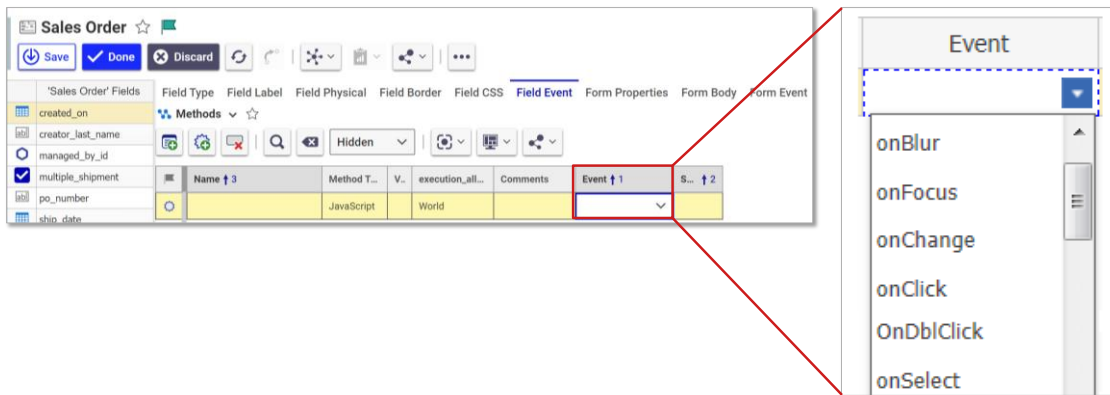
Form Keyboard Events

- **onKeyUp** – runs when key is released.
- **onKeyDown** – runs when key is pressed.

Form Mouse Events

- **onMouseOver** – runs when mouse pointer is moved over form body.
- **onMouseUp** – runs when left mouse button is released.
- **onMouseDown** – runs when left mouse button is clicked.
- **onMouseMove** – runs when mouse pointer is moved.

Reviewing Field Client Events



Reviewing Field Client Events

Each field on a form can be configured to respond to the following events:

- **onBlur** – runs when a user leaves a field.
- **onFocus** – runs when the cursor is moved into a field.
- **onChange** – runs when the value of a field is changed.
- **onClick** – runs when the user mouse clicks in a field.
- **OnDbClick** – runs when the user double clicks the mouse in a field.
- **onSelect** – runs when a user selects text in a field.

The field event onChange is used also for clicks inside dropdowns and checkboxes.

The field event onClick is typically used for buttons.

Examining the Context Item

- Each Client Method has access to the Innovator Context Item
- Context Item for Form and Field Event Methods is:

document.thisItem

- Item DOM Object is also available from:

document.item

- To obtain browser Window object (Form or Field object) use:

this



Examining the Context Item

The following references are also useful when creating client Methods to be used with Forms:

Syntax	Description
aras	Client Framework API object
document.item	An XML DOM object representing the Item in focus and used with the Client Framework API.
document.thisItem	The context Item object (must be used instead of "this" in Form and Field event client Methods)
document.itemID	Context Item ID
document.itemType	Context ItemType
window.handleItemChange("property",value)	The function to update the Item Property value (cache) and the HTML Form Field value
getFieldByName("element_name")	Returns a form element using the control name configured in the form editor.

Saving Form Data

HTML Form

Aras XML Cache (document.item)

```
<?xml version="1.0" encoding="UTF-8" standalone="yes"?>
<DesignRequest xmlns="http://schemas.aras.nl/DesignRequest" id="DR-000140">
  <Title>New Printer Style</Title>
  <Description>Customer requests a new style for a printer</Description>
  <Cost>350.00</Cost>
  <Classification>Customer</Classification>
  <CreatedOn>2022-01-28T00:00:00</CreatedOn>
  <RequestOwner>Change Specialist</RequestOwner>
  <RequestNumber>DR-000140</RequestNumber>
  <Customer>Brown Industries</Customer>
  <ProductDivision>Computing</ProductDivision>
  <ProductThumbnail>
    <img alt="Printer image" data-bbox="388 261 438 298"/>
  </ProductThumbnail>
  <CustomerPhone>888-456-2199</CustomerPhone>
  <ProductType>Printer</ProductType>
  <FileProperty>
    <File>
      <name>PrinterStyle.pdf</name>
      <size>102400</size>
      <mime-type>application/pdf</mime-type>
    </File>
  </FileProperty>
</DesignRequest>
```

Saving Form Data

Aras Innovator represents the current Item as a DOM object that stores property values. (You can view the cache object in the debugger by inspecting this.document.item).

As a user enters data the HTML Form captures that data in the form control. That data must also be set in the AML cache (document.item) in order for the data to be sent to the server.

In normal operations, the cache is updated each time a user leaves a field (field loses focus). When a user saves an Item the appropriate XML tags are defined with values to be submitted to the server.

A problem can occur however if you programmatically set the value of an Item using a Method associated with a Form or Field event.

Both the HTML control and the Item cache must be updated to the new value so that the Form presentation and data cache are the same. Aras provides a built-in function (described on the next page) to simplify this procedure.

Getting and Setting a Field Property Value

- **this.value**
 - Set or get value in HTML control only
- **document.thisItem.getProperty**
 - Retrieves value from the XML Cache
- **document.thisItem.setProperty**
 - Sets value to XML Cache
 - HTML control is not updated
- **window.handleItemChange(property, value)**
 - Sets value to XML Cache
 - Updates HTML control



Setting and Getting Data

Data can be obtained and manipulated in several ways using a Client Method.

this.value

Obtains data from the HTML Form (not the Item cache) and is useful in field onChange events to obtain the changed value of the current field.

document.thisItem.getProperty

Obtains data from the Item cache (not the Form control).

document.thisItem.setProperty

Assigns data to the Item cache (not the HTML Form).

window.handleItemChange

Each Aras Innovator Form contains a predefined JavaScript window function named handleItemChange that will set the value of a property in the cache as well as update the HTML control programmatically. You can use this function in your form client Methods to update Form fields and the Item cache using a single operation.

Creating a Field Event Method

▪ onClick (Button) Event Example

```
//Get the value of cost property
const cost = document.thisItem.getProperty("_cost", "0");

const taxcost = Number(cost) + (Number(cost) * 0.0625);
//cast

//Set calculated value to total_taxed property
window.handleItemChange("_total_taxed", taxcost);
```

Calculate Tax

Total Taxed Cost:



Creating a Field Event Method (onClick)

This example shows a simple calculation to obtain a Taxed Cost of a Design Request (assumes tax rate is 6.25 percent). A button has been added to the form to show the calculated result:

Cost 	Patent? ☐	Completion Date
Classification 	State 	Thumbnail Select an image...
Customer 		
Customer Phone 	Calculate Tax	Total Taxed Cost:

The Method above is then added to the onClick Field Event of the Button control:

Methods ▼ ☆							
					Hidden ▼		
	Name ↑ 3	Method T...	Ver	execution_all...	Comments	Event ↑ 1	S... ↑ 2
	CalculateTax	JavaScript	2	Administrators	Calculates tax of 6.25%	onClick	128

The Method obtains the value of the cost property from the context Item. The taxed cost is then calculated using simple JavaScript operators and assigned to the total_taxed property using the window.handleItemChange function.

Creating a Field Event Method

▪ onChange Event Example

```
const field_comp =
  getFieldComponentByName("_product_types");
const producttype = field_comp.getValue();
if (producttype==="TB") {
  window.handleItemChange("_patent_required", "1");
}
```

Patent? ☒ Product Types Tablet



Creating a Field Event Method (onChange)

This example will set the Patent required field to true (boolean "1") on a Design Request if the user selects the Product Type of Tablet (list value = "TB") from the Form. The Method is associated with the onChange event of the Product Types field:

Methods ▼ Hidden ▼ ▼ ▼ ▼							
	Name ↑ 3	Method T...	V..	execution_all...	Comments	Event ↑ 1	S... ↑ 2
	SetPatent	JavaScript	1	Administrators	Set Patent required for tablets	onChange	128

When capturing data from a Dropdown type, you can use a built-in function named **getFieldComponentByName** in a field event onChange Client Method to capture the changed value with the corresponding **getValue()** method. The Aras Item DOM is updated when the list field loses focus, so calling `document.thisItem.getProperty` in the onChange event of the current control will return the original field value (from the Item cache).

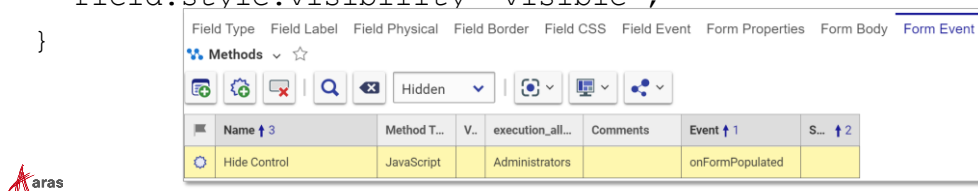
To get the current value of a Textbox control use the standard HTML function `this.value` to get the current value. Example below gets the current value of the "_title" field on a Design Request:

```
const t = this.value;
```

Creating a Form Event Method

- Hiding/showing a control:

```
const country = document.thisItem.getProperty("country");
const field = getFieldByName("address_state");
if (country!="United States") {
    field.style.visibility='hidden';
}
else {
    field.style.visibility='visible';
}
```



Creating a Form Event Method (onFormPopulated)

This example is associated with the ItemType “Customer” and shows how a Form event can be used to alter the user interface dynamically at runtime.

The **onFormPopulated** event is triggered each time the Form is refreshed on the screen. In this example the field “address_state” will be hidden if the country is not “United States”. Standard style attributes can be set in the client Method to alter the display and characteristics of the HTML Form dynamically.

When you create a Form in Aras Innovator you are also generating an HTML page that contains named controls. You can access those controls to change style or visibility on the control using a client Method with JavaScript. A standard method is provided on each Aras form named **getFieldByName** which will return the DIV element representing the control (and label) on the form. In this example, if the country property is not set to the “United States” then the address_state (DIV) on the Form is hidden from view.

This method can then be associated with the onFormPopulated event of a Form. Note the reference to the context Item uses the **document.thisItem** keyword.

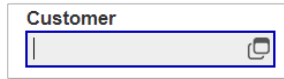
To access the individual HTML field elements within an item view form you can also use the following reference:

```
const f = document.getElementById("MainDataForm").address_state;
f.style.visibility='hidden';
```

OnSearchDialog Overview

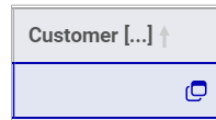
- Client Event that is triggered when a user:

- Clicks Ellipsis Button to Search for Item



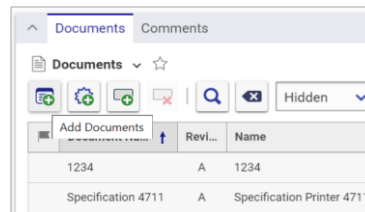
A screenshot of a form field labeled "Customer". To the right of the text input is a small square button with a downward-pointing arrow (ellipsis button).

- Clicks F2 Lookup Key on Searchable Field



A screenshot of a form field labeled "Customer [...]". To the right of the text input is a small square button with an upward-pointing arrow (lookup key).

- Adds a Related Item to a Parent Item



A screenshot of the OnSearchDialog search grid. It has tabs for "Documents" and "Comments". The "Documents" tab is active, showing a list of items with columns for "Add Documents", "Rev...", and "Name". The list contains two rows: one with "1234" and "A", and another with "Specification 4711" and "A".



OnSearchDialog Overview

The OnSearchDialog event is available to respond to the following user actions:

- User clicks the ellipsis [...] button or F2 lookup key on a form field to display an Item search grid.
- User clicks the Select items button and the related Item search grid is displayed.

In this example, the OnSearchDialog event restricts the selection options for Item properties.

Please note, that only Item properties provide the ability to search on another Item in the Aras Innovator client.

Search Options

▪ Prefill Search Criteria in the Item Search Grid

Select Items

Documents ▾

Q X Simple ▾

	Document Number	Revi...	Name	State
▾				Released
	MP0101-D0C-1	A	MakerBot Replicator Requirements ...	Released

▪ Create custom search result lists based on business rules

Select Items

Products ▾

Q X Hidden ▾

	Product Number	Name	Description
	DesignJet 2000 Large ...	Large Format Printer	Commercial high capacity large format
	DeskJet 400 Series	Inkjet Printer	Low cost inkjet printer for consumer and ...

Search Options

When any of the actions described on the previous page occurs you can:

- Prefill the Simple Search criteria in the Select Items grid dependent on criteria of your choice – this criterion can optionally be set to be read-only (cannot be changed by the user).
- Create a custom search result set based on business logic. The user is only able to select the Items that are defined by the custom logic. The search criteria bar is hidden and cannot be displayed (see Appendix for information on creating custom result sets).

Note

Using configuration, a filter value can be pre-set via the Property setting **Search Default** but will not provide conditions. Using the developing mechanism allows adding conditions as well as preventing users from changing the pre-set filter.

Predefining Search Criteria

- Choose properties to provide criteria
- Supply search criteria for properties
- Determine change access
 - Fixed – user cannot change default criteria supplied
 - Not Fixed – default is provided but user can override



Predefining Search Criteria

To predefine search criteria, you must decide which columns (properties) will be affected and what criteria you will prefill in the columns.

Criteria provided can be fixed so that a user is not able to change it.

In this unit, we create a Method that predefines the `owned_by_id` Property, so that only alias identities can be selected by the user.

Creating the Search Method

- Client-side JavaScript Method only
- Return Object and assign filter values
- Associate Method with OnSearchDialog event
- Syntax:

```
return {
    is_alias:{filterValue:"1",isFilterFixed:true}
};
```



Creating the Search Method

The search Method must be defined as client-side JavaScript. The Method creates a JavaScript object that is used to set the criteria for the grid before it is presented to the user. Using JavaScript Object Notation (JSON) you can indicate filter values and whether the filter can be changed by a user at runtime.

To Create the Search Method

1. Create a new Client Method and name it *SearchCriteria*.
2. Create assignment filters by supplying the property name as a named element. The Aras client expects an object array where each element is the name of property (grid column). Two attributes are available to set criteria and read only status in each element (filterValue, isFilterFixed).
3. Indicate the value of the search criteria using the filterValue setting.
4. Indicate whether the value can be changed by a user with the isFilterFixed setting (true or false).
5. Return the filter object to the client.

In this example, a Method named SearchCriteria is created with code as shown above. This method will predefine an Identity search to only show Alias Identities. The user will be unable to change the alias criteria. JavaScript object attributes are case-sensitive. Make sure to enter the attributes as shown in the example.

Associating Method to Event

The screenshot illustrates the process of associating a method to an event in the Aras application. It is divided into three numbered sections:

- Context Menu:** A right-click context menu is shown for the `owned_by_id` property. The menu includes options like `Open`, `Replace`, `Remove`, `Actions`, `Permissions`, and `Properties`. The `Properties` option is selected, and its sub-menu is open, showing `Open`, `Copy Row`, `Show Clipboard`, and `Promote`.
- Event Configuration Panel:** The `Event` configuration panel is shown. It includes a `Methods` dropdown, a `Hidden` dropdown, and a table of event associations.
- Event Association Table:** The table shows the association between the `SearchCriteria` method and the `OnSearchDialog` event.

Name ↑ 3	Method T...	V..	execution_all...	Comments	Event ↑ 1
SearchCriteria	JavaScript		Administrators		OnSearchDialog

Associating the Method to the OnSearchDialog Event

The search Method is then associated with the OnSearchDialog event on a property.

The property must be of data type Item.

To Define the OnSearchDialog Event

1. Open the desired ItemType in edit mode and locate a property that is exposed on a form and has the data type Item. Right click to view the context menu and select Properties > Open.
2. Set the Property to edit mode and add a new Event relationship on the property form and select the Method named SearchCriteria.
3. Choose **OnSearchDialog** as the event type.

In this example the **owned_by_id** property has been selected in the Part ItemType. The Method named SearchCriteria is assigned to the OnSearchDialog event.

Summary

In this unit, you learned how to apply client Methods to customize a solution.

You should now be able to:

- Customize New Item Functionality using Client Events
- Develop Form and Field Validation
- Predefine Search Criteria

Lab Exercise

Goal

Be able to create and use client Methods with the appropriate client events to customize a solution that uses the Aras Innovator client.

Scenario

In this exercise, you will create several client Methods to extend the behavior of the Innovator client. You will then associate these Methods with appropriate client events to provide custom business logic in a solution.

Client Event - Set Shipping Date on New Sales Order

1. When a user creates a new Sales Order the Shipping Date should be set to today's date by default.

A date property is always set using a "neutral" date string and must be formatted as "yyyy-mm-dd".

Example: "2010-06-10"

You can use the JavaScript Date() function to create a new Date object as shown below:

```
var d = new Date();
var today = d.toISOString().substring(0,10);
```

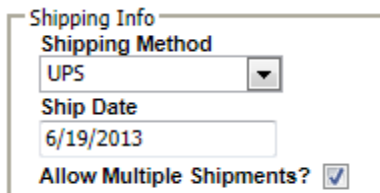
The date should be prefilled on the client before the form is loaded.

Which ItemType client event allows you to change the Context Item of a new Item after it has been returned from the server - but before the user sees the form?

Remember to return the modified context Item to the client in your Method.

Change the Value of a Property

2. If user checks the **Allow Multiple Shipments** box on a Sales Order form the Shipping Method should change to UPS, otherwise the Shipping Method should be set to Federal Ex.



Shipping Info

Shipping Method

UPS

Ship Date

6/19/2013

Allow Multiple Shipments? ☒

Use the `getProperty` method to get the value of the `multiple_shipment` property on the current item.

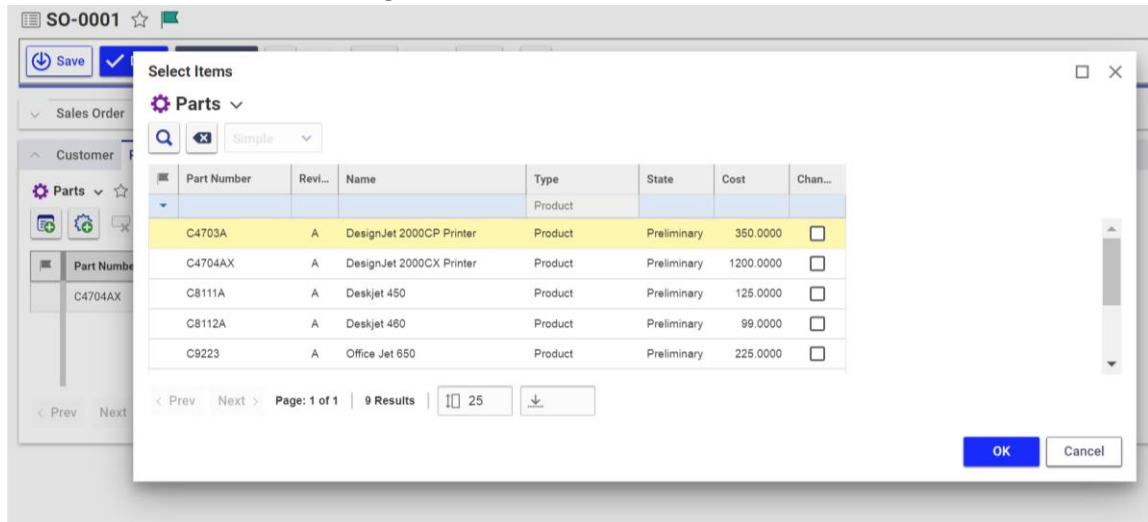
What syntax allows you to access the context Item on a Form or Field event?

If `multiple_shipment` is checked (true) then use a function described in this unit to change the `shipping_method` property value to "UPS". Otherwise set it to "Federal Ex".

Filter the Parts List

3. A user should only be able to select from Parts with the **Product** Type Classification when they add a Part to the Sales Order. You will need to create a client filter Method for the `related_id` property of the Sales Order Part relationship ItemType.

Create a client Method that creates a filter to set the **classification** property to Product. The user should not be able to change this value.



Edit the "Sales Order Part" relationship ItemType and attach this Method to the `onSearchDialog` event of the `related_id` property.

This page intentionally left blank.



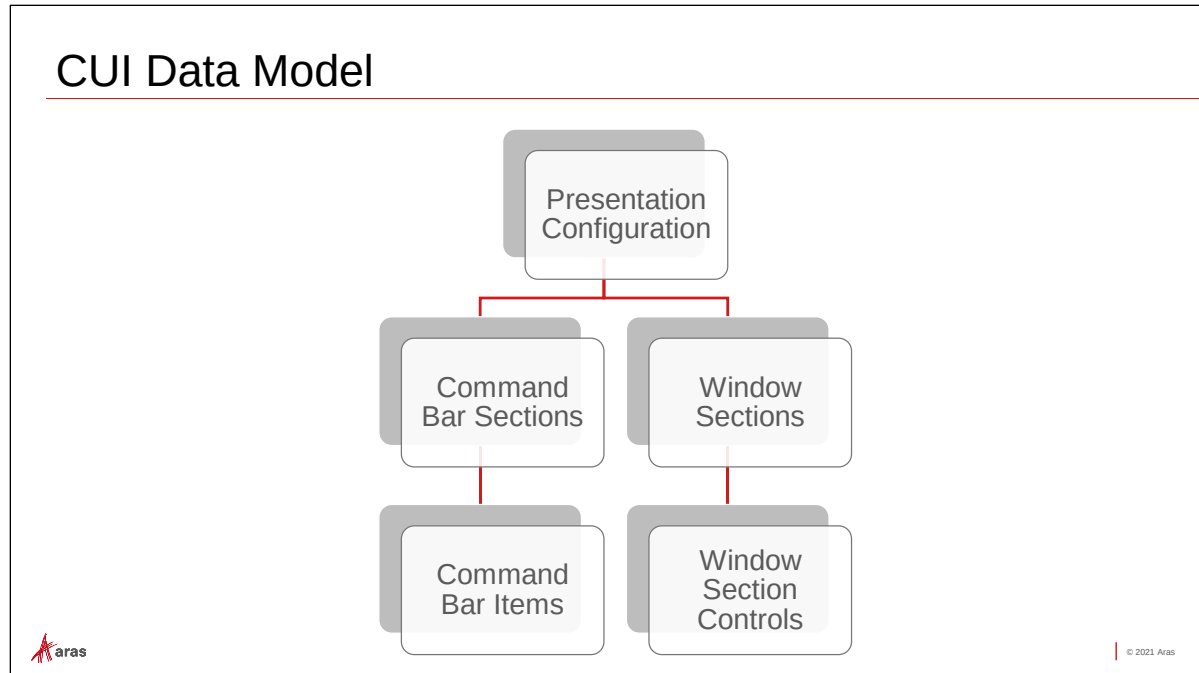
Unit 11 Configuring the User Interface

Overview

The Configurable User Interface (CUI) allows an administrator to customize the layout and behavior of the Innovator client. In this unit, you will learn how to reconfigure windows sections, add a new window accordion, and add controls that access the search grid.

Objectives

- Understand the scope of the Configurable User Interface (CUI)
- Identify CUI Window Section and Command Bar Controls
- Identify Command Bar Item actions
- Understand the CUI Item presentation hierarchy
- Rearrange Window Sections
- Add a new Item Accordion
- Add a new Command Bar Item
- Disable a Command Bar Item



CUI Data Model

The **Configurable User Interface (CUI)** allows an Aras administrator to define the layout and behavior of the Innovator client application. The configuration is based on a hierarchy of ItemTypes that define both the windows design as well as the individual controls that allow a user to interact with the application.

Presentation Configuration

A global **Presentation Configuration** acts as a container for the CUI configuration. The global presentation applies to all items in the application but can be overwritten or augmented by an ItemType presentation which is specific to a category of items (e.g., Part). One global presentation defines the standard out-of-the-box Aras web client.

Window Sections

A **Window Section** defines the overall layout of an item window and contains various window controls that will be displayed on the screen.

Window Section Controls

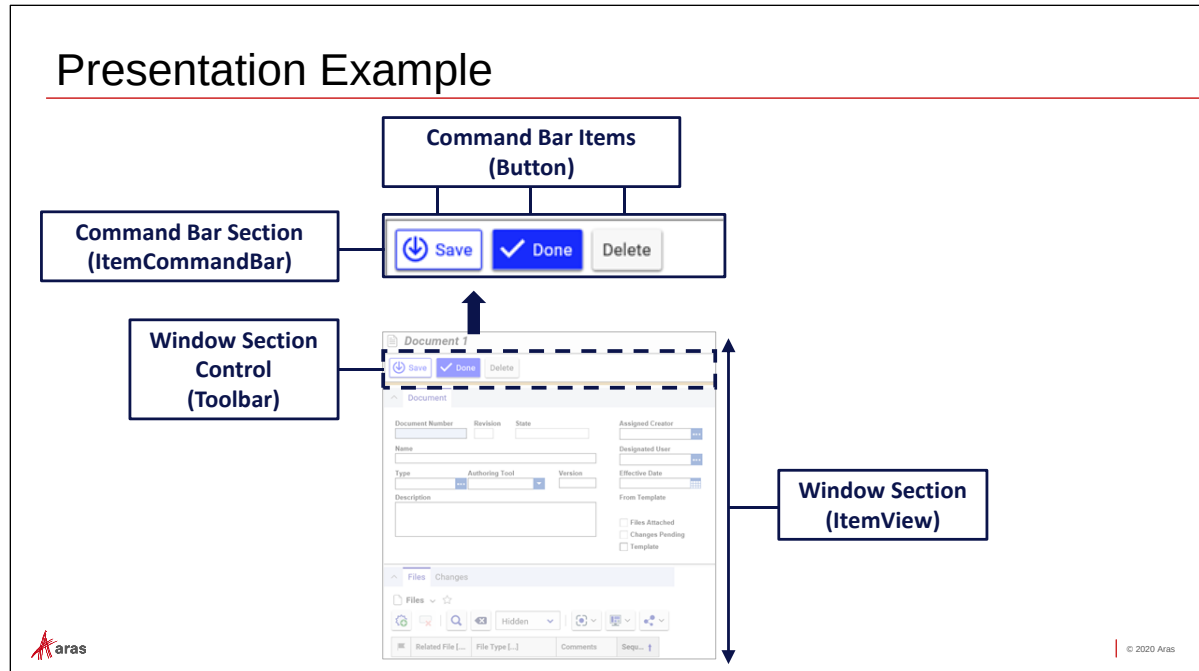
Window Section Controls determine what kind of the layout of a title bar, toolbar, tab control, form accordion element, and a relationship accordion element. Think of Window Section Controls as placeholders for content.

Command Bar Sections

Command Bar Sections define the content of a Window Section Control and contain a collection of one or more command bar items.

Command Bar Items

Command Bar Items define each individual control contained in a Command Bar Section. Examples include toolbar menus and buttons, windows shortcut keys, and context menu entries.



Presentation Example

The example above demonstrates how Window and Command Bar sections work together to create the presentation.

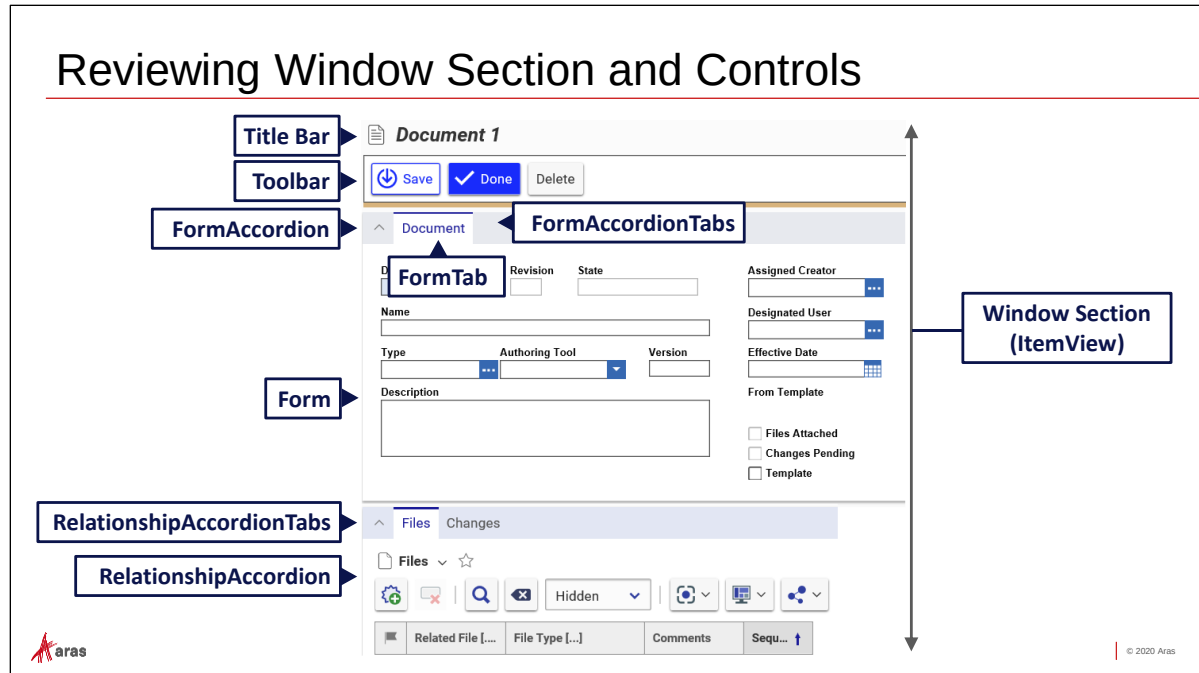
A Window Section defined as an ItemView represents the entire Item window.

Within each Window Section are Window Section Controls that define the possible elements that can be placed in a Window Section. In this example, a Window Section Control (Toolbar) has been created to display a toolbar.

The Window Section and Window Section Controls are placeholders that define the layout and positioning of the UI elements on the screen.

The content of the Window Section Control is defined using a Command Bar Section Control. In this example, a Command Bar Section Control (ItemCommandBar) defines a collection of toolbar buttons. Window Section Controls are assigned to a location in the user interface. There are currently over 60 locations available.

Each individual toolbar button is then defined using a Command Bar Item (Button) within that Command Bar Section control.



Reviewing Window Section Controls

Window Section controls describe the layout and positioning of elements within the Aras client. The layout of an item window is first defined in a Window Section named ItemView. The following Window Section controls are defined in a standard item window layout:

Title Bar

Defines the container for an item header.

Toolbar

Contains command bar section to allow a user to interact with the current item (save, delete, promote, etc.).

FormAccordion

A collapsible window section that displays an item Form and includes a tab control. A Window Section can define multiple FormAccordions, and each accordion can contain a FormAccordionTabs control.

FormAccordionTabs

Is associated with a FormAccordion control and contains one or more FormTab controls.

FormTab

Defines a tab associated with a Form and is associated with a FormAccordionTabs control.

Form

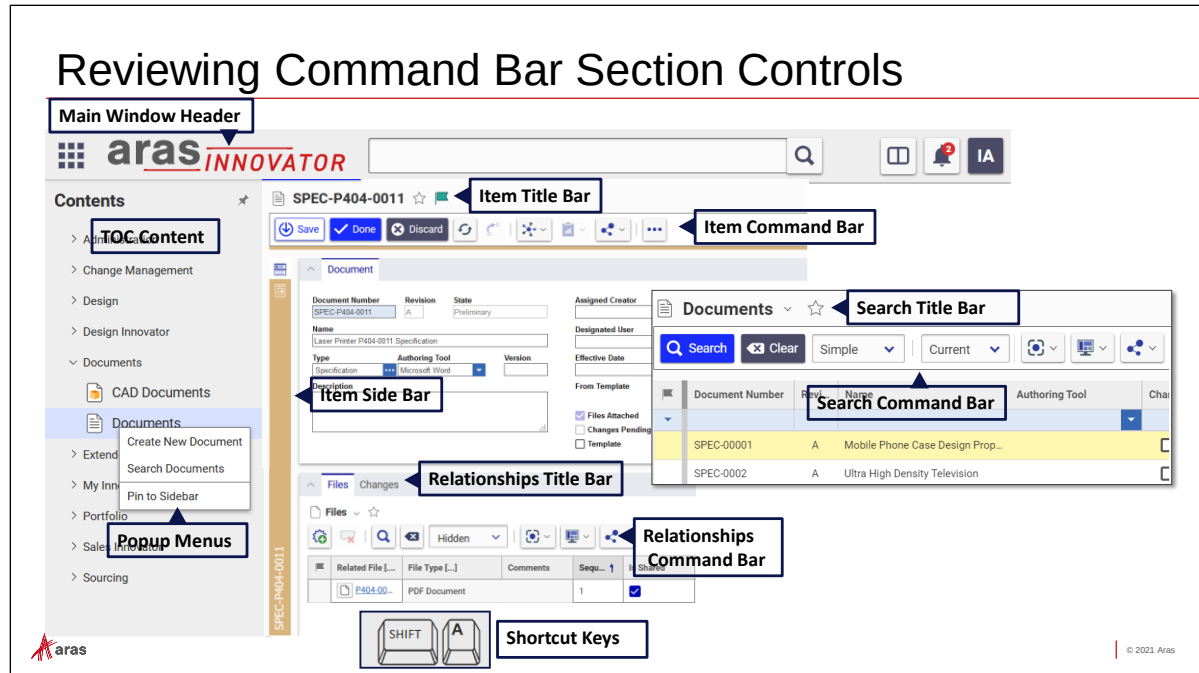
Defines the location of the item Form.

RelationshipAccordion

Displays one or more collapsible relationship grids.

RelationshipAccordion Tabs

Displays one or more tab controls that correspond to relationship grids.



Reviewing Command Bar Section Locations

A Command bar sections defines the content of the presentation and is assigned to a specific location. There are currently 60+ locations defined in the data model. Some primary locations include:

Main Window Header

Contains the navigation button, company logo as well as controls for enterprise search and the user menu.

TOC Content

Define the table of contents category labels.

Popup Menus

Define context menu entries available from the TOC, item grids and the item window.

Item Title Bar

Displays the keyed name of an item, associated icon, and a "save favorites" and claim control.

Item Command Bar (Toolbar)

Contain command bar items (menus, buttons, dropdowns, etc.) to allow a user to interact with the parent item.

Item Side Bar

Contains one or more side bar command buttons.

Relationship Title Bar

Displays the currently selected relationship type name, an associated icon and a "save favorites" control.

Relationships Command Bar (Toolbar)

Contains command bar items (menus, buttons, dropdowns, etc.) to interact with selected relationships.

Search Title Bar

Displays the ItemType label associated with the contents of the search grid.

Search Command Bar

Contains command bar item (menus, dropdowns, etc.) associated with the selected item(s) in the search grid.

Shortcut Keys

Contain windows shortcut keys to invoke a command.

Implementation Types

- Data Model
 - Create and configure controls using configurable CUI Items
 - No programming necessary
- Method
 - Define controls using Client or Server Methods and the Aras API



6 | © 2020 Aras

Implementation Types

The CUI supports two implementation types.

Data Model

Data Model type elements require no programming code and are manipulated by modeling the user interface with configurable items. You will learn how to create and configure data model elements in this session.

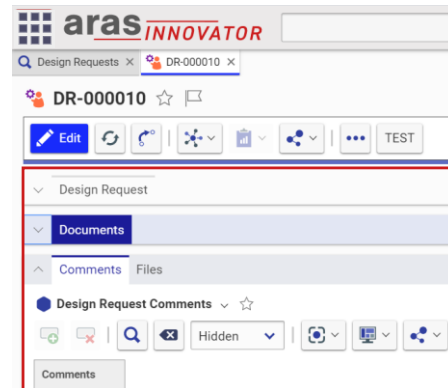
Method

Method type elements are created and populated dynamically with a program method that determines the look and behavior of the control.

Using the IOM, you can define and instantiate a control item or section that displays content based on changing conditions. The contents of the Table of Contents (TOC), which differs depending on the logged in user is a good example of the Method implementation.

Configuring Window Sections Overview

- Define or Modify an ItemView Section
- Define or Modify a Window Section Control
 - Determine Scope
 - Assign Sort Order Ranking
 - Qualify Identity Access and Classification



Configuring Window Sections Overview

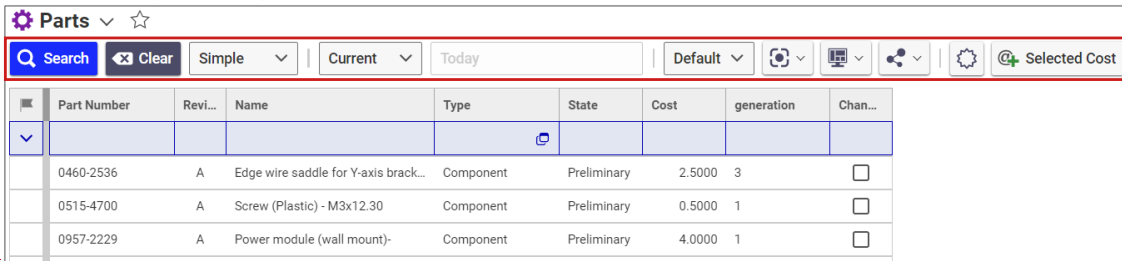
Window Sections are configured by defining (or modifying) an ItemView section which represents an item window. The ItemView section then contains a collection of Window Section Controls.

The appearance and order of the controls is determined by scope (global or ItemType) and a sort order ranking.

Controls are also qualified by identity and optionally item classification so that a window section only appears for a certain group of users based on the current classification of an item.

Configuring Command Bar Sections Overview

- Create or Modify a Command Bar Section
- Define or Modify a Command Bar Item
 - Determine Scope
 - Assign Sort Order Ranking
 - Qualify Identity Access and Classification



The screenshot shows the Aras Parts application interface. At the top, there is a command bar with a search icon, a 'Search' button, a 'Clear' button, and several filters: 'Simple', 'Current', 'Today', 'Default', and a 'Selected Cost' button. Below the command bar is a table with the following columns: Part Number, Revi..., Name, Type, State, Cost, generation, and Chan... The table contains three rows of data:

Part Number	Revi...	Name	Type	State	Cost	generation	Chan...
0460-2536	A	Edge wire saddle for Y-axis brack...	Component	Preliminary	2.5000	3	☐
0515-4700	A	Screw (Plastic) - M3x12.30	Component	Preliminary	0.5000	1	☐
0957-2229	A	Power module (wall mount)-	Component	Preliminary	4.0000	1	☐

Configuring Command Bar Sections Overview

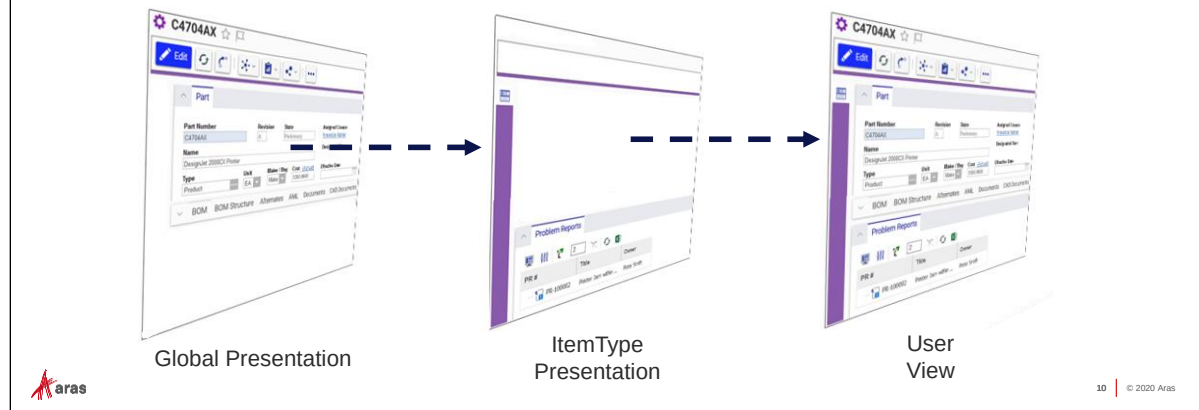
Command Bar sections are configured by defining (or modifying) a Command Bar Section (e.g., toolbar). The section then contains Command Bar Items (e.g., menus, buttons).

The appearance and order of the Command Bar items are determined by scope (global or ItemType) and a sort order ranking.

Command Bar Items can also be qualified by identity and item classification so that a command bar item only appears for a certain group of users based on the current classification of the item.

Reviewing Presentation Configuration Scope

- Global Client Presentation is applicable to all windows and controls
- ItemType Presentations augment the global configuration and can add, remove or replace controls based on the defined ItemType Client Style



Reviewing Presentation Configuration Scope

The user interface that is presented to a user at runtime is a combination of a global and an ItemType client presentation that are "layered" on top of each other. The global configuration dictates the appearance of the UI for all items, while the ItemType presentation can supplement/override the default configuration.

Control Actions

- **Add**
 - Add a CUI element to a Command Bar Section or Window Section.
- **Remove**
 - Remove a CUI item in a Command Bar Section or Window Section.
- **Replace**
 - Replace an existing CUI item with another item.
- **Clear All**
 - Remove all CUI items from a Command Bar Section or Window Section.



© 2020 Aras

Control Actions

Window Section Controls and Command Bar Items are added, removed, replaced, or cleared using an action instruction.

Add

Add a new (or existing) CUI element.

Remove

Remove a previously added item CUI element. Note that the element itself is NOT deleted from the database.

Replace

Substitute one CUI element for another.

Clear All

Remove all previously added CUI elements from a Command Bar Section or Window Section.

Action Ranking

- Each CUI Item Action is assigned a sort order value
- Actions are ranked and then executed in sort order
- Allows for positioning of controls in a section
- Allows for “override” of previous configurations based on rank order



	ItemType	Name	Sort Order	Action
▼	▼			▼
	Button	itemview.itemcommandbar.default.save	128	Add
	Button	itemview.itemcommandbar.default.done	256	Add
	Button	itemview.itemcommandbar.default.discard	450	Add
	Button	itemview.itemcommandbar.default.refresh	512	Add



© 2021 Aras

Action Ranking

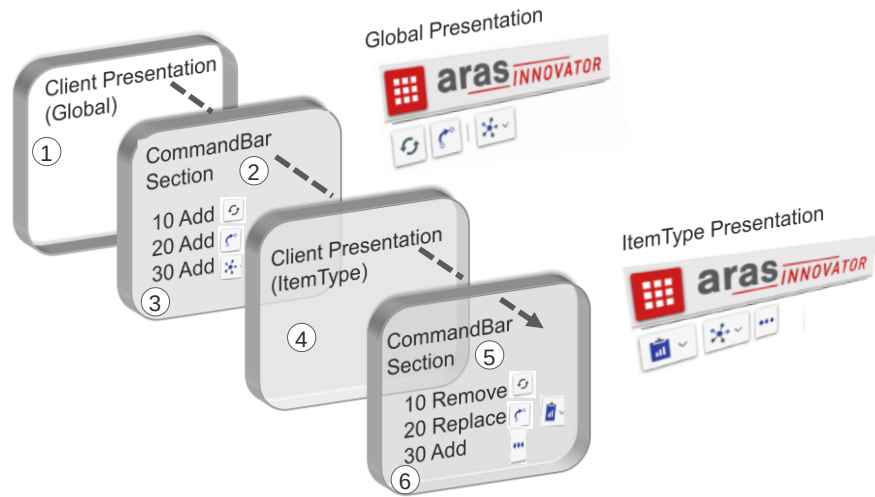
Actions are ranked by a sort order that indicates the order of execution within a section. The lowest order number executes first and so on.

In the case of the Add action, this dictates the position order of a command bar item or window section.

In the example above, a series of buttons are added to a toolbar in the sequence indicated by the sort order number.

Ranking is also useful for replacing or removing items. If the same item is removed after it has been added (higher rank) to a section, the control is hidden from view.

Reviewing Action Execution



12 | © 2020 Aras

Reviewing Action Execution

Action execution and ranking determine the rendering of the final view to a user.

In this example, the global presentation ① contains command bar section ② that contains three buttons that have been added to the section ③. This becomes the default view for all item windows that use this section.

An ItemType client presentation ④ is then configured to override the global command bar section ⑤ by removing, replacing, and adding a new control item ⑥.

Qualifying Identity Access and Classification

- CUI Item Actions can be qualified by Identity
 - The Command Bar Item Action will only execute if the currently logged in user belongs to the declared Identity
- Actions can also be qualified to run based on the value of an associated ItemType classification

Sort Order	Action	For Identity [...]	For Classification
10	Add	World	
32	Add	World	
64	Remove	Engineering	CAD Document
96	Add	Manufacturing	Component
128	Add	CRB	



© 2020 Aras

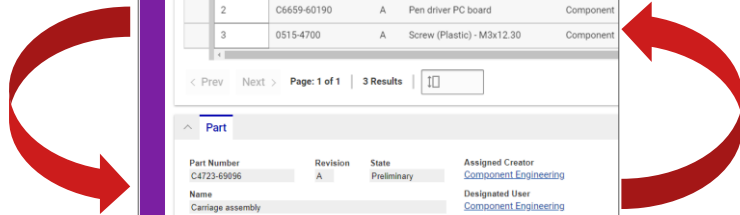
Qualifying Identity Access and Classification

In addition to order ranking, each action can be qualified by an identity and an item classification. At runtime, the qualifiers are evaluated to determine whether the action should be executed for the current user on the current item.

In the example, the first two Add actions will execute regardless of the user logged in to Innovator (World). The Remove action will only execute if the user is a member of Engineering, and the item is classified as CAD Document. The next Add action will only occur for members of Manufacturing on an item classified as Component. The final Add action will occur if user is a member of the CRB identity.

Modifying the Default ItemView

- Open Global Presentation
- Access Default ItemView
- Change Section Sort Order



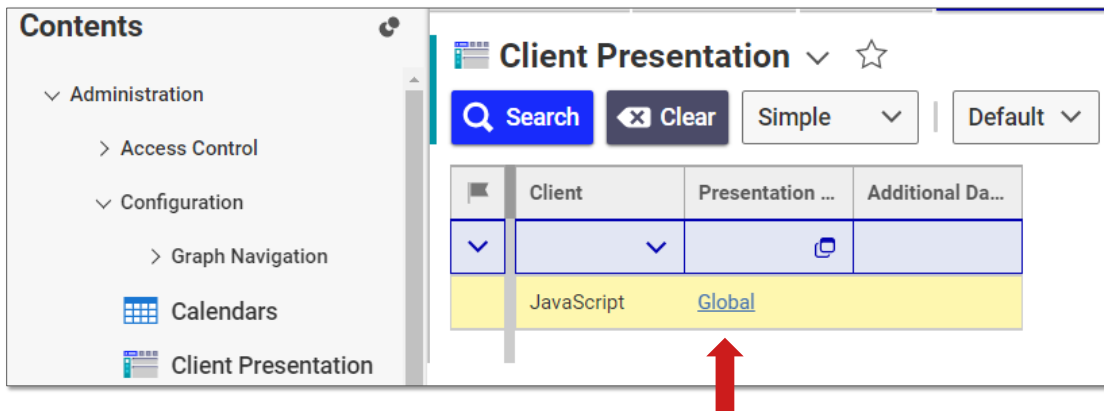
Modifying the Default ItemView

An ItemView contains a collection of Window Section Controls that define the layout of an item window. The global presentation contains a preconfigured ItemView that is used as a foundation for all item windows in the Innovator client.

In this example, we will reorder the window sections to show the relationships grid above the item form control.

Make sure a backup of the existing Aras database has been completed before making any changes to the user interface.

Opening Global Client Presentation



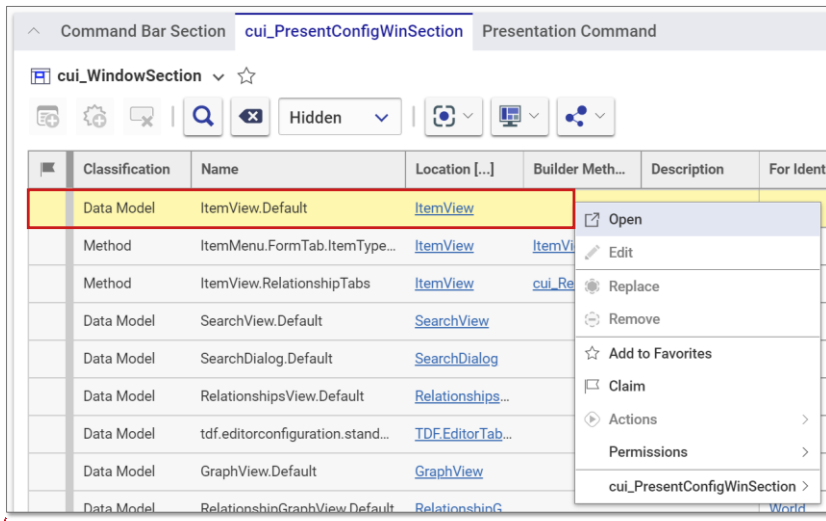
Opening Global Client Presentation

The global client presentation defines the default user interface for all items in the Innovator application. There is only one global presentation for the standard Innovator client, but additional presentations are possible for custom client implementations.

Try It ... Open the Global Presentation

1. Login to Aras Innovator as user admin (password: innovator).
2. Select Administration > Configuration > Client Presentation from the TOC and search for the presentation item.
3. Click the Global link in the Presentation column to open the presentation item.

Accessing Default ItemView



Accessing Default ItemView

The global presentation contains a default ItemView control that is used to render all item windows in the Innovator client.

Try It ... Access the Default ItemView

1. Select the cui_PresentConfigWinSection relationship tab.
2. Select and open the Window Section named ItemView.Default.

The Relationship Presentation Command is useful for advanced programming with CUI. You will find additional information provided by Aras labs at <https://community.aras.com/b/english/posts/using-cui-commands>.

Changing Section Sort Order

The screenshot displays the 'cui_WindowSectionControl' configuration window on the left and a BOM view on the right. The configuration window lists various controls with their sort orders. The BOM view shows a list of parts with their sequence numbers. A red arrow points from the 'ItemView.RelationshipAccordion' row in the configuration window to the BOM view.

Type	Name	Sort Order	Action
Toolbar Control	ItemView.TitleBar	128	Add
Toolbar Control	ItemView.Toolbar	256	Add
Accordion Element Control	ItemView.FormAccordion	384	Add
Tab Container Control	ItemView.FormAccordionTabs	512	Add
Tab Element Control	ItemView.FormTab	640	Add
Form Control	ItemView.Form	768	Add
Accordion Element Control	ItemView.RelationshipAccordion	300	Add
Tab Container Control	ItemView.RelationshipAccordionTabs	320	Add
Shortcuts Control	ItemView.Shortcuts	1152	Add

Seq.	Part Number	Rev.	Name	Type
1	C4704-60117	A	Line sensor lens cover	Component
2	C6659-60190	A	Pen driver PC board	Component
3	0515-4700	A	Screw (Plastic) - M3x12.30	Component

Changing Section Sort Order

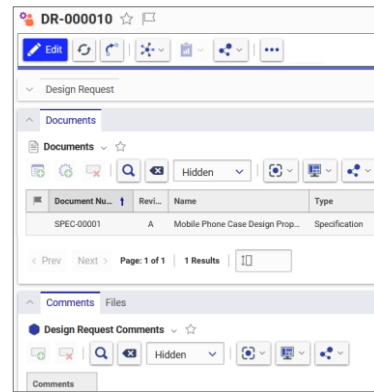
The layout of the item window can be modified by changing the sort order of the Window Section Controls. In this example, the relationship accordion and tab container will be positioned above the form accordion by changing the sort order of the two relationship sections to be less than the sort order of the ItemView.FormAccordion (384).

Try It ... Access the Default ItemView

1. Click the Edit button to make changes to the default ItemView.
2. Change the sort order of the ItemView.RelationshipAccordion to 300.
3. Change the sort order of the ItemView.RelationshipAccordionTabs to 320.
4. Click the Save button to update the configuration.
5. Open any Part item and the change in the layout.
6. To return the layout to its original state, set the ItemView.RelationshipAccordion sort order back to 896 and the ItemView.RelationshipAccordionTabs to 1024 and click the Done button.
7. Close the default ItemView window.

Adding a New ItemView Accordion

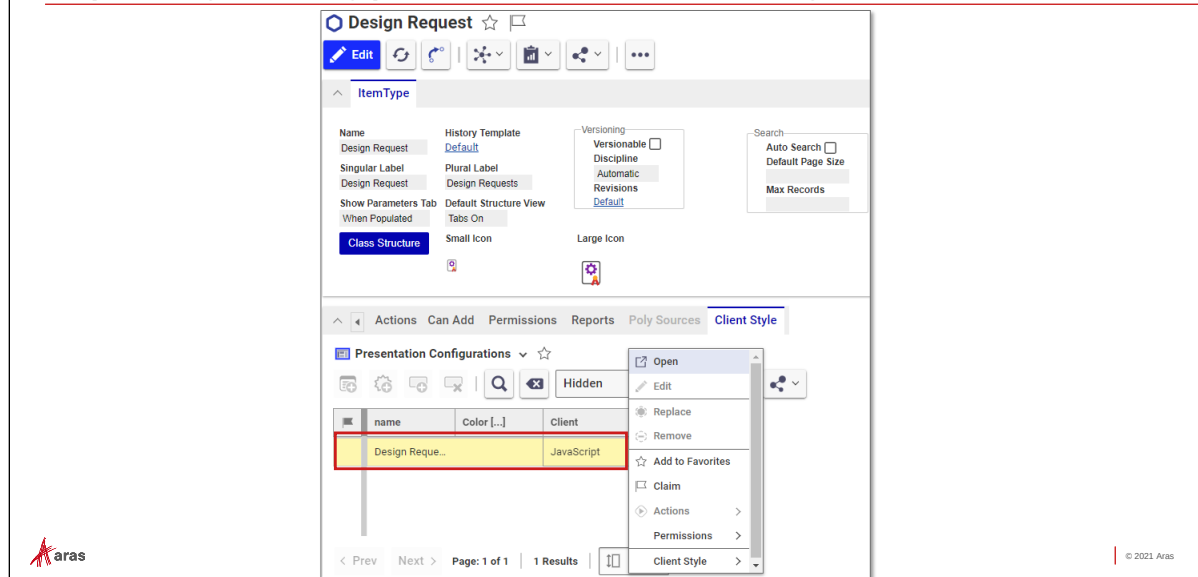
- Open ItemType Presentation Configuration
- Create New ItemView
 - Create New Accordion Element Control
 - Create New Tab Container Control
 - Create New Tab Element Control



Adding a New ItemView Accordion

An ItemView can contain one or more window accordion sections and those sections can be specific to an ItemType. In this example, a new ItemView is created and associated with the Design Request ItemType presentation. A new accordion is then defined that displays Documents that are associated with a Design Request item.

Opening ItemType Presentation Configuration



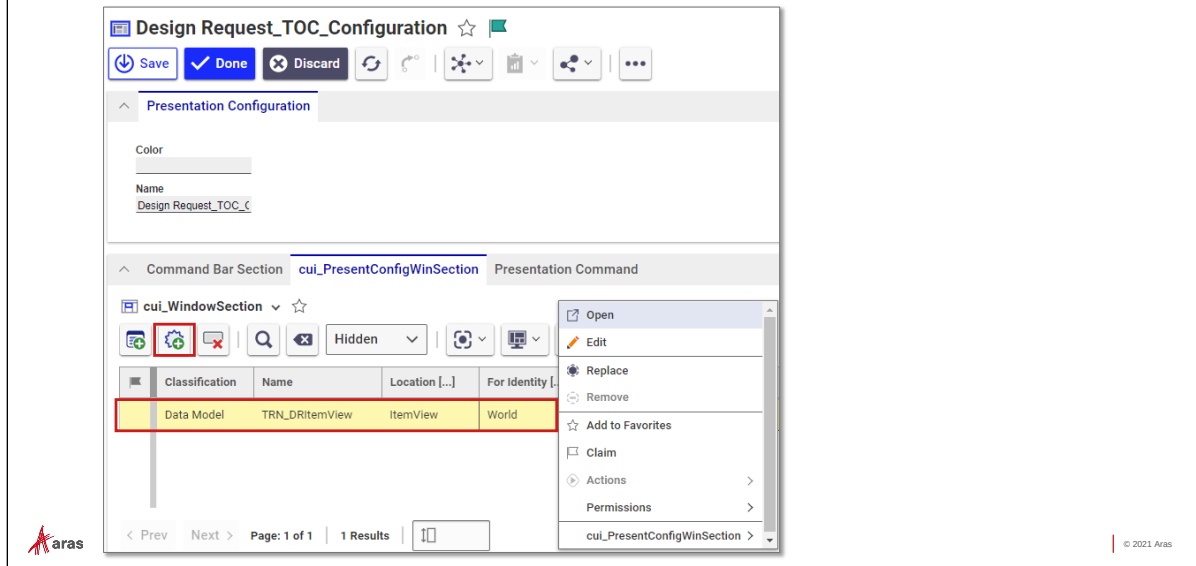
Opening ItemType Presentation Configuration

Each ItemType may contain a presentation configuration that supplements (or overrides) the Global presentation. In this example, we will open and modify the presentation associated with the Design Request ItemType.

Try It ... Open the ItemType Client Style Presentation

1. From the TOC, select Administration > ItemTypes and locate and open the Design Request ItemType template.
2. Choose the Client Style tab, select, and then open the presentation configuration.

Creating New Window Section in the ItemView



Creating New Window Section in the ItemView

A Window Section can be created for the ItemView of an ItemType to supplement the accordions available in the location ItemView contained in the global client presentation. We will create a new Window Section for the location ItemView that contains a new accordion section for Design Request items.

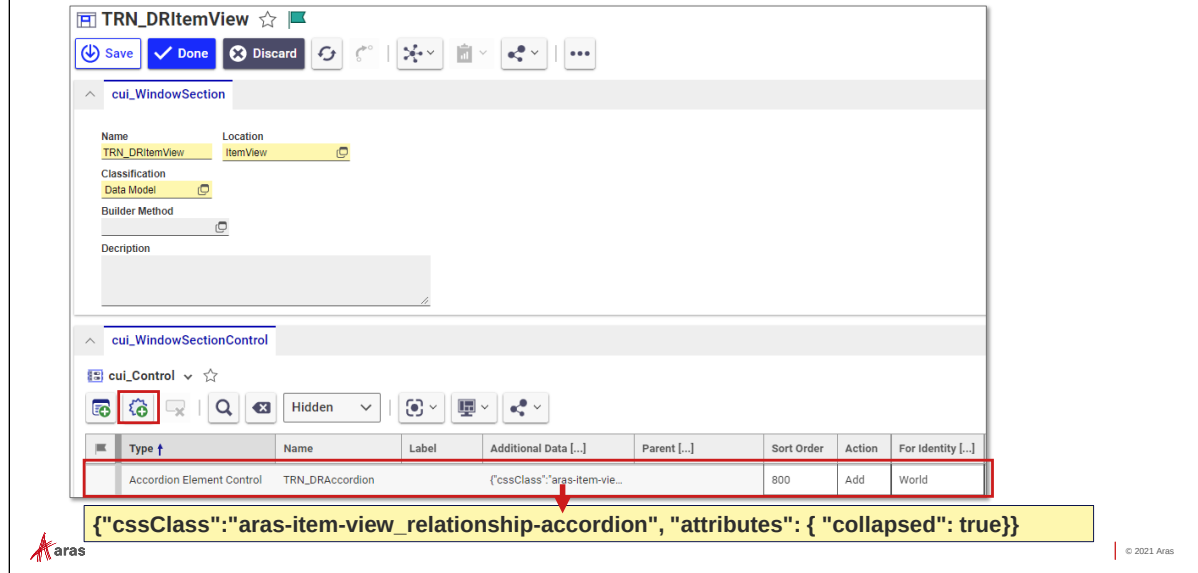
Try It ... Create New ItemView

1. Click the Edit button to make changes to the client presentation.
2. Select the cui_PresentConfigWinSection tab and then click the New cui_WindowSection button to create a new element.
3. Provide the following information in the grid:

Classification	Data Model
Name	TRN_DRItemView
Location	ItemView
For Identity	World

4. Right click and then select open from the context menu to continue.

Creating New Accordion Element Control



Creating New Accordion Element Control

An accordion Window Section Control is represented by the Accordion Element Control type.

Style information about the Accordion Element Control is entered in the Additional Data field using JavaScript Object Notation (JSON). Additional data varies depending on the type of control and is defined internally by Aras development.

In this example, the style is defined as a relationship accordion and the element is displayed in a collapsed state.

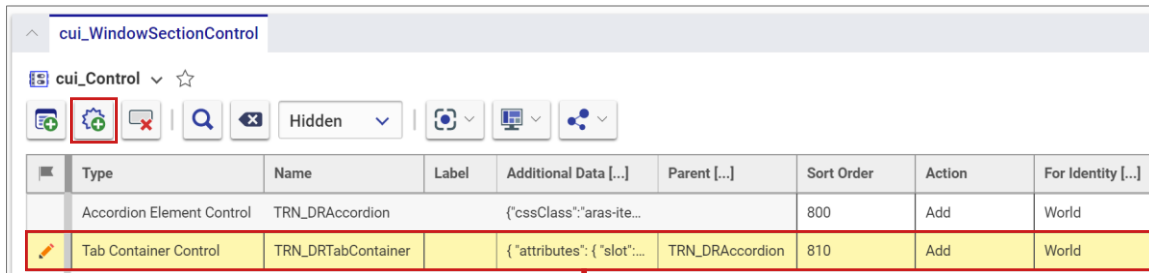
Try It ... Create a New Accordion Element Control

1. Locate the `cui_WindowSectionControl` tab and click the New `cui_Control` button to create a new element.
2. Provide the following information in the grid:

Type	Accordion Element Control
Name	TRN_DRAccordion
Additional Data	<code>{"cssClass": "aras-item-view_relationship-accordion", "attributes": { "collapsed": true}}</code>
Sort Order	800
Action	Add
For Identity	World

3. Click the Save button to create the first window section control.

Creating New Tab Container Control



Type	Name	Label	Additional Data [...]	Parent [...]	Sort Order	Action	For Identity [...]
Accordion Element Control	TRN_DRAccordion		{cssClass:"aras-ite...		800	Add	World
Tab Container Control	TRN_DRTabContainer		{"attributes": {"slot": "...	TRN_DRAccordion	810	Add	World

```
{"attributes": {"slot": "header"}}
```

Creating New Tab Container Control

The Tab Container Control defines the tab element for the accordion control. The Additional Data field provides the style instructions to display the standard tab control. The Parent field determines which accordion control should display the tab element.

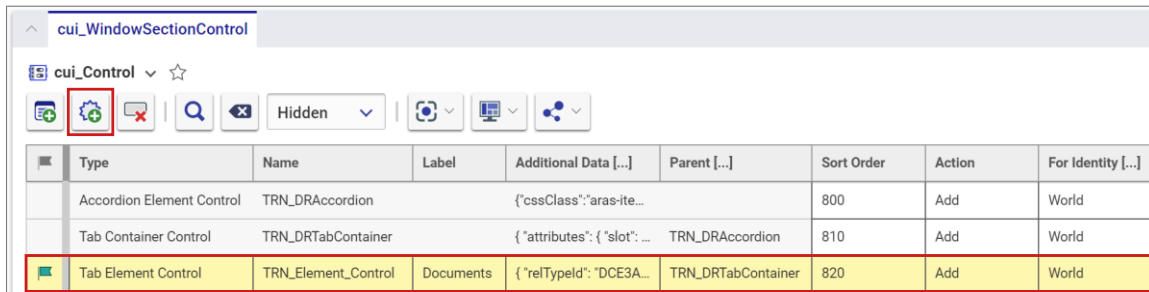
Try It ... Create a Tab Container Control

1. Click the New `cui_Control` button to create a new element.
2. Provide the following information in the grid:

Type	Tab Container Control
Name	TRN_DRTabContainer
Additional Data	{"attributes":{"slot":"header"}}
Parent	TRN_DRAccordion
Sort Order	810
Action	Add
For Identity	World

3. Click the Save button to update the configuration.

Creating New Tab Element Control



Type	Name	Label	Additional Data [...]	Parent [...]	Sort Order	Action	For Identity [...]
Accordion Element Control	TRN_DRAccordion		{ "cssClass": "aras-ite...		800	Add	World
Tab Container Control	TRN_DRTabContainer		{ "attributes": { "slot": ...	TRN_DRAccordion	810	Add	World
Tab Element Control	TRN_Element_Control	Documents	{ "relTypeId": "DCE3A...	TRN_DRTabContainer	820	Add	World

```
{ "relTypeId": "DCE3A7DB64314AA592601D13FFE8B6CE" }
```

Creating New Tab Element Control

The final element that is configured in the accordion is the Tab Element Control that provides instructions to populate the associated form area of the window section. The Parent field must be a configured Tab Container Control that we created in the previous step.

For this example, a RelationshipType that represents Design Request Documents in the training database will be used to display associated Documents for a Design Request item in a new accordion. Use the GUID of the RelationshipType named Design Request Document to assign the relTypeId attribute value in the additional data field.

Try It ... Create a Tab Element Control

1. Click the New cui_Control button to create a new element.
2. Provide the following information in the grid:

Type	Tab Element Control
Name	TRN_DRTabElement
Parent	TRN_DRTabContainer
Label	Documents
Additional Data	{ "relTypeId": "DCE3A7DB64314AA592601D13FFE8B6CE" }
Sort Order	820
Action	Add
For Identity	World

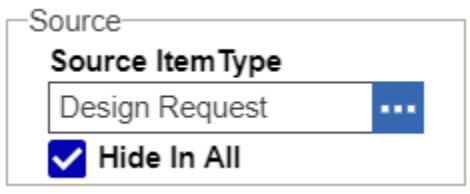
3. Click the Done button to update the configuration and close the Window Section Control tab.

4. Click the Done button to update the ItemType Client Presentation and then close all tabs.
5. Open an existing Design Request Item to view the new configuration.

Optional

To remove the Documents tab from the existing Design Request relationships accordion, locate and edit the Design Requests Document RelationshipType.

Click the Hide In All checkbox in the Source section to remove the tab from the standard relationship accordion.



Adding New SearchView Toolbar Button

- Create Click Method
- Open ItemType Client Style
- Create New Command Bar Section
- Create New Command Bar Button Item



© 2021 Aras

Adding New SearchView Toolbar Button

Command Bar Items allow you to define controls that allow a user to interact with an item. These controls include toolbars, menus, and toolbar buttons.

In this example, we will create a new toolbar button for the Part search command bar that displays the total cost of one or more selected Part items. We will define a new Click Method first that defines the behavior of the button.

Obtaining Selected Items from a SearchView Grid

The screenshot shows the Aras Innovator search grid. The grid has columns: Part Number, Rev..., Name, Type, State, Cost, generation, and Ch... The first three rows are highlighted in yellow, indicating they are selected. The code snippets and arrows illustrate how to access the grid and retrieve selected item IDs.

```
var topWnd =
aras.getMostTopWindowWithAras(window)
```

var myids = grid.getSelectedItemIDs(",")

var grid = topWnd.main.work.grid

Obtaining Selected Items from a SearchView Grid

Before we can define a new button, we need to create a Click Method that totals the cost of the selected Parts in a search grid. The Method needs to access the grid using programming features available to you as a developer.

Search Grid

The search grid is defined with the name `main.work.grid` and is a member of the `Aras GridContainerClass`. To access the grid, you must first get a handle to the top window of the Aras client. A built-in method named `aras.getMostTopWindowWithAras` is available to any Client Method (JavaScript only).

Get Selected Items

Once you have a handle to the search grid you can use the `GridContainerClass` method `getSelectedItemIDs` that returns a collection of item GUIDs separated by a specified delimiter. In this example, the delimiter is the comma (,) character.

Getting an Innovator Instance

Unlike other examples we have seen using Client Methods, an instance of Innovator is not readily available from the button Method context (this) and must be obtained using the reserved client method `aras.newIOMInnovator()`.

Try It ... Create New Click Method

1. Create a new Client Method named `TRN_SelectedPartCost`.
2. Examine and then use the example JavaScript code below to provide the logic.

```
// Access the top window in the Aras Client
const topWnd = aras.getMostTopWindowWithAras(window);

// Access the current search grid of type GridContainerClass
const grid = topWnd.main.work.grid;

// Retrieve GUIDs of selected rows in the search grid
const myids = grid.getSelectedItemIDs(",");

// Split the collection into an array for easy handling
const myarray = myids.split( ",");

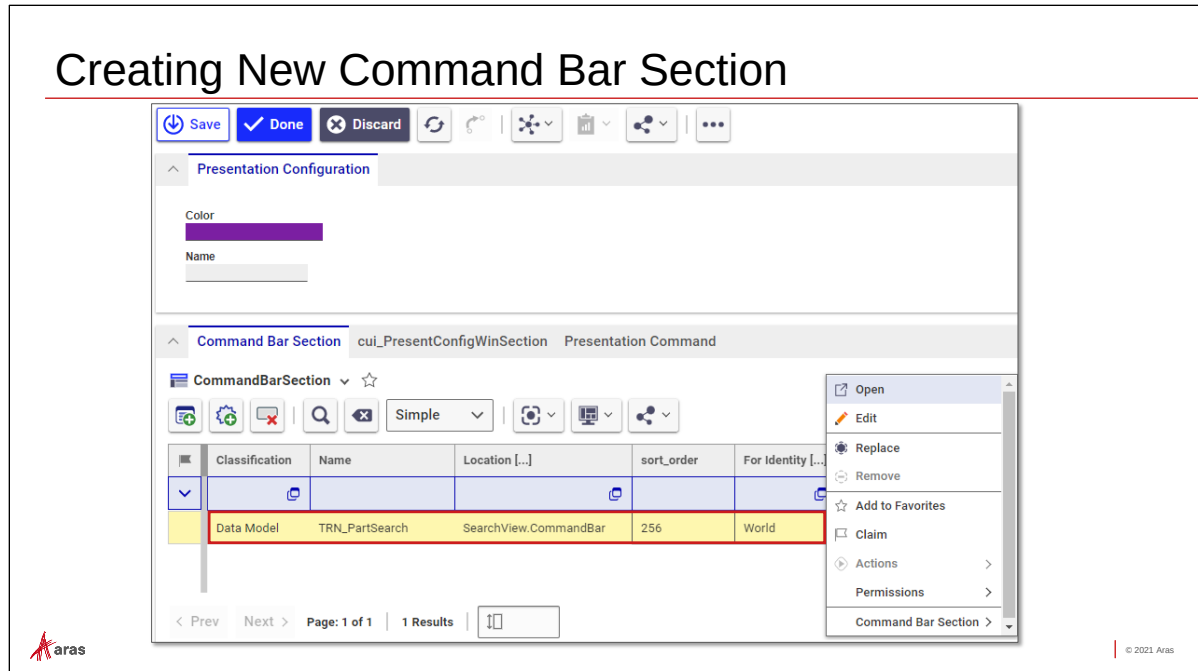
// Access an instance of the Innovator class
const inn = aras.newIOMInnovator();
var itm;
var totalcost=0;

// Get each item in a loop
for(i = 0; i < myarray.length; i++){

    itm = inn.getItemById("Part", myarray[i]);
    cost = itm.getProperty("cost","0");
    totalcost = Number(totalcost) + Number(cost);
}

// Show success prompt
aras.AlertSuccess("Total Cost: " + totalcost);
```

Creating New Command Bar Section



Creating New Command Bar Section

The item toolbar is presented as a Command Bar Section using the SearchView.CommandBar location. We will add a new section for the Part ItemType that supplements the existing toolbar configured in the global client presentation.

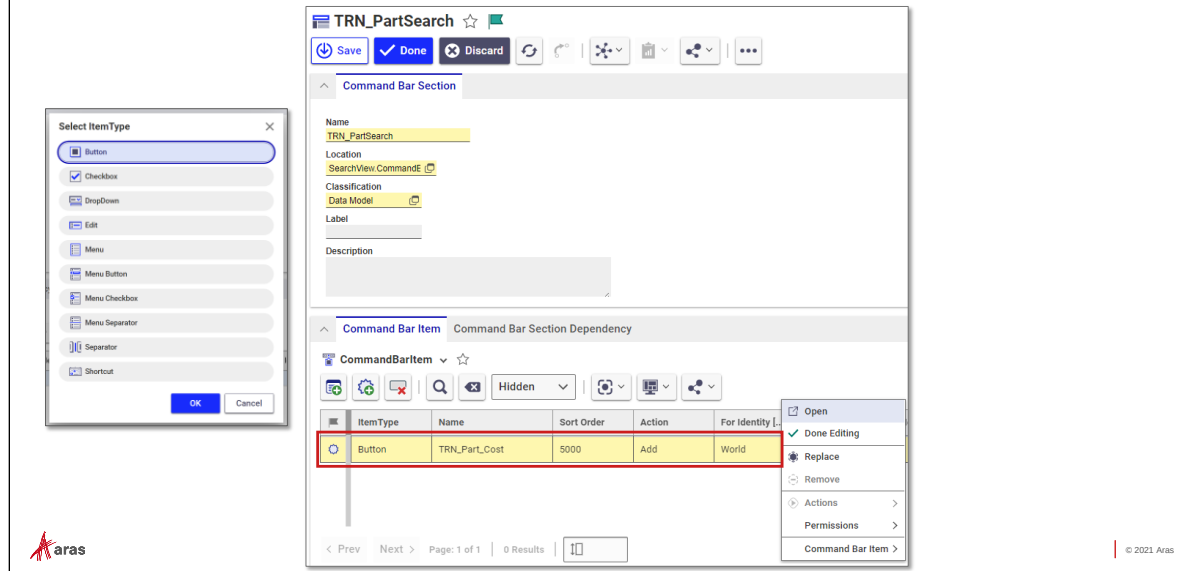
Try It ... Create New Command Bar Section

1. From the Part ItemType, open the client style presentation and click the Edit button to make changes.
2. Click the New Command Bar Section button and enter the following into the grid:

Classification	Data Model
Name	TRN_PartSearch
Location	SearchView.CommandBar
Sort Order	256
For Identity	World

3. Select and open the new Command Bar Section to continue.

Creating New Command Bar Button Item



Creating New Command Bar Button Item

Toolbar buttons are represented as Command Bar Items in the CUI. Each button is positioned using sort order ranking. In this example, the new button will be defined in an ItemType client presentation that supplements the global presentation. The combination of both presentations will determine the button placement order in the toolbar.

Try It ... Create New Command Bar Item

1. Locate the CommandBarItem tab and click the New Command Bar Item button to enter the following into the grid:

ItemType	Button (select from dialog)
Name	TRN_Part_Cost
Sort Order	5000
Action	Add
For Identity	World

2. Select and open the new Command Bar Item to continue.

Configuring Command Bar Button

The screenshot shows the configuration window for a command bar button named **TRN_Part_Cost**. The **Button** tab is active. The configuration fields are as follows:

- Name:** TRN_Part_Cost
- Label:** Selected Cost
- ARIA Label:** (empty)
- Tooltip Template:** Sum selected Part cost
- Command Alias:** (empty)
- Additional Data:** (empty text area)
- Init Method:** (empty)
- Click Method:** TRN_SelectedPartCost
- Image:** Select an image...
- Include Events:**
 - Select Item in TOC
 - Select Item in Grid
 - Update TearOff Window State
 - Update Preferences
- Additional Image:** Select an image...

Configuring Command Bar Button

Provide additional details about the button including a label, image, and the Click Method we created previously.

Try It ... Create New Command Bar Button

1. Enter **Selected Cost** in the Label field. This will be displayed as button text. You can also enter an ARIA (Accessible Rich Internet Applications) Label that is not displayed but used with assistive devices (e.g. text to speech).
2. Enter **Selected Cost** in the Tooltip Template field. This text will display when a user hovers over the button with the mouse.
3. Enter **TRN_SelectedPartCost** in the Click Method field. This is the Method created previously to provide the action for the button when clicked.
4. Click the **Select an image...** link to choose an icon for the button.
5. Click the **Done** button to save changes to the command bar button and close the tab.
6. Click the **Done** button to save changes to the command bar section and close the tab.
7. Click the **Done** button to save changes to the ItemType client style and close the tab.
8. To test the button, reopen the Part search grid and select several Parts that have cost value.
9. Click the new **Selected Cost** button to display the total cost of the selected parts.

Defining Initialization Methods

- Dynamically Modify a CUI Control
 - Hidden
 - Disabled
- Include Events
 - Select Item in TOC
 - Select Item in Grid
 - Update TearOff Window State
 - Update Preferences
 - Search State Change
 - Select Favorite
 - Update Notifications



© 2020 Aras

Defining Initialization Methods

Command Bar Items can be configured to execute an Initialization Method that can be used to modify the visibility or state of an existing control.

Two primary style attributes are available that can be returned from an Initialization Method:

hidden: if set to true the control is not displayed, if false the control is visible

disabled: if true the control is visible but not active, if false the control is active

Include Events

An Initialization Method can also be passed additional information about the client in the form of arguments. You must select the appropriate event(s) depending on how the Method will be used.

In the next demonstration, we will select the Search State Change event so that the Initialization Method will be aware of the current search mode in the search grid.

Select Item in TOC

An Item category is selected in the Table of Contents (which can be used to display and populate a TOC Popup Menu).

Select Item(s) in Grid

One or more items is highlighted in the current search grid.

Update Tearoff Window State

An item window is opened, saved, claimed/unclaimed or promoted.

Update Preferences

A user updates their preferences from the User menu.

Search State Change

A user changes the search mode on the search grid.

Select Favorite

A user selects/deselects an item as a favorite.

Update Notifications

A user notification is received by the client.

Split Screen

A user activates the Split Screen View.

Navigation Panel Visibility

The visibility of the navigation panel is modified.

Update SSVC Files

A user updates their Secure Social and Visual Collaboration files.

Update Favorites

A user updates the Items marked as favorites.

Arguments

Arguments to the initialization Method will vary depending on which Include Events are selected for the control.

All controls pass event argument information to a Method in a collection named options.

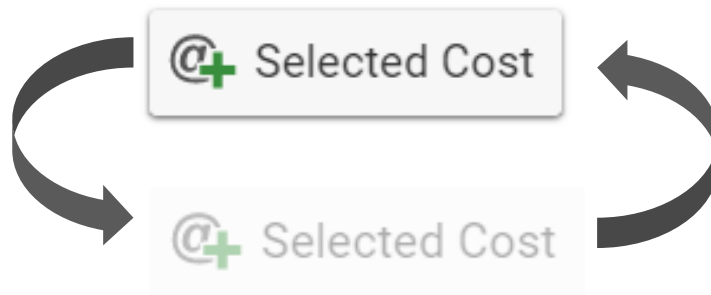
Common item options include:

- options.itemId
- options.itemTypeName
- options.item_classification

Other arguments can be passed to the Method based on the type of Include Event.

Disabling a Control

- Define an Initialization Method
- Assign to Control Init Method

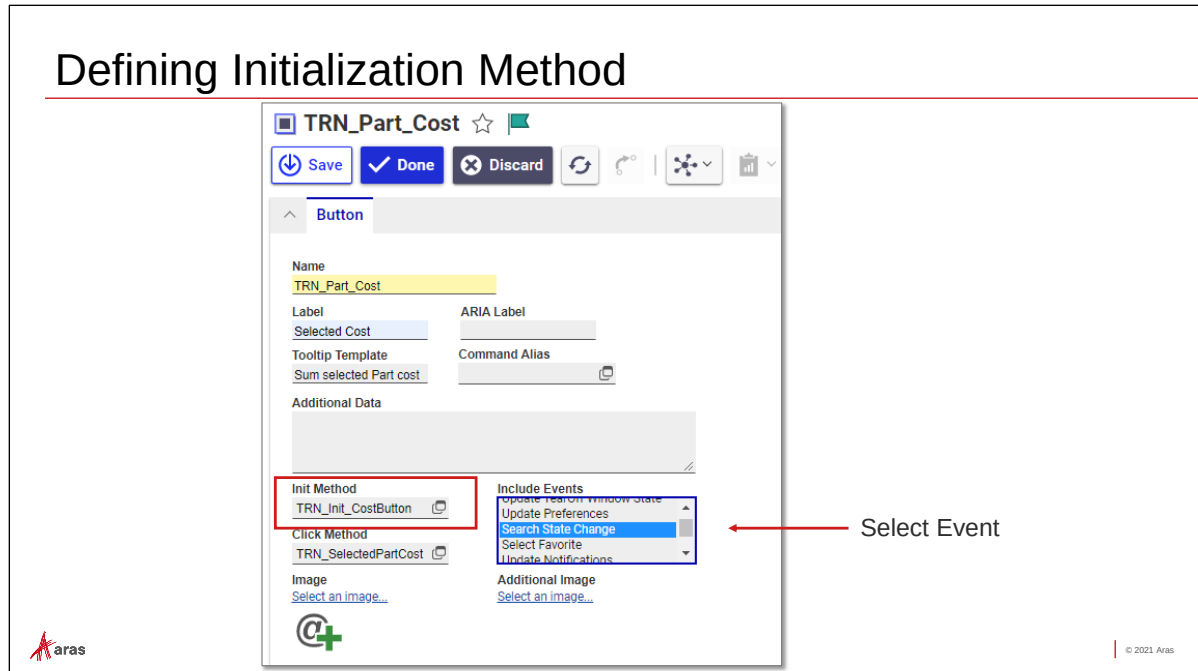


© 2020 Aras

Disabling a Control

In this example, we will disable the Selected Cost command button we created earlier in this unit if a user changes the search mode to anything except Simple. This requires an initialization Method be defined for the control as well as an Include Event.

Defining Initialization Method



Defining Initialization Method

The Search State Change Include Event provides an argument to the Initialization Method that indicates the current search mode. A new Method named TRN_Init_CostButton is created to enable/disable the control.

The argument variable `currentSearchMode.name` contains the current search mode name (Simple, Advanced, AML, Hidden).

Try It Define Initialization Method

1. Create a new Client Method named TRN_Init_CostButton that includes the following logic and code:

Logic:

If the `currentSearchMode.name` is equal to Simple, then set and return the disabled attribute equal to false, otherwise set it to true.

Code:

```
if (!currentSearchMode || currentSearchMode.name == "Simple") {
    return {disabled: false};
}
else
    return {disabled: true};
```

2. Locate and edit the TRN_Part_Cost command bar item created earlier in this unit.
3. Choose **Search State Change** for the Include Events field.
4. Assign the Client Method above to the Init Method field.
5. Save changes and then test the behavior in the Part search grid. (You may need to Clear Client Meta Data Cache or log out and in again to see the change).

Summary

In this session, you learned how to make changes to the Aras Innovator client using the Configurable User Interface (CUI). You should now be able to:

- ✓ Understand the scope of the Configurable User Interface (CUI)
- ✓ Identify CUI Window Section and Command Bar Controls
- ✓ Identify Command Bar Item actions
- ✓ Understand the CUI Item presentation hierarchy
- ✓ Rearrange Window Sections
- ✓ Add a new Item Accordion
- ✓ Add a new Command Bar Item
- ✓ Disable a Command Bar Item

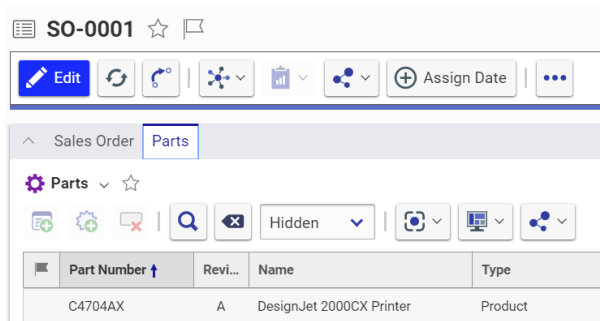
Lab Exercise

Goal

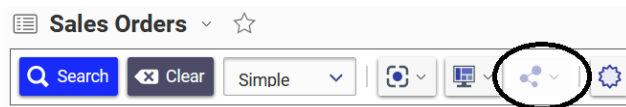
Define a new tab control in an existing accordion and disable a control based on programming logic.

Scenarios

1. A request has been made to make the following adjustments to the client user interface:
Add a new tab control to the top Sales Order accordion to show relationships to associated Parts. (Sales Order Part).



2. Disable the Share menu control on the Sales Orders search command bar if a user is not an administrator.



Steps:

Add a New Tab Control

1. Locate and open the Sales Order ItemType.
2. Access the Client Style tab and then open the existing Sales Order presentation configuration.
3. Select the cui_PresentConfigWinSection tab and create a new section named TRN_SOItemView for the classification Data Model, Location ItemView and Identity World.
4. Open the new section for edit and create a new cui_WindowSectionControl using the following criteria. You can obtain the GUID of the relTypeId by locating the Sales Order Part RelationshipType template and copying the Id to the clipboard.

Type	Tab Element Control
Name	TRN_SOTabParts
Parent	ItemView.FormAccordionTabs
Label	Parts
Additional Data	{"relTypeId": "274D1842F5B04B42BB1944B3F083A863"}
Sort Order	1024

Action	Add
For Identity	World

5. Remove the Parts tab from the standard relationship tabs by locating and opening the Sales Order Part ItemType. Click the Hide All button checkbox and save changes.

Disable Command Bar Item

1. Create a new Client Method named TRN_Init_ShareButton. Use the following logic to write the code:
 - a) Check the value of the options.itemTypeName argument. If it is not equal to "Sales Order" then return from the program.
 - b) Check to see if the current user is an administrator using the Aras Object function aras.isAdminUser().
 - c) If the user is an administrator, then return the attribute disabled set to false. Otherwise, return the attribute set to true.
2. Open the Global client presentation and locate the command bar section named **searchview.commandbar.default**.
3. Open the command section and locate the command bar item named **commonitems.commandbar.sharemenu**.
4. Edit the command bar item and assign the Method you created to the Init Method field.
5. Save all changes and then log out and in again as an admin and non-admin to test the configuration.



Unit 12 Using Federated Properties

Overview

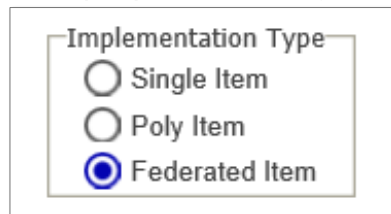
In this unit, you will learn how to use and store data in an external resource while still presenting the information to the user in the standard client user interface. Federation allows you to integrate an Aras Innovator solution with data that does not reside in the Innovator database.

Objectives

- Reviewing Federated Items and Properties
- Discuss Federated Data Sources
- Configuring a Federated Property
- Populating a Federated Property Value
- Saving a Federated Property Value
- Configuring a Federated ItemType

Federated Items and Properties

- Allow data to be accessed and shared with other systems
- Federated Properties display in grids and forms, but the value of the data may come from another system
- Federated Items have at least some of their properties stored in other database or system
- Data is provided to properties using server events



Federated Items and Properties

Federated Items and Properties are useful for allowing integration of data from other systems to be incorporated into a solution.

When you mark an ItemType as Federated you are indicating that at least one property of the Item will not be stored in the Innovator database. Remember that an ItemType that is created in Aras Innovator represents a table in the database and each property represents a column. In opposite there is no column in our database for federated properties.

With federation, you can seamlessly display and edit data and decide when and where that data should be retrieved or stored.

Federated Options

- **Mixed System**
 - Some of the data is stored in a remote system
 - ItemType is Federated
 - Certain properties are marked as Federated
 - Federated properties do not have a column in the ItemType database table
- **Separate System**
 - All data is stored in a remote system
 - ItemType is Federated
 - All properties of the ItemType are Federated
 - No data rows are created in the ItemType database table



Federated Options

You have two options when designing federation into a solution. Should some of the data reside in the Innovator database or should all of the data be located in an outside resource.

In the “separate system” option, no rows (Items) are created in the Innovator database and all data population must be accomplished with custom logic. System properties exist but are not used.

In a “mixed” environment some basic data may be stored in the system (e.g. Item ID , created_on, created_by) and data which currently resides in an outside resource can be retrieved (and updated) but not saved to the Innovator database.

Reviewing Possible Data Sources

- Web Services
 - Web Service Configuration
- SQL
 - External Relational Database
- IOM
 - Using API calls to remote system



Reviewing Possible Data Sources

In this unit, you use an external SQL Server database to retrieve and update data. However, any source of data that may be accessed through a Method can be used to populate or modify a federated property value of an Item, depending on the business requirements.

Working with a Federated Property Value - Mixed

- ItemType server events are available to populate data:
 - onAfterGet – used to populate Federated property
 - onAfterDelete – removes remote data when Item is deleted
 - onBeforeAdd – adds remote data to a remote system when Item is added
 - onBeforeUpdate – changes remote data on an Item edit



Federated Property Value – Mixed

To support federation, you respond to a set of server events on the ItemType. These events allow you intercept the AML messages being passed to and from the Innovator Server to create, retrieve, update and delete external data.

Working with a Federated Property - Separate

- All Item logic is replaced with developer code to obtain and return data to the remote system using:
 - onGet – replaces the standard retrieve logic
 - onAdd – replaces the standard add logic when a new Item is added
 - onDelete – replaces the standard delete logic
 - onUpdate – replaces standard edit logic

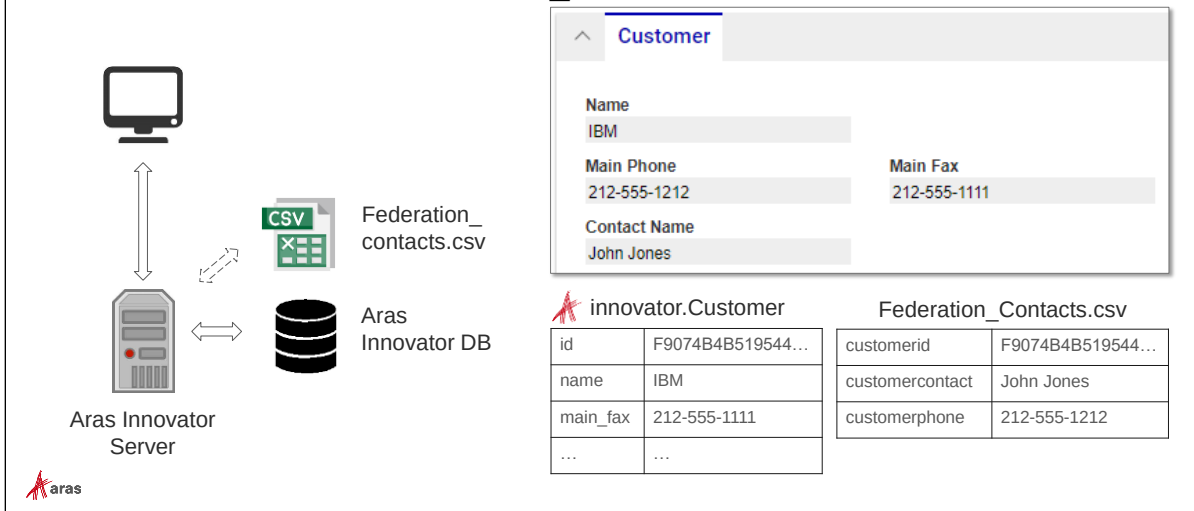


Federated Property Value – Separate

If no data is required to be stored in the Innovator database, you are required to replace the current retrieve/update logic with your own. These events completely replace the add, edit, get and delete standard behavior in Innovator. You are responsible for creating appropriate AML messages to and from the server.

Federated Example

- Customer table and Federation_Contact.csv



Federated Customer Example

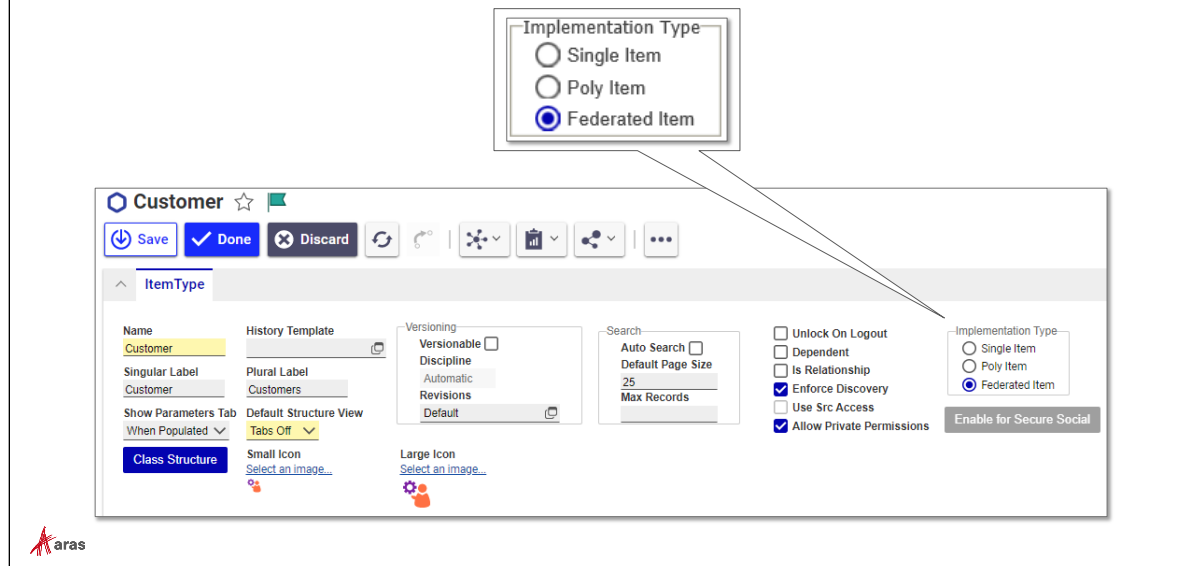
The standard solution database contains an ItemType named Customer that stores information about each customer including contact data. In this example, the information about the customer contact name and phone number will be stored in an external csv-file. This could be an existing sheet that is used by other applications and shared by the Aras Innovator.

The **Customer** ItemType will become a "mixed" Federated Item with the following data stored in a csv-file (Federation_Contacts.csv): The **customerid** will use the Aras Innovator Item **id** property to maintain a link between the Aras database and the csv-file. The columns **customername** and **customerphone** will store the federated property values of a Customer Item.

The mapping between the property name of Aras Customer ItemType and the csv-file columns is presented in following table:

Aras ItemType Customer property name	column name in csv-file
id	customerid
contact_name	customername
main_phone	customerphone

Configuring a Federated ItemType



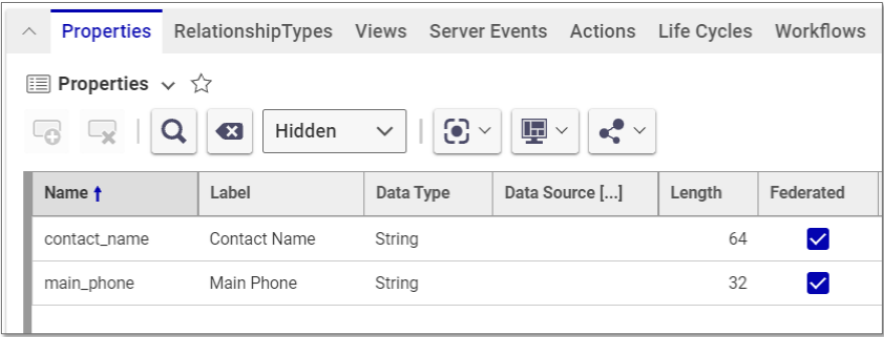
Configuring a Federated ItemType

Edit the ItemType and select the Federated Item radio button in the Implementation Type group.

Selecting this option has no effect on system behavior but is simply used as a marker or reminder that some or all of the data for this Item will be stored in a remote location. Associating Methods with the server events discussed on the previous pages dictates if an item or property is federated.

Configuring a Federated Property

- Enable Federated Property



The screenshot shows the 'Properties' tab in the Aras configuration tool. It features a table with columns: Name, Label, Data Type, Data Source [...], Length, and Federated. Two properties are listed: 'contact_name' and 'main_phone', both with 'String' data types and 'Federated' checkboxes checked.

Name ↑	Label	Data Type	Data Source [...]	Length	Federated
contact_name	Contact Name	String		64	☒
main_phone	Main Phone	String		32	☒



Configuring a Federated Property

Enable Federation using the "Federated" setting checkbox.

In this example, the `contact_name` and `main_phone` properties would no longer be stored in the Aras database.

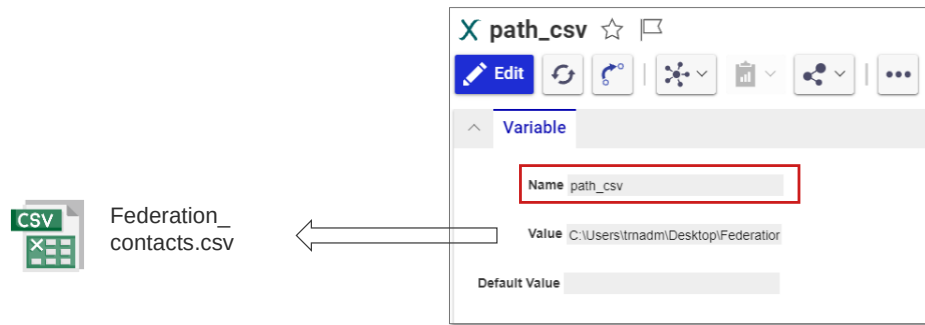
Warning

Choosing this option will remove the database columns for these properties from the Customer ItemType table in the Innovator database.

Do not change the datatype in the student database, as we will need these columns later.

Integrating variables in code

- Storing the path to the csv-file (same Server as Aras installation) in Value field of Variable Item



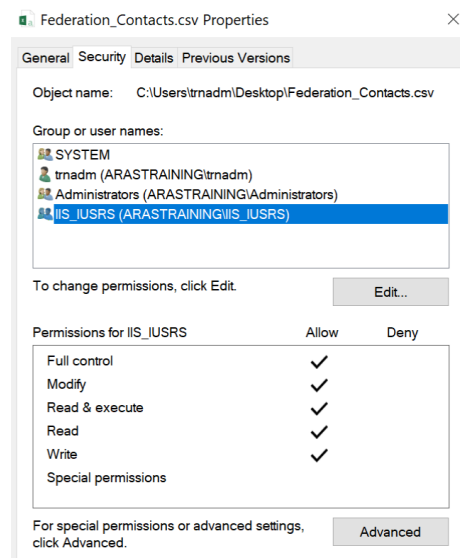
Working with Variables

Variable Items can be used to store values, which may be accessed by custom programming code. In this training example a Variable Item is used to specify the absolute path to the location of the csv-file (Federation_Contacts.csv). The method code examples later in the book used to retrieve or modify federated property values, obtain the path to the csv-file from the Value field of a Variable Item named "path_csv". The advantage is that the path to the file can be managed by one Variable Item centrally and does not need to be defined in multiple Method Items individually.

Therefore create a new Variable Item. The Item needs to be named "path_csv", as the method code later will access a Variable Item with the mentioned name. In the Value field of the Variable Item the path to the Federation_Contacts.csv needs to be specified.

Note

For the training example the Federation_Contacts.csv file needs to be copied to a location of the server, where the Aras Innovator is installed. Hence, its realization requires a local Aras installation. If the file is copied to a Windows specific User folder, the IIS_IUSRS needs to have reading and writing rights. In case missing, the file properties need to be adapted to add the IIS_IUSRS.



Accessing Federated Property values

- The retrieval and modification of federated property values is accomplished through Server Event Methods

Name ↑ 3	Method T...	V..	execution_all...	Template [...]	Comments	Event ↑ 1
_federation_get_csv	CSharp	7	World			onAfterGet
_federation_add_csv	CSharp	5	World			onBeforeAdd
_federation_delete_csv	CSharp	7	World			onBeforeDelete
_federation_update_csv	CSharp	6	World			onBeforeUpdate

← Display in Grid/ on Item
 ← Creation of Item
 ← Deletion of Item
 ← Update of Item

Server Event Method

Server Event Methods must be used to populate or modify a federated property value. They must be attached to the ItemType for whose Items they should establish a connection to the remote system – for the training example the csv-file.

Populating a Federated Property

To populate a federated property, the **onAfterGet** server event must be used.

Note that retrieving data may return more than one database row so your Method code should expect and handle this situation. Remember that federated property data can appear in relationship grids and the main search grids.

In this example, the Customer Item is being populated with external data from a csv-file. The `setProperty` method is used to set the appropriate values based on data retrieved from the external data source.

C#

```
// get path to csv file from Variable Item with the name "path_csv"
Item variablePath = this.newItem("Variable", "get");
variablePath.setProperty("name", "path_csv");
Item pathItem = variablePath.apply();
string path = pathItem.getProperty("value", "");
if (path=="")
{
    return this.getInnovator().newError("Path to csv file needs to be
defined on Variable item with name path_csv");
}

// read all lines of csv file and store them in dictionary
// customerid = key, customercontact and customerphone = values
var rows = new Dictionary<string, Tuple<string, string>>();
using (var reader = new StreamReader(path)) //StreamReader to read file
{
```

```

        while (!reader.EndOfStream)
        {
            var line = reader.ReadLine();
            var values = line.Split(',');
            rows.Add(values[0], new Tuple<string, string> (values[1],
values[2]));
        }
        reader.Close();
        reader.Dispose();
    }
    // loop through Collection and get federated property values for each
    Item from dictionary
    int count = this.getItemCount();
    for (int i = 0; i < count; i++)
    {
        Item idx_itm = this.getItemByIndex(i);
        string id = idx_itm.getID();
        if (rows.ContainsKey(id))
        {
            var props = rows[id];
            idx_itm.setProperty("contact_name", props.Item1);
            idx_itm.setProperty("main_phone", props.Item2);
        }
    }
    return this;

```

Adding New Data to a Remote System

To add a new data row to the external csv-file, you can use the **onBeforeAdd** or **onAfterAdd** server events.

In this example, a new row will be created in the csv-file (Federation_Contacts.csv).

C#

```

// get path to csv file from Variable Item with the name "path_csv"
Item variablePath = this.newItem("Variable", "get");
variablePath.setProperty("name", "path_csv");
Item pathItem = variablePath.apply();
string path = pathItem.getProperty("value", "");
if (path=="")
{
    return this.getInnovator().newError("Path to csv file needs to be
defined on Variable item with name path_csv");
}
//read out id and federated property values of new Item and append new
//row in csv file using a StreamWriter
using (StreamWriter streamw = File.AppendText(path))
{
    streamw.WriteLine($"{this.getID()},{this.getProperty("contact_name", "")}
,{this.getProperty("main_phone", "")}");
    streamw.Close();
    streamw.Dispose();
}
return this;

```


Updating Changes to the Remote System

To update an existing row in the external csv-file, the **onBeforeUpdate** or **onAfterUpdate** server events can be used to trigger this Method when an existing Item is saved.

C#

```
// get path to csv file from Variable Item with the name "path_csv"
Item variablePath = this.newItem("Variable", "get");
variablePath.setProperty("name", "path_csv");
Item pathItem = variablePath.apply();
string path = pathItem.getProperty("value", "");
if (path=="")
{
    return this.getInnovator().newError("Path to csv file needs to be
defined on Variable item with name path_csv");
}

//update federated property values on Item: append row if Item does not
//exist in file, otherwise replace values
string id = this.getID();
List<string> lines = new List<string>();
bool added = false;
using (var reader = new StreamReader(path))
{
    while (!reader.EndOfStream)
    {
        var line = reader.ReadLine();
        var values = line.Split(',');
        if (values[0].Contains(id))
        {
            values[1]=this.getProperty("contact_name", "");
            values[2]=this.getProperty("main_phone", "");
            line = string.Join(",", values);
            added = true;
        }
        lines.Add(line);
    }
}

if (!added) // adds a row if Item does not exist in the csv file
{
    lines.Add($"{id},{this.getProperty("contact_name", "")},{this.getPropert
y("main_phone", "")}");
}

File.WriteAllLines(path, lines);
return this;
```

Removing Data from the Remote System

The **onBeforeDelete** or **onAfterDelete** server events can be used to allow deletion of data from an external resource.

C#

```
// get path to csv file from Variable Item with the name "path_csv"
Item variablePath = this.newItem("Variable", "get");
variablePath.setProperty("name", "path_csv");
Item pathItem = variablePath.apply();
string path = pathItem.getProperty("value", "");
if (path=="")
{
    return this.getInnovator().newError("Path to csv file needs to be
defined on Variable item with name path_csv");
}

//get ID of Item and use it to delete row of matching customerid in csv
//file
string id = this.getID();
List<string> lines = new List<string>();
using (var reader = new StreamReader(path))
{
    while (!reader.EndOfStream)
    {
        var line = reader.ReadLine();
        var values = line.Split(',');
        if (!(values[0].Contains(id)))
        {
            lines.Add(line);
        }
    }
}
File.WriteAllLines(path, lines);
return this;
```

Summary

In this unit, you learned how to use Federated Items and Properties to integrate data external to the Innovator database.

You should now be able to:

- Configure a Federated Property
- Populate a Federated Property Value
- Save a Federated Property Value
- Configure a Federated ItemType

Lab Exercise

Goal

Purchase Order data for each Sales Order needs to be maintained in a separate csv-file. The csv-file "Federation_SalesOrder.csv" should be used to store the property values for the Shipdate and Purchase Order Number.

You should add (or update) the csv-file with the data entered by the user on the Sales Order Item (actions add, update, get, delete). Use the following Sales Order properties to map the data to the corresponding csv-file columns:

Aras Property	csv-file column
id	salesorderid
ship_date	shipdate
po_number	ponumber

Steps

1. Make a copy of the 4 Method Items provided in your student database to federate a Customer property. These are the Server Event Method code examples shown in this unit.
 2. Adjust the Methods to use the columns of Federation_SalesOrder.csv and Aras properties of ItemType Sales Order described in the table above.
 3. Attach each Method to an appropriate Server Event on the Sales Order ItemType
 4. Activate the *Federated* checkbox for the **ship_date** and **po_number** properties of ItemType Sales Order. What happens to any existing data in the Aras database?
-
5. Copy the "Federation_SalesOrder.csv" to a location on the server, where the Aras Innovator is installed. Adapt the file properties (in Windows) to allow the IIS_IUSRS to access and modify the csv-file.
 6. Create a new Variable Item and name it "path_salesorder_csv". In the Value field of the Variable Item specify the path to the csv-file.
 7. Create/update/read/delete a new Sales Order and test the configuration. Why will these Methods only work for Sales Orders?
-



Unit 13 Integrating a .NET Application

Overview

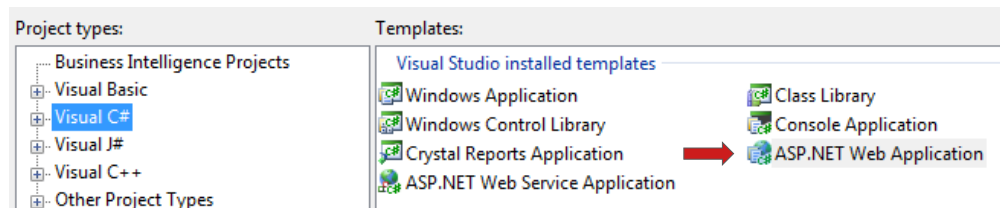
In this unit, you will learn how to install and reference the IOM in a .NET application to provide Innovator services to an outside application. As an alternative, you can also send standard SOAP message requests to Innovator Server and process the responses.

Objectives

- Using the IOM in a Visual Studio Project
- Calling Methods from a .NET Application

Using the IOM in Visual Studio

- Open the ASP .Net Web Application



Using IOM in Visual Studio

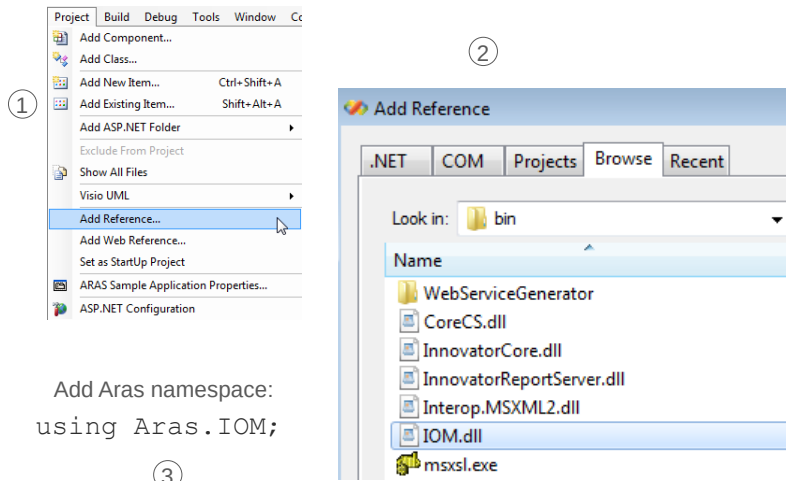
So far, in this course you focused on using the Aras Innovator standard client as a way for an end user to gain access to the system.

In this unit, you look at alternate ways to interact with the Innovator Server by creating an ASP.NET application that uses the IOM to interact with the system.

In this example, you'll use Visual Studio to create a simple ASP .NET web application that will interact with the Innovator Server.

A starting application has been provided, that works with the Sales Order items used in your lab exercises.

Adding IOM Reference



To Add IOM Reference to a Visual Studio Project

To gain access to the classes and methods we have discussed in this course requires adding a reference to the IOM.dll.

The IOM SDK has been created for use in developing external applications that connect to Aras Innovator. The SDK offers several different deployments including .NET, Windows Surface, IOS and Android compatible DLLs. The SDK is available as part of the Innovator installation media.

1. In an open Visual Studio project select Project > Add Reference... from the main menu.
2. Choose the Browse tab and select the IOM.dll from the appropriate IOM SDK directory (this example uses .NET).
3. Finally, add the namespace Aras.IOM to your project code to gain access to the IOM classes and methods.

Establishing Connection

1. Define the login credentials
 - User Id, Password, Database, Server URL
2. Establish a server connection
 - Using the IomFactory class
3. Login to the server using the connection
 - Using the HttpServerConnection class
4. Create Innovator instance
 - Using the IomFactory class

```
public class Global : System.Web.HttpApplication {
    {
        void Session_Start(object sender, EventArgs e)
        {
            // Code that runs when a new session is started
            //Define credentials
            ① string url = "http://localhost/Innovator12";
            string db = "DevelopingSolutions12";
            string user = "admin";
            string pw = "innovator";

            ② //MAKE A SERVER CONNECTION
            HttpServerConnection conn =
                IomFactory.CreateHttpServerConnection(url, db, user, pw);

            ③ //LOGIN
            Item login_result = conn.Login();

            //INNOVATOR CREATE
            ④ Innovator inn = IomFactory.CreateInnovator(conn);

            //STORE IN SESSION VARIABLES
            Session["innovator"] = inn;
            Session["cnx"] = conn;
        }
    }
}
```



Establishing Connection

This example shows a small project that will make a connection to the student development server.

The Visual Studio ASP.Net project uses the Global.asax page to define global settings initialized when the session begins. This example uses that page to define the Innovator variable that will be available to any page in the application.

C#:

```
public class Global : System.Web.HttpApplication {

    void Session_Start(object sender, EventArgs e)
    {
        // Code that runs when a new session is started
        //Define credentials
        string url = "http://localhost/Innovator12";
        string db = "DevelopingSolutions12";
        string user = "admin";
        string pw = "innovator";

        //MAKE A SERVER CONNECTION
        HttpServerConnection conn =
            IomFactory.CreateHttpServerConnection(url, db, user, pw);

        //LOGIN
        Item login_result = conn.Login();

        //INNOVATOR CREATE
        Innovator inn = IomFactory.CreateInnovator(conn);

        //STORE IN SESSION VARIABLES
        Session["innovator"] = inn;
        Session["cnx"] = conn;
    }
}
```


VB:

```

Public Class Global_asax
    Inherits System.Web.HttpApplication

    Private Sub Session_Start(sender As Object, e As EventArgs)
        ' Code that runs when a new session is started
        'Define credentials
        Dim url As String = "http://localhost/Innovator12"
        Dim db As String = "DevelopingSolutions12"
        Dim user As String = "admin"
        Dim pw As String = "innovator"
        'MAKE A SERVER CONNECTION
        Dim conn as HttpServerConnection =
        IomFactory.CreateHttpServerConnection(url, db, user, pw)
        'LOGIN
        Dim login_result As Item = conn.Login()
        'INNOVATOR CREATE
        Dim inn as Innovator = IomFactory.CreateInnovator(conn)
        'STORE IN SESSION
        Session("innovator") = inn
        Session("cnx") = conn
    End Sub
End Class

```

Note

As a best practice, it is a good idea to logout of a server connection when it is no longer needed. Use the Session_End method to call the connection Logout function.

```

void Session_End(object sender, EventArgs e){
    HttpServerConnection conn =
        (HttpServerConnection)Session["cnx"];
    conn.Logout();
}

```

Using IOM Item Methods

- All IOM and Innovator class methods discussed in class are now available
- Use the Innovator instance to create a new Item
- Microsoft Visual Studio “IntelliSense” automatically completes method syntax as you type

```
itm.setProperty(  
  ▲ 1 of 2 ▼ void Item.setProperty (string propertyName, string propertyValue)
```



Using IOM Item Methods

The IOM.dll assembly contains the Item and Innovator class methods we have used in the course.

Once a reference to the Innovator object is established in your program code, you can begin to access and modify Aras Innovator Items.

In addition, Visual Studio "Intellisense" makes coding easier as valid options are presented as you create your project code.

Application Example

```
//PRIVATE METHOD TO GET SESSION INNOVATOR
private Innovator getInnovator()
{
    return (Innovator)Session["innovator"];
}

//FIND BUTTON CLICK
protected void Button1_Click(object sender, EventArgs e)
{
    Item so = getInnovator().newItem("Sales Order", "get");
    so.setProperty("so_number", TextBox1.Text);
    Item result = so.apply();

    if (result.isError())
    {
        StatusMessage.Text = result.getErrorString();
        return;
    }
    TextBox2.Text = result.getProperty("state", "");
    TextBox3.Text = result.getProperty("po_number", "");
    DropDownList1.Text = result.getProperty("shipping_method", "");
    TextBox4.Text =
        result.getPropertyAttribute("managed_by_id", "keyed_name");
    string ms = result.getProperty("multiple_shipment");
    CheckBox1.Checked = (ms == "1") ? true : false;
    StatusMessage.Text = "Item loaded";
}
```

Application Example

The following code is placed on the ASP.Net page displayed above using the ASP.Net Button1 click method to retrieve the details of a Sales Order by the `so_number` property. A private method is added first to retrieve the Innovator instance saved in the Session object.

C#:

```
//PRIVATE METHOD TO GET SESSION INNOVATOR
private Innovator getInnovator()
{
    return (Innovator)Session["innovator"];
}

//FIND BUTTON CLICK
protected void Button1_Click(object sender, EventArgs e)
{
    Item so = getInnovator().newItem("Sales Order", "get");
    so.setProperty("so_number", TextBox1.Text);
    Item result = so.apply();

    if (result.isError())
    {
        StatusMessage.Text = result.getErrorString();
        return;
    }
    TextBox2.Text = result.getProperty("state", "");
    TextBox3.Text = result.getProperty("po_number", "");
    DropDownList1.Text = result.getProperty("shipping_method", "");
    TextBox4.Text =
        result.getPropertyAttribute("managed_by_id", "keyed_name");
    string ms = result.getProperty("multiple_shipment");
    CheckBox1.Checked = (ms == "1") ? true : false;
    StatusMessage.Text = "Item loaded";
}
```

VB:

```

Private Function getInnovator() As Innovator
    Return DirectCast(Session("innovator"), Innovator)
End Function

Protected Sub Button1_Click(sender As Object, e As EventArgs)
    Dim so As Item = getInnovator().NewItem("Sales Order","get")
    so.setProperty("so_number", TextBox1.Text)
    Dim result As Item = so.apply()

    If result.isError() Then
        StatusMessage.Text = result.getErrorString()
        Return
    End If
    TextBox2.Text = result.getProperty("state", "")
    TextBox3.Text = result.getProperty("po_number", "")
    DropDownList1.Text = result.getProperty("shipping_method", "")
    TextBox4.Text = result.getPropertyAttribute("managed_by_id", _
        "keyed_name")
    Dim ms As String = result.getProperty("multiple_shipment")
    CheckBox1.Checked = If((ms = "1"), True, False)
    Status.Message.Text = "Item loaded"
End Sub

```

Apply a Server Method

Earlier in this class you created a server Method named **Server-CalculateCost** to calculate the roll up cost of all Part items on a Sales Order item.

You can use the IOM `apply` method to execute this program and return a result to the solution:

```

Item costItm = so.apply("Server-CalculateCost");
string totalcost = costItm.getResult();
TextBox5.Text = totalcost;

```

VB

```

Dim costItm As Item = so.apply("Server-CalculateCost")
Dim totalcost As String = costItm.getResult()
TextBox5.Text = totalcost

```

Summary

In this unit, you learned how to integrate a .NET ASP application with the Innovator server using the IOM.

You should now be able to:

- Establish a connection to an Aras Innovator Server using a .NET application.
- Use IOM methods in a .NET ASP application to access or modify items.

Lab Exercise

Goal

Integrate an ASP.NET application with Aras Innovator.

Scenario

Build an ASP.NET application that allows a user to perform basic maintenance on a Sales Order. The application should be able to find, edit and delete Sales Orders using IOM methods you have used in the course.

1. Open the ASP.NET project named **SalesOrderApplication** in the Visual Studio Projects folder.
2. Provide the logic for the Edit and Delete buttons to allow a user to modify an existing Sales Order item. What attribute must you set to allow an edit or delete to an Item? _____.



Unit 14 Implementing RESTful Services

Overview

In this unit, you learn how to implement the Aras Innovator RESTful API that allows you to retrieve, add, update and delete items as well as execute server Methods stored in the Aras database. The API uses the Open Data Protocol (OData) which is an ISO/IEC approved Open Access Same-time Information (OASIS) standard. Data elements that are exchanged with the server are defined using the JavaScript Object Notation (JSON) format. This allows any clients that support this protocol to communicate easily with the Aras Innovator server.

Objectives:

- Defining RESTful Architecture
- Understanding the Open Data Protocol (Odata)
- Using JavaScript Object Notation (JSON) with the Aras Client
- Retrieving Items and Property Data
- Using Request Options to Filter Data
- Adding new Items
- Editing existing Items
- Deleting Items
- Executing Server Methods

Defining RESTful Architecture

- Web service style (Representational State Transfer)
- Provides interoperability between computers using the internet
- Communicate using HTTP Methods/ URL's
- Guiding constraints:
 - Uniform interface
 - Client-server architecture
 - Stateless
 - Cacheable
 - Support Layered Systems
 - Code on Demand (Optional)



Defining RESTful Architecture

REST stands for Representational State Transfer, a term defined by Roy Fielding in 2000. It is now a common architecture style for designing loosely coupled applications that interact using HTTP. REST does not enforce any rules regarding how it should be implemented at a lower level; it just defines high-level design guidelines, which are implemented by web services developers.

The Aras RESTful API has been created with these goals in mind and follows the basic constraints Fielding outlined in his original dissertation (see <https://restfulapi.net/rest-architectural-constraints/>).

Uniform interface

All resources should be accessible via a URI using a common approach (such as HTTP GET, POST, PATCH, DELETE) and follow a standardized set of syntax rules.

Client server architecture

The development of the client and the server must be able to evolve independently.

Stateless

Each request from client to server must contain all of the information necessary to understand the request and cannot take advantage of any stored context on the server. Session state is therefore kept entirely on the client.

Cacheable

If a response is cacheable, then a client cache is given the right to reuse that response data for later, equivalent requests.

Support Layered Systems

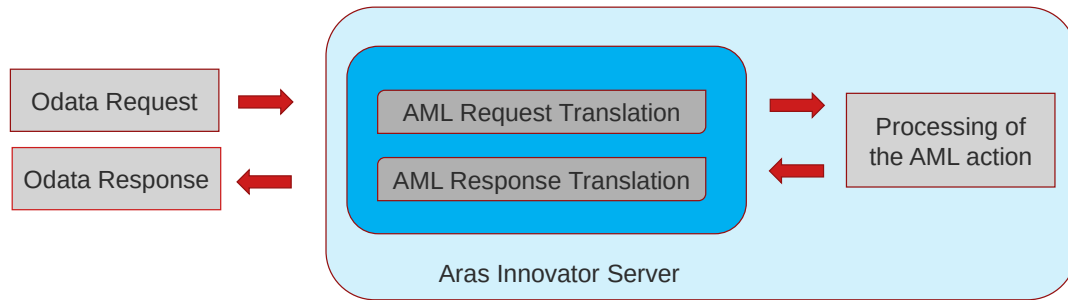
The architecture must support systems built in hierarchical layers so that different servers can work together to service a request.

Code On Demand (optional)

Allows the client to download and executing code in the form of applets or scripts.

Understanding Odata

- Data access protocol using REST architecture
- Built on the AtomPub protocol for retrieving and updating resources (OASIS standard)
- Aras Innovator Odata Interface:



Understanding Odata

The Open Data Protocol (OData) is a data access protocol for the web. OData provides a uniform way to query and manipulate data sets through CRUD operations (create, read, update, and delete).

Odata is built on the AtomPub protocol for retrieving and sending data and certified to follow the OASIS standard. It uses standardized technologies such as HTTP, Atom/XML, and JSON to send and receive data in a uniform way as well as define the data model being used.

The Aras implementation using Odata receives an Odata Request at the server and translates that request into a corresponding AML action. The result of the action is then translated back as an Odata response to the client.

Understanding the Aras AML command language is extremely helpful when constructing (and debugging) Odata requests to be consumed by the server.

Reviewing Odata URL

▪ Odata Request URL Format

`https://training.aras-cloud.com/sit-1-0-202111233/server/odata/Part?$top=2&$orderby=Name`

Service Root URL

Resource Path Query Options

Service Root URL

- `.../server/oData`
- Resource Path
 - ItemType or Method
- Query Options
 - Item Request Options



Reviewing the Odata URL

The Odata URL is constructed of three parts defined as:

Service Root

Contains the http protocol instruction as well as the server name (and port) and a service defined by the API.

Resource Path

Defines the Categories and Products to be consumed by a request. For the Aras API this is typically an ItemType name that is involved with the request.

Query Options

These options follow the query string indicator (?) and are implemented based on the API. Multiple query strings can be provided and are delimited by the ampersand character (&). In the Aras API these options include filtering, paging and selecting data.

Implementing HTTP Methods as Aras Actions

HTTP Method	Aras AML Action	Notes
GET	get	
POST	add	default for POST
POST	create	@aras.action: create
PUT	edit	update single property on an Item
PATCH	edit	default for PATCH
PATCH	Update	@aras.action: update
PATCH	Lock	@aras.action: lock
PATCH	Unlock	@aras.action: unlock
PATCH	Merge	@aras.action: merge
DELETE	Delete	default for DELETE
DELETE	Purge	@aras.action: purge



Implementing HTTP Methods as Aras Actions

Every OData request begins with an HTTP Method which is then used by the Aras API to determine its purpose.

You will review examples of these Methods in this unit and see how they are implemented to retrieve, add, edit, and delete items in the Aras database.

Testing API with Postman

- HTTP Client for testing Web Services
- Executes Aras RESTful API Requests
- Available as Packaged or Google in-browser App



- Download: www.getpostman.com



Testing API

Several 3rd party tools are available to help you create and test HTTP requests and review the responses.

A common open-source tool is Postman, which can be downloaded from www.getpostman.com as either a packaged application or an in-browser app.

This first part of this unit demonstrates the use of the API using this tool.

Authorizing Requests using OAuth Token

Generating a Token

POST `http://localhost/InnovatorServer/oauthserver/connect/token`

Params Authorization Headers (8) Body Pre-request Script Tests Settings

none form-data x-www-form-urlencoded raw binary GraphQL

KEY	VALUE	DESCRIPTION
grant_type	password	
scope	Innovator	
client_id	IOMApp	
username	admin	
password	607920b64fe136f9ab2389e371852af2	
database	PCR17DS	

```
{
  "access_token": "eyJhbGciOi...",
  "expires_in": 3600,
  "token_type": "Bearer",
  "scope": "Innovator"
}
```

Could not get response

SSL Error: Unable to verify the first certificate **Disable SSL Verification**

POST `https://training.aras-cloud.com/sit-1-0-202201132/oauthserver/connect/token`

Params Authorization Headers (9) Body Pre-request Script Tests Settings

Enable SSL certificate verification

Verify SSL certificates when sending a request. Verification failures will result in the request being aborted.

OFF Default: Settings

Authorizing Requests using OAuth Token

The Aras API requires that each request be authorized to access the server using a set of credentials. An access token is first obtained by providing a set of credentials to the Aras OAuthServer. This token is then used to authorize any requests made to the Aras server. The URL of the token provider can be obtained by issuing a request to `http(s)://servername/webalias/Server/OAuthServerDiscovery.aspx`. Currently, this returns the token provider URL shown above.

The following credentials must be provided in the request body to return a valid token:

Credentials	Description
grant_type	"password" indicates a valid Aras password will be provided in the request.
scope	The scope of information to be obtained = innovator.
client_id	A general application id named IOMApp is preconfigured in the OAuth.config file on the Aras server.
username	An Aras Innovator username to authorize requests to the server.
password	The Aras Innovator password of the provided username which must be encoded using an MD5 hash.
database	Name of the Aras Innovator SQL database.

The returned JSON response will contain a Bearer Access Token that expires in 1 hour by default (this can be modified in OAuth.config). This token can then be used on any valid REST request discussed on the following pages.

Note

If you see the message below, just click to Disable SSL Verification. Alternatively, disable SSL certificate verification in the Settings tab.

Could not get response

SSL Error: Unable to verify the first certificate **Disable SSL Verification**

POST `https://training.aras-cloud.com/sit-1-0-202201132/oauthserver/connect/token`

Params Authorization Headers (9) Body Pre-request Script Tests Settings

Enable SSL certificate verification

Verify SSL certificates when sending a request. Verification failures will result in the request being aborted.

OFF Default: Settings

Storing Token for Testing Requests



Storing Token for Testing Requests

To simplify the demonstration, the access token returned from the OAuth server is saved in a POSTMAN global environment variable named "innovator_token". The following script in the Tests tab sets the variable:

```
const jsonData = JSON.parse(responseBody);  
postman.setGlobalVariable('innovator_token', jsonData.access_token);
```

Retrieving All Items

The screenshot shows a REST client interface with the following details:

- Method:** GET (1)
- URL:** https://training.aras-cloud.com/sit-1-0-202111233/server/odata/Part
- Authorization:** Bearer To... (2) Provide Access Token
- Token:** {{innovator_token}}
- Body:**

```

1 {
2   "odata.context": "http://training.aras-cloud.com/sit-1-0-202111233/server/odata/$metadata#Part",
3   "value": [
4     {
5       "classification": "Component",
6       "created_on": "2021-08-25T10:48:59",
7       "current_state@aras.name": "Preliminary",
8       "generation": 1,

```
- Status:** 200 OK, Time: 1177 ms, Size: 99.32 KB
- Save Response:** Save Response

Retrieving All Items

The HTTP GET method is used to retrieve items from the database. The categories section of the URL specifies the kind of items to return.

To Retrieve All Items

1. To retrieve all items of a specific ItemType use the HTTP GET (1) method and provide the ItemType name in the Categories section of the URL.
2. Provide the Bearer Access token obtained from Aras OAuthServer to authorize the request. The request in this demonstration uses a POSTMAN global variable named "innovator_token" described on the previous page using the expression {{innovator_token}}.
3. Click the Send button to process the request. The response (items or error) will appear in the Body window at the bottom of the screen.

Paging Result Sets

- Specify PageSize using Prefer Key in Request Header

Authorization	Headers	Body	Pre-request Script
	Key	Value	
☒	Prefer	odata.maxpagesize=10	

- GET Result returns *skiptoken* to next page

```
GET http://localhost:80/Innovator12/server/odata/Part
1 {
2   "@odata.context": "http://localhost/Innovator11/server/odata/$metadata#Part",
3   "@odata.nextLink": "http://localhost/Innovator11/server/odata/Part?$skiptoken=Mza7MTA=",
4   "value": [
```

- Use *skiptoken* to specify next query page

```
GET http://localhost:80/Innovator12/server/odata/Part?$skiptoken=Mza&MTA=
```



Paging Requests

The Aras API supports the ability to retrieve large data sets in pages using a size attribute to determine how many rows are returned on each page (implements the AML page and pagesize attributes).

To Page Results

1. Add a key named Prefer and use the OData attribute maxpagesize to define how many rows will be contained in each page.
2. When the request is processed, an OData attribute named @odata.nextLink is returned with a *skiptoken* value, which is the current position for the start of the next GET query.
3. Specify the skiptoken value as a query string argument on the next request to get the next page.

When all data has been returned, the odata nextlink will not be present in the odata response. This is the indicator that this is the last page.

Using Paging Query Options

- Return item count (\$count)

```
GET http://localhost:80/Innovator12/server/odata/Part/$count
```

- Return items with count variable

```
GET http://localhost:80/Innovator12/server/odata/Part?$count=true
```

- Return Top (n) Items (\$top)

```
GET http://localhost:80/Innovator12/server/odata/Part?$top=10
```

- Sort Items (\$orderby)

```
GET http://localhost:80/Innovator12/server/odata/Part?$orderby=classification,name
```

- Skip Items (\$skip)

```
GET http://localhost:80/Innovator12/server/odata/Part?$skip=5
```



Using Page Query Options

Additional options are available while retrieving data to determine how many records have been returned as well as the ability to sort and specify which records to display.

Return Item Count

Use the \$count option to return the number of records returned from a query (raw value). Using the \$count=true query string returns the @odata.count variable with the value assigned.

Return Top Items

Returns the first set of items based on the value assigned. In the example above the first 10 Part items are returned in the response.

Sort Items

Use the \$orderby option to sort the records returned from the database based on property name(s) (comma delimited).

Skip Items

Skips over a defined number of items in the result set.

Page query options can be combined into meaningful expressions. For example:

```
http://localhost/Innovator12/server/odata/Part?$top=10&$skip=5&
$orderby=name
```

Returns a data set skipping the first 5 items and returns back the top 10 items from that set sorted by Part name.

Using Item Query Options

```
GET http://localhost:80/Innovator12/server/odata...
```

- Item by ID

```
/Part('C178DF9A420C4E8A8C9A058F27D662CF')
```

- Return a property value

```
/Part('C178DF9A420C4E8A8C9A058F27D662CF')/item_number
```

- Return a property raw value (\$value)

```
/Part('C178DF9A420C4E8A8C9A058F27D662CF')/item_number/$value
```



Using Item Query Options

The HTTP GET request supports several query options to locate an item using a GUID as well as select property data.

Item by Id

Specify a valid GUID string to retrieve the item.

Return a property value

Returns a JSON construct in the following format:

```
{
  "@odata.context": "http://localhost/Innovator11/server/odata/
$metadata#Part/item_number",
  "value": "0460-2536"
}
```

Return a property (raw value)

Returns the value as string

0460-2536

Using \$select Query Option

```
GET http://localhost:80/Innovator12/server/odata...
```

- Define Properties Returned

```
/Part?$select=name,description
```

- Return All Properties (including null values)

```
/Part?$select=*
```



Using \$select Query Option

You can also specify which property values should be returned from a query use the \$select option.

Define Properties Returned

Specify one or more property names (comma delimited).

Return All Properties

Using the * qualifier indicates all properties will be returned for each item, including properties that have no value (null). If you specify a query without a \$select filter Aras only returns properties that have a value assigned.

To select extended property values be sure to provide the "xp-" prefix.

Using \$filter Query Option

```
GET http://localhost:80/Innovator12/server/odata...
```

- Property Conditions

```
/Part?$filter=cost gt 100
```

- Logical Operators

```
/Part?$filter=cost gt 100 and classification eq 'Product'
```

- Contains String

```
/Part?$filter=contains(name, 'Cable')
```

- Starts with/Ends with String

```
/Part?$filter=endswith(name, 'Assembly')
```



Using \$filter Query Option

The \$filter query option is translated into AML property condition statements and supports the use of logical operators (and, or, not).

Property Conditions

A filter expression takes the following form:

```
$filter=propertyname queryoperator value
```

Standard Comparison Operators

eq	Equal
ne	Not equal
gt	Greater than
ge	Greater than or equal
lt	Less than
le	Less than or equal

Logical Operators

Filter expressions can take advantage of the AML logical operators to create more complex filter strings. You can combine filter expressions using the operator and, or and not (lower case). that specifying grouping preference is also supported using parentheses.

```
$filter=((cost gt 100) and (cost lt 1000)) or make_buy eq Make
```

Contains String

To implement an AML “like” condition three string functions are provided:

`contains (propertyname, comparevalue)`

Returns matching items if the specified compare value is found anywhere in the property value. This is similar to the AML `like='*string*'` expression. Otherwise, `"value": []` is returned.

`startswith (propertyname, comparevalue)`

Returns matching items if the compare value is found at the beginning of the property value string (`like='string*`). Otherwise, `"value": []` is returned.

`endswith (propertyname, comparevalue)`

Returns matching items if the compare value is found at the end of the property value string (`like='*string'`). Otherwise, `"value": []` is returned.

Using \$expand Query Option

```
GET http://localhost:80/Innovator12/server/odata...
```

- Return Item of Item Property

```
/Part?$expand=created_by_id
```

- Return Relationship Data

```
/Part?$filter=item_number eq 'C4703A'&$expand=Part BOM
```

- Return Related Item

```
/Part?$filter=item_number eq 'C4703A'&$expand=Part BOM($expand=related_id)
```



Using \$expand Query Option

The \$expand operator is used to retrieve the reference of a property of datatype Item as well as return relationship data.

Return Item of Item Property

Normally, if a result set is returned with a property of datatype Item only the GUID is returned in the response. To retrieve the referenced Item properties use the \$expand option specifying which property to be exposed.

The following query returns the created_by_id property for each Part item in expanded format:

```
http://localhost/Innovator12/server/odata/
Part?$select=created_by_id&$expand=created_by_id
```

Return Relationship Data

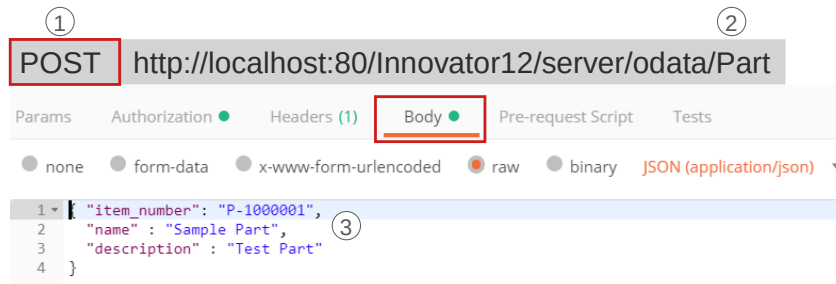
To return the relationship Items from a source Item, specify the Relationship ItemType name as the \$expand option. In the example above, Part# C4703A is returned with the Part BOM relationship data in the response. that related items are not returned by default (see below).

Return Related Item

To return the related Item from a relationship item use the nested \$expand option on the property *related_id*.

Adding New Items

▪ Add Item with Property Data



Adding New Items

The HTTP POST method can be used in the Aras API to create new items in the database. Creating a new item requires supplying a JSON definition of the properties to be assigned as part of the payload of the request.

To Add a New Item

1. Use the POST HTTP method to submit the request.
2. Assign the ItemType name to the Category portion of the URL.
3. Specify the payload in the Body of the request using JSON syntax to define the properties to be assigned. In the example above a new Part item is created with number P-100001 using the name and descriptions shown.

Adding New Items with Referenced Item

POST http://localhost:80/Innovator12/server/odata/Part

▪ Item with Referenced Item (Item Property)

```
1 { "item_number": "P-100003",
2   "name" : "Sample Part",
3   "owned_by_id@odata.bind": "Identity('DBA5D86402BF43D5976854B8B48FCDD1')"
```

▪ Item with New Referenced Item (Item Property)

```
1 { "item_number": "P-100004",
2   "name" : "Sample Part",
3   "owned_by_id" : { "name" : "Design Group",
4                     "is_alias" : "0" }
5 }
```



Adding New Items with Referenced Item

The Aras API also supports the ability to create “deep inserts” by allowing either a new or an existing item to be associated with an item property when the original item is created.

Item with Referenced Item

To associate an existing item to an item property use the @odata.bind attribute and specify the item to be related as the value. In the example above the owned_by_id property of a new Part item will be associated with the Identity with the specified GUID.

Item with New Referenced Item

To associate an item property with an item to be created specify the required properties of the item using the JSON syntax shown above. In this example, the owned_by_id property of the new Part will be associated with a new Identity named Design Group.

Adding Relationship Items

POST http://localhost:80/Innovator12/server/odata/Part

■ Creating Relationship to New Related Item

```

1 {
2   "item_number" : "P-100006",
3   "name" : "Sample Part",
4   "Part BOM" : {
5     {
6       "quantity" : "2",
7       "related_id" : {
8         "item_number" : "R-000002",
9         "name" : "Related Item Example"
10      }
11    }
12  }
13 }
```

■ Creating Relationship to Existing Related Item

```

1 { "item_number": "P-100006",
2   "name" : "Sample Part",
3   "Part BOM" : {
4     { "related_id@odata.bind": "Part('C178DF9A420C4E8A8C9A058F27D662CF')"}
5   }
6 }
```



Adding Relationship Items

Relationships can also be created in the same transaction as the source item.

Creating Relationship to New Related Item

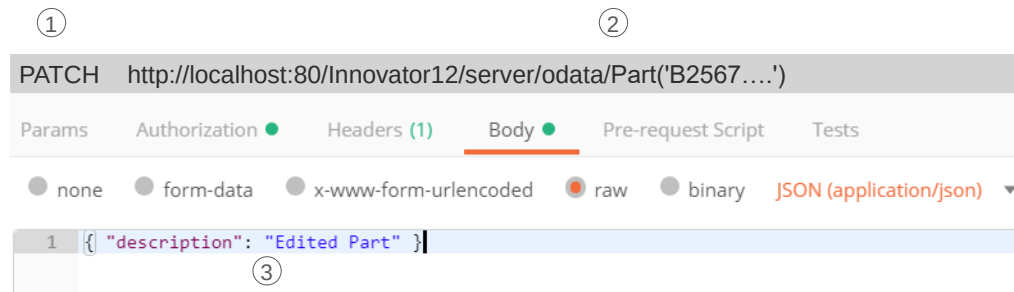
Specify the Name of the relationship ItemType as the JSON property name and use the embedded syntax shown to define the details of the relationship including the related item.

Creating Relationship to Existing Item

Use the same syntax as above using the @odata.bind attribute to specify the existing related item (Part).

Updating Items

▪ Editing Properties on Existing Item



Updating Items

Use the HTTP PATCH method to edit an existing item specified by the item GUID. that current implementation of the API only supports update of a single item.

To Update an Item

1. Use the HTTP PATCH method to submit the request.
2. Specify the item to be modified using the item GUID.
3. Identify the property(s) and value(s) to be changed in the Body payload using JSON notation.

The default implementation of the PATCH method performs an AML “edit” action on the specified item (claim, save, unclaim). The item must not be already claimed by another user or an error response will be returned.

Updating Items

▪ Updating Items with *lock* and *update* actions

```

PATCH http://localhost:80/Innovator12/server/odata/Part('B2567....')
1 {
2   "@aras.action": "lock"
3 }

PATCH http://localhost:80/Innovator12/server/odata/Part('B2567....')
1 {
2   "@aras.action": "update",
3   "description": "Edited Version of Sample Part"
4 }

PATCH http://localhost:80/Innovator12/server/odata/Part('B2567....')
1 {
2   "@aras.action": "unlock"
3 }
4 }
```



Updating Items

The Aras API also supports the ability to separately claim, save and unclaim an item using the @aras.action attribute.

To Update Items with lock and unlock

1. Specify the lock action on the first request using the @aras.action attribute.
2. Use the update action along with any changes to be made in the JSON payload.
3. Unclaim the item in the third request using the unlock action.

Use of the “edit” action is also supported. Claim and unclaim requests are unnecessary because they are defined in the AML edit action.

Updating Items

▪ Changing Single Property

The screenshot displays two sequential PUT requests in a REST client interface. The first request, labeled '1', is a PUT to the URL `https://training.aras-cloud.com/sit-1-0-202111233/server/odata/Part('C178DF9A420C4E8A8C9A058F27D662CF')/make_buy`. The 'Body' tab is selected, showing a JSON payload: `{ "value": "Buy" }`. The second request, labeled '2', is a PUT to the URL `https://training.aras-cloud.com/sit-1-0-202111233/server/odata/Part('B26F41E971FF4388B9B33F537DC86CB6')/make_buy/$value`. The 'Body' tab is also selected, showing a raw text payload: `Make`. Both requests are configured with 'JSON' as the body type. The Aras logo is visible in the bottom left corner of the interface.

Updating Items

The Aras API also supports the ability to update a single property by using the HTTP PUT method.

Two forms of the PUT method are defined:

1. Specify the item with GUID and then follow with the name of the property to be changed. The Body payload is then supplied with the value attribute using JSON notation.
2. Specify the item with GUID and then follow with the name of the property and the \$value query operator. The Body payload should contain the raw data value to be used for the edit.

Both use cases assume the item is already claimed – see the previous examples to claim the item first.

Deleting Items

- Deleting Existing Item (all versions)

```
DELETE http://localhost:80/Innovator12/server/odata/Part('B2567....')
```

- Purging an Item Generation

```
DELETE http://localhost:80/Innovator12/server/odata/Part('B2567....')
```

```
1 {
2   "@aras.action" : "purge"
3 }
```

- Deleting a Relationship

```
DELETE http://localhost:80/Innovator12/server/odata/Part('B2567....')/
Part Document/$ref$Id=Part Document('C3924...')
```

- Clear Item Property

```
DELETE http://localhost:80/Innovator12/server/odata/Part('B2567....')/
owned_by_id/$ref
```



Deleting Items

To delete an item requires the GUID be supplied in the request using the HTTP DELETE method. If an item is versioned, using the DELETE method without the purge @aras.action deletes all versions of the item.

Deleting Relationships

Relationships can be deleted from a source item using the \$ref option which then defines which relationship to be removed. that this does not delete the related item and the source item must be claimed before a relationship can be deleted.

Clear Item Property

To remove an item association on a property of type Item use the \$ref option as shown above. This removes the reference from the item property but does not delete the referenced item.

Executing Server Methods

- Server Methods Exposed as Odata Actions

- Execute a Method with Method Context

```
POST http://localhost:80/Innovator12/server/odata/method.TRNGetUsers
```

```
POST http://localhost:80/Innovator12/server/odata/Sales Order("D4356...")
/method.Server-CalculateCost
```

- Execute a Method using Request Payload

```
POST http://localhost:80/Innovator12/server/odata/method.Server-CalculateCost
```

```
1 {
2   "@odata.type" : "http://host/data/$metadata#Sales Order",
3   "id" : "D4D83BC7382C41A8B29B8C3747A5E1AB"
4 }
```



Executing Server Methods

Aras server Methods are exposed as Odata actions in the Aras API using the HTTP POST method. You can execute a Method using the context of the Method item or specify an Item that the Method should act on.

Execute a Method with Method Context

Prefix the name of the Method Item with the namespace method (lowercase) to execute the code. The context of the running Method will be the Method Item itself.

Execute a Method with a Specific Item Context

Provide the Item to be acted on in the string and then append with the method namespace and Method Item name.

Execute a Method using Request Payload

You can also specify the Item to be acted on using JSON notation in the request Body payload.

Implementing in .NET Code

▪ Sales Order Application

The screenshot shows the ARAS Application interface for the Sales Order Application. The header includes 'ARAS APPLICATION' and navigation links 'Home' and 'About'. The main section is titled 'SALES ORDER APPLICATION' and contains a 'Sales Order#' input field with 'Find', 'Update', and 'Delete' buttons. Below this are two sections: 'Sales Order Info' with fields for 'Status', 'PO#', and 'Manager', and 'Shipping Info' with a 'Shipping Method' dropdown, a 'Multiple Shipments' checkbox, and a 'Total Cost' field. Red arrows point from the 'Find' button to 'Get Item', from the 'Delete' button to 'Delete Item', and from the 'Update' button to 'Update Item'.

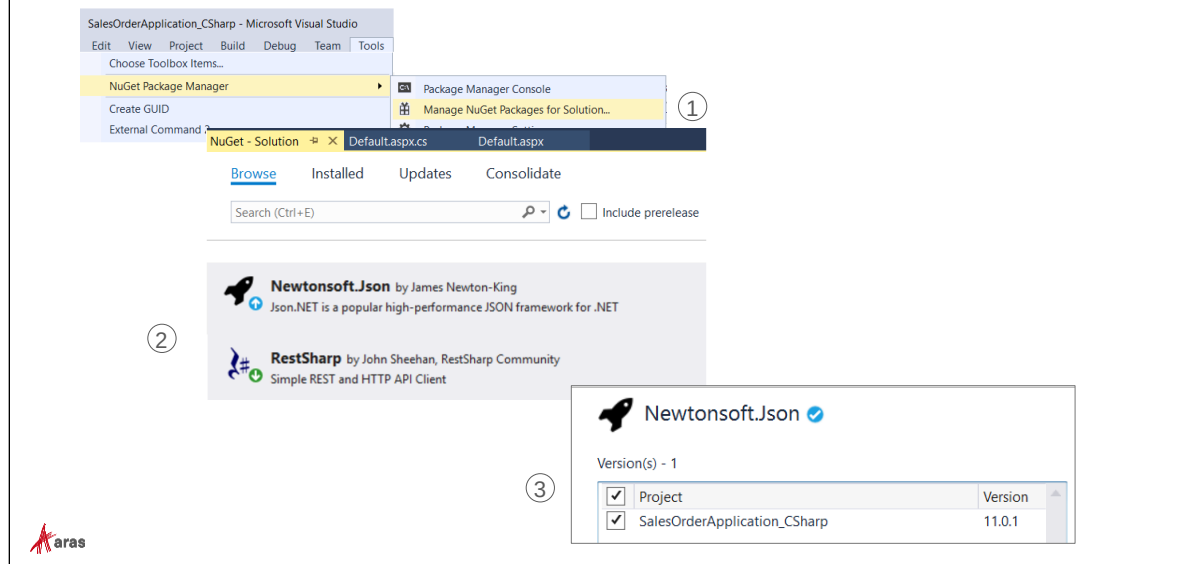
- Get Item
- Delete Item
- Update Item

Implementing in .NET Code

A Sample C# project has also been provided with the course materials to provide logic for a simple application that can read and write Sales Orders in the DevelopingSolutions12 database.

Open the sample application provided to examine an example of using RESTful services as part of a simple web application.

Referencing Additional Packages



Using JSON.Net

The example project includes some useful utilities and program classes provided in two Nuget packages named Newtonsoft.JSON and RESTSharp. Several utilities allow you to serialize object data into JSON notation and vice versa.

The sample application provided in class already has these packages installed. If you are starting a new Project, you will need to add these references to use the same sample code.

To Add Newtonsoft.JSON and RESTSharp packages to a Project

1. Open the Sales Order Application example project in Visual Studio.
2. Select Tools > NuGet Package Manager > Manage Nuget Packages for Solutions from the main menu.
3. Browse for the Newtonsoft.Json and RESTSharp packages and then click the Project checkbox to add the packages to your project.

Project Summary

- Data Classes to Serialize/De-serialize JSON
- Programming Methods
 - GetOAuthToken
 - Uses a predefined class named Credentials to provide Aras login information to generate a valid Bearer Access Token
 - GetSalesOrder
 - Retrieves a specified Sales Order (by number) using HTTP Get
 - EditSalesOrder
 - Modifies an existing Sales Order using HTTP Patch
 - ExecuteServerMethod
 - Executes an Aras Server Method using HTTP Post



Project Summary

Open the provided sample C# project to examine one way to access the Aras Server using RESTful services.

Data Classes

The application defines a series of data classes to allow JSON information to be converted into .Net objects.

A Credentials class also predefines the login credentials for an Aras user to allow the generation of an OAuth token. These values are hard coded in the application but could be provided as part of a login screen.

Programming Methods

The following methods have been provided in the sample application to demonstrate how to generate an access token and retrieve and modify Sales Order items in the DevelopingSolutions12 database.

GetOAuthToken

This method demonstrates how to pass the values obtained from the predefined Credentials class to the Aras OAuthServer to obtain a valid Bearer access token.

GetSalesOrder

This method is used to define an HTTP Get request to the server to obtain information about a specific Sales Order. The JSON data returned is captured in a data class to allow assignment to Visual Studio controls (textbox, etc).

Edit Sales Order

This method is used to modify an existing Sales Order item using an HTTP Patch request. User data is serialized into a JSON object as part of the payload for the request.

ExecuteServerMethod

This method demonstrates how an Aras Server Method can be called using a RESTful service using an HTTP Post request.

Summary

In this unit, you learned how to the Aras RESTful services API.

You should now be able to:

- Explain the RESTful Architecture using OData
- Retrieve Items and Property Data
- Add new Items
- Edit existing Items
- Delete Items
- Execute Server Methods
- Use JavaScript Object Notation (JSON) with the Aras Client

Lab Exercise

Goal

Use the Aras REST API to retrieve, add or edit items. Implement the “delete” feature in the Sales Order application example project.

Steps

Using the Postman client app:

1. Submit a request using the Aras REST API to find all Sales Orders where the shipping_method is Federal Express and the multiple_shipment is TRUE.
2. Create a new Sales Order where the shipping_method is UPS, multiple_shipment is TRUE and po_number is “12345”.
3. Update the Sales Order item above and change the po_number to “54321”.
4. Delete the Sales Order item above (modified in step 3).



Unit 15 Importing Data with Batch Loader

Overview

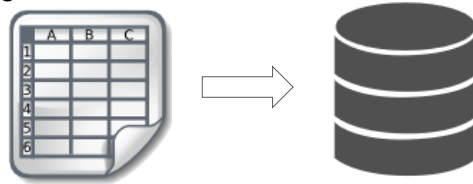
In this unit, you will learn how to use the Batch Loader utility to migrate data from an outside resource into the Innovator database. The Batch Loader is a subscriber only.

Objectives

- Reviewing the Batch Loader Features
- Working with AML Templates
- Using the BatchLoader Tool
- Importing Data from a File
- Using Command Line BatchLoader
- Creating AML Templates

Reviewing Batch Loader Features

- Separate Executable designed to import data from external flat files
- Transfers data into AML statements to load new Items
- Two modes available
 - Windows Application
 - Command Line



Reviewing Batch Loader Features

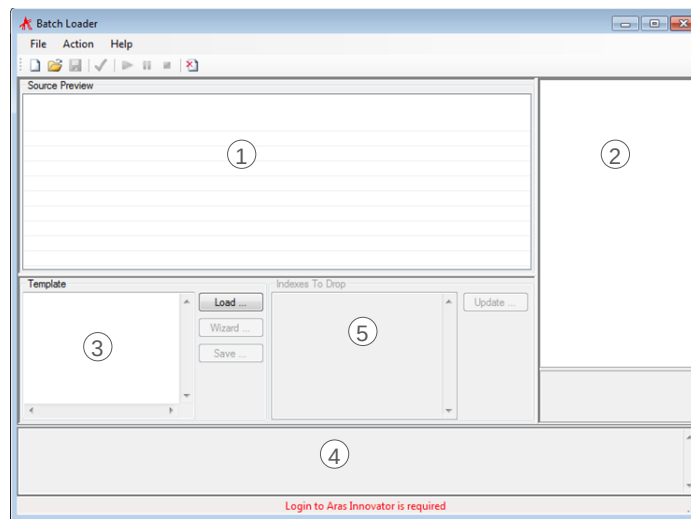
The Aras Batch Loader tool is a utility for loading data from a flat file into Aras Innovator. This tool transforms the flat file data into AML for loading directly into Aras Innovator much like an alternative client.

The Batch Loader has two distinct modes for loading data. The first, more direct method is through execution of the Batch Loader utility at the command line. This allows for rapid data loading into an Aras Innovator instance based upon a configuration pre-defined in an xml file. The use of the command line executable also makes it suitable for use in a daily batch job through the use of Windows Scheduler.

Note

The Batch Loader is a subscriber only tool. You will need to obtain a license key from Aras in order to operate the tool.

Using the Batch Loader Application



Using the Batch Loader Application

The Batch Loader tool is divided into 5 areas:

- ① **Preview Pane**
Shows a preview of the flat file data that will be loaded into the Innovator database.
- ② **Configuration Pane**
You define the file to be processed as well as appropriate parameters in this pane.
- ③ **Template**
The AML Template is displayed in this pane. AML templates are used to build AML instructions for the server to process the flat file data.
- ④ **Status**
Status messages are presented in the status pane as the tool processes a file.
- ⑤ **Indexes to Drop**
To improve performance on very large imports, you can disable database indexes using an alter index statement in SQL Server. Contact the Aras support via subscriber portal or partner portal respectively if you need more information/recommendations regarding this option.

Completing the Configuration Page

Database Options	
Server	http://localhost/innovator12
Database	DevelopingSolutions12
User	root
Load Options	
DataFilePath	C:\TrainingFiles\customers.csv
Delimiter	.
WorkerProcesses	1
Threads	1
LinesPerProcess	100
Encoding	Unicode (UTF-8)
FirstRow	1
LastRow	-1
Log Options	
LogFilePath	C:\Program Files (x86)\BatchLoader\log\batchload.log
LogLevel	2
Preview Options	
PreviewRows	10
MaxColumns	10
FirstRowIsColumnNames	False

 aras

Completing the Configuration Page

You configure what file will be processed as well as other parameters to the tool in the Configuration Pane. The following fields are available:

Server – The connection URL for Aras Innovator. If all the defaults were taken during the Innovator installation, the path should be something like: http://localhost/InnovatorServer. If unsure, please check in IIS to get the exact path. This URL should not include reference to the /Client folder

Database – The database to which the data will be loaded. Selecting this field after defining the connection URL will make this a pick list of available databases.

User – The user login to be used for connecting to and loading data into the database.

DataFilePath – The fully qualified path to data file that contains the data to be loaded into the database. Use the Customers.csv file provided by the instructor.

Delimiter – The delimiter used to separate data, usually a tab (\t), or a space (), or a comma (,).

WorkerProcesses – The number of worker processes to be used by the Batch Loader while loading data. Recommend using the default of 1.

Threads – The number of threads per worker process. Recommend using the default of 1.

LinesPerProcess – The number of lines in the data file that will be loaded by a single worker process. If the worker process finishes processing all its lines and the data file has more lines to be processed, then a new worker process will be started.

Encoding – Encoding (or codepage number) of data file

FirstRow – The number of the row where the actual data starts. Sometimes the first row is used for row headings. In that case, the data will start in the second row. (See FirstRowIsColumnNames property below.)

LastRow – The number of the row where the actual data stops. Default of -1 indicates that the file should be read until the end of the file

LogFilePath – The fully specified file name of the log file where all information and errors are to be written by the Batch Loader.

LogLevel – The Level of detail included in the logging.

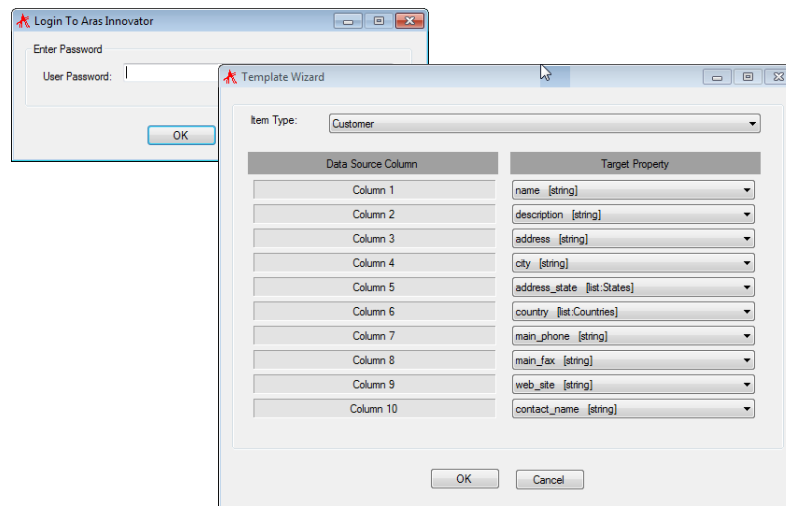
- 1 – Low, recommended for automated jobs with low risk of failure. Details about start and stop, and how many items succeeded.
- 2 – Medium, recommended for use while developing new Batch load job. Logs details about failure, as well as details logged in low mode.
- 3 – High, recommended for debugging. Provides detail and AML about every line loaded.

PreviewRows – The number of rows visible in the Preview Pane.

MaxColumns – The number of columns in the Preview Pane.

FirstRowIsColumnNames – When set to True, the Batch Loader starts parsing the data file from the second row after the FirstRow value.

Creating the Data Mappings



Using the Template Wizard to Map Data

Click the *Wizard* button in the AML Template Pane to start the Data Mapping Wizard. You will be required to enter the password for the User specified in the Configuration Pane before you can begin.

1. Select the ItemType to indicate the type of Item to be created from the imported data.
2. Use the Target Property dropdown list to choose a target property for each Column in the delimited flat file. You can view the contents of the file in the Preview Pane to help you to map the data.
3. Click OK when mapping is complete.

Reviewing the Batch Loader Template

```
Template
<Item type='Customer' action='add'>
  <name>@1</name>
  <description>@2</description>
  <address>@3</address>
  <city>@4</city>
  <address_state>@5</address_state>
  <country>@6</country>
  <main_phone>@7</main_phone>
  <main_fax>@8</main_fax>
  <web_site>@9</web_site>
```



The Batch Loader Template

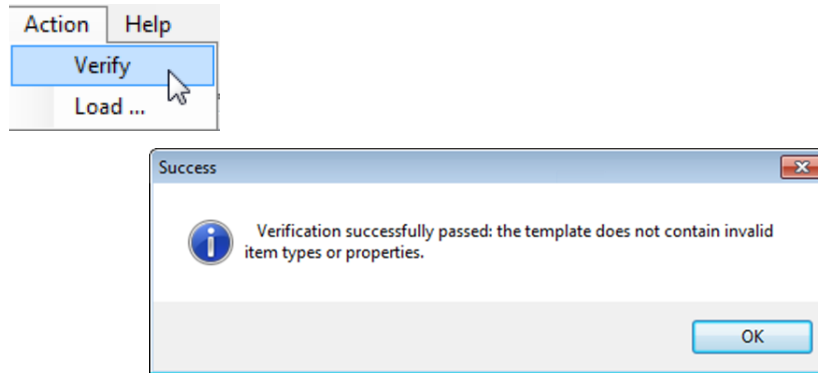
The AML Template is used to indicate to the server how data should be added (or merged) into the Innovator database. The template uses standard AML statements you have learned about in this course.

Each data column from the flat file is marked with an “@” sign followed by a number. In the example above, the first column of the file is expected to have the “name” field.

You can modify this template file and also save it by clicking the Save key in the AML Template Pane.

Verifying the Template

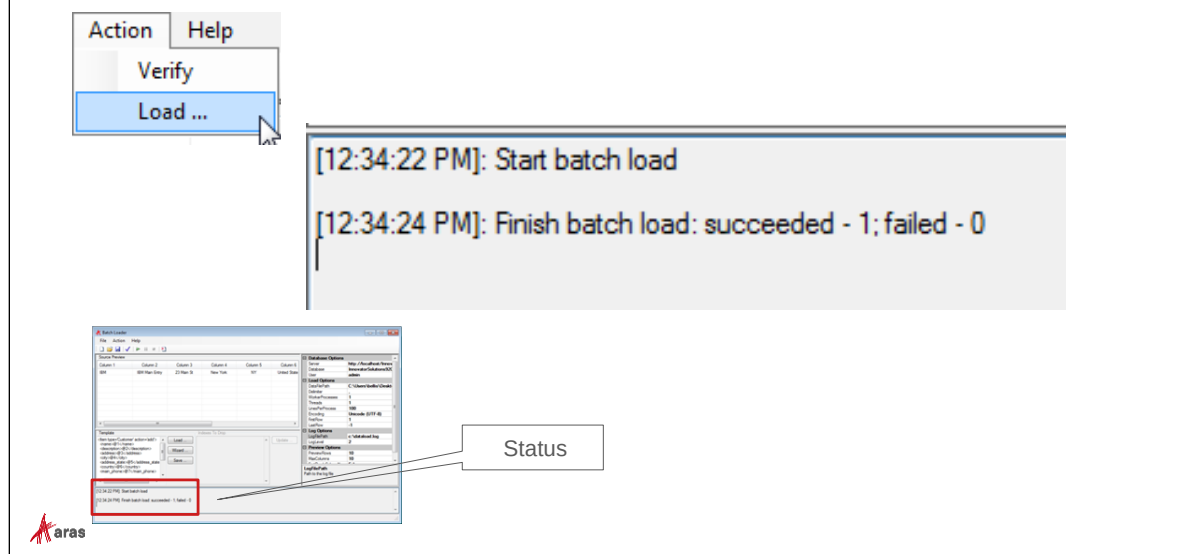
- Use Verify Action to check template for problems



Verify the Template

To assist with defining a correct AML Template, the Verify action is available to check that the AML syntax presented is correct. Any error messages will be displayed in a message box. If no errors are found, the success message displayed above appears.

Loading Data



Loading Data

Use the Load... action to begin the import process. The Status Pane will display the data as it is imported and indicate success or failure.

Remember that a data log (configured in the Configuration Pane) is also available for review when the import is completed.

Importing Relationships

- Define an external file that relates two Items together:

	A	B
1	Document Number	Design Request
2	SPEC-0001	DR-000030



Importing Relationships

Along with single items, you can also import Relationship data.

One simple way to define relationships is to create a “join” spreadsheet that indicates the source and related Item based on some unique property.

In this example, a spreadsheet has been created that denotes a Document with item_number SPEC-0001 that should be related to a Design Request DR-000030.

Defining Relationship Mapping

Item Type: Design Request Document ▼

Data Source Column	Target Property
Column 1	related_id [item:Document] ▼
Column 2	source_id [item:Design Request] ▼



Define Relationship Mapping

Using the Template Wizard described previously, choose a Relationship ItemType and then match the columns of the spreadsheet with the `related_id` and `source_id` properties of the relationship.

In this example, the Document is the related Item and the Design Request is the source.

Reviewing Relationship Template

Source Preview	
Column 1	Column 2
SPEC-0001	DR-000030

Template
<pre><Item type='Design Request Document' action='add'> <related_id> <Item type='Document' action='get' select='id'> <item_number>@1</item_number> </Item> </related_id> <source_id> <Item type='Design Request' action='get' select='id'> <item_number>@2</item_number> </Item> </source_id> </Item></pre>



Reviewing the Template

Review the template to make sure that the AML syntax is correct (also check with the Verify action).

In this example, the item_number property is used to find the Document and Design Request in the database and relate them together using the Design Request Document relationship.

Using Command Line Mode

- Use the BatchLoaderCMD.exe executable
- Allows for scheduled imports using previously saved configurations
- Server connection configured using an XML file
- Uses Configuration Template Files to process import data



Using Command Line Mode

The Batch Loader can also be executed from the windows command line. The command line version of the Batch Loader allows for the use of a known configuration that can be used in a regularly scheduled process (e.g. adding new Users to the database). Combined with the Scheduler Service, the command line Batch Loader can also import data at scheduled time intervals.

BatchLoaderCMD Arguments

- BatchLoaderCMD -d {datafilename} [arguments]
- Most arguments can be contained in an XML configuration file (called with the -c switch)
- File delimiter character must be defined in the XML config file
- Example:

```
BatchLoaderCMD -d c:\import.csv -c c:\config.xml
               -t c:\template.xml
```



BatchLoaderCMD Arguments

The BatchLoaderCMD executable has no user interface but accepts a number of switch arguments which can provide connection and configuration information.

Usage: BatchLoaderCmd.exe -d {data file} [optional arguments]

where:

-d {data file} - path to a delimiter-separated data file

Optional arguments can either be specified in the command line or in a configuration file.

-c {config file} - path to configuration file; default is 'config.xml' if no file is specified

-t {template file} - path to template file; default 'template.xml'

-l {log file} - path to log file

-ll {log level} - verbosity of logging (1 - 3); default 1

-e {encoding} - name of data file encoding; e.g. 'us-ascii'

-s {url} - Innovator server URL

-db {db name} - Database name

-u {user name} - Innovator user login name

-p {user password} - Innovator user password

-th {threads} - Number of threads in a worker process; default 1

-pr {processes} - Number of worker processes; default 1

-lpp {lines} - Lines processed by each worker process; default 1000

-fr {row number} - First row to process in the data file; default 1

-lr {row number} - Last row to process in the data file; default -1

Note

Most of these parameters are optional and are generally included in the xml configuration file. However, the command line arguments will override any values in the configuration file.

Sample Configuration File

```
<BatchLoaderConfig>
  <server>http://localhost/InnovatorServer</server>
  <db>InnovatorSolutions</db>
  <user>admin</user>
  <password>innovator</password>
  <max_processes>1</max_processes>
  <threads>1</threads>
  <lines_per_process>250</lines_per_process>
  <delimiter>\t</delimiter>
  <encoding>utf-8</encoding>
  <first_row>2</first_row>
  <last_row>-1</last_row>
  <log_file>C:\User</log_file>
  <log_level>3</log_level>
</BatchLoaderConfig>
```

Example Template File

- Add new Users with password "innovator":

```
<BatchLoaderPrototype>
  <Item type="User" where="login_name='@1'" action="add">
    <login_name>@1</login_name>
    <password>607920b64fe136f9ab2389e371852af2</password>
    <logon_enabled>@2</logon_enabled>
    <first_name>@3</first_name>
    <last_name>@4</last_name>
    <email>@5</email>
  </Item>
</BatchLoaderPrototype>
```



Example Template File

AML Templates define the instructions for loading data from an external file.

This example shows how a series of Users could be added from a file with 5 columns of data defined. The password has been hard coded using an MD5 hash string.

Note

If a template will be used with the command line version of the Batchloader then the <BatchLoaderPrototype> tags shown above are required. If the template will be used in the windows version, these tags should be removed.

Additional examples are available in the *Aras Innovator Batch Loader* documentation.

Summary

In this unit, you learned how to use the Batch Loader utility to add new data to an Innovator database from an outside file.

You should now be able to:

- Create an AML Template File
- Import Items and Relationships from an External File
- Use the Graphical Tool for Import
- Use Command Line Batch Loading

Lab Exercise

Goal

Be able to use the Batch Loader to import data from an external source into Aras Innovator using flat files.

Scenario

In the following exercises, you will use both the client utility and the command line interfaces to import new users into the system.

Exercise 1

Use the provided spreadsheet named “users.csv” and configure the import operation using the Batch Loader client UI.

Steps

1. The following spreadsheet (users.csv file) has been provided to import new users.

	A	B	C	D
1	Login Name	Password	First Name	Last Name
2	Test1	607920b64	User1	Test1
3	Test2	607920b64	User2	Test2

2. Start and configure the Batch Loader utility (configuration and data template) to import the records from the comma delimited file.
3. Run the Import and verify the new records are displayed on the Aras Innovator Client interface.

Exercise 2

Import users with the Batch Loader command line interface. Use the data mapping template you saved in Exercise 1 above to import user records contained in the “users2.csv” provided import file.

Steps

1. Open “users2.csv” to analyze its data structure. Edit the data mapping template you saved in Exercise 1, considering the identical data structure in both exercises, but add the right tags for command line execution.
2. Create the configuration file. Refer to the BatchLoaderCMD Arguments slide above.
3. Place all files in the same directory (for easier command line composition) and run the import.
4. Verify the new records are now displayed on the Aras Innovator Client interface.



Unit 16 Configuring the Scheduler Service

Overview

In this unit, you learn how to set up a windows service that will execute a specified Innovator Method periodically based on configured settings. This is useful for sending reminders or processing escalations.

Objectives

- Reviewing the Scheduler Service
- Installing and Configuring the Service
- Monitoring the Service

Reviewing the Scheduler Service

- Executable Program Installed as a Windows Service
- Connects to Aras Innovator Server
- Runs one or more Methods at Scheduled Intervals
- Useful for:
 - Sending reminders (needed)
 - Processing timed escalations (needed)
 - Running nightly reports (optionally)
 - Performing scheduled file replications (optionally)



© 2022 Aras

The Scheduler Service

The Aras Innovator Service is a Windows executable program that runs as a Windows Service. It is used to automatically run Innovator Methods on a periodic schedule, as specified in the InnovatorServiceConfig.xml file.

A single running instance of the Aras Innovator Service can be configured to periodically connect to any number of Aras Innovator Servers over the network, and for each Aras Innovator Server, run any number of Methods. The standard Windows Event Viewer is used for logging of program status and error messages.

Installing the Scheduler Service

- Components:
 - Innovator Service Executable
 - InstallService Batch File
 - UninstallService Batch File
 - InnovatorServiceConfig.xml



To Install the Scheduler Service

1. Copy the InnovatorService.exe file and the two BAT files into any Folder.
2. Copy the file InnovatorServiceConfig.xml into the same folder.
3. Run the BAT file InstallService.bat as Administrator. This program runs the standard .NET utility named INSTALLUTIL. This program will leave a set of .NET log files in the working directory which should be checked for error messages, and then deleted.
4. You will be prompted for a username and password. This is the user that will have privileges to run the Windows Service. Use a fully qualified username (domain_name\user_name). If the username and password are not valid the installation will fail.
5. Access the Services Management Console to make sure the service named “Innovator Service” has been added to system. You can then configure when to start the service, set to Automatic, etc. You can also edit the username and password that will execute this service or use the Local System Account.

Configuring the Service – InnovatorServiceConfig.xml

```
<?xml version="1.0" encoding="utf-8" ?>
<innovators>
  <innovator>
    <server>SERVER </server>
    <database>DATABASE</database>
    <username>USERNAME</username>
    <password>PASSWORD</password>
    <http_timeout_seconds>600</http_timeout_seconds>
    <job>
      <method>METHOD NAME</method>
      <months>MONTH STRING</months>
      <days>DAY STRING</days>
      <hours>HOUR STRING</hours>
      <minutes>MINUTE STRING</minutes>
    </job>
  </innovator>
  <eventLoggingLevel>LOGGING LEVEL</eventLoggingLevel>
  <intervalMinutes>INTERVAL</intervalMinutes>
</innovators>
```



Configuring InnovatorServiceConfig.xml

The following parameters are available in the InnovatorServiceConfig.xml file:

<innovator> tag is used to specify an Aras Innovator server to connect to. There can be multiple **<innovator>** tags in the configuration file.

<server> is the HTTP url of the Innovator server.

<database> is the ID of the database in the InnovatorServerConfig.xml.

<username> is the login name that should be used for running the Methods.

<password> is the password for the login account, in plain-text.

<method> is the Method name in the Innovator instance.

<months> is the month of the year that the Method should be run. Supports a comma delimited list of 1 through 12. January is month=1 and December is month=12.

<days> is the day of the week that the Method should be run. Supports a comma delimited list. Sunday is day=0 and Saturday is day=6. A special value, "last", is used to indicate the last day of the current month.

<hours> is the hours of the day that the Method should be run. Supports a comma delimited list of hour values from 0 through 23.

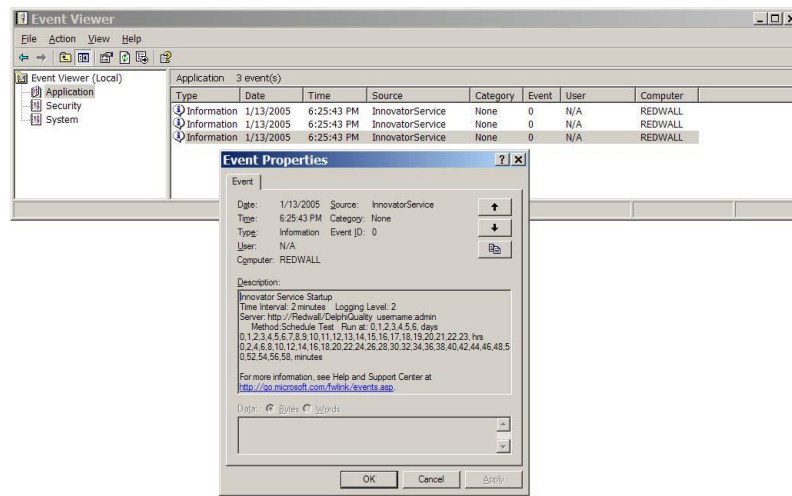
<minutes> is the minute of the hour that the Method should be run. Supports a comma delimited list of minute values from 0 through 59. A special value, "once", is used to indicate the job should be run only "once" when the <hours> criteria is met, but on which minute is not important.

<eventLoggingLevel> sets the level of detail for messages that are sent to the Windows Event monitor.

- 0 = Service startup announcement and error messages from Innovator only
- 1 = Announces the start of every Method run
- 2 = Detailed reporting of every run cycle.

<intervalMinutes> sets the Windows timer to control how often the InnovatorServiceConfig file is polled and the <job> timing logic is tested. Any value from 1 to 60 is supported.

Monitoring the Service – Event Viewer



Monitoring the Scheduler Service

Use the Windows Event Viewer to track when the Service is executed.

The level of logging information displayed is configured in the InnovatorServiceConfig.xml file.

Example

```
<innovators>
  <innovator>
    <server>http://localhost/Innovator12</server>
    <database>DevelopingSolutions12</database>
    <username>admin</username>
    <password>innovator</password>
    <http_timeout_seconds>600</http_timeout_seconds>
    <job>
      <method>Schedule Test</method>
      <months>*</months>
      <days>*</days>
      <hours>*</hours>
      <minutes>0</minutes>
    </job>
  </innovator>
  <eventLoggingLevel>2</eventLoggingLevel>
  <intervalMinutes>1</intervalMinutes>
</innovators>
```



Example Scheduled Email Delivery

In this example, the Method *Schedule Test* is executed every hour. 15 minutes. The Service Interval check is set for 1 minute and detailed reporting (service level 2) is defined.

You can use more than one entry for the timing related tags by using the comma as separator.

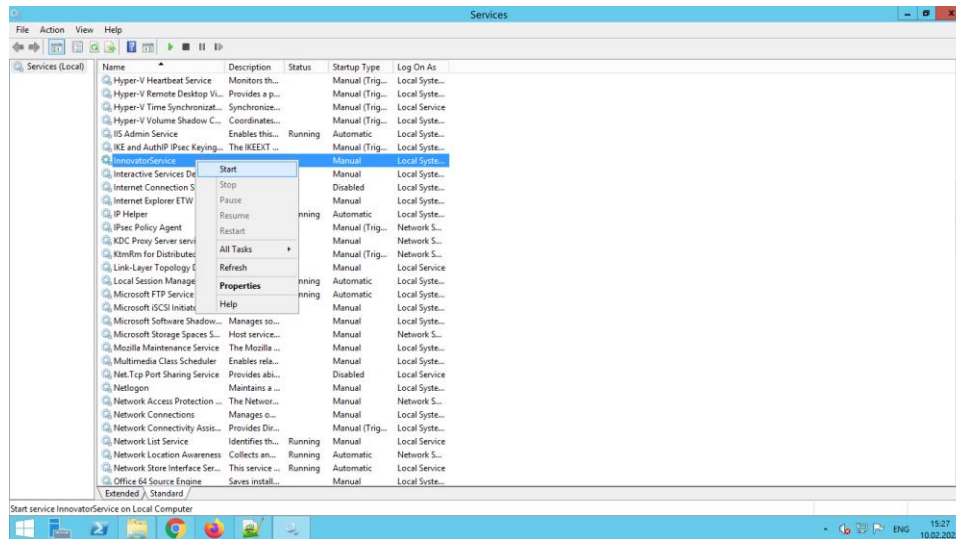
Example: If you want the Method to be executed every 15 minutes you can use the following syntax
 <minutes>0,15,30,45</minutes>

Note

Two standard server Methods are also provided in the Solutions database to check for workflow reminders as well as escalations. The example below shows two sample jobs that can be included in the configuration file to run these recommended Methods on a periodic basis:

```
<job>
  <method>Send Email Reminders</method>
  <months>*</months>
  <days>*</days>
  <hours>0</hours>
  <minutes>2</minutes>
</job>
<job>
  <method>Check Escalations</method>
  <months>*</months>
  <days>*</days>
  <hours>0</hours>
  <minutes>12</minutes>
</job>
```

The Scheduler Service



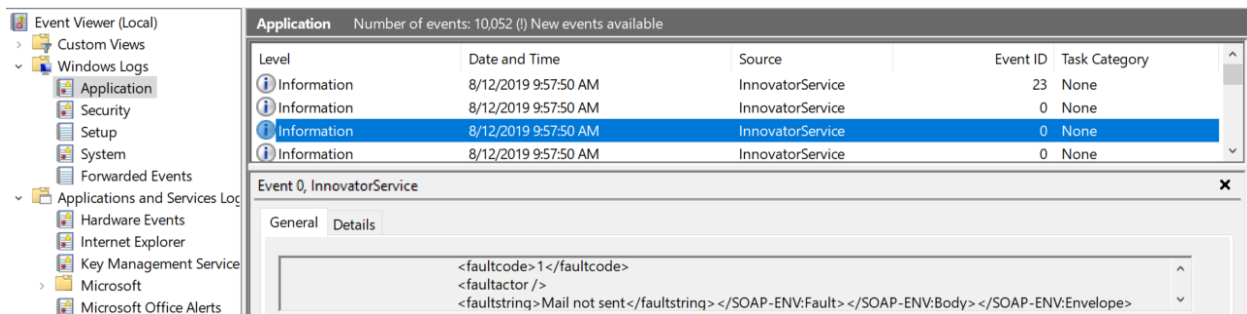
The Scheduler Service

The Scheduler Service can be found with the local Services with the name InnovatorService.

When you start it or when the service is started and the configured time is reached the Method will run.

If the method fails an exception is raised using `newError()` in the Innovator class.

Exceptions can be inspected in the Windows Event Viewer Application log:



Summary

In this unit, you learned how to install and configure the Scheduler Service.

You should now be able to:

- Install the Scheduler Service
- Configure the Scheduler Service to execute Methods periodically
- Monitor the Scheduler Service

Lab Exercise

Goal

Be able use install and use the Scheduler Service to execute a Method based on a time schedule.

Scenario

In this exercise, you will create a simple server Method that logs a message to the debug log each time the Method is executed. You will then install and configure the Scheduler Service to call this Method and check the log to make sure the service is running.

Steps

1. Create a server Method named Write Debug Log that creates a message in the debug log. Use the WriteDebug method (CCO class) you learned about earlier in this course to write a message to the log each time the Method is executed. Test to make sure the message is written to the log.
2. Add a job tag to the InnovatorServiceConfig.xml file to execute the Method.
3. Set the time interval to run the Method once every 2 minutes.
4. Start (or restart) the Innovator Service and check the log file for results.

Appendix A Custom Search Result Sets

Overview

There may be occasions when it is necessary to restrict the search criteria on the Item search grid. The grid appears when a user clicks the ellipsis button on a form field representing an Item property - or when a user searches for a related Item when building a new relationship.

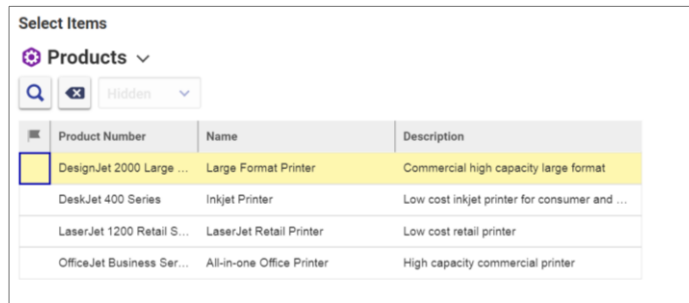
The OnSearchDialog event is invoked in these situations and is available to allow you to predefine the search criteria for an Item search, or to build a custom search result set based on your own business logic.

Objectives

- Defining Custom Search Result Sets

Defining Custom Result Sets

- Define custom business logic to define a result list for a user
- Client side method must create a comma delimited list of item id's
- ID list is set on the Query object
- User cannot access the search criteria bar



Defining Custom Result Sets

To control the results in the Item search grid you can create a Method that populates the grid with Items based on your own business logic. The search criteria bar will be hidden from the user and they will not be able to change the criteria.

This option is useful for presenting a restricted set of Items based on the value of other Items or conditions. In this unit, we will create a Method that prevents a user from selecting a Product unless a Model (relationship) has been established for the Product Item.

The client-side method works directly with the Query object to indicate what results should be displayed to the user. The method must create a comma delimited list of valid Item id's that are passed to the Query object when a user performs an Item search.

Using Method Parameters

- Parameters passed to calling Method:
 - inDom
 - Contains the AML request for the search grid
 - inArgs.windowContext
 - Window that invoked the search
 - inArgs.itemTypeName
 - ItemType name of items to be searched
 - inArgs.itemContext
 - The Item from which the search is being invoked
 - inArgs.itemSelectedID
 - If search is invoked from Relationship Grid – the id of the row



Using Method Parameters

Two objects are passed to the custom search Method which allows you to determine the current object the user is working on as well as other runtime criteria as shown above.

Using Method Parameters

- Parameter that must be set in the Method to define the result set
 - inArgs.QryItem
 - Contains "idlist" attribute that is set to comma delimited list of Item id's



Using Method Parameters

The Method must set the **idlist** attribute of the QryItem object with a valid comma-delimited list of Item id's. The Items will then be presented to the user in the search grid for selection.

Creating the Custom Result Method

- Product Search Example
 - User can only choose from Products that have a Model
- Retrieve a collection of Products that have an assigned Model relationship
- Build a comma-delimited list of id's based on the result
- Set the inArgs.QryItem attribute to define the result set



Creating Custom Result Method

The following Method example named ProductModelSearch demonstrates how to set the parameter object QryItem with a valid idlist.

The goal of this example is to prevent a user from selecting a Product that does not have an assigned Model. Only Products that have an assigned Model relationship will be displayed in the search grid. You can easily customize this query to return a different result for different use cases.

Programmatically supplying items to the search grid prevents the user from overriding the search criteria.

Example source code**//FIND PRODUCTS with MODELS ONLY**

```

const prods = this.newItem("Product", "get");
const relationship = this.newItem("Model", "get");
relationship.setPropertyCondition("item_number", "is not null");
prods.addRelationship(relationship);
const result_prods = prods.apply();

//GET THE COUNT OF PRODUCT ITEMS RETURNED
const count = result_prods.getItemCount();

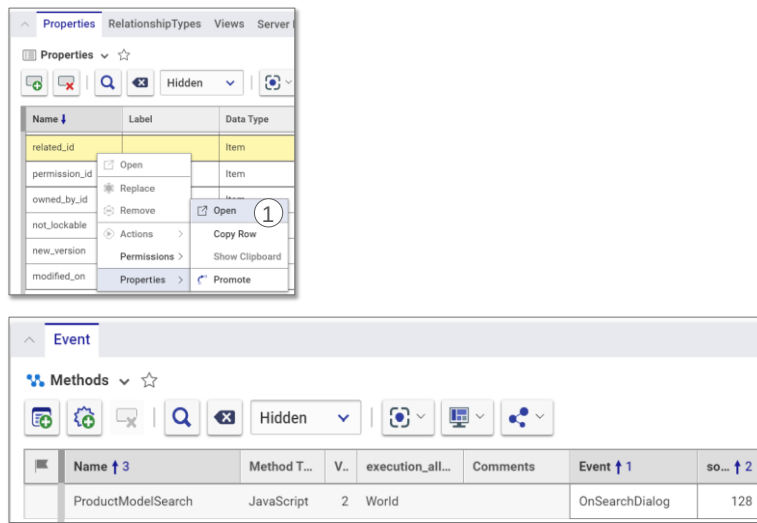
//CREATE A COMMA DELIMITED STRING OF ID'S BY ADDING EACH ID
//TO A SIMPLE ARRAY
const idArray = new Array();
for (i=0; i < count; i++) {
    const item = result_prods.getItemByIndex(i);
    idArray[i]=item.getID();
}

//USE THE JOIN FUNCTION TO CONVERT THE ARRAY TO A COMMA DELIMITED
//STRING
const idlist=idArray.join(",");

//***SET THE idlist ATTRIBUTE ON THE QRYITEM TO THE LIST
inArgs.QryItem.item.setAttribute("idlist", idlist);

```

Associating Method to Event



Associating Custom Method to OnSearchDialog Event

In this example, we want the custom search Method described on the previous page to be invoked when a user tries to establish a new relationship to a Product from a Change Request.

Remember that all relationship Items are configured with two System Properties: `source_id` and `related_id`.

In this example, we want to invoke the custom search Method when the user attempts to add or change the **related_id** of the **Change Request Product** relationship.

To Associate the Custom Method to the OnSearchDialog Event

For this example, open the Change Request Product relationship ItemType for editing and locate the property named `related_id`.

1. Display the Property form by selecting Properties > open from the context menu.
2. Add the custom Method and select the **OnSearchDialog** event.

Summary

In this unit, you learned how to customize Item searches using the OnSearchDialog Event.

You should now be able to:

- Create a Method that sets defines a result set in an Item search grid based on business logi

Appendix B Creating Form Dialogs

Overview

You can use Form Items that are created in Aras Innovator as custom dialog boxes in a solution. This unit explains the steps to create and process a dialog box that uses a standard Aras Form.

Objectives

- Creating a Form dialog
- Populating the dialog
- Returning information from the dialog

Creating Display Dialog Actions

- Use Forms created in Aras Innovator
- Forms can be displayed as an Action result
- Dialog can display (or edit) item data
- Dialog can be sized using form attributes
- Button and Field Events can return data to the server



Creating Display Dialog Actions

You can create your own custom dialogs that take advantage of the Aras Innovator Form Item to display and edit data in a solution.

Dialogs can be used as a result of selecting an Action menu or triggered by a client Method to provide additional information or capture additional data.

Steps to Create an Action Dialog Box

- 1) Create a Form with named fields and optional button(s)
- 2) Create a Client Method to display the Form as a dialog box and process results
- 3) Create a Client Method to populate the fields of the dialog box
- 4) Create a Client Method to return data from the Dialog back to Innovator server
- 5) Create an Action that calls the Method created in Step 2



Steps to Create an Action Dialog Box

Action Dialog Boxes use standard Aras Innovator forms which you display programmatically. You can use them to display additional information about an Item or to provide custom editing capabilities.

The basic steps to create an Action Dialog Box include:

1. Creating a form that will be displayed as the dialog. You can add named fields, buttons and any other HTML controls that will be used in the dialog.
2. Creating a client Method that will show the dialog box based on an Action event and process the results.
3. Creating a client Method on the form that will populate data in the HTML controls.
4. Creating a client Method on the form to return results to the calling Method Step 2 above.
5. Creating an Action that calls the Method created in Step 2 above.

Step 1 – Create the Form

The screenshot shows a dialog box titled "Part 0515-4700" with a close button (X) in the top right corner. Inside the dialog, there is a text area labeled "Partdescription" with a circled number 1 next to it. Below the text area are two blue buttons: "Save" (with a circled number 2) and "Cancel" (with a circled number 3). The Aras logo is visible in the bottom left corner of the dialog box.

Create the Form

In this example, a Form is created that will be used as an Item action assigned to the Part ItemType. This will allow a user to edit a Part description without opening the Part item.

First, create a new Form item that will be used to display (or edit) property data.

In this example, a Form has been created named *PartEditDescription* with three elements:

- ① **Part Description Text Area**
A Text Area control has been added to the Form and named *partdescription*.
- ② **Save Button**
Field event code will be added to close the window and return the text from the *partdescription* Text Area.
- ③ **Cancel Button**
Field event code will be added to close the window with no results.

The dialog title will be assigned the current part number.

Step 2 – Display Form Method

- Define Working Variables
- Get Form Width and Height
- Define Dialog Parameter Object
- Call `ArasModules.Dialog.show()` method
- Define Callback function from show method



Display Form Method

The following Client Method named *ActionPartDescriptionEdit* shows how to call an Aras dialog that uses a Form item. The Method also uses several functions available in the aras object library.

Define Working Variables

Several variables are established to define which Form will be used as well as the part number. An aras object function retrieves the topmost window in the Aras client so that the dialog is displayed correctly. In this example, the Method also makes sure that the item is not already locked.

```
if (this.isLocked()) {
    aras.AlertError("Unclaim item first!")
    return;
}

const formName = "PartEditDescription";
const itemtype = this.getAttribute("type");
const keyedname = this.getProperty("keyed_name");
const title = itemtype + " " + keyedname
const part = this;
const win = aras.getMostTopWindowWithAras(window);
```

Get Form Width and Height

An aras object function is used to retrieve the form height and width values to size the dialog appropriately.

```
const formNd = aras.getItemByName('Form', formName);
const width = aras.getItemProperty(formNd, 'width');
const height = aras.getItemProperty(formNd, 'height');
```

Define Dialog Parameter Object

The Dialog function requires a parameter object to define which form will be displayed. The current part item is also passed to the dialog using the *item* parameter.

```
const param = {
  title: title,
  formId: formNd.getAttribute('id'),
  aras: aras,
  dialogWidth: width,
  dialogHeight: height,
  content: 'ShowFormAsADialog.html',
  item: this
};
```

Call ArasModules.Dialog.show()

The show method displays the dialog using the param object above and establishes a "promise" to invoke the function named callback when the user closes the dialog.

```
(win.main || win).ArasModules.Dialog.show('iframe',
param).promise.then(
  //If promise resolved then invoke callback function
  function (dialogResult) {
    callback(dialogResult);
  }
);
```

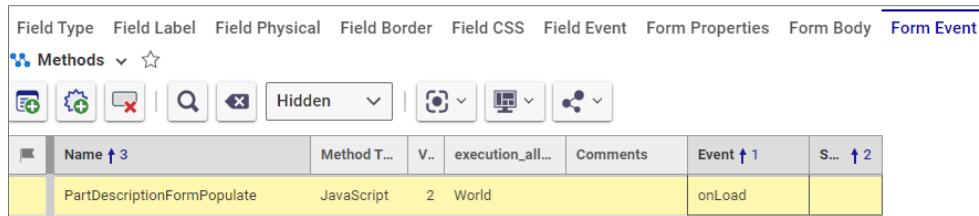
Define Callback Function

A function is defined to process the results of the dialog. In this example the description property of the current part item is updated with the results from the dialog.

```
function callback(results){
  if (!results) return;
  part.setAction("edit");
  part.setProperty("description", results);
  const p = part.apply();
  if (p.isError()){
    aras.AlertError(p.getErrorString());
  }
}
```

Step 3 – Form onLoad Method

- Retrieve values from passed Context Item
- Assign values to Form elements



Form onLoad Method

The following Client Method named *PartDescriptionFormPopulate* will be used to populate the dialog elements with data from the passed context Item. The current part will then be available in the *document.thisItem* DOM variable and property data can be retrieved to populate form elements. In this example the description of the current part is used to populate the *partdescription* Text Area control.

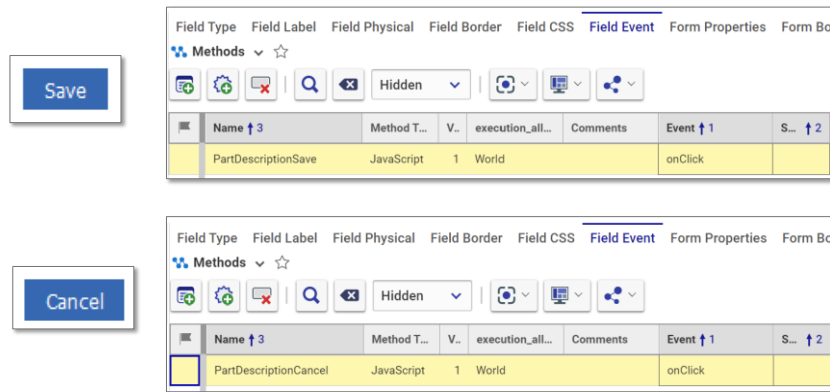
```
const description = document.thisItem.getProperty("description","");

//Set value on form element
document.getElementById("MainDataForm").partdescription.value =
description;
```

Assign this Method to the onLoad event of the *PartEditDescription* Form.

Step 4 – Return Data Method

- Retrieve data from form control
- Return dialogArguments (with data)



Return Data Method

The following Client Method named *PartDescriptionSave* will return the data entered into the *partdescription* Text Area control.

```
const returndata =
document.getElementById("MainDataForm").partdescription.value;

window.parent.dialogArguments.dialog.close(returndata);
```

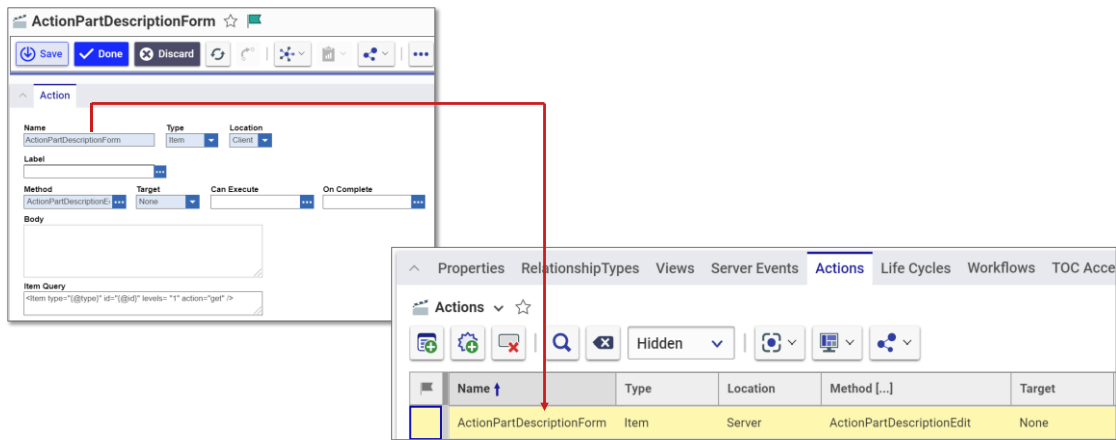
An additional Client Method named *PartDescriptionCancel* will simply close the window with no results returned.

```
window.parent.dialogArguments.dialog.close();
```

Assign the *PartDescriptionSave* Method to the onClick Field Event of the Save button.

Assign the *PartDescriptionCancel* Method to the onClick Field Event Cancel button.

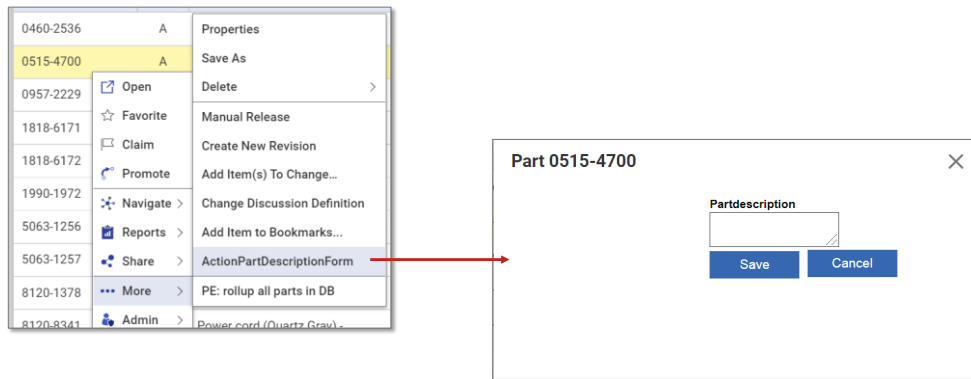
Step 5 – Create Item Action



Create Item Action

To display the dialog from a client Action you will need to create an Action of type Item and supply the *ActionPartDescriptionEdit* Method described earlier. This Action should then be assigned to the Part ItemType.

Using the Action



Using the Action

Once complete, this Action can now be used as an alternate way to quickly edit a Part description without opening the Part item window.

Summary

In this unit, you learned how to create a custom dialog using a Form Item.

You should now be able to:

- Create a dialog that uses a Form Item
- Populate and return data from the dialog
- Return results from the dialog

This page intentionally left blank.

Appendix C Customizing a Form

Overview

When you create a Form in Aras Innovator, the form is rendered as an HTML page at runtime. You can modify the attributes of the form or perform custom operations on HTML controls using client Methods.

Objectives

- Customize logic on form HTML controls

Creating Custom HTML Form Controls



- Form/field events and Client Methods define the values and behavior of the control
- Can be bound to a property using a Client Method



Creating Custom HTML Form Controls

You can define your own controls on an Item Form by selecting the HTML Control element in the Form Editor toolbar.

HTML Controls allow you to provide your own HTML source code to define the visual display and how the control will interact with the user.

Because forms use standard HTML with JavaScript you can define how the control will behave and appear to the user.

Building a Dynamic DropDownList

- List is created dynamically from any data source
 - Create a Form Dropdown element
 - Create a Form Event Method to populate the list
 - Create a Field Event Method to capture result



Building a Dynamic Dropdown List

This example shows how to create a dynamic dropdown list that can be populated from any data source using a client Method. In this example, we will be using a collection of Items from the Aras database. Also, in this course, you learned how to use federated data to access outside resources.

To Build a Dynamic Dropdown List

You must:

1. Place the HTML dropdown control on the Form and supply the appropriate actions to be performed.
2. Create a Form Event Method to populate the control using JavaScript.
3. Optionally, create a Form Event Method to capture the selected result into a property.

Building a Dynamic Dropdown List

- Create the HTML Dropdown Control



- Create a property to hold the selected value
- Populate Control
 - Create a Client Method for the onFormPopulated Event
- Capture Result
 - Create Client Method to set a property for the onChange Event



Building a Dynamic Dropdown List

Defining the HTML Form Control

1. Create a string property to hold the value of the selected choice. In this example, the property is named *vendor*.
2. Create an HTML Form element by selecting the dropdown control from the Form control toolbar menu.
3. Name the control and place it on the form in an appropriate location. In this example, the control is named *VendorList*.

Populating the Control

1. Create a Client Method to populate the control using Javascript.
2. Associate this Method with the onFormPopulated Form event. In this example, the dropdown will be populated with Vendor items. The vendor name is displayed to the user and the selected vendor id is set in the vendor property.

```
//get dropdown Control Element
const select = getFieldComponentByName('VendorList');
const itm = document.thisItem;
const opt=[ ];

//get Vendors
const v = itm.newItem("Vendor", "get");
v.setAttribute("select", "name");
returnItm = v.apply();

//populate the select list with options
for (let i = 0; i < returnItm.getItemCount(); i++) {
  const vendor = returnItm.getItemByIndex(i);
  const str_name = vendor.getProperty("name");
```



```
        opt.push({ label: str_name, value: str_name});  
    }  
    select.component.setState({list: opt,value:''});  
  
    //Set dropdown value to current property value  
    select.value = itm.getProperty("vendor", "");
```

Capturing the Selection

1. Create a Client Method to set the vendor property. Associate this Method with the onChange event of the dropdown field.

```
window.handleItemChange("vendor", this.value);
```

Changing Field Attributes

- Change Colors, Set Focus

Customer



Name

Bike Land Inc.

Main Phone

555-1212

Contact Name

K Smith



Changing Field Attributes

You can obtain a handle to any form control and the set styles or attributes using standard JavaScript. This example shows how to set the focus on a specific field (Name) and change the field color.

Place the following code in a Form onLoad event:

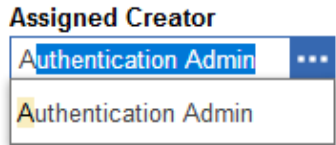
```
//get the 'name' text field control
const name_field = document.getElementById("MainDataForm").name;

//change the color of the name field
name_field.style.backgroundColor="#808080";

//Put focus on the name field (cursor) when form opens
name_field.select();
```

Populating Item Field using Type-Ahead

- Dynamically populate an Item type field when user enters characters



- Create Client Method
- Assign to Form onFormPopulatedEvent



Populating Item Field using Type-Ahead

The Item field type contains a built-in feature that allows a user to enter in characters that are used as part of a query to display a selectable list of matches.

To override the standard query requires a custom client Method that is associated with the Form onFormPopulated event.

The following client Method will filter out any identities that are not considered individuals (is_alias = 1) on the owned_by_id property of a Part.

1. Create a client Method to override the standard query:

```
const item = getFieldComponentByName('owned_by_id');
item.component.request = function() {
  //Arguments passed to Method:
  const itemType = this.state.itemType;
  const maxCount = this.state.maxItemsCount;
  const label = this.state.label;

  //Create a query based on new filter
  const r = document.thisItem.newItem(itemType, "get");
  r.setAttribute("select", "keyed_name");
  r.setProperty("is_alias", "1");

  //Make soap call using AML string of request
  return ArasModules.soap(r.toString(), {async: true});
}
```

2. Associate this Method with the Part Form onFormPopulated event.

Summary

In this unit, you learned how to define custom logic for HTML form controls.

You should now be able to:

- Create custom logic for HTML form controls

Appendix D Tracking Login and Logout

Overview

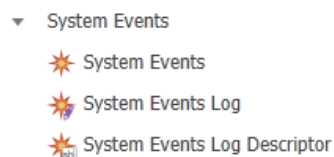
Three system events can be configured to track successful user login, failed login and user logout. You can provide conditional business logic to extend (or prevent) these events with a server Method. A class utility is also available to create System Log Items which track login activity.

Objectives

- Overview of System Events
- Configuring System Events
- Creating a System Log Item with Server Method.

System Events

- Allow tracking and validation of
 - Failed Login
 - Successful Login
 - Logout
- Related to System Events Handler Method
- Handler Methods can create a System Events Log Item defined by a System Events Log Descriptor Template



System Events

System Events allow you track successful logins, failed logins as well as session logouts. Each event can be associated with a System Events Handler Method that can access the current login name as well as other session information.

An additional System Events Log Descriptor can be created for each event type that contains a log level number (to determine when logging should occur) as well as a custom log message. Using a provided IOM Method you can create System Events Log Items in the database based on the configuration settings in the Log Descriptor.

Logging System Events

- Define a new System Event for each Event Type
 - Failed Login
 - Successful Login
 - Logout
- Create a System Events Log Descriptor for each Event Type
- Define one or more Method handlers for each System Event
 - Create System Events Log Items

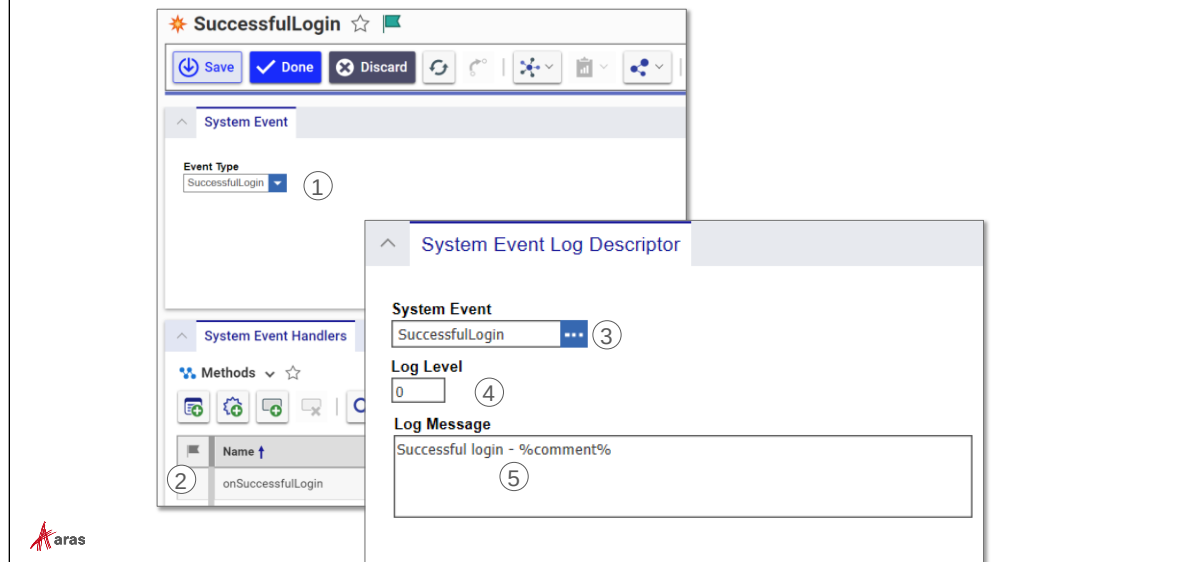


Logging System Events

To log a system event, you must perform the following steps:

1. Create a new System Events Item and select the appropriate Event Type.
2. Create a System Events Log Descriptor Item for each Event Type to be configured.
3. Create one or more server Methods for each System Event to handle the event. Typically, the Method will create a System Events Log Item to indicate successful or unsuccessful logins.

Configuring System Events



Configuring System Events

To configure a System Event:

1. Select Administration > System Events from the TOC to create a new event Item. Select the Event Type from the dropdown.
2. Select one or more server Methods that will be executed when this event is triggered.
3. Select Administration > System Events Log Descriptor from the TOC to create a new descriptor Item. Choose the Event Type from the selection field.
4. Enter a log level number. This number indicated if the event should be triggered or not and is compared to the System Variable named *SystemEventLogLevel*. If the log level number is greater than or equal to the system event log level the event will be triggered, otherwise it will be ignored.
5. Enter a log message that will appear in the System Events Log Item when it is created by the system event handler Method. The following variables are available for substitution at runtime: %login_name%, %ip_address%, %method_name%, %event_type%, %session_id%, %comment%.

Tracking Successful Login

```
//Store Last Successful Login Name
Item itm = this.newItem("Variable", "edit");
itm.setAttribute("where", "[variable].name='LastLogin'");
itm.setProperty("value", eventData.LoginName);
itm.apply();

//Create System Events Log
CCO.SystemEventLogger.CreateSystemLogRecord("SuccessfulLogin"
, "onSuccessfulLogon", eventData.LoginName, "Login
Successful");
return this;
```



Tracking Successful Login

The example above demonstrates an event handler Method for a SuccessfulLogin event.

This method first stores that name of the login in a System Variable named LastLogin.

The Method then creates a new System Events Log Item using the CreateSystemLogRecord method available in the CCO (Core Context Object) class.

The CreateSystemLogRecord takes the following arguments:

- Event Type (string) – "SuccessfulLogin", "FailedLogin", or "Logout" are valid system event types.
- Method Name (string) – name of the handler Method.
- Login Name (string) – the user login name. A reserved object (eventData) contains an attribute named LoginName which contains the current user's login.
- Comment (string) – additional comments that are added the System Events Log Item (if a comment has not been provided in the System Events Log Descriptor).

Summary

In this unit, you learned how to configure Systems Events for tracking login and logout.

You should now be able to:

- Configure a System Event and Log Entry
- Create a System Event Method

Appendix E Working with Relationship Grids

Overview

Relationships are presented to the user as a tabbed grid on a standard tear-off window. You can react to grid events using row and cell callback Methods to provide additional business logic for the grid.

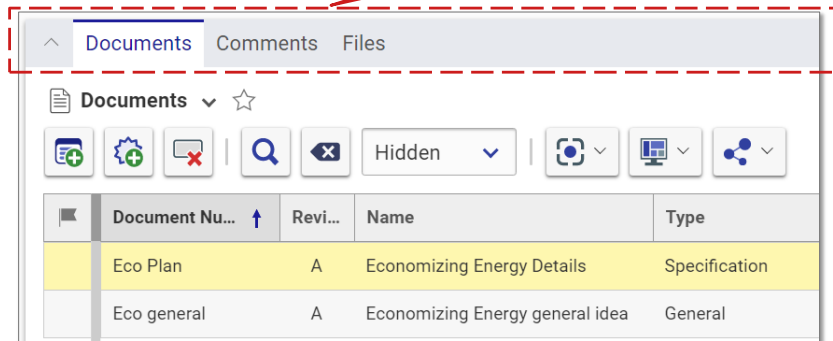
Objectives

- Access the Tab Bar Control
- Access the Grid Control
- Create Custom Grid Row/Cell Callback Methods

Working with Relationship Tabs

- Useful for disabling or removing tab from view in Relationship Grid
- Uses the relTabbar Control

`parent.relationships.relTabbar`



Working with Relationship Tabs

The standard Aras Innovator Form always contains a grid with a set of tab controls to display relationship information to the user.

You can control the tab(s) by disabling or hiding/showing the tabs.

The relTabbar control is accessed from the DOM using the expression *parent.relationships.relTabbar*

Tab Bar Methods

- **SetTabVisible** (*tabID*, *boolean*)
 - Hides (or shows) a Tab in a Grid
- **SetTabEnabled** (*tabID*, *boolean*)
 - Disables/Enables a Tab
- **GetSelectedTab**();
 - Returns id of user selected tab
- **GetTabId**("tab label");
 - Returns tab id label name



Tab Bar Methods

Several tab bar client methods are available to manipulate the tab bar controls. Examples of these methods are demonstrated on the following pages.

Hide or Disable a Tab

- Hide a tab (Form onLoad Event)

```
const tabbar = parent.relationships.relTabbar;  
const tabid = tabbar.GetTabId("Documents");  
tabbar.SetTabVisible(tabid, false);
```

- Disable a tab (Form onLoad Event)

```
const tabbar = parent.relationships.relTabbar;  
const tabid = tabbar.GetTabId("Documents");  
tabbar.SetTabEnabled(tabid, false);
```



Hide or Disable a Tab

These examples are designed to be work with the Form onLoad Event.

The SetTabVisible method allows you to hide or show the tab while the SetTabEnabled method enables/disables a tab control. Use the boolean true or false as the second argument to indicate whether a tab should be shown/enabled (true) or hidden/disabled(false).

Note

This example uses the GetTabId method to select a control by label name. You can also substitute this method with the GetSelectedTab() function to act on the tab currently selected by a user.

GetSelectedTab() returns the tab id of the actively selected tab.

```
const tabid = tabbar.GetSelectedTab();
```

Example

The following example hides a tab labeled *Documents* when a Form opens. The Method is associated with the OnLoad Form Event. A JavaScript setInterval has been added to allow for the form to fully display before the tab operation is executed.

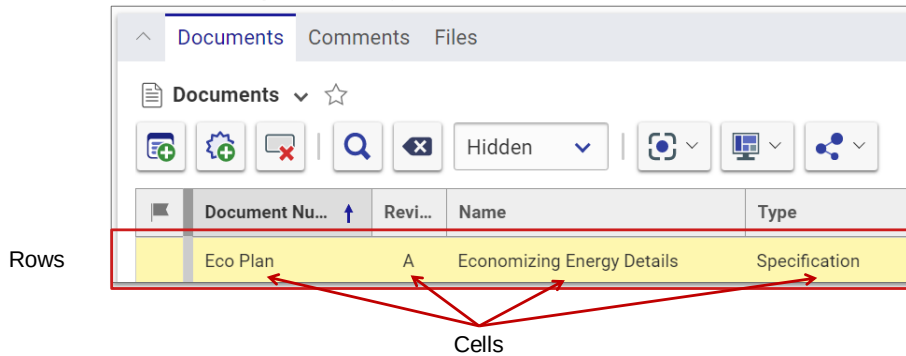
```
document.hideTab = function() {
    const showTab = false;
    const tabbar = parent.relationships.relTabbar;
    const tabID = tabbar.GetTabId("Documents");
    if (!tabID) {
        aras.AlertError("Cannot access tab");
    }
    else {
        tabbar.SetTabVisible(tabID, showTab);
    }
    return;
};

const interval = setInterval(checkTab, 1000);

function checkTab() {
    //Check if Tabs are ready
    if (parent.relationships.relTabbar !== null )
    {
        clearInterval(interval);
        document.hideTab();
    }
}
```

Working with Relationship Grids

- Standard Tear-off Window contains a Relationship Grid
- Represented by the Grid Control
- Reacts to User Input using Grid Events



Working with Relationship Grids

Relationship grids represent relationships from the current source Item. You can modify and provide custom logic in the standard relationship grid by accessing the grid control.

Relationships Window Layout

parent.relationships

parent.relationships.reltabbar

parent.relationships.iframesCollection[tab_id].contentWindow.grid

Document Nu...	Rev...	Name	Type	State
Eco Plan	A	Economizing Energy Details	Specification	Preliminary
Eco general	A	Economizing Energy general Idea	General	Preliminary

Relationships Window Layout

Each tear-off window can contain a relationships grid, which allows users to manipulate relationship data from the form.

When a user selects a tab, a corresponding iFrame appears based on the id of the tab. A content window is nested in each iFrame which contains the corresponding relationship grid data in rows and cells as shown.

Relationship Grid Methods

- **GetCellValue**
 - Return value of a cell
- **cells**
 - Set or get value of a cell
- **getRowCount**
 - Get number of grid rows



Relationship Grid Methods

The relationship grid container allows you to either set or get the value of a cell as well as obtain the number of rows in the grid.

The methods shown above are current as of Version 10 SP1 of Aras Innovator and may be amended in future releases.

Accessing Grid Control

- Create Method for Form onLoad Event

```
document.TabRowCount = function() {
    const tabbar = parent.relationships.relTabbar;
    const tabid = tabbar.GetSelectedTab();
    const ifrm = parent.relationships.iframesCollection[tabid];
    const grid = ifrm.contentWindow.grid;
    const count = grid.getRowCount();
    return count;
};
```

- Call from any Field Event

```
const numrows = document.TabRowCount();
document.thisItem.setProperty("row_count", numrows);
```



Loading JavaScript Functions with onLoad Event

In this example, a client Method is defined that creates a JavaScript function that will return the number of rows in a grid. Note that the function will reside on the “page” representing the form in the browser but not be executed until called by a Field Event.

The Field Event code can then call this function anytime the grid count is required by a control.

Note

The example above accesses a tab selected by the user. The tab id is obtained by calling the GetSelectedTab function of the relTabbar control. Each grid is contained in an iFrame collection which is indexed by the corresponding tab id.

Creating Custom Grid Row/Cell Callbacks

- Row Grid Events respond to user interaction with the grid control rows
(delete, insert, select)
- Cell Grid Events respond to user interaction within a relationship grid edit cell
- Associated Methods receive arguments relative to the grid control
- Context Item is available using:

parent.thisItem



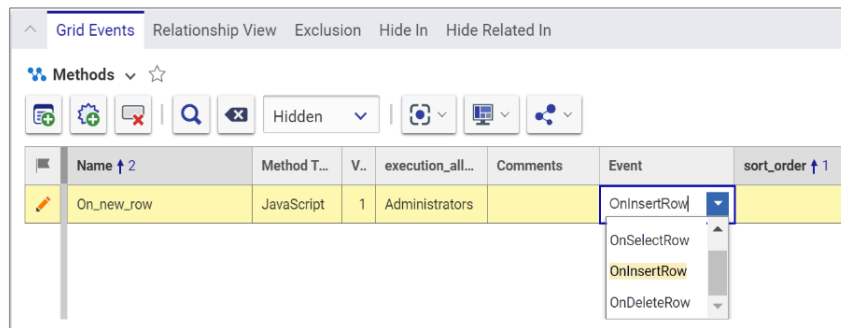
Custom Grid Row/Cell Callbacks

To respond to a user's interaction with a Relationship grid, you create client Methods and associate them with events on the Grid Rows and Cells.

Because the grid is considered a child of the current HTML document you can use the `parent.thisItem` reference to access the context of the current Item associated with the grid.

Configuring Row Events

- Row Grid Events defined on the RelationshipType
 - OnSelectRow
 - OnInsertRow
 - OnDeleteRow



Configuring Row Events

Row events are defined on the RelationshipType template in the Grid Events tab.

Three events allow you respond to the insertion, selection or deletion of a grid row by a user.

Grid Row Event Method Arguments

- Method receives the following arguments:
 - **relationshipID** = ID for the Relationship Item. This is also the selected row ID in the Grid control
 - **relatedID** = the ID for the related Item. (Blank for null relationships)
 - **gridApplet** = handle to the Grid control



Grid Row Event Method Arguments

The OnInsertRow, OnSelectRow and OnDeleteRow all receive three arguments, which allow you to programmatically access the current Related Item, the current Relationship as well as the grid control itself.

Defining a Grid Row Event Method (OnInsertRow)

```
const team =
  parent.thisItem.getPropertyAttribute("team_id", "keyed_name");
const msg = "Assigned team: " + team;
const i = gridApplet.getColumnIndex("assigned_team_D");
const x = gridApplet.cells(relationshipID, i).setValue(msg);

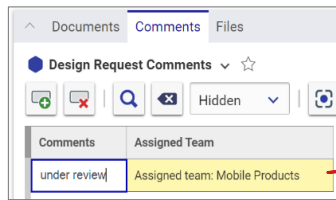
const relItem = parent.thisItem.getItemsByXPath("//Item[@id='" +
  relationshipID + "']");
relItem.setProperty("assigned_team", msg);
```

Get Property
from Parent Item

Set Grid Cell
Value

Get Item in dom
cache to set value

Column set in
control and cache



Defining a Grid Row Event Method

In this example, the cells method of the grid control is used to set the value of the column when a new row is inserted. The column index number is determined by the sort order of the properties assigned to the relationship and is zero based. The index number can also be retrieved from the column name – the column is named automatically with the property name appended with "_D". (See notes below on how columns are named.)

The relationshipID is passed to the grid row event as a standard argument and is used to locate the relationship Item in the cached DOM by using the Item method `getItemsByXPath`.

Finally, the property is set in the client cache for the relationship so when the user saves the parent Item this information will be saved to the database as well.

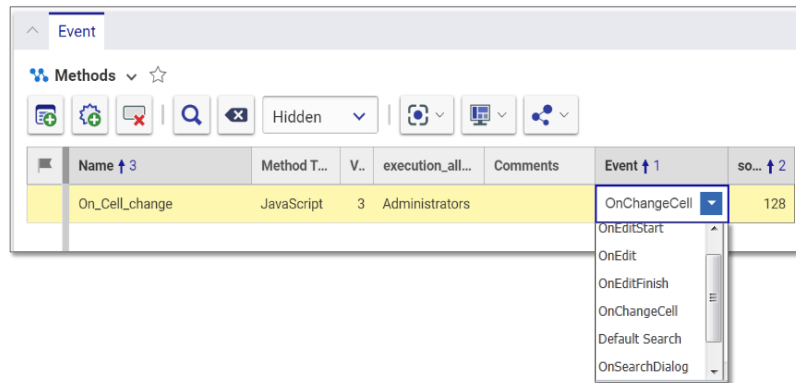
Notes

You must set both the control value as well as the Item cache property for the value to be successfully displayed and saved to the database.

Relationship column names are derived by taking the name of the property shown in the column and appending an underscore and a letter. If the column refers to a property of the *related* item, then `_R` is appended to the property name. Otherwise, if the property refers to a *relationship* property `_D` is appended instead.

Configuring Cell Events

- Cell Grid Events defined on the **Property** of the corresponding Grid Cell
 - onEditStart
 - onEditFinish
 - onChangeCell
 - onSearchDialog



Configuring Cell Events

Each cell in the relationship grid is also capable of responding to a series of events while a user interacts with the grid.

A grid cell is represented as a property value of either the relationship item or the related item based on the configuration of the relationship.

Access the appropriate ItemType and right click on property that represents the relationship cell. Select View Property to access the Event tab and select the appropriate event.

Note

Default Search is a deprecated event – use *onSearchDialog* for any new projects.

Grid Cell Event Method Arguments

- Method receive the following arguments:
 - **relationshipID** = ID for the Relationship Item. This is also the selected row ID in the Grid control
 - **relatedID** = the ID for the related Item. (Blank for null relationships)
 - **gridApplet** = handle to the Grid control
 - **propertyName** = name of the Property for the currently selected cell
 - **colNumber** = zero based column position number in the grid of the current cell



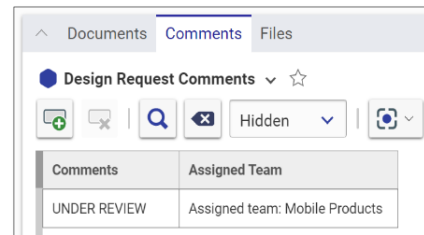
Grid Cell Event Method Arguments

The client Method receives a set of standard parameters that can be used to identify the current relationship, the related Item, the current Property name of the cell as well as its column number (zero based).

Defining a Grid Cell Method (OnChangeCell)

```
const g = gridApplet;
const val = g.GetCellValue(relationshipID, colNumber);
const uperval = val.toUpperCase();
g.cells(relationshipID, colNumber).setValue(uperval);

const relitm =
  parent.thisItem.getItemsByXPath("//Item[@id='" + relationshipID
    + "']");
relitm.setProperty("_comments", uperval);
```

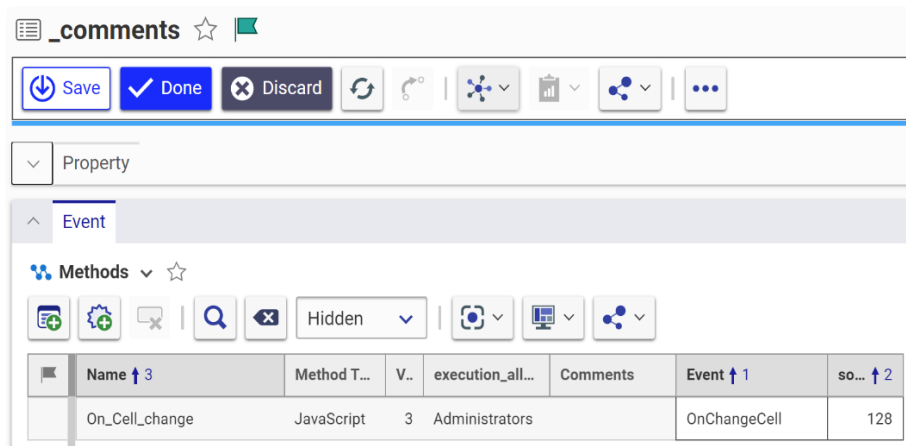


Defining a Grid Cell Method

In this example, the value of the current cell is retrieved using the `GetCellValue` method. The letters of the string are then set in upper case using the JavaScript `toUpperCase` function.

The modified string is then set back into the same cell using the `cells` method and the DOM is updated so the change will be reflected when the parent Item is saved to the database.

This Method is then attached to the `_comments` property of the Design Request Comments ItemType using the `OnChangeCell` event.



Summary

In this unit, you learned how to create define custom logic for Relationships grids.

You should now be able to:

- Access the tabs in the Relationship Grid
- Create callback Methods for row and cell events in the Relationship grid

This page intentionally left blank.



Appendix F Using Message Resources

Overview

In this unit, you will learn how to retrieve and build new message resources for Client and Server Methods. Embedding prompt and error message text in program code leads to more difficult maintenance and the inability to display messages in multiple languages. The Aras Innovator platform provides two approaches to centralize messages displayed from both Client and Server Methods.

Objectives

- Benefits of Message Resources
- Retrieving messages in a Client Method
- Retrieving messages in a Server Method
- Creating new client UI resource messages
- Creating new server UserMessages

Benefits of Message Resources

- Text messages displayed in errors and prompts can be centralized vs. hard coded in Method source code
- Provide a way to localize/internationalize your applications by automatically determining which language to display based on the user's locale
- Message maintenance is simplified and changes can be made independent of the calling Method code



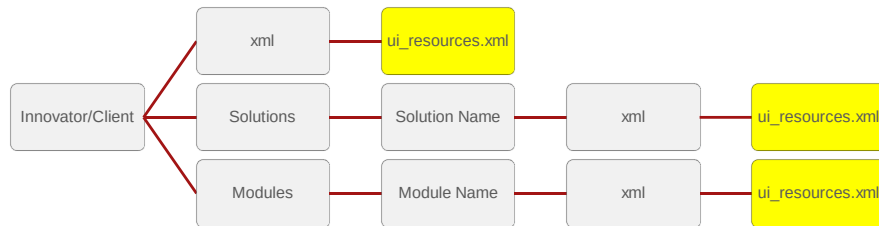
Benefits of Message Resources

Text messages can be displayed from either a Client or Server Method as a result of either a prompt or error message. Embedding the text messages into program code can cause future maintenance issues and prevents the use of multiple language support in enterprise applications.

By centralizing text messages into a resource library makes it easier to maintain (and translate) text messages that will be displayed from a program.

In this unit, you will learn how the Aras Innovator platform stores and retrieves text message resources for both Client (JavaScript) and Server (VB/C#) Methods.

Client Message Resources



Client Message Resources

Messages that will be displayed from a Client (JavaScript) Method are stored on the Aras server in XML formatted resource files using the directory structure shown above. The message resource file is always named *ui_resources.xml*.

Messages in the *ui_resource* file under *Innovator/Client/xml* are considered "root" messages and are not tied to any specific application.

Messages in *ui_resource* files within the subfolders under the *Solutions* folder are specific to individual applications. Each subfolder contains a *ui_resources* file appropriate for that application. Aras standard Solutions folders include (*PLM* (Product Engineering), *Project* (Project Management), and *CE* (Component Engineering)).

Additionally the *Modules* folder contains subfolders with *ui_resources* files that contain messages specific to different components in the Aras platform.

Note

Be careful not to make any changes to the existing folders and files created during the Aras installation. Modifying these files can cause problems with the Aras core platform as well as the standard applications.

Client Resource File Layout

```
<ui_resources>
  <resource key = "keyname" value = "message text {n}" />
</ui_resources>
```



Client Resource File Layout

The `ui_resources.xml` file defines a key name identifier and message text.

Message resources are contained within an XML tag named `<ui_resources>`.

Each message entry is contained in a `resource` tag with a *keyname* and *value* attribute.

Keyname

The *keyname* attribute contains the lookup key for this entry.

Value

The *value* tag contains the message text to be displayed.

Data

The calling program can also pass string arguments to a message to make it more versatile. For a message to display the passed data, one or more text placeholders can be defined using the curly brace syntax as shown above. Place holders are numbered starting with 0. Subsequent placeholders need to be numbered in sequence.

The following message would accept two arguments from the calling program:

```
"The {0} property must have a value greater than {1}"
```


Retrieving Client Messages

- `aras.getResource(location, keyname, [data]);`
- *location* options
 - Empty string "" (root)
 - Solution directory (e.g PLM, Project, CE)
 - Specific directory (..\Modules\aras.innovator.CMF)
- Examples
 - `aras.getResource('', "aras_object.edit_again");`
 - `aras.getResource("PLM", "createnewrevision.confirm");`
 - `aras.getResource("../Modules\aras.innovator.CMF", "new_item_selected")`



Retrieving Client Messages

The Aras Object provides a method to retrieve message resource defined for the client. The *getResource* method can retrieve messages from several locations. The standard locations are root level messages, messages specific to a Solution and messages specific to an Aras Module.

To retrieve a message from the root level, the location must be the empty string "". If a Solution name is provided, the message is retrieved from the corresponding Solutions subfolder. A specific directory location can also be provided as shown in the Modules example above.

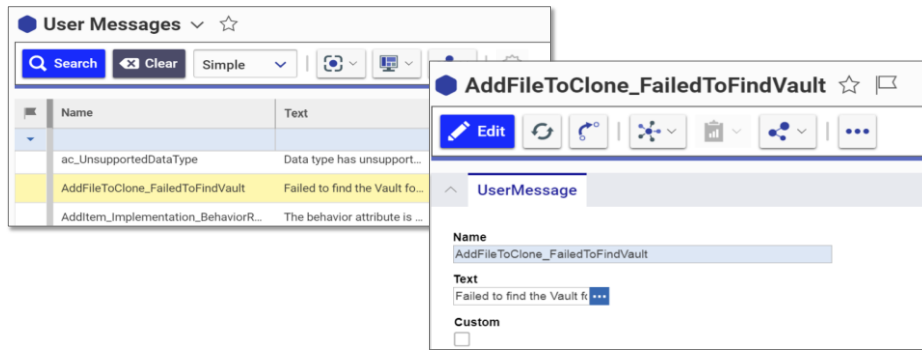
Example

The following Client Method displays an error message using a root level message:

```
const ITname = "Part";
const it = this.newItem("ItemType", "add");
it.setProperty("name", ITname);
it.setProperty("structure_view", "tabs off");
const final_it = it.apply();
if (final_it.isError()){
    const m = aras.getResource('',
        "itemtype_methods.itemtype_already_exists", ITname);
    aras.AlertError(m);
}
```

Server Message Resources

■ UserMessage Items



Server Message Resources

Message resources for server Methods are stored in the database in an Item of type UserMessage. To view the existing messages in the database, edit the UserMessage ItemType and adjust the TOC Access. A default view has been created for the Item which lets you see the keyname and text message. A boolean Custom property can be set for messages you add to the database.

Retrieving Server Messages

- `CCO.ErrorLookup.Lookup("keyname" [,data]);`

- **Examples:**
 - `CCO.ErrorLookup.Lookup("EvaluateActivity_InternalError");`
 - `CCO.ErrorLookup.Lookup("ExecuteMethodByName_methodNotFound",
method_name);`



Retrieving Server Messages

To retrieve a message string from a UserMessage item, a method is provided named Lookup in the CCO.ErrorLookup class which is part of the Aras platform framework. A keyname and optional data string(s) can be provided to retrieve and display a store message.

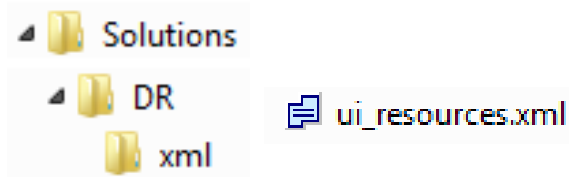
Similar to client messages, the message text can also contain text placeholders {0}, {1}, etc. which then display data strings passed in the Lookup method.

Example

```
if (x == null) {
    string m =CCO.ErrorLookup.Lookup("EvaluateActivity_InternalError");
    return this.getInnovator().newError(m);
}
```

Creating a New Client Resource

- Define new Solutions folder
- Create `ui_resources.xml` file
- Provide keyname and message text



```
<resource key = "dr_calendar_required" value = "Calendar is required!"/>
```



Creating a New Client Resource

The best practice to add you own application client messages is to create a new folder under the Solutions folder.

In the example above, a new subfolder named *DR* is created with a corresponding *xml* subfolder. A new resource file must be created and named *ui_resources.xml*.

Finally, a new keyname/message is added to the file:

```
<ui_resources>
  <resource_key="dr_calendar_required"
    value="Calendar is required!" />
</ui_resources>
```

To call this resource in a Client Method use the Arass `getResource` method:

```
const m = aras.getResource('DR', " dr_calendar_required");
```

Creating a New Server Resource

- Create new UserMessage item
- Provide keyname and message text
- Select "Custom" option

The screenshot shows a web form titled "DR_patent_required". At the top, there are three buttons: "Save" (with a download icon), "Done" (with a checkmark icon), and "Delete". Below these buttons is a tab labeled "UserMessage". Under the tab, there are three sections: "Name" with a text input field containing "DR_patent_required", "Text" with a text input field containing "Patent is required!" and a blue ellipsis button to its right, and "Custom" with a checked checkbox.



Creating a New Server Resource

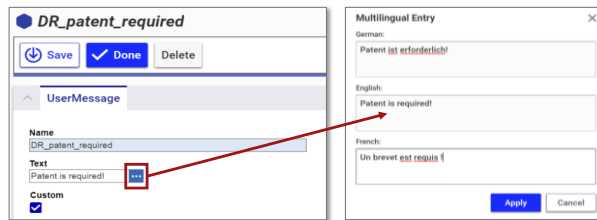
To create a new server message, create a new UserMessage item. You may need to modify the Can Add and Permissions on the ItemType. Enter a key name and message text. To isolate your application messages from the Aras standard messages, check the Custom box.

To retrieve this message in a Server Method, use the Aras Lookup method:

```
string m = CCO.ErrorLookup.Lookup("dr_patent_required");
```

Supporting Multiple Languages

- **Client**
 - Create xml.[country code] folder
 - Provide language specific text with same keyname
- **Server**
 - Provide translated text in multilingual property of UserMessages item



Supporting Multiple Languages

The Aras platform supports multiple languages – for more information about configuring multiple languages download the PDF document *Aras Innovator - Configuring Internationalization* from the aras.com website.

Client

To support multiple languages on the Aras client you must create a separate xml subfolder under the root, Solutions or Modules folders as described earlier. Append the ISO country code to the xml folder name using a period before the country name.

Example

xml.de – would be used to hold a ul_resources.xml file using the same keynames as the English file with messages translated in German.

Server

When a new language is configured on the Aras platform, a new column is available in the UserMessage table to hold the translated text. Click the lookup button next to the Text field on the UserMessage form to display the multilingual entry dialog box. Translated data can be provided in the dialog.

Summary

In this unit, you learned how to retrieve and create message text stored in resource libraries.

You should now be able to:

- Retrieve messages in a Client and in a Server Method
- Create new client UI resource messages
- Create new server User Message Items

This page intentionally left blank.



Appendix G Passing Event Parameters

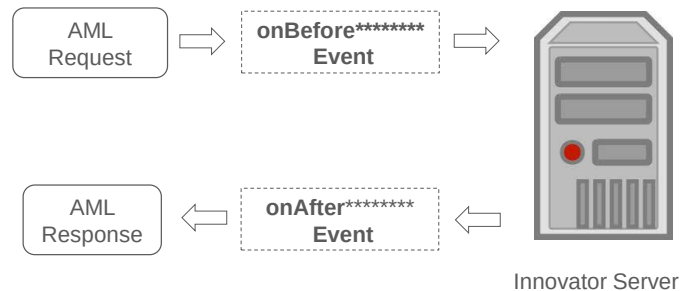
Overview

All server Methods (only) have access to a reserved class named `RequestState` to allow information to be passed between Methods when they are invoked from an event. Server Methods are typically associated with system events that execute before and after an interaction with the Aras Innovator server. With this facility, you can pass information from `onBefore*****` to `onAfter***` server events on the same (or different) Items, check for the existence of event parameters and remove or clear parameters as necessary.

Objectives

- Reviewing `onBefore` and `onAfter` Server Events
- Implementing the `RequestState` Class
- Adding `RequestState` Object Keys
- Retrieving `RequestState` Object Keys
- Clearing `RequestState`
- Examining Example Use Cases

Reviewing onBefore/onAfter Server Events



Reviewing onBefore and onAfter Server Events

Server Methods are associated with one or more server events to prevent, replace or extend the standard behavior of Aras Innovator.

onBefore**** Event

This collection of events execute before the AML request from the client is passed to the Innovator server and are typically used to validate data based on custom business logic (e.g. onBeforeAdd, onBeforeUpdate, onBeforeDelete, etc.).

onAfter**** Event

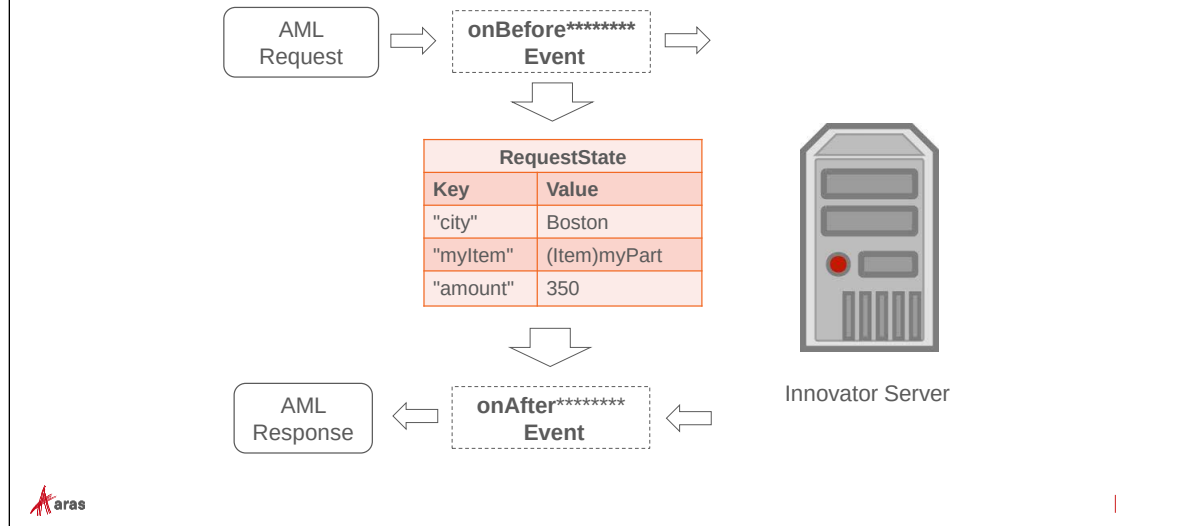
This collection of events execute after the server interaction has executed and have access to the information returned from the Aras Innovator server (e.g. onAfterAdd, onAfterUpdate, onAfterDelete).

on***** Event

This collection of events replaces the standard behavior defined by Aras Innovator completely. It is the responsibility of the Method developer to create the appropriate request and format the response.

In this unit, you will learn how to assign values to a reserved class named RequestState that is available to store and retrieve values between Methods called from different events.

Reviewing RequestState Class



Reviewing the RequestState Class

The RequestState class is an implementation of the Aras.Server.Core.IContextState interface and is available to any server Method created in Aras Innovator.

The RequestState class contains an Object hash table that can be assigned key/values using a series of class methods. A typical example is to assign one or more values in a Method associated with an onBefore**** event. Values can then be retrieved using a server Method associated with an onAfter**** event.

For example, when an Item is deleted it may be useful to store information about the Item before it is removed from the database and then use that information in a later call to the server after the Item is removed (and no longer available).

Note

The RequestState values are only available during a database transaction (between onBefore**** event through onAfter**** event).

Adding RequestState Keys

- `RequestState.Add(Object key, Object value)`
- `RequestState[Object key] = Object value`

```
RequestState.Add("city", "Boston");
```

```
RequestState.Add("itemKey", myItem);
```

```
RequestState.Add("amount", 350);
```

"city"	"Boston"
"itemKey"	(Item)myItem
"amount"	350



Adding RequestState Keys

To add a session key, use the Add method and supply the key name and key value or assign a key to a value. Note that the keys and corresponding values support any Object type.

In the example above, three key/value pairs have been assigned to the RequestState. The first key named "city" is set to the string of "Boston". The second key named "itemKey" is assigned to an Aras Innovator Item that has been declared in the program. The third key refers to an integer value.

Notes

These examples all use a string as the key name. You can use any Object type to define the key. An exception is thrown if you attempt to add a key and it already exists in the collection using the Add method. Assigning a value to an existing key directly (as in the "amount" example shown above) overwrites the current value.

Key assignments respect the Method sort order on a configured event. If two Methods have been assigned to the same event and both assign a value to the same key, the last Method that executes in the sort order will replace the value (assuming it already exists).

Retrieving RequestState Values

- Object value = RequestState[Object key];

```
string city = (string)RequestState["city"];
Item itm = (Item)RequestState["itemKey"];
int total = (int)RequestState["amount"];
```

"city"	"Boston"
"itemKey"	(Item)myItem
"amount"	350



|

Retrieving RequestState Values

In a subsequent Method, you retrieve the values using the appropriate key.

In the example above, the values set on the previous page are retrieved into their appropriate variable types. Note the use of casting to ensure the appropriate type is returned from the RequestState.

Note

Null (or *Nothing* in VB) is returned if you attempt to retrieve a key that does not exist in the collection.

VB.NET

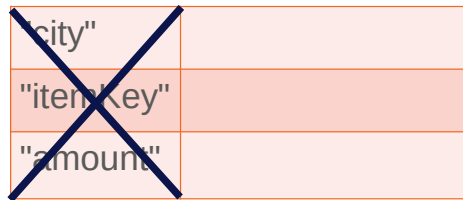
```
Dim city As String = DirectCast(RequestState("city"), String)
Dim itm As Item = DirectCast(RequestState("itemKey"), Item)
Dim total As Integer = CInt(RequestState("amount"))
```

Clearing RequestState

- `RequestState.Remove(Object key);`
- `RequestState.Clear();`

```
RequestState.Remove("city");
```

```
RequestState.Clear();
```



"city"	
"itemKey"	
"amount"	



Clearing RequestState Values

The RequestState class persists between the onBefore through onAfter server events. You can also remove the key/values if they are no longer needed.

To remove each key individually you can use the Remove method and supply the appropriate key name.

To clear all key/value pairs use the Clear method, which will remove all values from the RequestState.

VB.NET

```
RequestState.Remove("city")
RequestState.Clear()
```

Checking for Keys

- boolean value = RequestState.Contains(Object key);
- integer value = RequestState.Count;

```
if (RequestState.Contains("city")) ...
if (RequestState.Count < 1) ...
```

"city"	"Boston"
"itemKey"	(Item)myItem
"amount"	350

} 3



Checking for Keys

You can check to see if a key exists in a collection as well as determine how many key/value pairs have been assigned.

Use the Contains method to determine if a key exists in the collection. A true or false is returned based on the search.

Use the Count property to determine how many key/value pairs have been assigned.

The following C# example adds a new key if it does not already exist. Otherwise, the key is reassigned:

```
string key = "amountKey";
string value = 350;
if (RequestState.Contains(key))
{
    RequestState[key] = value;
}
else
{
    RequestState.Add(key, value);
}
return null;
```

VB.NET

```
Dim key As String = "amountKey"
Dim value As String = 350
If RequestState.Contains(key) Then
    RequestState(key) = value
Else
    RequestState.Add(key, value)
End If
Return Nothing
```

Use Case: Item Versioning

- **onBeforeVersion**
 - Obtain current property values before item is versioned
 - Set RequestState keys to original values
- **onAfterVersion**
 - Retrieve RequestState property values
 - Compare values and prevent versioning if certain values have changed



Use Case Example: Item Versioning

This example demonstrates how to use the RequestState class with a versioned Item. The following Method has been created for the onBeforeVersion event of a Document Item. The current document number as well as the name is stored in the RequestState collection.

```
string doc_number = this.GetProperty("item_number");
string name = this.GetProperty("name");

RequestState.Add("docnum", doc_number);
RequestState.Add("docname", name);

return null;
```

VB.NET

```
Dim doc_number As String = Me.GetProperty("item_number")
Dim name As String = Me.GetProperty("name")

RequestState.Add("docnum", doc_number)
RequestState.Add("docname", name)

Return Nothing
```


A second server Method is created to compare the original values of the Document with any changes. In this example, an error is returned if the user attempts to change the name or document number in the new version:

```
string previous_number = (string)RequestState["docnum"];
string previous_name = (string)RequestState["docname"];

if (previous_number != this.GetProperty("item_number") ||
    previous_name != this.GetProperty("name")) {
    return this.getInnovator().newError("Cannot change doc number or name
in versions");
}
RequestState.Clear();
return this;
```

VB.NET

```
Dim previous_number As String = _
    DirectCast(RequestState("docnum"), String)
Dim previous_name As String = _
    DirectCast(RequestState("docname"), String)

If previous_number <> Me.GetProperty("item_number")
    OrElse previous_name <> Me.GetProperty("name") Then
    Return _
        Me.getInnovator().newError("Cannot change doc number or name in
versions")
End If
RequestState.Clear()
Return Me
```

The server Methods are then configured on the Document ItemType with the appropriate server events:

^ Properties RelationshipTypes Views Server Events Actions Life Cycles Workflows Client Events Can Add Permissions								
Methods ▼ ☆								
     Hidden ▼  ▼  ▼  ▼								
	Name ↓	Method T...	Ver	execution_all...	Comments	Event	Sort Order	Event Version
	_Event Before Version	CSharp	3	World		onBeforeVersion	256	Version 1
	_Event After Version	CSharp	2	World		onAfterVersion	128	Version 1

Use Case: Item Deletion

- **onBeforeDelete**
 - Obtain property values of items before it is deleted
 - Set RequestState key equal to item and it's values
- **onAfterDelete**
 - Retrieve RequestState item after item is deleted
 - Send e-mail with message text confirming item is deleted (with property data)



Use Case Example: Item Deletion

In this example, an email message is generated when an Item is deleted from the system. The first Method obtains the information about the Product to be deleted and caches it in the RequestState collection before the Item is deleted. Note that the value stored is an Object of type Item.

```
//Retrieve the desired property values before deleting item
Item itm = this.newItem(this.getType(), "get");
itm.setID(this.getID());
itm.setAttribute("select", "item_number,name");
itm = itm.apply();

RequestState.Add("itmKey", itm);
return this;
```

VB.NET

```
Dim itm As Item = Me.newItem(Me.[getType](), "get")
itm.setID(Me.getID())
itm.setAttribute("select", "item_number,name")
itm = itm.apply()

RequestState.Add("itmKey", itm)
Return Me
```

The second Method checks to make sure the itemKey has been set and then retrieves the cached information. An email message is then generated using the email method of the Aras IOM Item class.

```
if (!RequestState.Contains("itmKey")) {
    return null;
}

Item deleted_item = (Item)RequestState["itmKey"];

Item identityItem = this.newItem("Identity", "get");
```

```

identityItem.setProperty("name", "Innovator Admin");
identityItem = identityItem.apply();

Item emailItem = this.newItem("EMail Message", "get");
emailItem.setProperty("name", " EventDeleteMessage ");
emailItem = emailItem.apply();

bool sent = deleted_item.email(emailItem, identityItem);

RequestState.Remove("itmKey");
return this;

```

VB.NET

```

If Not RequestState.Contains("itmKey") Then
    Return Nothing
End If

Dim deleted_item As Item = _
    DirectCast(RequestState("itmKey"), Item)

Dim identityItem As Item = Me.newItem("Identity", "get")
identityItem.setProperty("name", "Innovator Admin")
identityItem = identityItem.apply()

Dim emailItem As Item = Me.newItem("EMail Message", "get")
emailItem.setProperty("name", "EventDeleteMessage")
emailItem = emailItem.apply()

Dim sent As Boolean = deleted_item.email(emailItem, identityItem)

RequestState.Remove("itmKey")
Return Me

```

The Product ItemType is configured to use both Methods.

^ Properties RelationshipTypes Views Server Events Actions Life Cycles Workflows Client Events Can Add Permissions								
Methods ▼ ☆								
     Hidden   								
	Name	Method T...	Ver	execution_all...	Comments	Event	Sort Order ↑	Event Version
	_Event After Delete	CSharp	1	World		onAfterDelete	128	Version 1
	_Event Before Delete	CSharp	1	World		onBeforeDelete	256	Version 1

The following EMail Message is used to send the email:

Email Message 	Name EventDeleteMessage	Body Plain Product Number \${Item[@type='Product']/item_number} was deleted
	From User Innovator Admin	Product Number \${Item[@type='Product']/name} is no longer available
	Subject Product \${Item[@type='Product']/item_number}	Body Html

Use Case: Related Items

- **onBeforeUpdate (Parent Item)**
 - Obtain property values of parent item
 - Set RequestState keys equal to property values
- **onBeforeAdd (Related Item)**
 - Retrieve RequestState property values
 - Set property values of related item equal to property values of parent item



Use Case Example: Related Items

In this example, two different Items are used with the RequestState collection. When a user creates a new Model from a Product Item the Model number is automatically prefixed with the Product Number.

The following Method is created to obtain the Product number and cache it in the RequestState before the Product is updated.

```
string prodnum = this.getProperty("item_number");
RequestState["prodnum"] = prodnum;
return this;
```

VB.NET

```
Dim part As String = Me.getProperty("item_number")
RequestState["prodnum"] = prodnum
Return Me
```

The second Method retrieves the new Model number and the current Product before the new Model is added to the system and prefixes the Model number with the Product number:

```
if (!RequestState.Contains("prodnum")) return this;
string prodnumber = (string)RequestState["prodnum"];
string modelnum = this.getProperty("item_number", "");
this.setProperty("item_number", prodnumber + "-" + modelnum);
RequestState.Remove("prodnum");
return this;
```

VB.NET

```
If Not RequestState.Contains("prodnum") Then
    Return Me
End If
Dim prodnumber As String = DirectCast(RequestState("prodnum"), String)
Dim modelnum As String = Me.getProperty("item_number", "")
Me.setProperty("item_number", prodnumber & "-" & modelnum)
RequestState.Remove("prodnum")
Return Me
```

The first Method is configured on the Product ItemType:

Properties	RelationshipTypes	Views	Server Events	Actions	Life Cycles	Workflows	Client Events	Can Add	Permissions
Methods ▼ ☆ ⚙️ ⚙️ ⚙️ 🔍 ✖️ Hidden ▼ 🔄 🖥️ 🔗									
Name	Method T...	Ver	execution_all...	Comments	Event	Sort Order	Event Version		
_Product Model Add	CSharp	2	World		onBeforeAdd	128	Version 1		
_Product Model Add	CSharp	2	World		onBeforeUpdate	384	Version 1		

The second Method is configured on the Model ItemType:

Properties	RelationshipTypes	Views	Server Events	Actions	Life Cycles	Workflows	Client Events	Can Add	Permissions
Methods ▼ ☆ ⚙️ ⚙️ ⚙️ 🔍 ✖️ Hidden ▼ 🔄 🖥️ 🔗									
Name	Method T...	Ver	execution_all...	Comments	Event	Sort Order	Event Version		
_Model Add	CSharp	2	World		onBeforeAdd	256			

Result

SN10005 ☆ 🚩
💾 Save ✅ Done ❌ Discard 🔄 🔄 🔗 🗑️ 🔗 ⋮

Product
 Product Number
 SN10005
 Name
 Smart Phone (Standard)
 Description

Models
Models ▼ ☆
⚙️ ⚙️ | 🔍 ✖️ Hidden ▼ | 🔄 🖥️ 🔗

Model Number	Name	Release Number	Version Number
SN10005-100A	16GB Plasma Display		

Summary

In this unit, you learned how to pass parameters between server event Methods.

You should now be able to:

- Implement the RequestState Class with Server Events
- Add RequestState Object Keys
- Retrieve RequestState Object Keys
- Clear the RequestState of key/value

This page intentionally left blank.

Appendix H Creating Methods with Aras VS Methods Plugin

Overview

In this unit, you learn how to install the Aras Visual Studio plugin, which allows you to create or edit server side Methods in the Visual Studio IDE. You can then take advantage of all the features built into VS to develop code more efficiently including active syntax checking and 'intellisense' code completion. Methods can then be saved in the Aras database or exported directly to an AML Export Package.

Objectives

- Reviewing Features of Aras Visual Studio Plugin
- Installing the Aras VS Plugin
- Creating a New Method
- Saving a Method to the Database
- Saving a Method to an AML Package
- Opening a Method from the Database
- Opening a Method from an AML Export Package

Reviewing Features of VS Plugin

- Allows Aras Methods to be created and modified using the Visual Studio IDE
- Visual Studio provides a full set of development features including :
 - Intellisense code completion
 - Instant error checking
 - Just in time debugging
- Server Methods can be created and saved to either the Aras database or directly to an Aras package



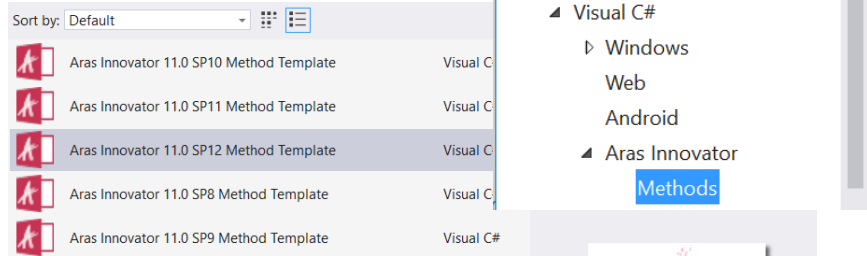
Reviewing Features of Visual Studio Plugin

The Aras Visual Studio plugin allows you to leverage the power of the VS IDE to help create error-free code more quickly. As you enter code in the IDE editor, built-in syntax checking checks for errors as you type. Microsoft Intellisense code completion also prompts you with possible choices from the Aras IOM make your code more accurate with less typing.

Currently the plugin only works with C# Server Methods. Methods can be created/edited and save/retrieved from the Aras Innovator database or an Aras export package.

Installing the Aras Plugin

- Download and Install Plugin from Visual Studio Marketplace
- Create New C# Project Using Plugin
- Select appropriate Template based on Service Pack



Installing the Aras Plugin

The plugin is available as an online template, which is available from the Visual Studio marketplace at: <https://marketplace.visualstudio.com/items?itemName=ArasCorporation.ArasInnovatorVisualStudioMethodPlugin>

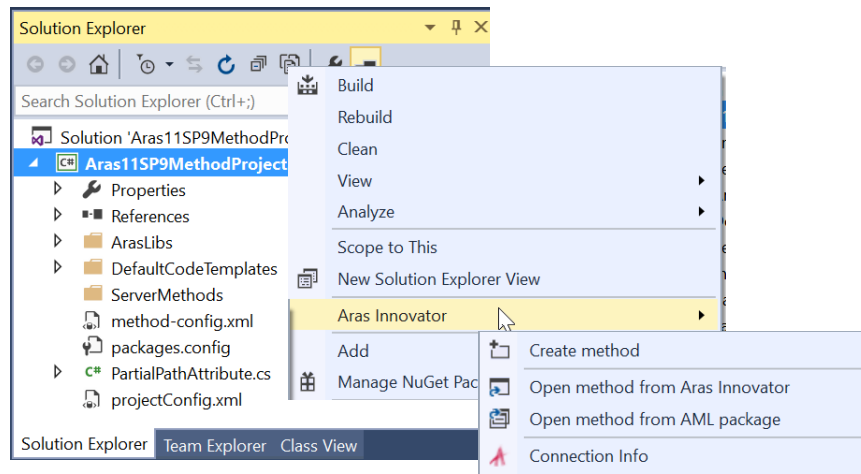
You can also install the plugin from Visual Studio directly from the Tools > Extensions and Features menu.

When the download completes execute the install file to add the extension to the Visual Studio platform.

To Start Using the Plugin

1. Create a new Visual C# project in Visual Studio.
2. Search for "Aras Innovator->Methods" to locate the templates.
3. Select the template that matches the service pack of your Aras installation to open the project.
4. Build the solution to update the associated resources.

Accessing the Plugin



Accessing the Plugin

To use the plugin, you will need to make a connection to the current installation of Aras Innovator (or to an existing Package export file).

To Access the Plugin

1. Using the mouse, right click on the project in the Solution Explorer and choose Aras Innovator from the menu.
2. A menu appears that allows you to create a new Method or open an existing Method.

Creating a New Method

① Method Type: Server Language: CSharp ②

③ Template Name: CSharp Event Data: None ④

⑤ ☒ Template Preview

⑥ Execution allowed to: World ...

⑦ Package: SampleProject

⑧ Method Name: SampleMethod

OK Cancel

Creating a New Method

Before you can create a new Method, you must provide login credentials. Enter URL, database name and user/password credentials to connect to the server.

Server URL: http://localhost/TRAIN-dev-sprint1

Database: ANDTRAINV14-TRAIN-dev-sprint1

Login: admin

Password:

Login

Once connected the Create new method dialog appears with the following fields:

- ① **Method Type**
Currently only Server Methods are supported by the plugin.
- ② **Language**
Currently C Sharp is the only supported language.
- ③ **Template**
Currently C Sharp is the only supported template.
- ④ **Event Data**
If this Method will be associated with a Server Event you can choose the event to receive additional event data.
- ⑤ **Template Preview**

Displays a preview of the source code that will be generated along with code you add to the Method.

⑥ **Execution Allowed To**

Indicates who will have access to run this Method.

⑦ **Package**

Name of the Export Package that will contain this Method. In this example a project named SampleProject is used.

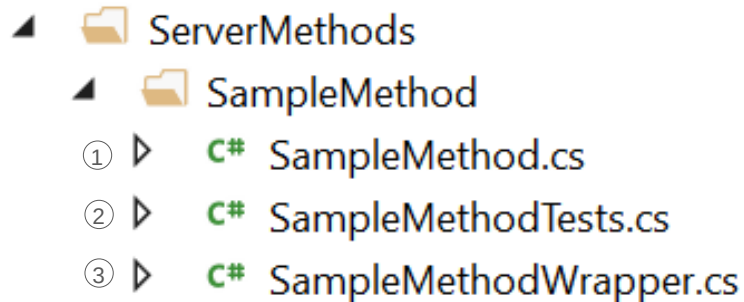
⑧ **Method Name**

Name of the Method to be created. In this example the Method is named SampleMethod.

Note

You can also review connection information once a connection has been established by selecting the Connection Info menu entry.

Reviewing Generated Files



Reviewing Generated Files

The Visual Studio plugin creates three files in the project that represent the server Method.

① **Source File**

Contains the definition of the server Method and a reserved region for you to enter source code that will normally be entered in the Aras Method Code Editor.

② **Test File**

A template is created that can be used later to build a unit test for this Method.

③ **Method Wrapper**

The wrapper code is extracted from the Aras method-config.xml file and represents the Aras framework that is used for all server Methods. The wrapper code calls the source file defined above and makes the source file easier to read.

Providing Method Code

```
//start your code inside region MethodCode - DO NOT CHANGE CODE ABOVE

#region MethodCode

    Item u = this.newItem("User", "get");

    return u.apply();

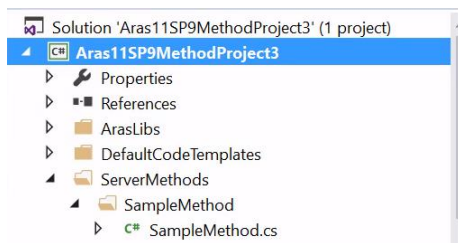
#endregion MethodCode

// end your code inside region MethodCode - DO NOT CHANGE CODE BELOW
```



Providing Method Code

The Method Template creates a new CSharp source file (.cs) within the Visual Studio project. You can access this file from the Solution Explorer.



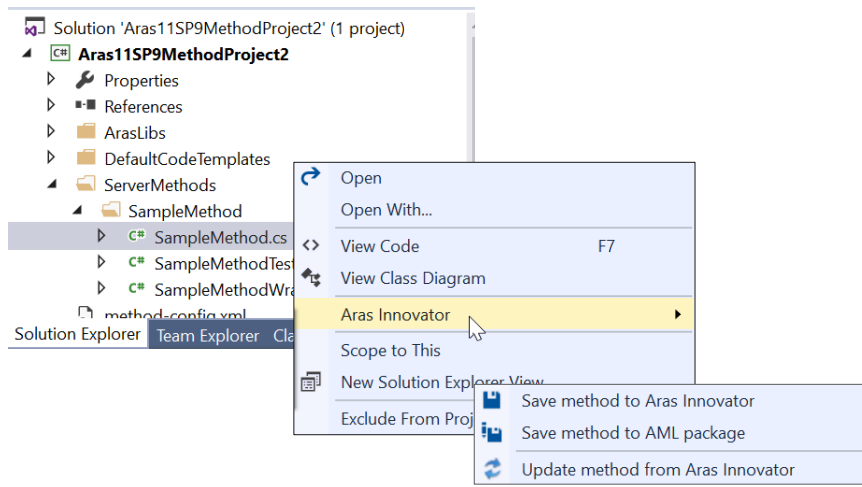
A region in the file is reserved for your code and is marked with comments. You should only make changes within the marked region.

Note

If you make changes outside of the marked region the Method may not compile successfully.

In this example, the SampleMethod has been modified and will return a list of User items from the database.

Saving New Method



Saving New Method

Once the code has been added, you can save the new Method to the Aras Innovator database or to an AML Export package.

To Save New Method

1. Make sure to save the Method file in the Visual Studio project using File > Save (Ctrl + S)
2. Right click on the .cs file and select Aras Innovator and the appropriate Aras save option. For this example, the Method will be saved to the Aras Innovator database.

Saving to Innovator

Save method to Aras Innovator

Connection Info

1 Server URL: http://localhost/BOBTRN-dev-sprint1 Database: ANDTRAINV14-BOBTRN- User: admin 4 Edit Connection Info

Method Info

5 Method Type: server 6 Language: C#

7 Template Name: CSharp 8 Event Data: None

9 Method Preview

10 Execution allowed to: World ...

11 Package: SampleProject

12 Method Name: SampleMethod

OK Cancel

Saving to Innovator (Database)

When you choose to save a Method to Innovator (database) the dialog above appears with the following fields:

- ① **Server URL**
Location of the current Aras instance where Method will be saved
- ② **Database**
Database name to store Method
- ③ **User**
Aras user id
- ④ **Edit Connection Info**
Allows you to change the login credentials and server/database locations
- ⑤ **Method Type**
Server is currently the only supported type.
- ⑥ **Language**
C Sharp is currently the only supported language
- ⑦ **Template Name**
CSharp is currently the only supported template
- ⑧ **Event Data**
If this Method will be associated with a server event, select the event to receive additional data
- ⑨ **Template Preview**

Displays a preview of the source code that will be generated along with code you add to the Method.

⑩ **Execution Allowed To**

Indicates who will have access to run this Method

⑪ **Package**

Name of the Export Package that will contain this Method

⑫ **Method Name**

Name of the Method to be saved

Saving to Package

Saving to Package

You can also save a Method directly to an Aras Export Package. The customer repository contains a directory named AML Packages that stores any additional packages that you add to a project. You can use the plugin to export directly to an Aras Export Package in this location vs. having to export the Method from the Innovator database.

The following fields are provided to save to an AML Export Package:

- ① **Package Path**
File director to save the Method export. This is typically the AML Package directory of the customer repository.
- ② **Method Type**
Server is currently the only supported type.
- ③ **Language**
C Sharp is currently the only supported language
- ④ **Template Name**
CSharp is currently the only supported template
- ⑤ **Event Data**
If this Method will be associated with a server event, select the event to receive additional data
- ⑥ **Template Preview**
Displays a preview of the source code that will be generated along with code you add to the Method.
- ⑦ **Execution Allowed To**
Indicates who will have access to run this Method

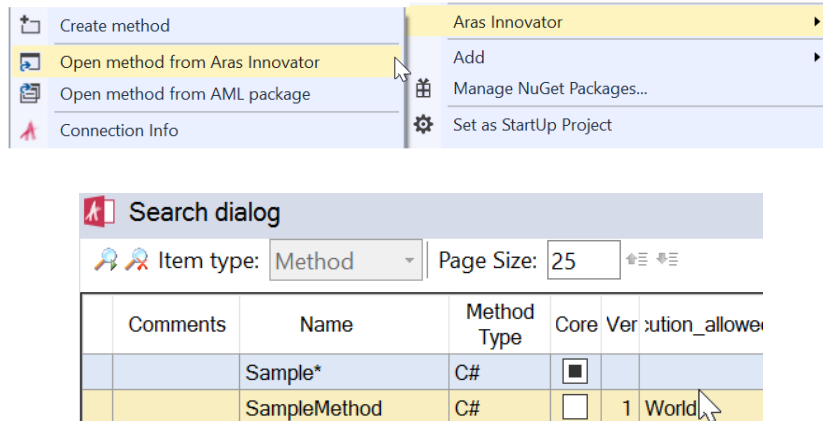
- ⑧ **Package**
Name of the Export Package that will contain this Method
- ⑨ **Method Name**
Name of the Method to be saved

The package folder will be created if it does not exist and the Method will be exported to an AML file with the same name as the Method.

Note

The manifest file *imports.mf* in the targeted directory will be modified automatically to include the new package information. You should review this file once the save is complete to insure the package order is correct.

Opening Method From Innovator



Opening Method from Innovator

The Aras plugin can also be used to open and edit existing Methods stored in the Innovator database or an AML Export Package file.

To Open a Method from Innovator

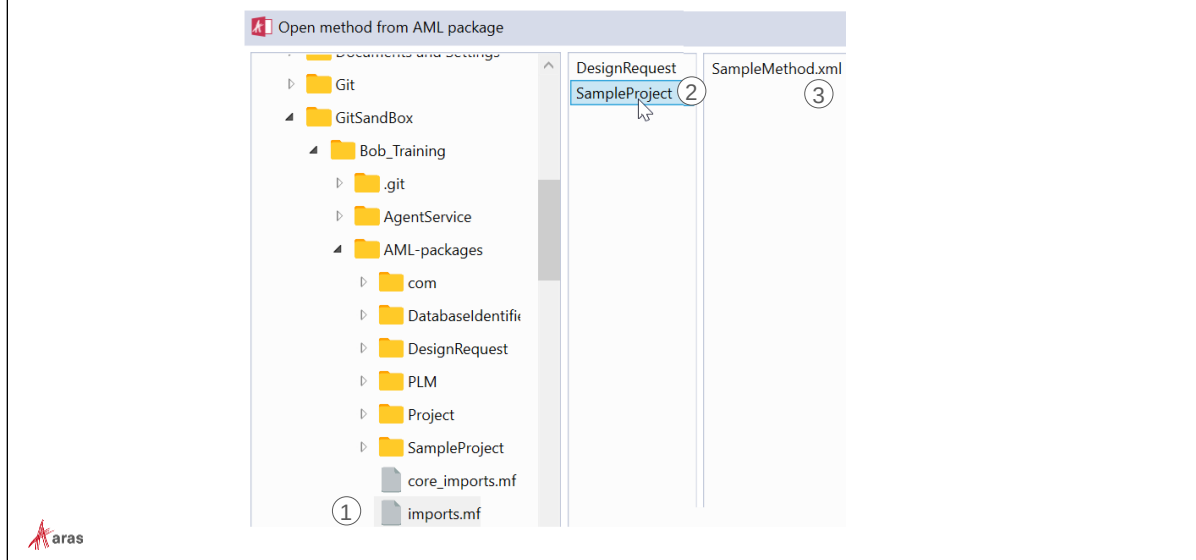
1. Select Aras Innovator > Open method from Aras Innovator from the context menu.
2. Locate the Method using the simple search dialog.
3. Click OK to retrieve the Method from the database.

To Open and Update the Current Method from Innovator

If you currently have a Method open in Visual Studio you can retrieve any updates that have been made in the database using the Update method from Aras Innovator action.

1. Select Aras Innovator > Update method from Aras Innovator from the plugin context menu.
2. Confirm the Method to be retrieved and click OK to refresh the Method source code in Visual Studio.

Opening Method From Package



Opening Method from Package

You can also open a Method from an existing Aras AML Export Package file.

To Open a Method from a Package

After selecting Aras Innovator > Open method from package from the context menu:

1. Select the import manifest file for the current exports. This file is typically located in the AML-packages directory of the customer repository.
2. Select the appropriate package from the second column of the dialog.
3. Select the appropriate AML (xml) file containing the Method.
4. Click OK to retrieve the Method into Visual Studio.

Summary

In this unit, you learned how to install and use the Aras Visual Studio plugin for Method development.

You should now be able to:

- Install the Aras VS Plugin
- Create a New Method
- Save a Method to the Innovator Database
- Save a Method to an AML Package
- Open a Method from the Database
- Open a Method from an AML Export Package



Lab Solutions

Overview

Solution code for all Review Questions and Lab Exercises.

Visual Basic solutions are shown in *italicized text*.

Unit 2 Solutions: Exploring AML

Review Questions

- *AML respects Permissions based on your user profile. You cannot remove an item if you do not have the proper Permissions.*
- *<Relationships> tag.*
- *You must supply a where clause (or id, idlist).*

1.

```
<Item type="Sales Order" action="get">
  <multiple_shipment>1</multiple_shipment>
  <shipping_method>UPS</shipping_method>
</Item>
```

2.

```
<Item type="Sales Order" action="get">
  <or>
    <multiple_shipment>1</multiple_shipment>
    <shipping_method>UPS</shipping_method>
  </or>
</Item>
```

3.

```
<Item type="Sales Order" action="get">
  <Relationships>
    <Item type="Sales Order Customer" action="get"
      select="related_id">
      <related_id>
        <Item type="Customer" action="get" select ="city">
          <city>New York</city>
        </Item>
      </related_id>
    </Item>
  </Relationships>
</Item>
```

4.

```
<Item type="Sales Order" action="get">
  <ship_date condition="gt">2013-05-01T00:00:00</ship_date>
  <Relationships>
    <Item type="Sales Order Customer" action="get"
      select="related_id">
      <related_id>
        <Item type="Customer" action="get" select ="city">
          <city>New York</city>
        </Item>
      </related_id>
    </Item>
  </Relationships>
</Item>
```

5.

```

<Item type="Sales Order" action="add">
  <managed_by_id>
    <Item type="Identity" action="get">
      <keyed_name>Peter Smith</keyed_name>
    </Item>
  </managed_by_id>
  <shipping_method>UPS</shipping_method>
</Item>

```

6.

```

<Item type="Sales Order" action="add">
  <managed_by_id>
    <Item type="Identity" action="get">
      <keyed_name>Peter Smith</keyed_name>
    </Item>
  </managed_by_id>
  <shipping_method>UPS</shipping_method>
  <Relationships>
    <Item type="Sales Order Customer" action="add">
      <related_id>
        <Item type="Customer" action="get">
          <name>Brown Industries</name>
        </Item>
      </related_id>
    </Item>
    <Item type="Sales Order Part" action="add">
      <quantity>1</quantity>
      <related_id>
        <Item type="Part" action="get">
          <item_number>C4703A</item_number>
        </Item>
      </related_id>
    </Item>
  </Relationships>
</Item>

```

7.

```

<Item type="Sales Order" action="edit"
  where="[sales_order].multiple_shipment='1'">
  <shipping_method>UPS</shipping_method>
</Item>

```

8.

```

<Item type="Sales Order" action="edit"
  where="[sales_order].multiple_shipment='1'">
  <shipping_method>UPS</shipping_method>
  <Relationships>
    <Item type="Sales Order Remarks" action="add">
      <comment_text>Modified by AML Administrator</comment_text>
    </Item>
  </Relationships>
</Item>

```

Unit 3 Solutions: Creating a Method

Review Questions

- Client Methods use JavaScript and execute in the browser. Server Methods use either C# or VB.NET and execute on the Aras server.
- Server Methods must return an Item(s). Client Methods can return an Item but it is not mandatory unless the use case warrants it.
- AML
- newItem(), setProperty(), setAttribute(), apply();

1.

```
Item so = this.newItem("Sales Order", "get");
so.setProperty("multiple_shipment", "1");
so.setProperty("shipping_method", "UPS");
return so.apply();
```

```
Dim so As Item = Me.newItem("Sales Order", "get")
so.setProperty("multiple_shipment", "1")
so.setProperty("shipping_method", "UPS")
Return so.apply()
```

2.

```
const so = this.newItem("Sales Order", "get");
so.setAttribute("select", "so_number");
so.setProperty("multiple_shipment", "1");
so.setProperty("shipping_method", "UPS");
return so.apply();
```

3.

```
Item so = this.newItem("Sales Order", "add");
so.setProperty("shipping_method", "Federal Express");
return so.apply();
```

```
Dim so As Item = Me.newItem("Sales Order", "add")
so.setProperty("shipping_method", "Federal Express")
Return so.apply()
```

4.

```
string d = DateTime.Now.ToString("yyMMddhhmmss");
Item so = this.newItem("Sales Order","add");
so.setProperty("shipping_method","Federal Express");
so.setProperty("po_number",d);
return so.apply();
```

```
Dim d As String = DateTime.Now.ToString("yyMMddhhmmss")
Dim so As Item = Me.newItem("Sales Order", "add")
so.setProperty("shipping_method", "Federal Express")
so.setProperty("po_number", d)
Return so.apply()
```

5.

```

Item so = this.NewItem("Sales Order", "edit");
so.SetAttribute("where", "[sales_order].multiple_shipment='1'");
so.SetAttribute("doGetItem", "0");
so.SetProperty("shipping_method", "UPS");
return so.apply();

Dim so As Item = Me.NewItem("Sales Order", "edit")
so.SetAttribute("doGetItem", "0")
so.SetAttribute("where", "[sales_order].multiple_shipment='1'")
so.SetProperty("shipping_method", "UPS")
Return so.apply()

```

Unit 4 Solutions: Debugging

1. Add the appropriate debug statement to the method **DebugMe**.

```

if (System.Diagnostics.Debugger.Launch())
    System.Diagnostics.Debugger.Break();
Innovator inn = this.getInnovator();
Item itm = this.NewItem("Sales Order", "get");
itm.SetProperty("so_number", "SO-0005");
Item result = itm.apply();

string po = result.GetProperty("po_number");

return inn.NewResult(po);

System.Diagnostics.Debugger.Break()
Dim inn As Innovator = Me.getInnovator()

Dim itm As Item = Me.NewItem("Sales Order", "get")
itm.SetProperty("so_number", "SO-0005")
Dim result As Item = itm.apply()

Dim po As String = result.GetProperty("po_number")

Return inn.NewResult(po)

```

Unit 5 Solutions: Custom Actions

1.

```
string x = System.Environment.MachineName;
return this.getInnovator().NewResult(x);
```



```
Dim x As String = System.Environment.MachineName
return Me.getInnovator().NewResult(x)
```
2. Uses the **Server EditOrders** Method (unit 3, step 5)
3.

```
this.SetAttribute("action", "edit");
this.SetProperty("ship_date", "__now()");
return this.apply();
```

Unit 6 Solutions: Innovator Class

```
string option = this.GetProperty("option", "");
Innovator i = this.getInnovator();
string uid = i.getUserID();
Item user = i.getItemById("User", uid);
user.SetAttribute("action", "edit");
user.SetProperty("starting_page", option);
return user.apply();
```

```
Dim [option] As String = Me.GetProperty("option", "")
Dim i As Innovator = Me.getInnovator()
Dim uid As String = i.getUserID()
Dim user As Item = i.getItemById("User", uid)
user.SetAttribute("action", "edit")
user.SetProperty("starting_page", [option])
Return user.apply()
```

Unit 7 Solutions: Item Class

Calculate Cost:

```
Innovator inn = this.getInnovator();

decimal total_cost = 0;
decimal unit_cost = 0;
decimal line_cost = 0;
int quantity = 0;
int count = 0;

this.fetchRelationships("Sales Order Part");
Item relationships = this.getRelationships("Sales Order Part");
count = relationships.getItemCount();
for (int i = 0; i < count; i++) {
    Item partRel = relationships.getItemByIndex(i);
    quantity =
        Convert.ToInt16(partRel.GetProperty("quantity", "0"));
    Item part = partRel.getRelatedItem();
    unit_cost =
        Convert.ToDecimal(part.GetProperty("cost", "0"));
    line_cost = quantity * unit_cost;
    total_cost = total_cost + line_cost;
}
return inn.newResult(total_cost.ToString());
```

```

Dim inn As Innovator = Me.getInnovator()

Dim total_cost As Decimal = 0
Dim unit_cost As Decimal = 0
Dim line_cost As Decimal = 0
Dim quantity As Integer = 0
Dim count As Integer = 0

Me.fetchRelationships("Sales Order Part")
Dim relationships As Item = Me.getRelationships("Sales Order Part")
count = relationships.getItemCount()
For i As Integer = 0 To count - 1
    Dim partRel As Item = relationships.getItemByIndex(i)
    quantity = _
        Convert.ToInt16(partRel.getProperty("quantity", "0"))
    Dim part As Item = partRel.getRelatedItem()
    unit_cost = _
        Convert.ToDecimal(part.getProperty("cost", "0"))
    line_cost = quantity * unit_cost
    total_cost = total_cost + line_cost
Next
Return inn.newResult(total_cost.ToString())

```

Unit 8 Solutions: Server Event Methods

1. ItemType Validation Method, use the server events OnBeforeAdd + onBeforeUpdate

```

Innovator inn = this.getInnovator();
string shipmethod = this.getProperty("shipping_method", "UPS");
string po_number = this.getProperty("po_number", "");
if (shipmethod=="Federal Express") {
    if (po_number=="") {
        return inn.newError("PO# Required");
    }
}
return this;

```

```

Dim inn As Innovator = Me.getInnovator()
Dim shipmethod As String = Me.getProperty("shipping_method", "UPS")
Dim po_number As String = Me.getProperty("po_number", "")
If shipmethod = "Federal Express" Then
    If po_number = "" Then
        Return inn.newError("PO# Required")
    End If
End If
Return Me

```

2. Lifecycle Validation Method, define a Pre-Method for the transition from New to In Progress

```

Innovator inn = this.getInnovator();
string shipdate = this.getProperty("ship_date", "");
if (shipdate=="") {
    return inn.newError("Shipping Date Required");
}
return this;

```

```

Dim inn As Innovator = Me.getInnovator()
Dim shipdate As String = Me.getProperty("ship_date", "")
If shipdate = "" Then
    Return inn.newError("Shipping Date Required")
End If
Return Me

```

3. Apply Method to onActivate event of Reject Order Workflow Activity

```

Item order = this.apply("Get Controlled Item");

Item rel =
    order.createRelationship("Sales Order Remarks", "add");
rel.setProperty("comment_text", "Sales Order Rejected");
return order.apply();

Dim order As Item = Me.apply("Get Controlled Item")
Dim rel As Item = _
    order.createRelationship("Sales Order Remarks", "add")
rel.setProperty("comment_text", "Sales Order Rejected")
Return order.apply()

```

Unit 9 Solution: Aras Object Functions

```

if (this.getLockStatus() != 0) {
    aras.AlertError("Item is locked!")
    return;
}

this.setAttribute("action", "edit");
this.setProperty("ship_date", "__now()");
return this.apply();

```

Unit 10 Solutions: Client Event Methods

1. Apply to the client **OnAfterNew** Event.

```

const d = new Date();
const today = d.toISOString().substring(0,10);

this.setProperty("ship_date", today);
return this;

```

2. Apply to the field **onChange** Event.

```

const multiple =
    document.thisItem.getProperty("multiple_shipment", "0");
if (multiple==1) {
    window.handleItemChange("shipping_method", "UPS");
}
else {
    window.handleItemChange("shipping_method", "Federal Express");
}
// document.thisItem allows accessing the context Item on a Form or Field event

```


3. Edit the "Sales Order Part" relationship ItemType and attach this Method to the onSearchDialog event of the related_id property.

```
return {classification:{filterValue:"Product",isFilterFixed:true}};
```

Unit 11 Solutions: Configuring the User Interface

Exercise 1 solution steps

Add a New Tab Control

1. Locate and open the Sales Order ItemType.
2. Access the Client Style tab and then open the existing Sales Order presentation configuration.
3. Select the cui_PresentConfigWinSection tab and create a new section named TRN_SOItemView for the classification Data Model, Location ItemView and Identity World.
4. Open the new section for edit and create a new cui_WindowSectionControl using the following criteria. You can obtain the GUID of the relTypeId by locating the Sales Order Part RelationshipType template and copying the Id to the clipboard.

Type	Tab Element Control
Name	TRN_SOTabParts
Parent	ItemView.FormAccordionTabs
Label	Parts
Additional Data	{ "relTypeId": "274D1842F5B04B42BB1944B3F083A863" }
Sort Order	1024
Action	Add
For Identity	World

5. Remove the Parts tab from the standard relationship tabs by locating and opening the Sales Order Part ItemType and clicking the "Hide All" checkbox option.
6. Save the changes.

Exercise 2 solution steps

Disable Command Bar Item

1. Create a new Client Method named TRN_Init_ShareButton, as shown on step 6 below.
Use the following logic to write the code:
 - a. Check the value of the options.itemTypeName argument. If it is not equal to "Sales Order" then return from the program.
 - b. Check to see if the current user is an administrator using the Aras Object function aras.isAdminUser().

- c. If the user is an administrator, then return the attribute disabled set to false. Otherwise, return the attribute set to true.
2. Open the Global client presentation and locate the command bar section named `searchview.commandbar.default`.
3. Open the command section and locate the command bar item named `commonitems.commandbar.sharemenu`.
4. Edit the command bar item and assign the Method you created to the Init Method field.
5. Save all changes and then log out and in again as an admin and non-admin to test the configuration.
6. Client Method: **TRN_Init_ShareButton**

```

        if (options.itemTypeName!='Sales Order') return;

        if(aras.isAdminUser() )
        {
            return {disabled: false};
        }
        else
        {
            return {disabled: true};
        }

```

Unit 12 Solutions: Federation

```

//Add Federated Data
// get path to csv file from Variable Item with the name "path_csv"
Item variablePath = this.newItem("Variable","get");
variablePath.setProperty("name","path_salesorder_csv");
Item pathItem = variablePath.apply();
string path = pathItem.getProperty("value","");
if (path=="")
{
    return this.getInnovator().newError("Path to csv file needs to be
defined on Variable item with name path_csv");
}
//read out federated property values and id of new Item and append new
//row in csv file
using (StreamWriter streamw = File.AppendText(path))
{

    streamw.WriteLine($"{this.getID()},{this.getProperty("ship_date","")},{
this.getProperty("po_number","")}");
    streamw.Close();
    streamw.Dispose();
}
return this;

//Update Federated Data
// get path to csv file from Variable Item with the name "path_csv"
Item variablePath = this.newItem("Variable","get");

```

```

variablePath.setProperty("name","path_salesorder_csv");
Item pathItem = variablePath.apply();
string path = pathItem.getProperty("value","");
if (path=="")
{
    return this.getInnovator().newError("Path to csv file needs to be
defined on Variable item with name path_csv");
}
//update federated property values on Item: append row if Item does not
//exist in file, otherwise replace values
string id = this.getID();
List<string> lines = new List<string>();
bool added = false;
using (var reader = new StreamReader(path))
{
    while (!reader.EndOfStream)
    {
        var line = reader.ReadLine();
        var values = line.Split(',');
        if (values[0].Contains(id))
        {
            values[1]=this.getProperty("ship_date","");
            values[2]=this.getProperty("po_number","");
            line = string.Join(",",values);
            added = true;
        }
        lines.Add(line);
    }
}
if (!added) // adds a row if Item does not exist in the csv file
{
    lines.Add($"{{id}},{{this.getProperty("ship_date","")}},{{this.getProperty("
po_number","")}}");
}
File.WriteAllLines(path, lines);
return this;

//Get Federated Data
// get path to csv file from Variable Item with the name "path_csv"
Item variablePath = this newItem("Variable","get");
variablePath.setProperty("name","path_salesorder_csv");
Item pathItem = variablePath.apply();
string path = pathItem.getProperty("value","");
if (path=="")
{
    return this.getInnovator().newError("Path to csv file needs to be
defined on Variable item with name path_csv");
}
// read all lines of csv file and store them in dictionary
var rows = new Dictionary<string, Tuple<string, string>>();
using (var reader = new StreamReader(path))
{
    while (!reader.EndOfStream)
    {
        var line = reader.ReadLine();
        var values = line.Split(',');
    }
}

```

```

        rows.Add(values[0], new Tuple<string, string> (values[1],
values[2]));
        //throw new Exception(values);
    }
    reader.Close();
    reader.Dispose();
}
// loop through Collection and get federated property values for each
//Item
int count = this.getItemCount();
for (int i = 0; i < count; i++)
{
    Item idx_itm = this.getItemByIndex(i);
    string id = idx_itm.getID();
    if (rows.ContainsKey(id))
    {
        var props = rows[id];
        idx_itm.setProperty("ship_date", props.Item1);
        idx_itm.setProperty("po_number", props.Item2);
    }
}
return this;

//Delete Federated Data
// get path to csv file from Variable Item with the name "path_csv"
Item variablePath = this.newItem("Variable", "get");
variablePath.setProperty("name", "path_salesorder_csv");
Item pathItem = variablePath.apply();
string path = pathItem.getProperty("value", "");
if (path=="")
{
    return this.getInnovator().newError("Path to csv file needs to be
defined on Variable item with name path_csv");
}
//get ID of Item and use it to delete row in csv file
string id = this.getID();
List<string> lines = new List<string>();
using (var reader = new StreamReader(path))
{
    while (!reader.EndOfStream)
    {
        var line = reader.ReadLine();
        var values = line.Split(',');
        if (!(values[0].Contains(id)))
        {
            lines.Add(line);
        }
    }
}
File.WriteAllLines(path, lines);
return this;

```

Unit 13 Solutions: Integrating a .NET application

Edit Sales Order:

```
{
//MODIFY SALES ORDER

    string so_number = TextBox1.Text;
    Item so = getInnovator().newItem("Sales Order", "edit");
    so.setAttribute("where", "[sales_order].so_number='" +
        so_number + "'");
    so.setProperty("po_number", TextBox3.Text);
    so.setProperty("shipping_method", DropDownList1.Text );
    int box = CheckBox1.Checked ? 1 : 0;
    so.setProperty("multiple_shipment", box.ToString());
    Item result = so.apply();

    if (result.isError())
    {
        StatusMessage.Text = result.getErrorString();
        return;
    }
    StatusMessage.Text = "Changes Saved";
}

Protected Sub Button2_Click(sender As Object, e As EventArgs)
    'MODIFY SALES ORDER

    Dim so_number As String = TextBox1.Text
    Dim so As Item = getInnovator().newItem("Sales Order","edit")
    so.setAttribute("where", "[sales_order].so_number='" & _
        so_number & "'")
    so.setProperty("po_number", TextBox3.Text)
    so.setProperty("shipping_method", DropDownList1.Text)
    Dim box as Integer = If((Checkbox1.Checked), 1, 0)
    so.setProperty("multiple_shipment", box.ToString())
    Dim result as Item = so.apply()

    If result.isError() Then
        StatusMessage.Text = result.getErrorString()
        Return
    End If
    StatusMessage.Text = "Changes Saved"
End Sub
```

Delete Sales Order:

```

protected void Button3_Click(object sender, EventArgs e)
{
    //Delete SO
    string so_number = TextBox1.Text;
    Item so = getInnovator().newItem("Sales Order", "delete");
    so.setAttribute("where", "[sales_order].so_number='" +
        so_number + "'");
    Item result = so.apply();

    if (result.isError())
    {
        StatusMessage.Text = result.getErrorString();
        return;
    }
    StatusMessage.Text = "Order Removed";
}

Protected Sub Button3_Click(sender As Object, e As EventArgs)
    'Delete SO
    Dim so_number As String = TextBox1.Text
    Dim so As Item = _
        getInnovator().newItem("Sales Order", "delete")
    so.setAttribute("where", "[sales_order].so_number='" & _
        so_number & "'")
    Dim result as Item = so.apply()

    If result.isError() Then
        StatusMessage.Text = result.getErrorString()
        Return
    End If
    StatusMessage.Text = "Order Removed"
End Sub

```

Unit 14 Solutions: RESTful Services

1. GET

```
URL_SERVER/server/odata/Sales Order?$filter=shipping_method eq 'Federal Express' and multiple_shipment eq 1
```

2. POST

```
URL_SERVER/server/odata/Sales Order
```

```
{
  "po_number" : "12345",
  "shipping_method" : "UPS",
  "multiple_shipment" : "1"
}
```

3. PATCH

```
URL_SERVER/server/odata/Sales Order('[GUID FROM PREVIOUS RESPONSE]')
{
  "po_number" : "54321"
}
```

4. DELETE

```
URL_SERVER/server/odata/Sales Order('[GUID FROM PREVIOUS RESPONSE]')
```

Unit 15 Solutions: Importing Data with Batch Loader

- Exercise 1 – Importing users with the Batch Loader client interface.

Steps

- Use the provided import file “users.csv” for this exercise. Open the file and analyze its data structure.

	A	B	C	D
1	Login Name	Password	First Name	Last Name
2	Test1	607920b6	User1	Test1
3	Test2	607920b6	User2	Test2

- b. Configure the Batch Loader utility to import the file, as shown below. Some of the options, within the red boxes below, may be different to accommodate individual environment requirements.

Database Options	
Server	http://127.0.0.1/InnovatorServer/
Database	PCR17DS
User	admin
Load Options	
DataFilePath	C:\WAREHOUSE\SANDBOX\users.csv
Delimiter	.
WorkerProcesses	1
Threads	1
LinesPerProcess	100
Encoding	Unicode (UTF-8)
FirstRow	1
LastRow	-1
Log Options	
LogFilePath	C:\Vras Utility Batch Loader\Logfiles\BatchLoaderLog.log
LogLevel	3
Preview Options	
PreviewRows	10
MaxColumns	10
FirstRowIsColumnN	True

- c. The Source Preview panel should look like the picture below:

Source Preview			
Login Name	Password	First Name	Last Name
Test1	607920b64fe136f9ab2389e371852af2	User1	Test1
Test2	607920b64fe136f9ab2389e371852af2	User2	Test2

- d. Use the Wizard to properly map the fields, as shown below:


Template Wizard
—
□
×

Item Type: User

Data Source Column	Target Property
Login Name	login_name [string]
Password	password [md5]
First Name	first_name [string]
Last Name	last_name [string]

- e. Save the mapping template for the next exercise.



- f. Use the 'green arrow' on the Batch Loader toolbar to perform the import operation. Observe job execution and completion at the bottom panel of the Batch Loader utility user interface.
- g. Open Batch Loader log file to see import operation details.
- h. Verify users were added from the Aras Innovator client user interface.
2. Exercise 2 – Import users with the Batch Loader command line interface. Edit the data mapping template you saved in Exercise 1 above to import users contained in the “users2.csv” provided import file.

Basic syntax of the Batch Loader command line interface.

BatchLoaderCMD -d [path to source flat file] -c [path to config file] -t [path to template file]

Sample config file and template file are shown below:

Configuration file

```
<BatchLoaderConfig>
  <server>fill in server URL</server>
  <db>fill in database</db>
  <user>admin</user>
  <password>innovator</password>
  <max_processes>1</max_processes>
  <threads>1</threads>
  <lines_per_process>100</lines_per_process>
  <delimiter>,</delimiter>
  <encoding>utf-8</encoding>
  <first_row>2</first_row>
  <last_row>-1</last_row>
  <log_file>path to logfile folder</log_file>
  <log_level>3</log_level>
</BatchLoaderConfig>
```

Template file

```

<BatchLoaderPrototype>
  <Item type='User' action='add'>
    <login_name>@1</login_name>
    <password>@2</password>
    <first_name>@3</first_name>
    <last_name>@4</last_name>
  </Item>
</BatchLoaderPrototype>

```

Unit 16 Solutions: Scheduler**1. Service Method to call:**

```

string msg = "Service method executed";
CCO.Utilities.WriteDebug("ServerDebug", msg);
return this;

```

2. Config file settings:

```

<?xml version="1.0" encoding="utf-8" ?>
<innovators>
  <innovator>
    <server>http://localhost/Innovator12</server>
    <database>DevelopingSolutions12</database>
    <username>admin</username>
    <password>innovator</password>
    <http_timeout_seconds>600</http_timeout_seconds>
    <job>
      <method>Write Debug Log</method>
      <months>*</months>
      <days>*</days>
      <hours>*</hours>
      <minutes>2,4,6,8, ...</minutes>
    </job>
  </innovator>
  <eventLoggingLevel>2</eventLoggingLevel>
  <intervalMinutes>2</intervalMinutes>
</innovators>

```

3. Start (or restart) the Innovator Service from Windows Services under Administration.

Index

A

actions	
argument passing	123
creating	102
defined	100
dialog box, custom	343
Generic	104
Item	106
Item Query	107
ItemType	105
addRelationship method	140
AML	
add example	46
and/or tag	32
attributes, passing	125
built-in actions	43
condition property attribute	33
creating custom actions	119
date and time	34
delete example	52
edit example	48
item attributes	40
Item Properties	39
item property tags	30
item tag attributes (primary)	28
order by attribute	42
promoteItem example	51
related_id tag	36
relationships tag	35
reverse lookup	38
select attribute	42
where attribute	42
API on-line help	63
appendItem method	144
apply method	71, 73, 130
applyAML method	114
applyMethod method	114, 120
applySQL method	114
aras	
AlertSuccess/AlertError	178
Confirm/Prompt	179
evalMethod	180
evalMethodItem	180
get paths	181
get URLs	181
get user info	183
get working directory	181
Aras Object (Client)	176
aras.Object	176

B

batchloader	
AML template	311
command line	317
example template	320
loading data	313
main screen	307
overview	306
relationship data	314
template wizard	310

C

cancelw action	151
CCO Utilities class methods	
WriteDebug	94
clear caches	92
client events	
field	195
form	193
form and field	193
form event method	201
grid/row	378
ItemType	189, 191
OnSearchDialog	202
tabs	368
clone method	130
closew action	151
comparing IOM and AML	71
context item	65
context item, on a form	196
createPropertyItem method	134

D

debugging	
client	83
server	84, 86
displaying prompt/confirm window	179
document.thisItem context	196
document.thisItem.getProperty method	198
document.thisItem.setProperty method	198
DOM object	197
dropdown list, dynamic from database	355
dynamic dropdown list	356

E

Email method	150
--------------	-----

F

federated ItemType	256
federated properties	250
federated property	257
federated property events	254
federation, using events	259
fetchDefaultPropertyValues method	134
fetchFileProperty	136
fetchLockStatus method	150
File methods	136
Form dialog box	341

G

generic methods	119
getNewId method	116
Get Controlled Item example	165
getAction method	133
getAttribute method	125, 133
getBaseUrl method	181
getDatabase method	183
getErrorCode method	149
getErrorDetail method	149
getErrorSource method	149
getErrorString method	149
getId method	133
getIdentityList method	183
getInnovator method	77
getItemById method	116
getItemByIndex method	144
getItemByKeyedName method	116
getItemByXPath method	144
getItemCount method	144
getItemInDom method	116
getLoginName method	183
getNextSequence method	116
getProperty method	134
getPropertyAttribute method	134
getPropertyCondition method	134
getPropertyItem method	134
getRelatedItem method	142
getRelationships method	142
getServerBaseUrl method	181
GetTabId method	370
getType method	133
getUserId method	118
getUserId method	183
getUserType method	183
getWorkingDir method	181
grid cell method	382
grid control	
accessing	375
callbacks	376
cell events	380
customizing	372
row events	377
grid/row event method	379

H

handleItemChange	198
hide/show tabs	370
HTML, custom controls	354

I

Innovator class methods	
applyAML	114
applyMethod	114, 120
applySQL	114
getInnovator	77
getItemById	116
getItemByKeyedName	116
getItemInDom	116
getNewId	116
getNextSequence	116
getUserID	118
newError	116
newItem	74, 75, 116
newResult	78, 116
ScalcMD5	118
instantiatew method	150
isAdminUser method	183
isCollection method	132
isEmpty method	132
isError method	132
isLocked method	132
isLogical method	132
isNew method	132
isRoot method	132
Item class methods	
addRelationship	140
appendItem	144
apply	71, 73, 130
boolean methods (is...)	132
clone	130
collection methods	144
createPropertyItem	134
createRelationship	141
Email	150
extended methods	150
fetchDefaultPropertyValues	134
fetchFileProperty	136
fetchLockStatus	150
getAction	133
getAttribute	125, 133
getErrorCode	149
getErrorDetail	149
getErrorSource	149
getErrorString	149
getId	133
getItemByIndex	144
getItemByXPath	144
getItemCount	144
getLockStatus	150
getProperty	134
getPropertyAttribute	134

- getPropertyCondition 134
 - getPropertyItem 134
 - getRelatedItem 142
 - getRelationships 142
 - getResult 150
 - getType 133
 - instantiatew 150
 - loadAML 130
 - lockItem 150
 - logical methods (and/or) 146
 - newItem 71, 73, 148
 - newOr/newAnd/newNot 146
 - newXMLDocument 148
 - promote 150
 - relationship overview 138
 - removeAttribute 133
 - removeItem 144
 - removeLogical 146
 - removeProperty 134
 - removePropertyAttribute 134
 - setAction 133
 - setAttribute 73, 133
 - setErrorCode 149
 - setErrorDetail 149
 - setErrorSource 149
 - setErrorString 149
 - setFileProperty 136
 - setID 133
 - setNewID 133
 - setProperty 71, 73, 134
 - setPropertyAttribute 134
 - setPropertyCondition 134
 - setPropertyItem 134
 - setRelatedItem 140
 - setType 133
 - ToString 150
 - unlockItem 150
 - item factory 66
 - item types, supported 76
- L**
- loadAML method 130
 - lockItem method 150
 - logging 94
 - logging server events 95
 - logging system events 363
- M**
- ME keyword 65
 - Message Resources
 - client 387
 - client create new 392
 - client file layout 388
 - client retrieving 389
 - localization 394
 - server 390
 - server create new 393
 - server retrieving 391
 - method
 - creating 67
 - editor *Solution Studio Editor*
 - invoking 61
 - testing 72
 - method types 60
 - methodology, Aras 14
 - methods
 - generic 119
 - Multilingual messages 394
- N**
- newAnd/newOr/newNot method 146
 - newError method 79, 116, 149
 - newItem 74
 - newItem method 71, 73, 116, 148
 - newResult method 78, 116
 - newXMLDocument method 148
- O**
- OnSearchDialog event 202
- P**
- parent.thisItem context 376
 - programming references 64
 - promote method 150
- R**
- removeAttribute method 133
 - removeItem method 144
 - removeLogical method 146
 - removeProperty method 134
 - removePropertyAttribute method 134
 - RequestState Class
 - add 400
 - clear 402
 - contains 403
 - count 403
 - deletion use case 406
 - overview 399
 - related items use case 409
 - remove 402
 - retrieve 401
 - versioning use case 404
 - returning an Item 66
- S**
- ScalcMD5 method 118
 - scheduler service
 - configure 326
 - installing 325
 - monitoring 328
 - overview 324

[illegible]

ARAS University

www.aras.com/training

